

Data Structures

UNIT-1:

Basic concepts - Algorithm Specification, Data Abstraction , Performance analysis - time complexity and space complexity, Asymptotic Notation - Big O, Omega and Theta notations, Introduction to Linear and Non Linear data structures.

Linear list – Singly linked list implementation, insertion, deletion and searching operations on linear list, Circular linked list implementation, Doubly linked list implementation, insertion, deletion and searching operations. Applications of linked lists.

Data Structure:

- **Data Structure** is a way to store and organize data in a computer so that it can be used efficiently.
- A data structure is seen as a logical concept that address two fundamental concerns.
 - i) How the data will be stored, and
 - ii) What operations will be performed on it.
- There are two types of data structures-Linear and Non-Linear Data Structures.
- Examples:Arrays,Stack,Queues,Trees, etc..

Algorithm Specification

Algorithm:

- An **Algorithm** is a finite set of instructions that, if followed, accomplishes a particular task.
- **Characteristics of an Algorithm:**
 - i. **INPUT:** Zero or more quantities are externally supplied.
 - ii. **OUTPUT :** At least one quantity is produced.
 - iii. **DEFINITENESS :** Each instruction is clear and unambiguous.
 - iv. **FINITENESS :** If we trace out the instructions of an algorithm, then for all cases, the algorithm terminates after a finite number of steps.
 - v. **EFFECTIVENESS :** An algorithm is also generally expected to be effective. This means that all of the operations to be performed in the algorithm must be sufficiently basic that they can in principle be done exactly and in a finite length of time.

Algorithm Specification....

➤ **Algorithm can be described in three ways:**

- **Natural language like English:** In this any natural language is used to describe the algorithm. This is informal way of describing the algorithm.
- **Graphic representation called flowchart:** This method will work well when the algorithm is small and simple.
- **Pseudo-code Method:** In this method, we should typically describe algorithms as program, It uses the structural conventions of a normal programming language, but is intended for human reading rather than machine reading. There is no need to follow all the syntax rules of a programming language.

➤ **Data Abstraction:**

Abstraction means displaying only essential information and hiding the details. Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation.

Performance Analysis

➤ **Performance analysis** of an **algorithm** depends upon two factors i.e. amount of memory used and amount of compute time consumed on any CPU.

➤ It can be done using following two complexities:

i) **Space Complexity:**

➤ It is a function of the space needed by the algorithm to run to completion.

➤ It is calculated as sum of following two components:

i) **Fixed Component**- This includes space of simple variables, constants and instruction space. It does not depend on input and output characteristics.

ii) **Variable Component**- This includes space of reference variables and the recursion stack space. It depends on input and output characteristics.

$$\text{Space}(A) = \text{Fixed Component}(A) + \text{Variable Component}(A)$$

Performance Analysis....

ii) Time Complexity:

- It is a function of time needed by the algorithm to complete its execution.
- It is calculated by counting the no. of elementary steps performed by algorithm to completes its execution.

Example 1 :Computing Space and Time complexity of an algorithm:

```
1. Algorithm Sum(a,n)
2. {
3.   s=0.0;
4.   for i=1 to n do
5.     s=s+a[i];
6.   return s;
7. }
```

Space Complexity:

Fixed Components are: s - 1 word

i - 1 word

n- 1 word

Total=3 words

Variable Component are: a - n words

Space Complexity=(3+n) words= $O(n)$

Performance Analysis....

Time Complexity:

1. Algorithm Sum(a,n)	No. of elementary steps
2. {	
3. s=0.0; -----	1
4. for i=1 to n do -----	n+1
5. s=s+a[i]; -----	n
6. return s; -----	1
7. }	
Total-----	2n+3

Time Complexity is given as $O(n)$

Performance Analysis....

Example 2:

	No. of elementary steps
1. Algorithm Add(A,B,n)	
2. {	
3. for(i=0;i< n;i++) -----	n+1
4. {	
5. for(j=0;j< n;j++)-----	n*(n+1)
6. {	
7. C[i,j]=A[i,j]+B[i,j] -----	n*n
8. }	
9. }	
10. }	

Total----- $n+1+n*(n+1)+n*n=2n^2+ 2n+1$

Time Complexity= $O(n^2)$

Space Complexity is : Fixed components are i,j,n-----each 1 word=3 words

Variable components are A- $n*n$,B- $n*n$ and C- $n*n=3n^2$

Total= $3n^2+3$

Space complexity= $O(n^2)$

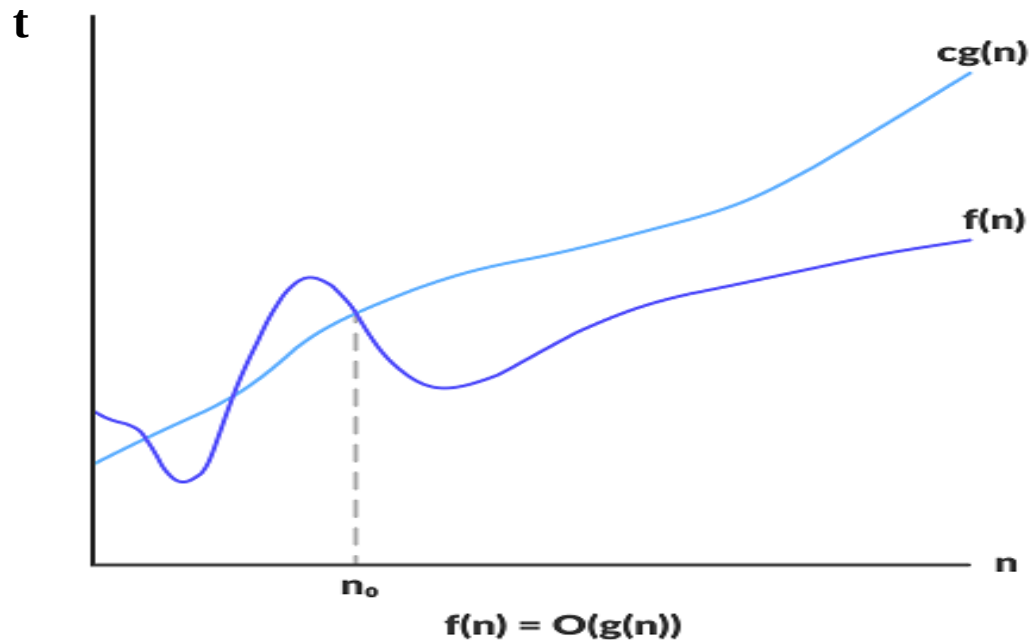
Asymptotic Notations

- Asymptomatic analysis of an algorithm refers to defining the mathematical boundation of its runtime performance.
- Using this we can conclude the best case, average case and worst case scenario of an algorithm.
- The time required by an algorithm falls under three types:
 - **Best case:** Minimum time required for program execution
 - **Average case:** Average time required for program execution.
 - **Worst case:** Maximum time required for program execution.
- Asymptomatic notations are the expressions that are used to represent the complexity of an algorithm.
- Types of Asymptomatic notations are:
 - Big-Oh Notation (O)**
 - Big-Omega Notation(Ω)**
 - Big -Theta Notation (Θ)**

Asymptotic Notations...

➤ Big-Oh Notation (O)

- It describes the worst case scenerio,it represents the upper bound running time complexity of an algorithm .

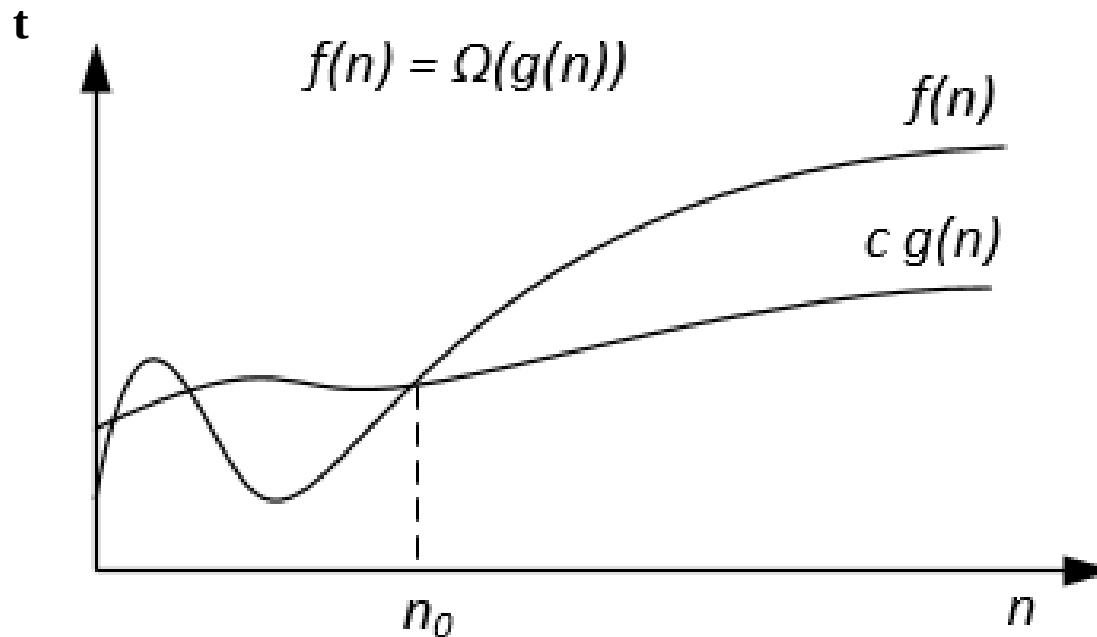


- if $f(n) \leq cg(n) \forall n \geq n_0$ where $c > 0$ and $n_0 \geq 1$ then we can say that $f(n) = O(g(n))$.

Asymptotic Notations...

➤ Big-Omega Notation(Ω)

- It describes the best case scenerio,it represents the lower bound running time complexity of an algorithm .

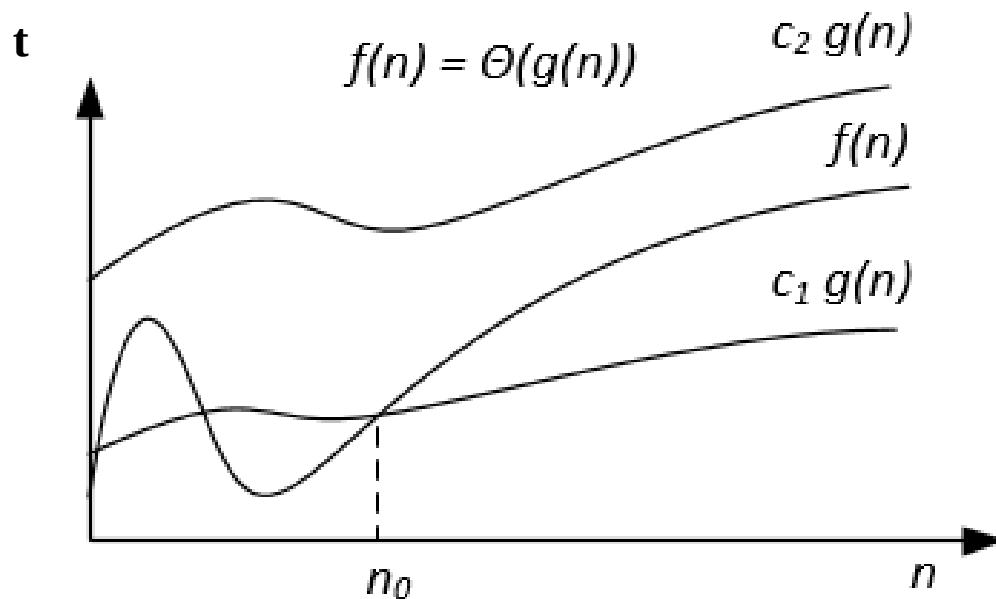


- if $f(n) \geq c \cdot g(n) \forall n \geq n_0$ where $c > 0$ and $n_0 \geq 1$ then we can say that $f(n) = \Omega(g(n))$.

Asymptotic Notations...

➤ Big -Theta Notation (Θ)

- It describes the average case scenerio,it represents the lower bound and upper bound of an algorithm .



- if $c_1 g(n) \leq f(n) \leq c_2 g(n) \forall n \geq n_0$ where $c_1, c_2 > 0$ and $n_0 \geq 1$ then we can say that $f(n) = \Theta(g(n))$.

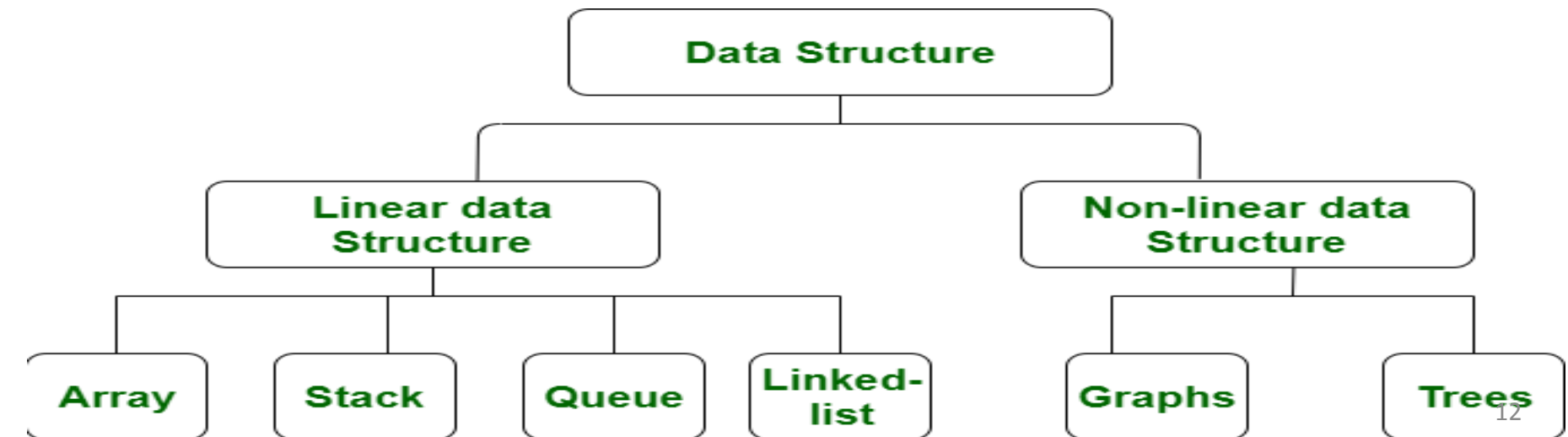
Linear & Non-linear Data Structure

➤Linear Data Structure:

- In this data elements are arranged sequentially or in linear fashion.
- It involves single level ,therefore we can traverse all the elements in single run only.
- Linear data structures are easy to implement because computer memory is arranged in a linear way.

➤Non -Linear Data Structure:

- In this data elements are not arranged sequentially or linearly .
- It does not involve single level, therefore, we can't traverse all the elements in single run.
- They are not easy to implement in comparison to linear data structure.
- It utilizes computer memory efficiently in comparison to a linear data structure.



Linear & Non-linear Data Structure....

➤ Difference between Linear and Non-Linear Data structures :

Sr. No.	Key	Linear Data Structures	Non-linear Data Structures
1	Data Element Arrangement	In linear data structure, data elements are sequentially connected and each element is traversable through a single run.	In non-linear data structure, data elements are hierarchically connected and are present at various levels.
2	Levels	In linear data structure, all data elements are present at a single level.	In non-linear data structure, data elements are present at multiple levels.
3	Implementation complexity	Linear data structures are easier to implement.	Non-linear data structures are difficult to understand and implement as compared to linear data structures.
4	Traversal	Linear data structures can be traversed completely in a single run.	Non-linear data structures are not easy to traverse and needs multiple runs to be traversed completely.
5	Memory utilization	Linear data structures are not very memory friendly and are not utilizing memory efficiently.	Non-linear data structures uses memory very efficiently.
6	Examples	Array, List, Queue, Stack.	Graph,Tree.

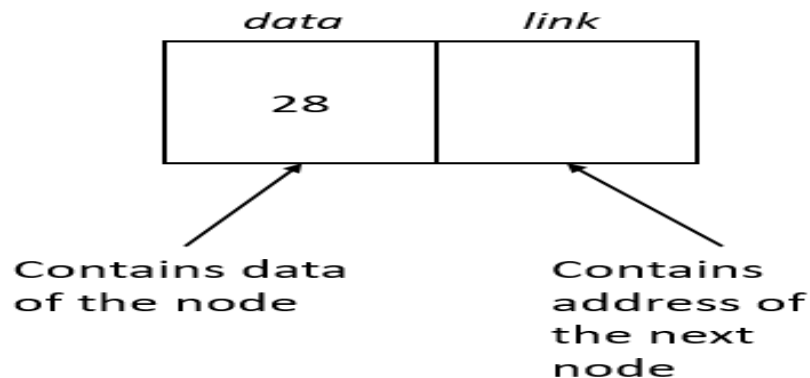
Linear / Linked List

➤ Types of linear/linked lists:

- 1) Single Linked List
- 2) Doubly Linked List
- 3) Circular Linked List

➤ Single Linked List :

- A linked list is a linear collection of data elements. These data elements are called nodes.
- In single linked list every node contains two fields, data field and link field -a pointer to the next node/address of next node.



Single Linked List....

- The node in a single linked list is declared as

```
struct node  
{  
  int data;  
  struct node *next;  
};
```

- The last node address field in the single linked list contains NULL.



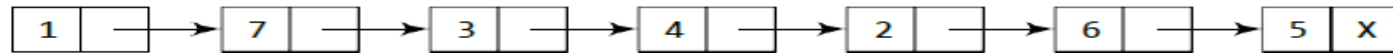
- **Operations performed on a single linked list:**

- I. Insertion
- II. Deletion
- III. Searching
- IV. Traversing

Single Linked List....

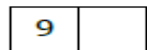
➤ Insertion:

Case 1: The new node is inserted at the beginning

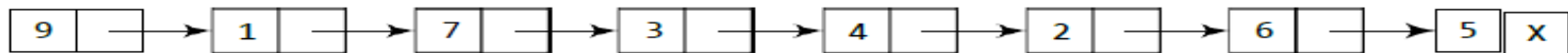


START

Allocate memory for the new node and initialize its DATA part to 9.

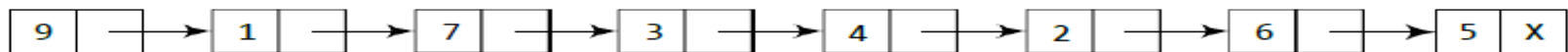


Add the new node as the first node of the list by making the NEXT part of the new node contain the address of START.



START

Now make START to point to the first node of the list.



START

Step 4: SET NEW_NODE -> DATA = VAL

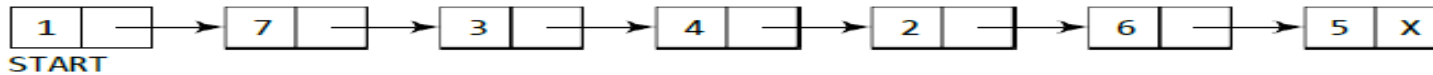
Step 5: SET NEW_NODE -> NEXT = START

Step 6: SET START = NEW_NODE

Step 7: EXIT

Single Linked List....

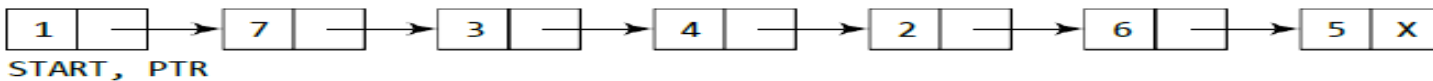
Case 2: The new node is inserted at the end



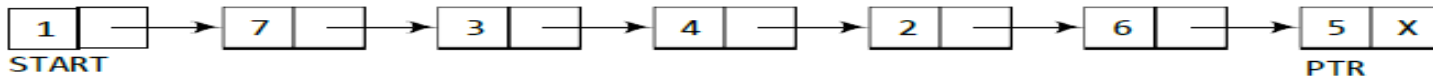
Allocate memory for the new node and initialize its DATA part to 9 and NEXT part to NULL.



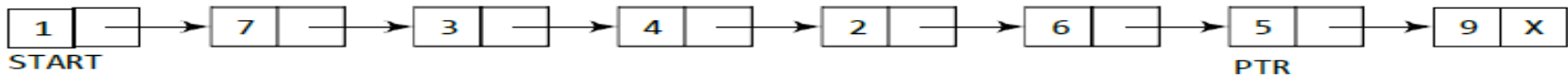
Take a pointer variable PTR which points to START.



Move PTR so that it points to the last node of the list.



Add the new node after the node pointed by PTR. This is done by storing the address of the new node in the NEXT part of PTR.



Step 4: SET NEW_NODE -> DATA = VAL

Step 5: SET NEW_NODE -> NEXT = NULL

Step 6: SET PTR = START

Step 7: Repeat Step 8 while PTR -> NEXT != NULL

Step 8: SET PTR = PTR -> NEXT

[END OF LOOP]

Step 9: SET PTR -> NEXT = NEW_NODE

Step 10: EXIT

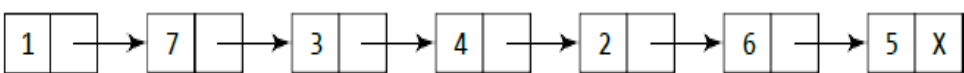
Single Linked List....

Case 3: The new node is inserted after a given node

Allocate memory for the new node and initialize its DATA part to 9.

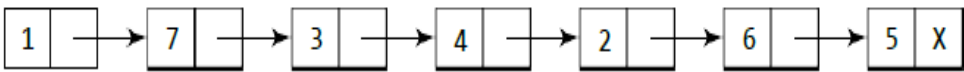


Take two pointer variables PTR and PREPTR and initialize them with START so that START, PTR, and PREPTR point to the first node of the list.

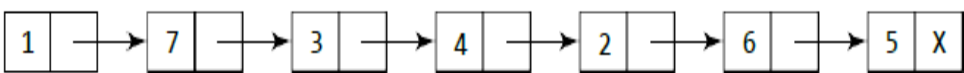


START
PTR
PREPTR

Move PTR and PREPTR until the DATA part of PREPTR = value of the node after which insertion has to be done. PREPTR will always point to the node just before PTR.

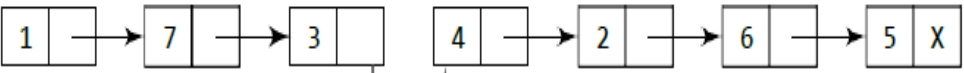


START PREPTR PTR

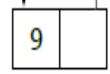


START PREPTR PTR

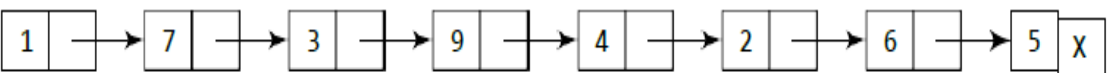
Add the new node in between the nodes pointed by PREPTR and PTR.



START PREPTR PTR



NEW_NODE



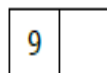
START

Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET PTR = START
Step 6: SET PREPTR = PTR
Step 7: Repeat Steps 8 and 9 while PREPTR -> DATA != NUM
Step 8: SET PREPTR = PTR
Step 9: SET PTR = PTR -> NEXT
 [END OF LOOP]
Step 10: PREPTR -> NEXT = NEW_NODE
Step 11: SET NEW_NODE -> NEXT = PTR
Step 12: EXIT

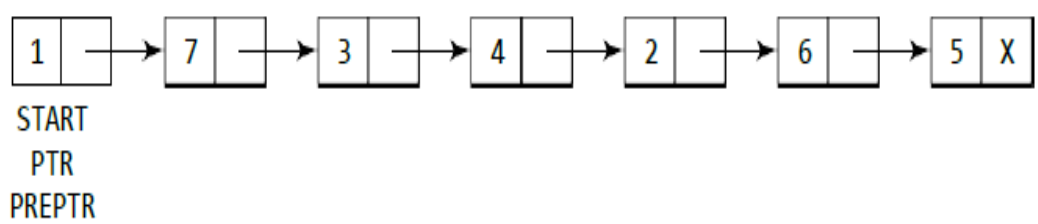
Single Linked List....

Case 4: The new node is inserted before a given node

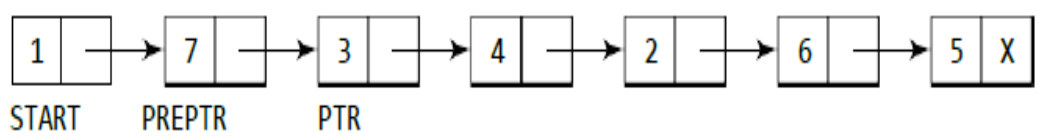
Allocate memory for the new node and initialize its DATA part to 9.



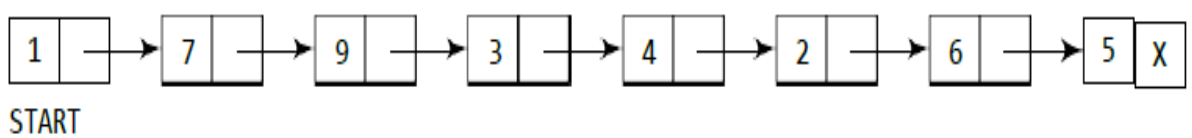
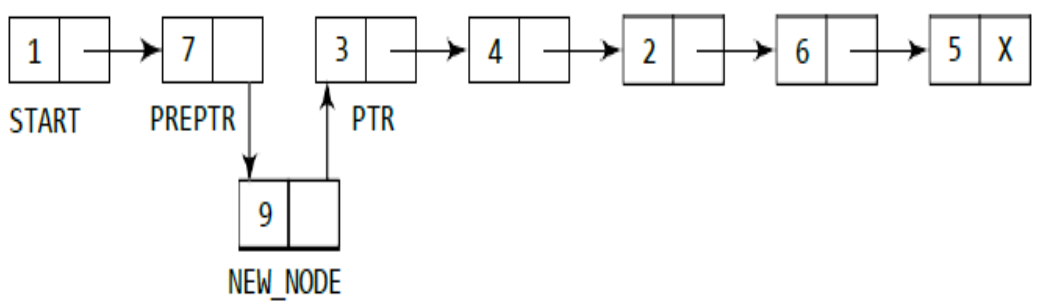
Initialize PREPTR and PTR to the START node.



Move PTR and PREPTR until the DATA part of PTR = value of the node before which insertion has to be done. PREPTR will always point to the node just before PTR.



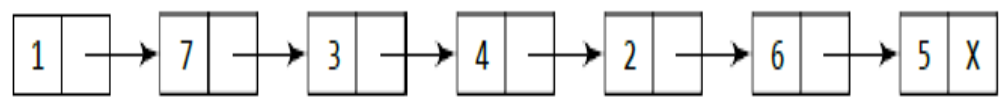
Insert the new node in between the nodes pointed by PREPTR and PTR.



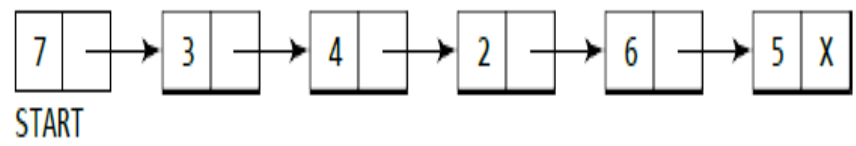
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET PTR = START
Step 6: SET PREPTR = PTR
Step 7: Repeat Steps 8 and 9 while PTR -> DATA != NUM
Step 8: SET PREPTR = PTR
Step 9: SET PTR = PTR -> NEXT
[END OF LOOP]
Step 10: PREPTR -> NEXT = NEW_NODE
Step 11: SET NEW_NODE -> NEXT = PTR
Step 12: EXIT

Single Linked List....

➤ Deletion : Case 1: The first node is deleted



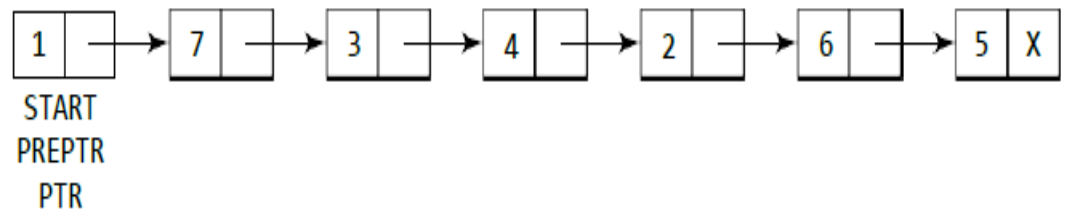
START
Make START to point to the next node in sequence.



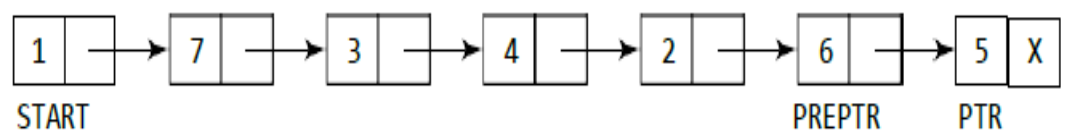
- Step 1: IF START = NULL
Write UNDERFLOW
Go to Step 5
[END OF IF]
- Step 2: SET PTR = START
- Step 3: SET START = START -> NEXT
- Step 4: FREE PTR
- Step 5: EXIT

Case 2: The last node is deleted

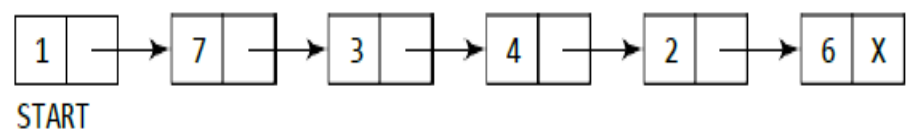
Take pointer variables PTR and PREPTR which initially point to START.



Move PTR and PREPTR such that NEXT part of PTR = NULL. PREPTR always points to the node just before the node pointed by PTR.



Set the NEXT part of PREPTR node to NULL.

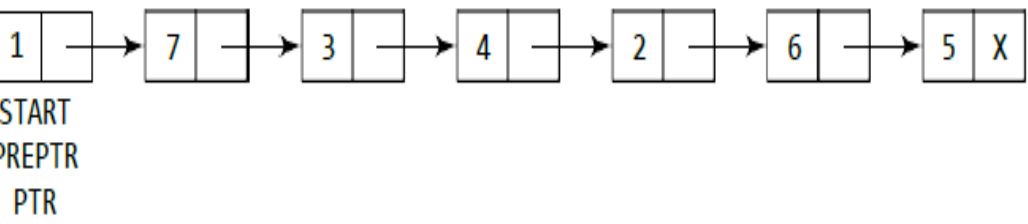


- Step 1: IF START = NULL
Write UNDERFLOW
Go to Step 8
[END OF IF]
- Step 2: SET PTR = START
- Step 3: Repeat Steps 4 and 5 while PTR->NEXT != NULL
- Step 4: SET PREPTR = PTR
- Step 5: SET PTR = PTR->NEXT [END OF LOOP]
- Step 6: SET PREPTR -> NEXT = NULL
- Step 7: FREE PTR
- Step 8: EXIT

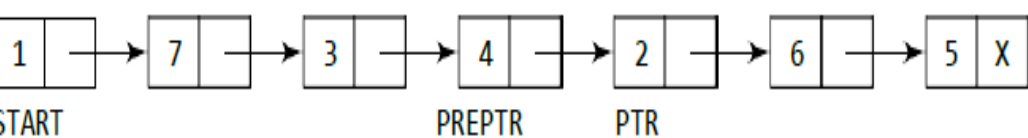
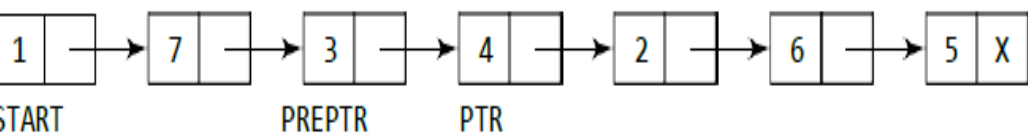
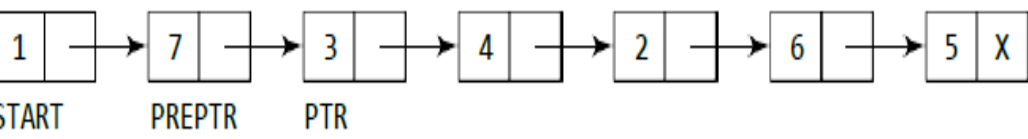
Single Linked List....

Case 3: The node after a given node is deleted

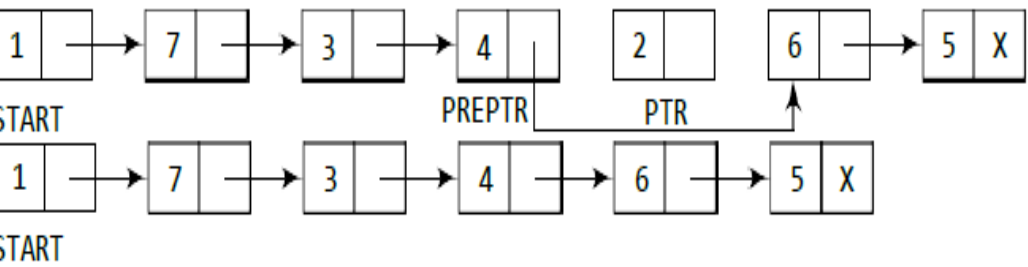
Take pointer variables PTR and PREPTR which initially point to START.



Move PREPTR and PTR such that PREPTR points to the node containing VAL and PTR points to the succeeding node.



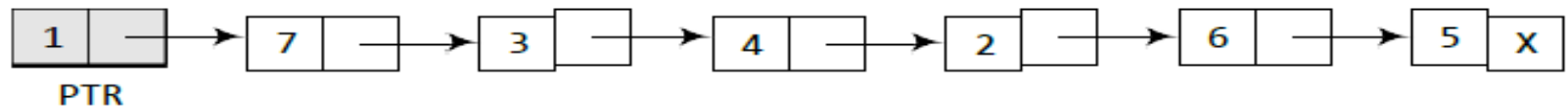
Set the NEXT part of PREPTR to the NEXT part of PTR.



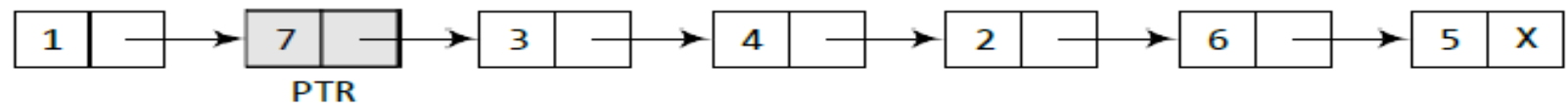
Step 1: IF START = NULL
 Write UNDERFLOW
 Go to Step 10
 [END OF IF]
Step 2: SET PTR = START
Step 3: SET PREPTR = PTR
Step 4: Repeat Steps 5 and 6 while PREPTR -> DATA != NUM
Step 5: SET PREPTR = PTR
Step 6: SET PTR = PTR -> NEXT [END
 OF LOOP]
Step 7: SET TEMP = PTR
Step 8: SET PREPTR -> NEXT = PTR -> NEXT
Step 9: FREE TEMP
Step 10: EXIT

Single Linked List....

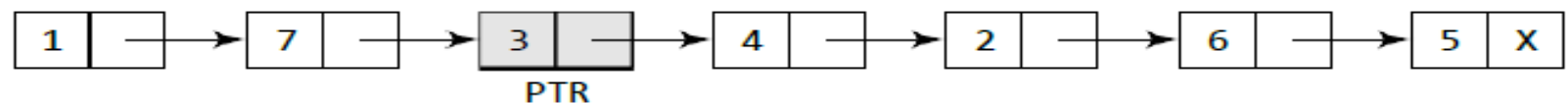
➤ Searching:



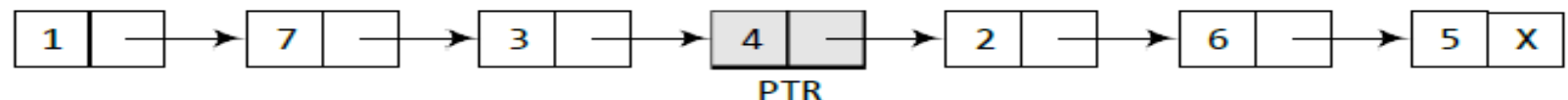
Here PTR -> DATA = 1. Since PTR -> DATA != 4, we move to the next node.



Here PTR -> DATA = 7. Since PTR -> DATA != 4, we move to the next node.



Here PTR -> DATA = 3. Since PTR -> DATA != 4, we move to the next node.



Here PTR -> DATA = 4. Since PTR -> DATA = 4, POS = PTR. POS now stores the address of the node that contains VAL

```
Step 1: [INITIALIZE] SET PTR = START
Step 2: Repeat Step 3 while PTR != NULL
Step 3:   IF VAL = PTR -> DATA
           SET POS = PTR
           Go To Step 5
         ELSE
           SET PTR = PTR -> NEXT
         [END OF IF]
       [END OF LOOP]
Step 4: SET POS = NULL
Step 5: EXIT
```

Single Linked List....

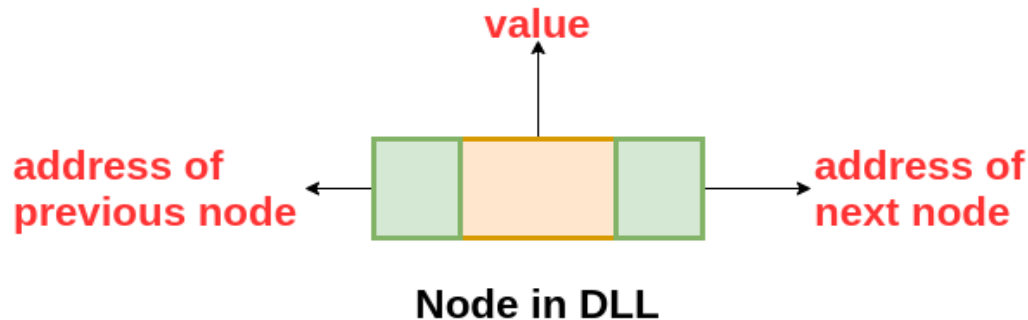
➤ Traversing:

Traversing a linked list means accessing the nodes of the list in order to perform some processing on them.

```
Step 1: [INITIALIZE] SET PTR = START  
Step 2: Repeat Steps 3 and 4 while PTR != NULL  
Step 3:         Apply Process to PTR -> DATA  
Step 4: SET PTR = PTR -> NEXT [END OF  
        LOOP]  
Step 5: EXIT
```

Doubly Linked List

- A doubly linked list or a two-way linked list is a more complex type of linked list which contains a pointer to the next as well as the previous node in the sequence.
- It is a collection of node , in which each node will contain three fields- a pointer to the previous node ,data, a pointer to the next node.



- The declaration of node in double linked list is given as

```
struct node
{
    struct node *prev;
    int data;
    struct node *next;
};
```

- The PREV field of the first node and the NEXT field of the last node will contain NULL.
- The NEXT field is used to traverse the list in forward direction and PREV field is used to traverse the list in backward direction.

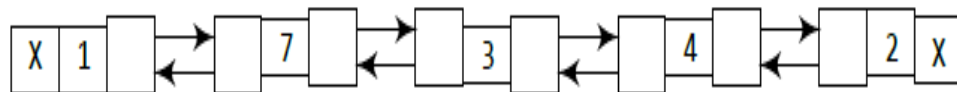
Doubly Linked List....

➤ Operation on Doubly linked list are

- I. Insertion
- II. Deletion
- III. Searching
- IV. Traversing

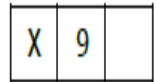
➤ Insertion:

Case 1: The new node is inserted at the beginning.

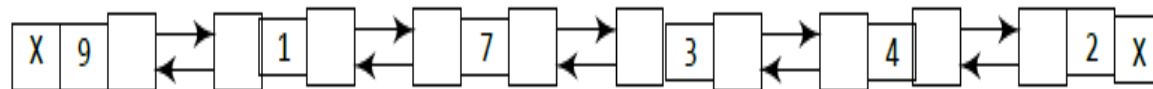


START

Allocate memory for the new node and initialize its DATA part to 9 and PREV field to NULL.



Add the new node before the START node. Now the new node becomes the first node of the list.

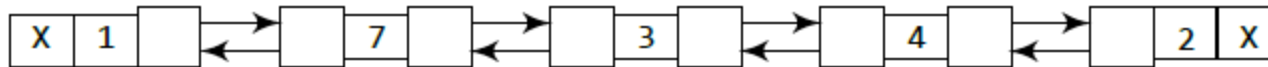


START

```
SET NEW_NODE -> DATA = VAL
SET NEW_NODE -> PREV = NULL
SET NEW_NODE -> NEXT = START
SET START -> PREV = NEW_NODE
SET START = NEW_NODE
EXIT
```

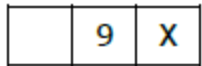
Doubly Linked List....

➤ Case 2: The new node is inserted at the end.

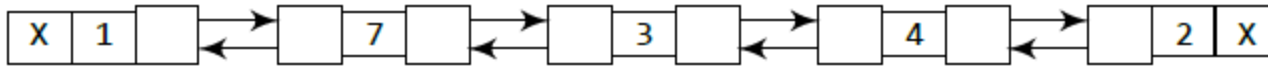


START

Allocate memory for the new node and initialize its DATA part to 9 and its NEXT field to NULL.

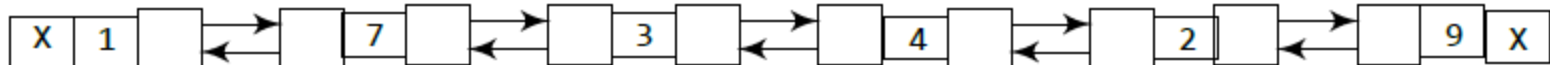


Take a pointer variable PTR and make it point to the first node of the list.



START, PTR

Move PTR so that it points to the last node of the list. Add the new node after the node pointed by PTR.



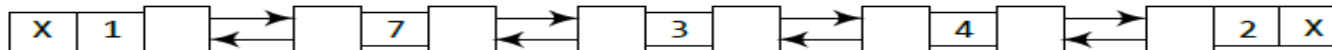
START

PTR

```
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET NEW_NODE -> NEXT = NULL
Step 6: SET PTR = START
Step 7: Repeat Step 8 while PTR -> NEXT != NULL
Step 8:     SET PTR = PTR -> NEXT
           [END OF LOOP]
Step 9: SET PTR -> NEXT = NEW_NODE
Step 10: SET NEW_NODE -> PREV = PTR
Step 11: EXIT
```

Doubly Linked List....

➤ **Case 3: The new node is inserted after a given node.**

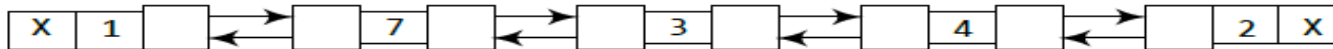


START

Allocate memory for the new node and initialize its DATA part to 9.

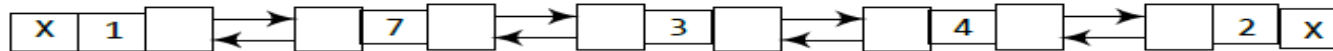


Take a pointer variable PTR and make it point to the first node of the list.



START, PTR

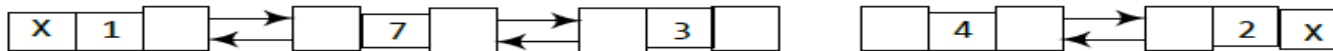
Move PTR further until the data part of PTR = value after which the node has to be inserted.



START

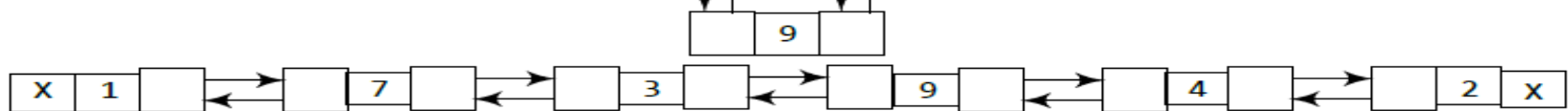
PTR

Insert the new node between PTR and the node succeeding it.



START

PTR



START

Step 4: SET NEW_NODE -> DATA = VAL

Step 5: SET PTR = START

Step 6: Repeat Step 7 while PTR -> DATA != NUM

Step 7: SET PTR = PTR -> NEXT

[END OF LOOP]

Step 8: SET NEW_NODE -> NEXT = PTR -> NEXT

Step 9: SET NEW_NODE -> PREV = PTR

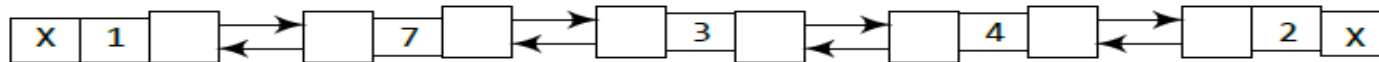
Step 10: SET PTR -> NEXT = NEW_NODE

Step 11: SET PTR -> NEXT -> PREV = NEW_NODE

Step 12: EXIT

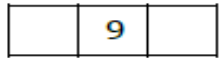
Doubly Linked List....

➤ Case 4: The new node is inserted before a given node.

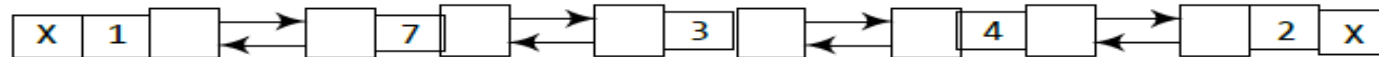


START

Allocate memory for the new node and initialize its DATA part to 9.

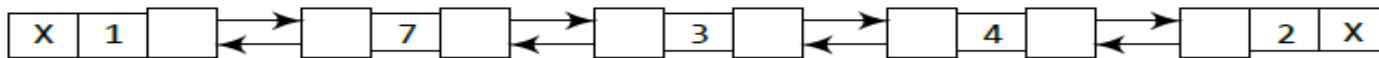


Take a pointer variable PTR and make it point to the first node of the list.



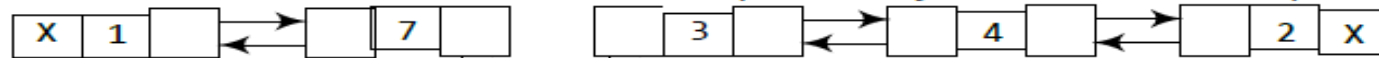
START, PTR

Move PTR further so that it now points to the node whose data is equal to the value before which the node has to be inserted.

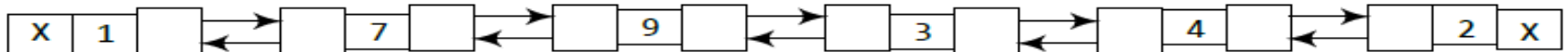
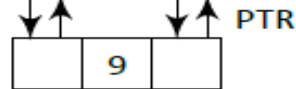


START

Add the new node in between the node pointed by PTR and the node preceding it.



START



START

Step 4: SET NEW_NODE -> DATA = VAL

Step 5: SET PTR = START

Step 6: Repeat Step 7 while PTR -> DATA != NUM

Step 7: SET PTR = PTR -> NEXT

[END OF LOOP]

Step 8: SET NEW_NODE -> NEXT = PTR

Step 9: SET NEW_NODE -> PREV = PTR -> PREV

Step 10: SET PTR -> PREV = NEW_NODE

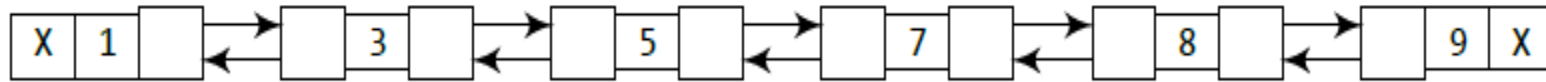
Step 11: SET PTR -> PREV -> NEXT = NEW_NODE

Step 12: EXIT

Doubly Linked List....

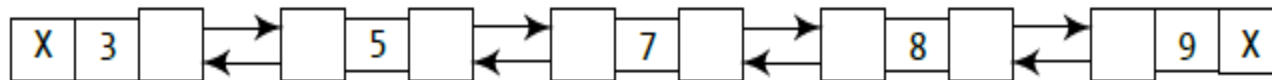
➤ Deletion:

Case 1: The first node is deleted.



START

Free the memory occupied by the first node of the list and make the second node of the list as the START node.



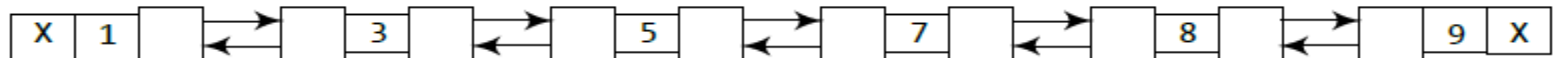
START

```
Step 1: IF START = NULL
        Write UNDERFLOW
        Go to Step 6
    [END OF IF]
Step 2: SET PTR = START
Step 3: SET START = START -> NEXT
Step 4: SET START -> PREV = NULL
Step 5: FREE PTR
Step 6: EXIT
```

Doubly Linked List....

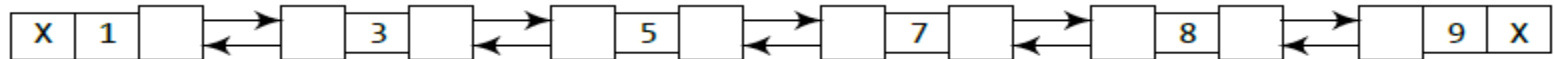
➤ Deletion:

Case 2: The last node is deleted.



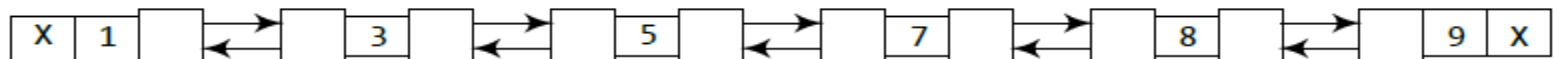
START

Take a pointer variable PTR that points to the first node of the list.



START, PTR

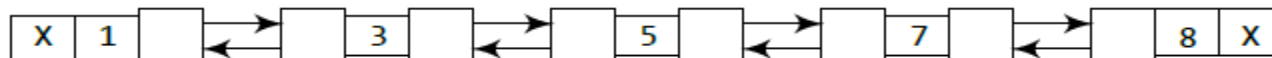
Move PTR so that it now points to the last node of the list.



START

PTR

Free the space occupied by the node pointed by PTR and store NULL in NEXT field of its preceding node.



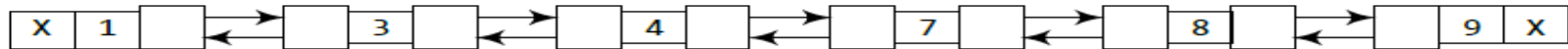
START

```
Step 1: IF START = NULL
        Write UNDERFLOW
        Go to Step 7
    [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Step 4 while PTR -> NEXT != NULL
Step 4:     SET PTR = PTR -> NEXT
    [END OF LOOP]
Step 5: SET PTR -> PREV -> NEXT = NULL
Step 6: FREE PTR
Step 7: EXIT
```

Doubly Linked List....

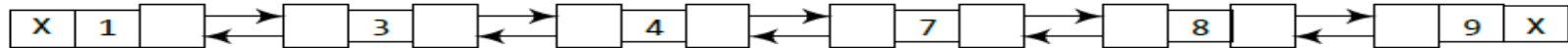
➤ Deletion:

Case 3: The node after a given node is deleted.



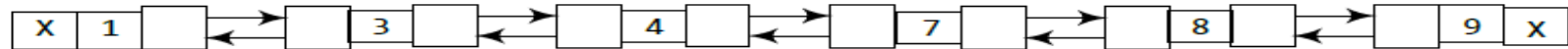
START

Take a pointer variable PTR and make it point to the first node of the list.



START, PTR

Move PTR further so that its data part is equal to the value after which the node has to be inserted.



START

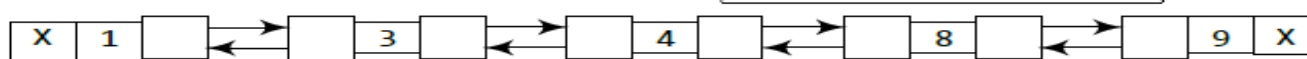
PTR

Delete the node succeeding PTR.



START

PTR



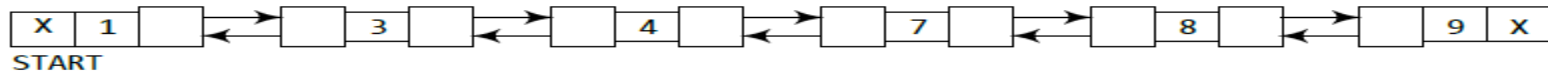
START

```
Step 1: IF START = NULL
        Write UNDERFLOW
        Go to Step 9
    [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Step 4 while PTR -> DATA != NUM
Step 4:     SET PTR = PTR -> NEXT
    [END OF LOOP]
Step 5: SET TEMP = PTR -> NEXT
Step 6: SET PTR -> NEXT = TEMP -> NEXT
Step 7: SET TEMP -> NEXT -> PREV = PTR
Step 8: FREE TEMP
Step 9: EXIT
```

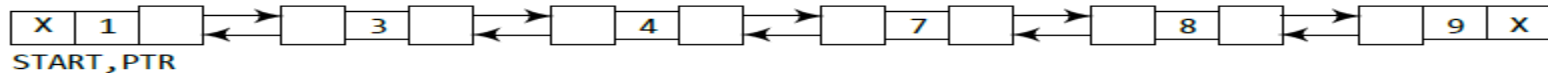
Doubly Linked List....

➤ Deletion:

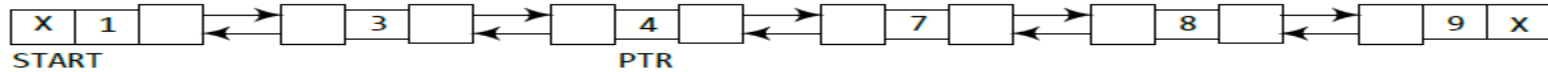
Case 4: The node before a given node is deleted.



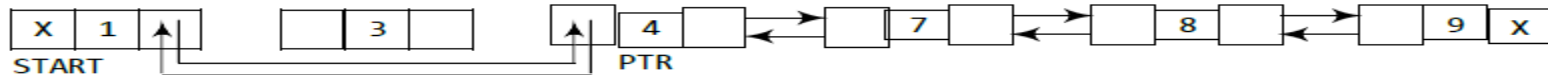
Take a pointer variable PTR that points to the first node of the list.



Move PTR further till its data part is equal to the value before which the node has to be deleted.



Delete the node preceding PTR.



```
Step 1: IF START = NULL
        Write UNDERFLOW
        Go to Step 9
    [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Step 4 while PTR -> DATA != NUM
Step 4: SET PTR = PTR -> NEXT
    [END OF LOOP]
Step 5: SET TEMP = PTR -> PREV
Step 6: SET TEMP -> PREV -> NEXT = PTR
Step 7: SET PTR -> PREV = TEMP -> PREV
Step 8: FREE TEMP
Step 9: EXIT
```

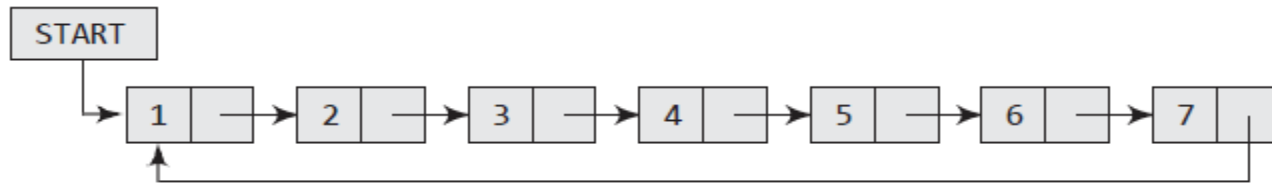
Circular Linked List

➤ Circular Linked List Types:

- I. Circular Single Linked List
- II. Circular Doubly Linked List

➤ Circular Single Linked List:

- In a circular single linked list, the last node contains a pointer to the first node of the list i.e the last node address field contains the address of the first node.



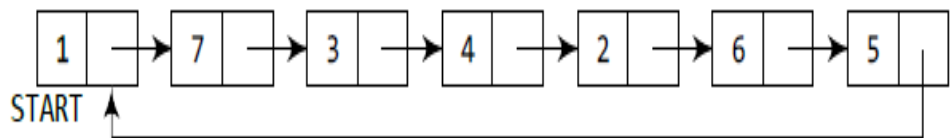
- Operation on circular single linked list are:

- I. Insertion
- II. Deletion
- III. Searching
- IV. Traversing

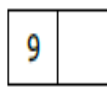
Circular Single Linked List.....

➤ Insertion:

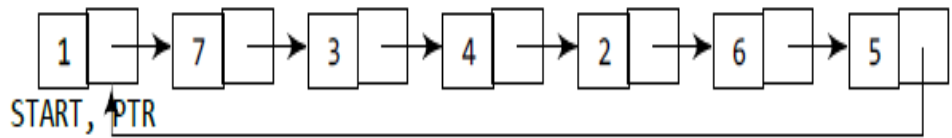
Case 1: The new node is inserted at the beginning of the circular linked list.



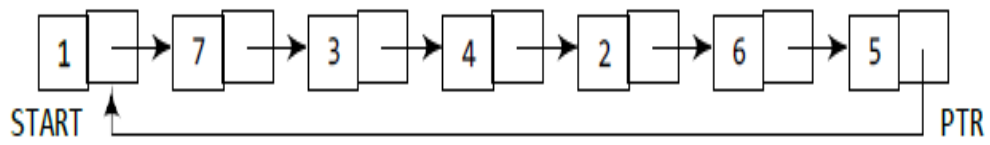
Allocate memory for the new node and initialize its DATA part to 9.



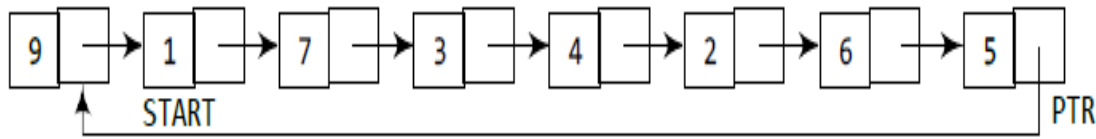
Take a pointer variable PTR that points to the START node of the list.



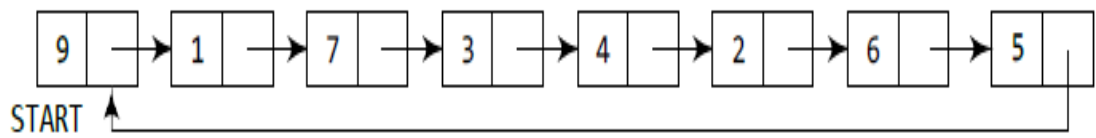
Move PTR so that it now points to the last node of the list.



Add the new node in between PTR and START.



Make START point to the new node.

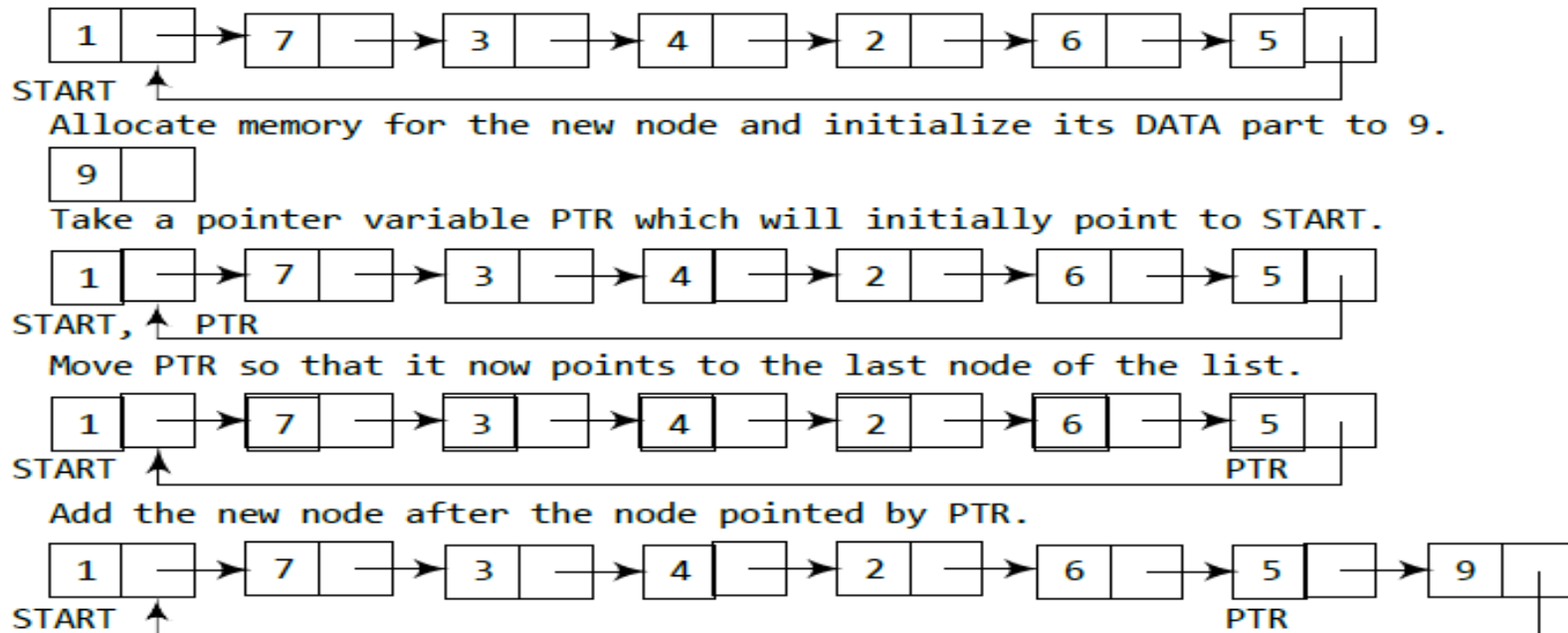


Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET PTR = START
Step 6: Repeat Step 7 while PTR -> NEXT != START
Step 7: PTR = PTR -> NEXT
[END OF LOOP]
Step 8: SET NEW_NODE -> NEXT = START
Step 9: SET PTR -> NEXT = NEW_NODE
Step 10: SET START = NEW_NODE
Step 11: EXIT

Circular Single Linked List.....

➤ Insertion:

Case 2: The new node is inserted at the end of the circular linked list.

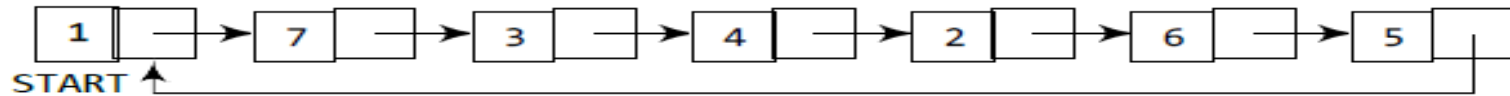


```
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET NEW_NODE -> NEXT = START
Step 6: SET PTR = START
Step 7: Repeat Step 8 while PTR -> NEXT != START
Step 8:     SET PTR = PTR -> NEXT
          [END OF LOOP]
Step 9: SET PTR -> NEXT = NEW_NODE
Step 10: EXIT
```

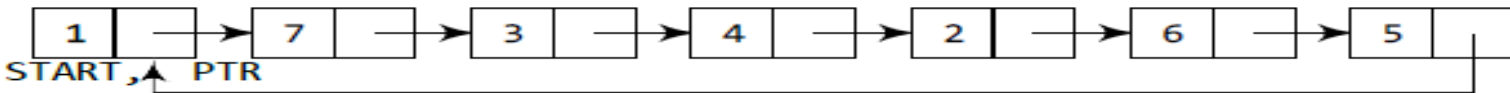
Circular Single Linked List.....

➤ Deletion:

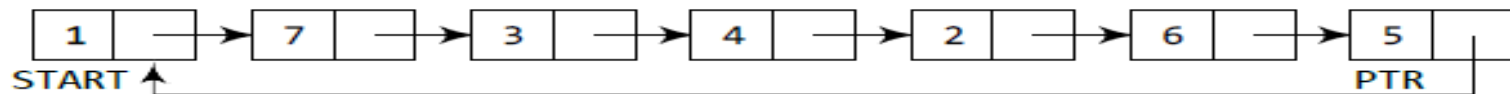
Case 1: The first node is deleted.



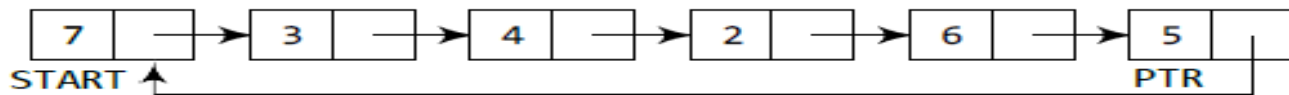
Take a variable PTR and make it point to the START node of the list.



Move PTR further so that it now points to the last node of the list.



The NEXT part of PTR is made to point to the second node of the list and the memory of the first node is freed. The second node becomes the first node of the list.

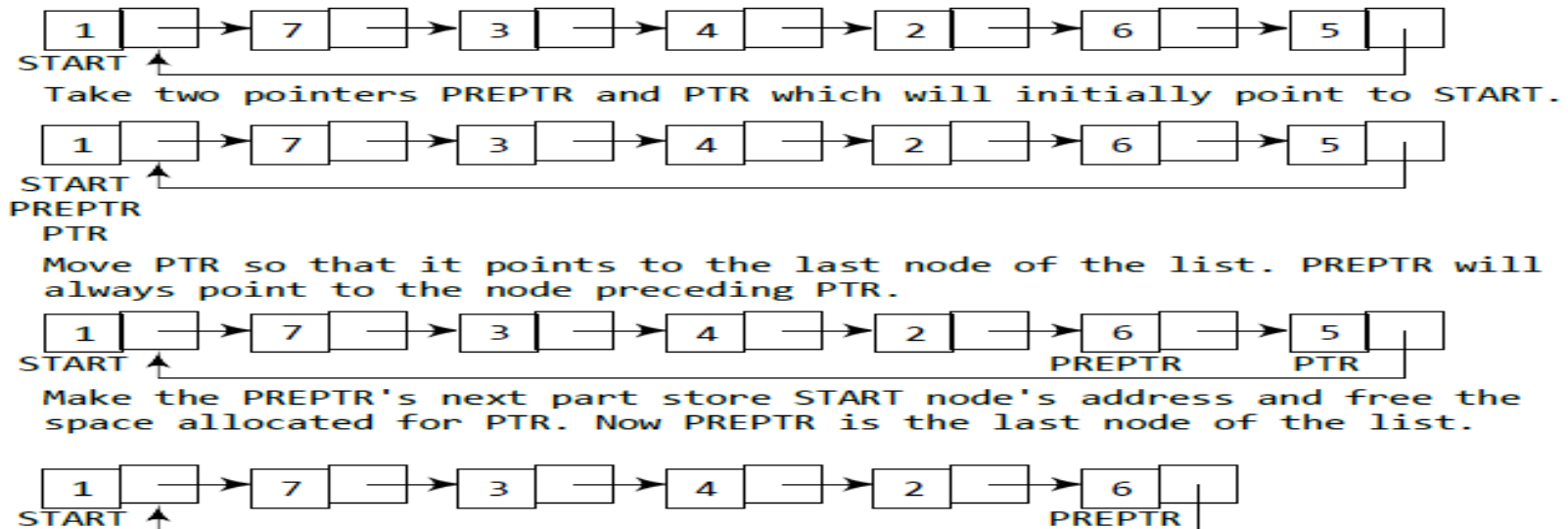


```
Step 1: IF START = NULL
        Write UNDERFLOW
        Go to Step 8
    [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Step 4 while PTR -> NEXT != START
Step 4:     SET PTR = PTR -> NEXT
    [END OF LOOP]
Step 5: SET PTR -> NEXT = START -> NEXT
Step 6: FREE START
Step 7: SET START = PTR -> NEXT
Step 8: EXIT
```


Circular Single Linked List.....

➤ Deletion:

Case 2: The last node is deleted.



```
Step 1: IF START = NULL
        Write UNDERFLOW
        Go to Step 8
    [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Steps 4 and 5 while PTR->NEXT != START
Step 4:     SET PREPTR = PTR
Step 5:     SET PTR = PTR->NEXT
    [END OF LOOP]
Step 6: SET PREPTR->NEXT = START
Step 7: FREE PTR
Step 8: EXIT
```

Circular Doubly Linked List

➤ Circular Doubly Linked List:

- Circular doubly linked list doesn't contain NULL in any of the node.
- The last node NEXT field of the list contains the address of the first node of the list.
- The first node PREV field of the list contain address of the last node of the list.

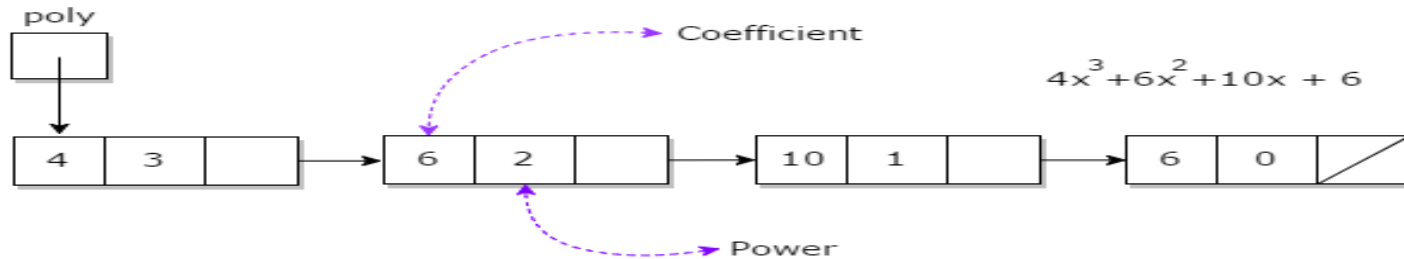


➤ Operations:

- I. Insertion
 - II. Deletion
 - III. Searching
 - IV. Traversing
- Implementation is more complex than other linked lists.

Applications of Linked Lists

➤ Polynomial Representation



- Implementation of different data structures like stack ,queues etc.
- Dynamic Memory allocation.
- Used in Image viewer-previous and next images are linked , hence we can access by using next and previous buttons.
- In Operating System, all the running applications are kept in a circular linked list and the OS gives a fixed time slot to all for running.
- Used in web browser to access the next and previous web pages while browsing.

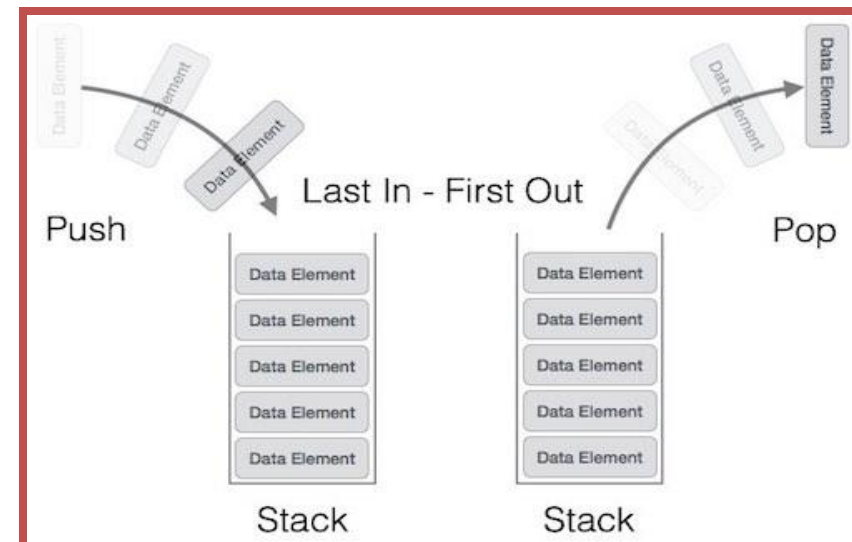
UNIT - II

Stacks-Operations, array and linked representations of stacks, stack applications-infix to postfix conversion, postfix expression evaluation, recursion implementation.

Queues-operations, array and linked representations. Circular Queue operations, Dequeue, applications of queue.

Stack :

- A stack is a linear data structure, which works on the principle of LIFO/FILO.
- Stack is closed at one end and opened at other end.
- The elements can be added and removed from the stack only at the top(opened end).
- Basic Operations performed on stack are:
 - **PUSH-Insertion**
 - **POP-Deletion**
 - **PEEK>Returns top most element**
- Stack maintains a variable called TOP, if $TOP = -1$ then the stack is empty.

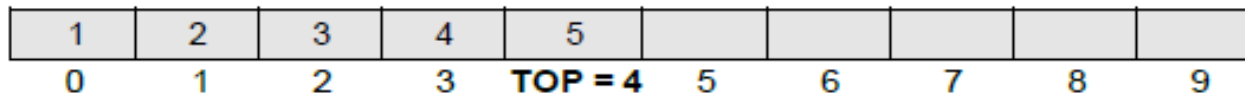


Stack...

- When an element is pushed into the stack TOP value is incremented by 1 i.e TOP++.
- When an element is popped from the stack TOP value is decremented by 1 i.e TOP--.
- Stack can be implemented in two ways:
 - I. Stack Implementation using Arrays
 - II. Stack Implementation using Linked List

➤ Stack Implementation using Arrays:

- Stack can be represented using array as

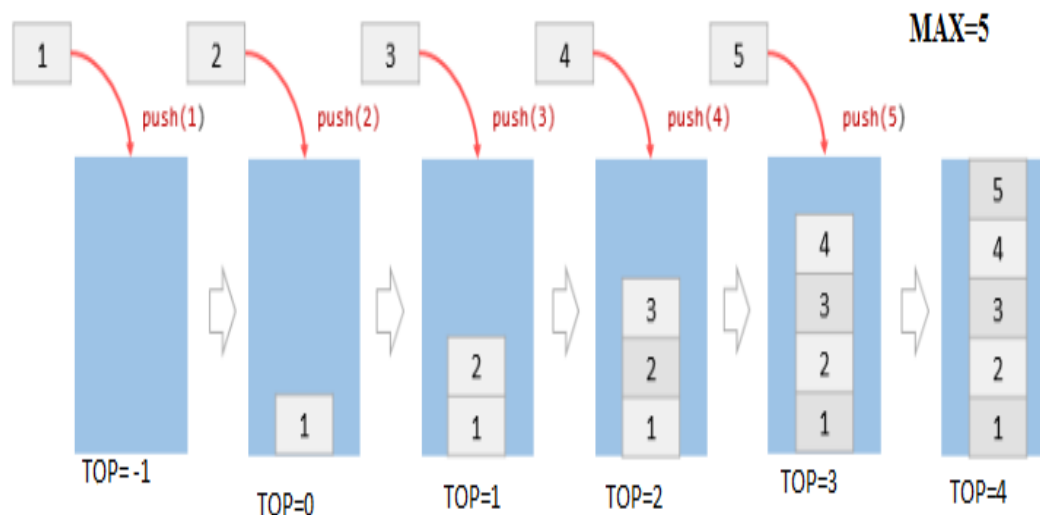


- Here, the maximum size(MAX) of the stack is 10 i.e it can hold up to 10 elements.
- Basic Operations performed on stack implemented using arrays are:
 - PUSH
 - POP
 - PEEK

Stack Implementation using Array...

➤ PUSH:

- The push operation is used to insert an element into the stack. The new element is added at the topmost position of the stack.
- While inserting the value, we must first check if $TOP = MAX - 1$, because if that is the case, then the stack is full and no more insertions can be done. This condition is called as **stack overflow**.
- If the above condition fails then we can perform the insertion into the stack, then TOP value is incremented by 1.

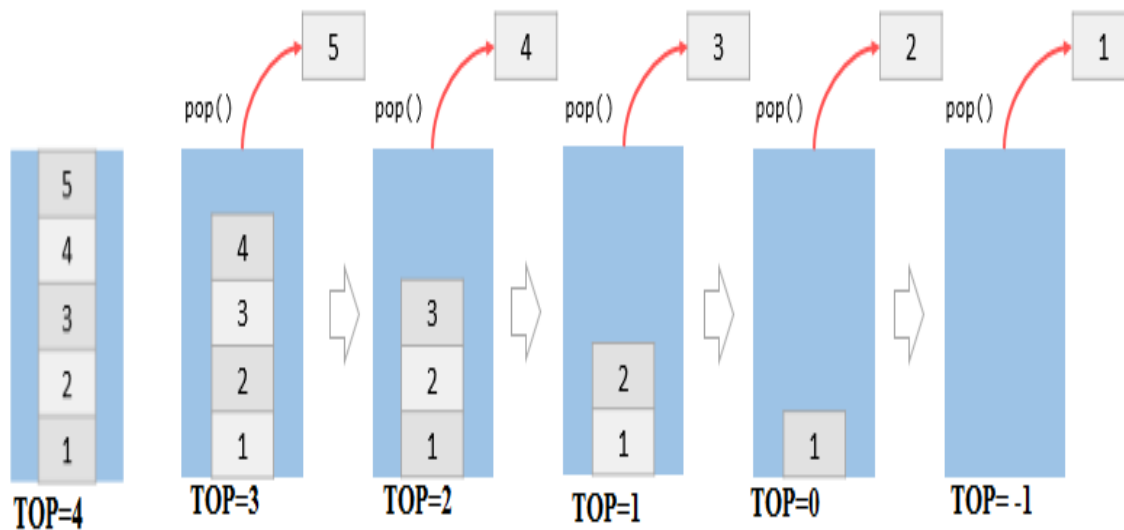


```
Step 1: IF  $TOP = MAX - 1$   
        PRINT "OVERFLOW"  
        Goto Step 4  
        [END OF IF]  
Step 2: SET  $TOP = TOP + 1$   
Step 3: SET  $STACK[TOP] = VALUE$   
Step 4: END
```

Stack Implementation using Array...

➤ POP:

- The POP operation is used to delete an element from top of the stack.
- Before deleting an element we have to check whether $TOP = -1$, if that is the case POP operation is not possible. This condition is called as **stack underflow**.
- Other wise we can pop an element from the stack by decrementing the TOP by 1.

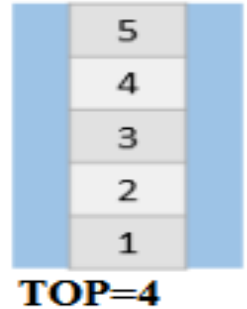


```
Step 1: IF TOP = NULL
        PRINT "UNDERFLOW"
        Goto Step 4
      [END OF IF]
Step 2: SET VAL = STACK[TOP]
Step 3: SET TOP = TOP - 1
Step 4: END
```

Stack Implementation using Array...

➤ PEEK:

- PEEK is an operation that returns the value of the topmost element of the stack without deleting it from the stack.
- If we perform PEEK operation on the stack, **it will first check whether the stack is empty or not**, if stack is not empty then it returns the top most element in the stack i.e 5 otherwise it has to print stack is empty.

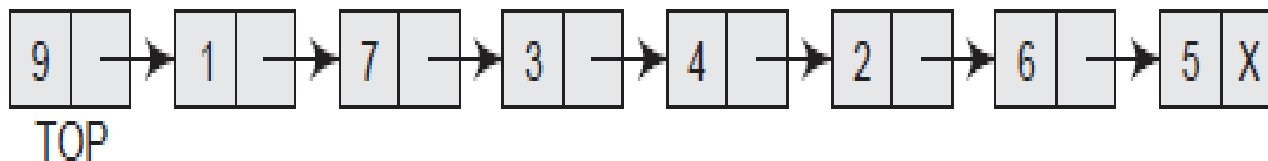


```
Step 1: IF TOP = NULL  
        PRINT "STACK IS EMPTY"  
        Goto Step 3  
Step 2: RETURN STACK[TOP]  
Step 3: END
```


Stack Implementation using Linked List

➤ Stack Implementation using Linked List:

- In a linked stack, every node has two parts—one that stores data and another that stores the address of the next node.
- if the stack size cannot be determined in advance, then linked representation is used.
- The START pointer of the linked list is used as TOP.
- All insertions and deletions are done at the node pointed by TOP.
- If TOP = NULL, then it indicates that the stack is empty.
- The linked representation of a stack is :



- A linked stack supports all the three stack operations, that is, push, pop, and peek.

Stack Implementation using Linked List....

➤Operations on a Linked Stack :

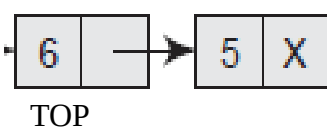
➤PUSH:

➤The push operation is used to insert an element into the stack. The new element is added at the topmost position of the stack.

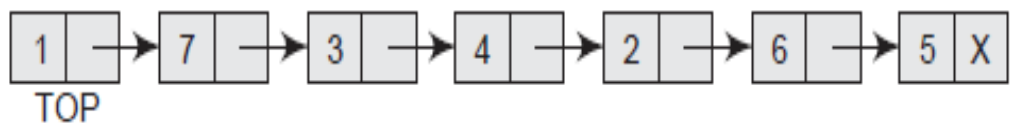
Push 5



Push 6



similarly Push 2,4,3,7,1 we get



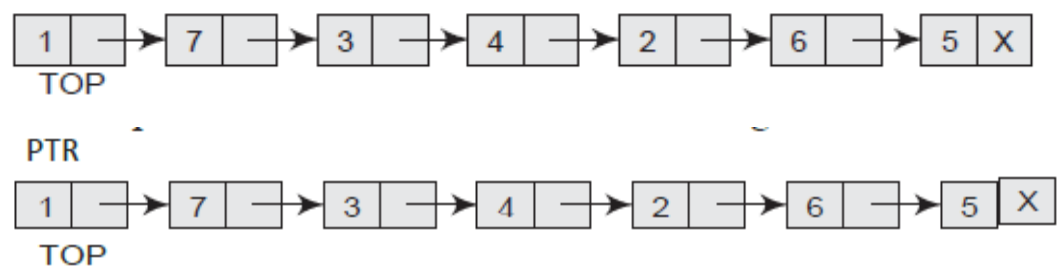
```
Step 1: Allocate memory for the new node and name it as NEW_NODE
Step 2: SET NEW_NODE -> DATA = VAL
Step 3: IF TOP = NULL
        SET NEW_NODE -> NEXT = NULL
        SET TOP = NEW_NODE
    ELSE
        SET NEW_NODE -> NEXT = TOP
        SET TOP = NEW_NODE
    [END OF IF]
Step 4: END
```

Stack Implementation using Linked List....

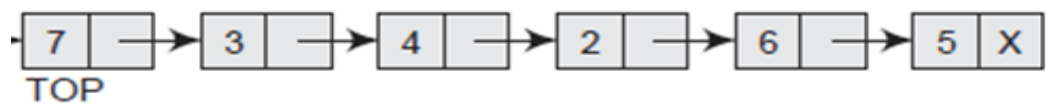
➤POP

- The pop operation is used to delete the topmost element from a stack.
- Before deleting the element, we must first check if TOP=NULL (stack empty).
- If an attempt is made to delete a value from a stack that is already empty, an **Underflow message** is printed.
- If TOP!=NULL , then top most element is deleted from the stack i,e in this case element 1 is deleted from the stack.

```
Step 1: IF TOP = NULL
        PRINT "UNDERFLOW"
        Goto Step 5
    [END OF IF]
Step 2: SET PTR = TOP
Step 3: SET TOP = TOP -> NEXT
Step 4: FREE PTR
Step 5: END
```



Now the new top element is 7.



Stack Implementation using Linked List....

➤ PEEK

- It is used to print the top element of stack.
- If TOP=NULL returns stack is empty, else prints the top element in the linked stack.



- If Peek operation is performed on the above stack ,it will return element 1.

```
Step 1:if(top==NULL)
        return "stack is empty";
Step 2: else
        return top ->data;
```

Applications of Stack

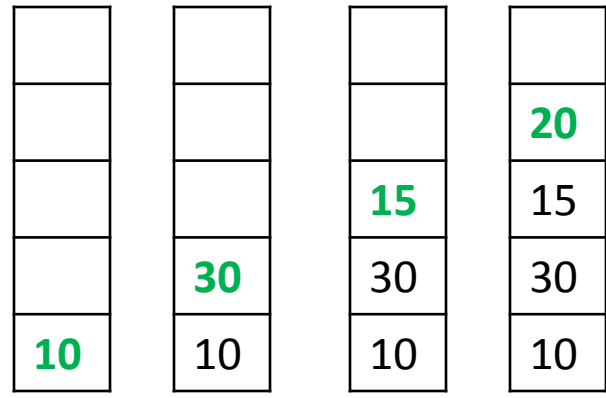
➤The applications of the stack are

1. Reversing a list

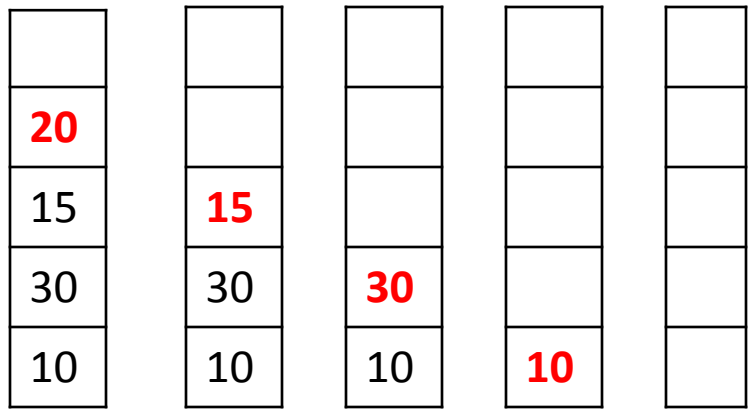
A list of numbers can be reversed by reading each number from an array starting from the first index and pushing it on a stack. Once all the numbers have been read, the numbers can be popped one at a time and then stored in the array starting from the first index.

Example: 10,30,15,20

Push:



Pop:



Reverse List: 20 15 30 10

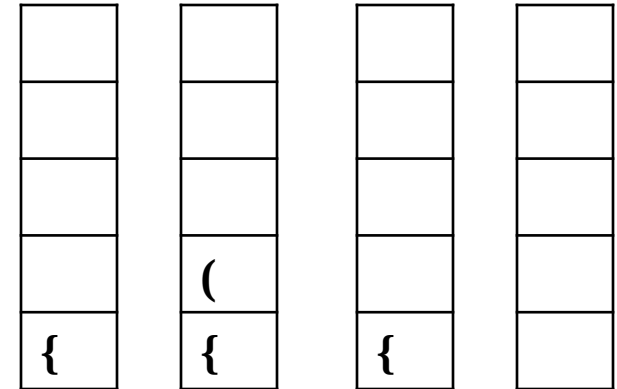
Applications of Stack...

2. Parentheses checker :

- Stacks can be used to check the validity of parentheses in any algebraic expression.
- An algebraic expression is valid if for every open bracket there is a corresponding closing bracket.

Example: { A + (B – C) }

- Here we scan the expression from left to right, when open bracket comes push it on to the stack ,when closed bracket comes pop the corresponding closed bracket from the stack.
- Finally if the stack is empty then parenthesis are well balanced other wise not well balanced.



3. Conversion of an infix expression into a postfix expression

- **Expression:** It is collection of operands and operator which can be evaluated to single value.
- **Different types of Expressions:**
 1. **Infix Expression**-operators is in between their operands. Ex:5+2
 2. **Prefix Expression**-operator is followed by its operands. Ex:+52
 3. **Postfix Expression**- operands are followed by its operator. Ex: 52+

Conversion of an infix expression into a postfix expression...

Example:

Infix: $5*4/2+7-8$

Prefix: $- + / * 5 4 2 7 8$

Postfix: $5 4 * 2 / 7 + 8 -$

$= (5*4)/2+7-8$

$= ((5*4)/2) + 7 - 8$

$= ((((5*4) / 2) + 7) - 8)$

➤ Algorithm to convert an infix expression into a postfix expression:

Step 1: Add ")" to the end of the infix expression

Step 2: Push "(" on to the stack

Step 3: Repeat until each character in the infix notation is scanned

IF a "(" is encountered, push it on the stack

IF an operand (whether a digit or a character) is encountered, add it to the postfix expression.

IF a ")" is encountered, then

a. Repeatedly pop from stack and add it to the postfix expression until a "(" is encountered.

b. Discard the "(" . That is, remove the "(" from stack and do not add it to the postfix expression

IF an operator 0 is encountered, then

a. Repeatedly pop from stack and add each operator (popped from the stack) to the postfix expression which has the same precedence or a higher precedence than 0

b. Push the operator 0 to the stack

[END OF IF]

Step 4: Repeatedly pop from the stack and add it to the postfix expression until the stack is empty

Step 5: EXIT

Conversion of an infix expression into a postfix expression...

Example: Infix Expression: $A - (B / C + (D \% E * F) / G) * H$

Step 1: $A - (B / C + (D \% E * F) / G) * H$)

Steps 2-5:

Infix Character Scanned	Stack	Postfix Expression
	(	
A	(	A
-	(-	A
(	(- (	A
B	(- (	A B
/	(- (/	A B
C	(- (/	A B C
+	(- (+	A B C /
(	(- (+ (	A B C /
D	(- (+ (	A B C / D
%	(- (+ (%	A B C / D
E	(- (+ (%	A B C / D E
*	(- (+ (* %	A B C / D E %
F	(- (+ (* %	A B C / D E % F
)	(- (+	A B C / D E % F *
/	(- (+ /	A B C / D E % F *
G	(- (+ /	A B C / D E % F *
)	(-	A B C / D E % F * G
*	(- *	A B C / D E % F * G / +
H	(- *	A B C / D E % F * G / + H
)		A B C / D E % F * G / + H * -

Postfix Expression : $A B C / D E F * \% G / + H * -$

Applications of Stack...

3. Evaluation of a postfix expression

- The computer first converts the infix expression into the equivalent postfix notation and then evaluates the postfix expression.
- Both these tasks—converting the infix notation into postfix notation and evaluating the postfix expression—make extensive use of stacks as the primary tool.

Example: The infix expression $9 - ((3 * 4) + 8) / 4$ can be written as $9\ 3\ 4\ *\ 8\ +\ 4\ /\ -$ in postfix notation.

Algorithm for Evaluation of a postfix expression :

```
Step 1: Add a ")" at the end of the
        postfix expression
Step 2: Scan every character of the
        postfix expression and repeat
        Steps 3 and 4 until ")" is encountered
Step 3: IF an operand is encountered,
        push it on the stack
        IF an operator O is encountered, then
        a. Pop the top two elements from the
           stack as A and B as A and B
        b. Evaluate B O A, where A is the
           topmost element and B
           is the element below A.
        c. Push the result of evaluation
           on the stack
        [END OF IF]
Step 4: SET RESULT equal to the topmost element
        of the stack
Step 5: EXIT
```

Character Scanned	Stack
9	9
3	9, 3
4	9, 3, 4
*	9, 12
8	9, 12, 8
+	9, 20
4	9, 20, 4
/	9, 5
-	4

Applications of Stack...

4. Conversion of an infix expression into a prefix expression

Algorithm:

Step 1: Reverse the infix string. Note that while reversing the string you must interchange left and right parentheses.

Step 2: Obtain the postfix expression of the infix expression obtained in Step 1.

Step 3: Reverse the postfix expression to get the prefix expression

Example:

Infix expression: $(A - B / C) * (A / K - L)$

Step 1: Reverse the infix string.

while reversing the string we must interchange left and right parentheses.

$(L - K / A) * (C / B - A)$

Step 2: Obtain the corresponding postfix expression of the infix expression obtained as a result of Step 1.

$(L - K / A) * (C / B - A)$

$= (L - (K / A)) * (C / B - A) = (L - (K / A)) * ((C / B) - A) = ((L - (K / A)) * (C / B)) - A$

Postfix Expression is : $L K A / - C B / A - *$

Step 3: Reverse the postfix expression to get the prefix expression

Therefore, the **prefix expression is:** $* - A / B C - / A K L$

Applications of Stack...

5. Evaluation of a prefix expression

Algorithm:

Step 1: Accept the prefix expression
Step 2: Repeat until all the characters in the prefix expression have been scanned

- (a) Scan the prefix expression from right, one character at a time.
- (b) If the scanned character is an operand, push it on the operand stack.
- (c) If the scanned character is an operator, then
 - (i) Pop two values from the operand stack
 - (ii) Apply the operator on the popped operands
 - (iii) Push the result on the operand stack

Step 3: END

Example: **Prefix: + - 2 7 * 8 / 4 12**

Character scanned	Operand stack
12	12
4	12, 4
/	3
8	3, 8
*	24
7	24, 7
2	24, 7, 2
-	24, 5
+	29

Applications of Stack...

6. Recursion

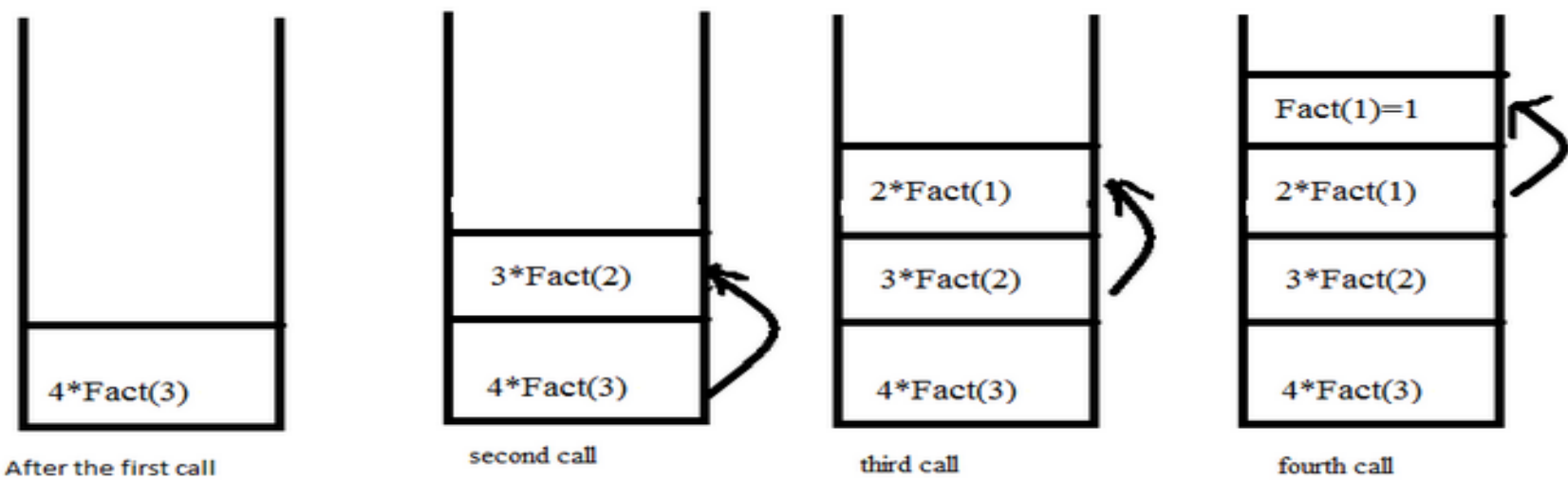
- The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called as recursive function.
- Since a recursive function repeatedly calls itself, it makes use of the system stack to temporarily store the return address and local variables of the calling function.
- Every recursive solution has two major cases. They are
 1. **Base case:** In which the problem is simple enough to be solved directly without making any further calls to the same function.
 2. **Recursive case:** In which first the problem at hand is divided into simpler sub-parts. Second the function calls itself but with sub-parts of the problem obtained in the first step. Third, the result is obtained by combining the solutions of simpler sub-parts.
- For the factorial function,
 1. Base case is when $n = 1$, because if $n = 1$, the result will be 1 as $1! = 1$.
 2. Recursive case of the factorial function will call itself but with a smaller value of n , this case can be given as $\text{Fact}(n) = n \times \text{Fact}(n-1)$.

Recursive Function for Factorial:

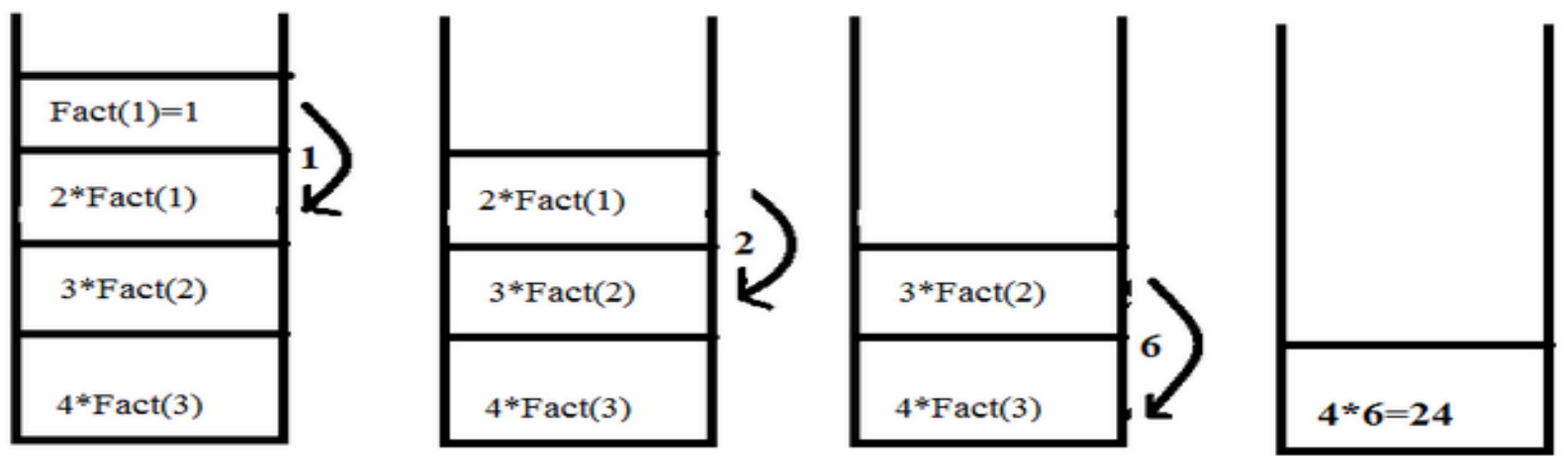
```
int Fact(int n)
{
    if(n==1)
        return 1;
    else
        return (n * Fact(n-1));
}
```

Recursion...

When function call happens previous variables gets stored in stack



Returning values from base case to caller function



Queue

➤ Queue:

- A Queue is a linear data structure that works on the principle of FIFO.
- It is opened at both the ends (Rear and Front Ends).
- The elements can be inserted at the rear end and deleted from the front end.



- Queue maintains two variables Front and Rear. Initial value of Front and Rear are -1.
- Basic operations performed on queue are:
 - **Insertion**-Inserts the element at rear end of the queue .
 - **Deletion**- deletes the element from front end of the queue.

Queue...

➤ Implementation of queue:

- Array Implementation of queue
- Linked List Implementation of queue

➤ Array Implementation of queue:

- Queues can be represented using linear arrays.

➤ Queue Operations:

➤ Insertion:

- This operation is used to insert an element into the queue at rear end. Before inserting an element we have to check whether the queue is full or not . if

queue is already full it has to display

queue overflow otherwise it permits

insertions.

- While inserting the first element both rear

and front are set to zero, from second

insertion onwards only rear will be incremented by 1.

```
Step 1: IF REAR = MAX-1
        Write OVERFLOW
        Goto step 4
    [END OF IF]
Step 2: IF FRONT = -1 and REAR = -1
        SET FRONT = REAR = 0
    ELSE
        SET REAR = REAR + 1
    [END OF IF]
Step 3: SET QUEUE[REAR] = NUM
Step 4: EXIT
```

Array Implementation of Queue ...

Insertion Example:

MAXSIZE=6

0	1	2	3	4	5

FRONT= -1

REAR= -1

0	1	2	3	4	5
10					

FRONT= 0

REAR= 0

0	1	2	3	4	5
10	20				

FRONT= 0 REAR= 1

0	1	2	3	4	5
10	20	30			

FRONT= 0

REAR= 2

0	1	2	3	4	5
10	20	30	40	60	35

FRONT= 0

REAR= 3

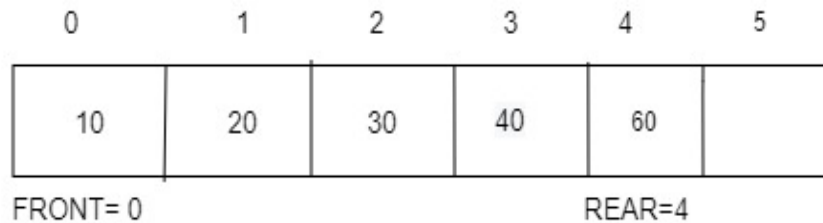
REAR= 4

REAR= 5

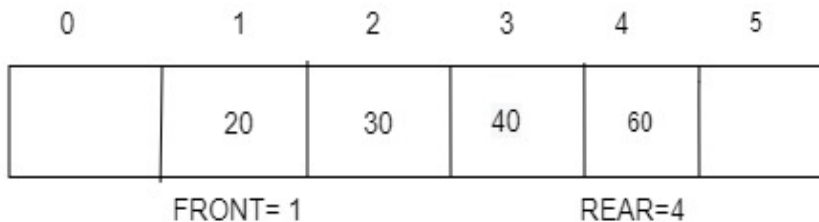
Array Implementation of Queue ...

 Deletion:

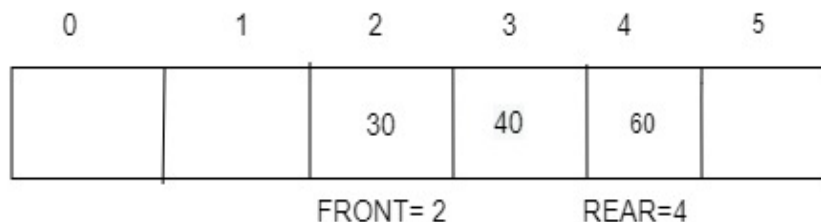
- This operation is used to delete an element from front end of the queue.
- While deleting an element ,we have to check for underflow condition. An underflow occurs if $\text{front} = -1$ or $\text{front} > \text{rear}$.i.e we cannot delete an element from empty queue.
- When an element is deleted, Front is incremented by 1.
- Rear value does not change while deleting an element from queue.
- Example:



Delete



Delete



```

Step 1: IF FRONT = -1 OR FRONT > REAR
        Write UNDERFLOW
    ELSE
        SET VAL = QUEUE[FRONT]
        SET FRONT = FRONT + 1
    [END OF IF]
Step 2: EXIT

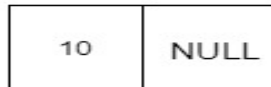
```

Linked List Implementation of Queue

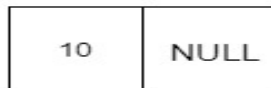
➤ Insertion:

- The new element is added as the last element of the queue.
- Initially FRONT=NULL and REAR = NULL.
- if FRONT=NULL, then the queue is empty.
- There is no overflow condition in linked list implementation of queue.
- **Example:**

PTR

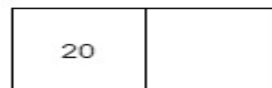


FRONT
REAR

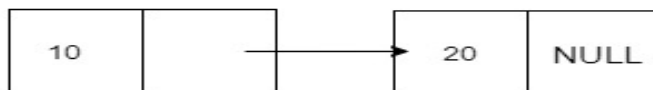


FRONT
REAR

PTR



PTR



FRONT

REAR

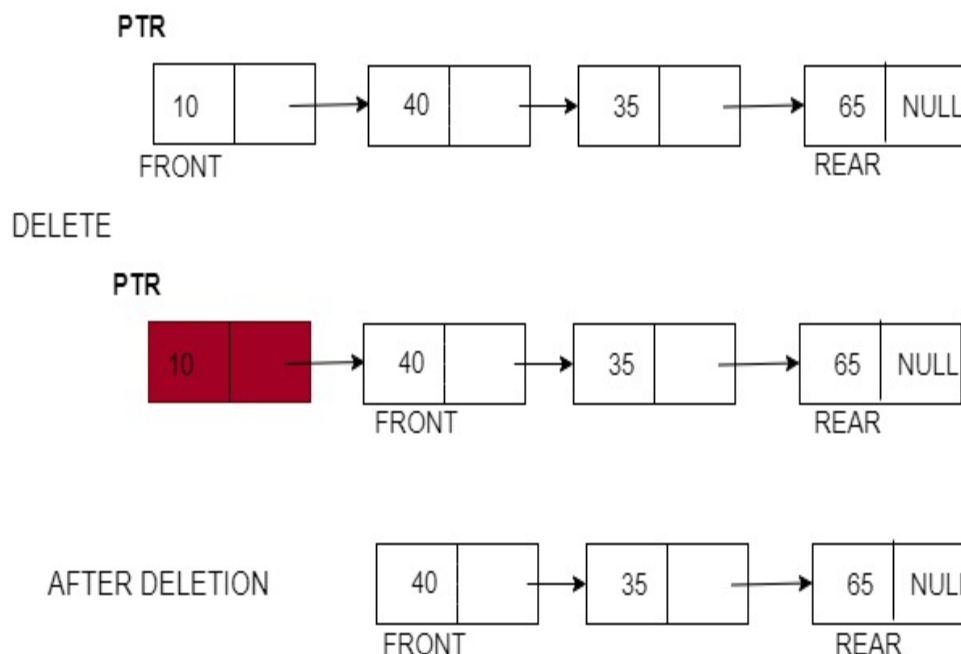
```
Step 1: Allocate memory for the new node and name it as PTR
Step 2: SET PTR -> DATA = VAL
Step 3: IF FRONT = NULL
        SET FRONT = REAR = PTR
        SET FRONT -> NEXT = REAR -> NEXT = NULL
    ELSE
        SET REAR -> NEXT = PTR
        SET REAR = PTR
        SET REAR -> NEXT = NULL
    [END OF IF]
Step 4: END
```

Linked List Implementation of Queue

➤ Deletion:

- The delete operation is used to delete the element that is first inserted in a queue, i.e., the element whose address is stored in FRONT.
- Before deleting the value, we must first check if FRONT=NULL because if this is the case, then the queue is empty and no more deletions can be done.
- If an attempt is made to delete a value from a queue that is already empty, an underflow.

➤ Example:



```
Step 1: IF FRONT = NULL
        Write "Underflow"
        Go to Step 5
    [END OF IF]
Step 2: SET PTR = FRONT
Step 3: SET FRONT = FRONT -> NEXT
Step 4: FREE PTR
Step 5: END
```

Types of Queue

➤ Circular Queue:

- In linear queues, insertions can be done only at one end called the REAR and deletions are always done from the other end called the FRONT.

54	9	7	18	14	36	45	21	99	72
0	1	2	3	4	5	6	7	8	9

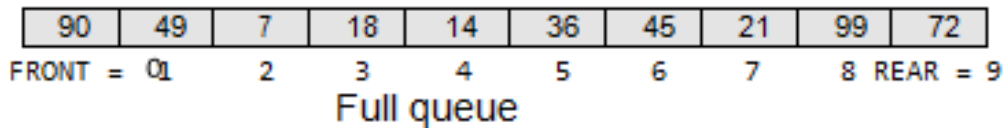
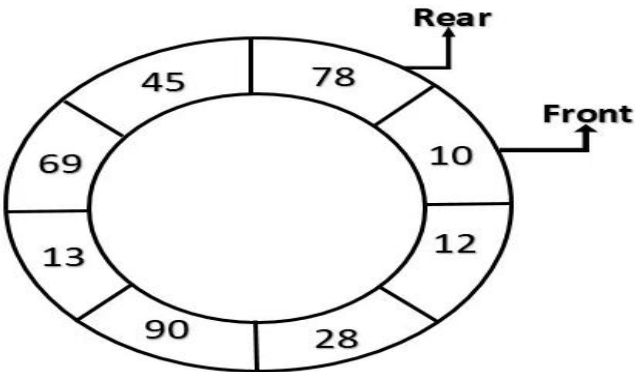
- Suppose if we delete first two elements i.e 54 and 9 from the above queue, we get

		7	18	14	36	45	21	99	72
0	1	2	3	4	5	6	7	8	9

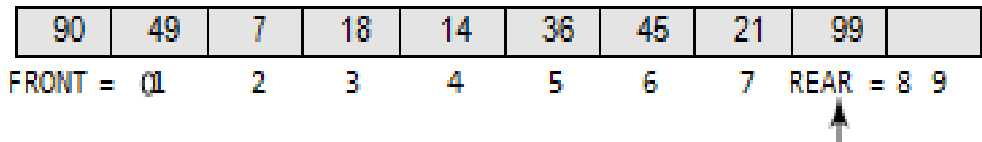
- After deletion, the space cannot be reused for inserting the new elements even though there is space available, the overflow condition still exists because the condition $REAR = MAX - 1$ still holds true. This is a major drawback of a linear queue.
- To resolve this problem, we have two solutions.
 - First, shift the elements to the left so that the vacant space can be occupied and utilized efficiently. But this can be very time-consuming, especially when the queue is quite large.
 - Second, to use a circular queue.

Circular Queue...

- In the circular queue, the first index comes right after the last index.
- The circular queue will be full only when $FRONT = 0$ and $REAR = \text{Max} - 1$.
- Operations performed on circular queue are
 - Insertion
 - Deletion
- Insertion:
 - For insertion, we now have to check for the following three conditions:
 1. If $front=0$ and $Rear=MAX-1$, then circular queue is full.

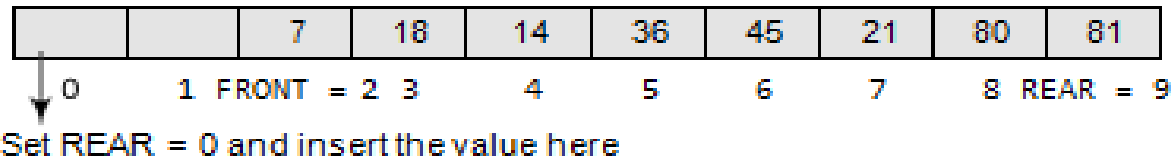


2. If $REAR \neq MAX - 1$, then REAR will be incremented and the value will be inserted.



Circular Queue...

3. If $\text{FRONT} \neq 0$ and $\text{REAR} = \text{MAX} - 1$, then it means that the queue is not full. So, set $\text{REAR} = 0$ and insert the new element there.



```
Step 1: IF FRONT = 0 and Rear = MAX - 1
        Write "OVERFLOW"
        Goto step 4
    [End OF IF]
Step 2: IF FRONT = -1 and REAR = -1
        SET FRONT = REAR = 0
    ELSE IF REAR = MAX - 1 and FRONT != 0
        SET REAR = 0
    ELSE
        SET REAR = REAR + 1
    [END OF IF]
Step 3: SET QUEUE[REAR] = VAL
Step 4: EXIT
```

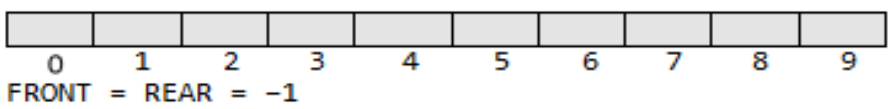
Circular Queue...

➤ Deletion:

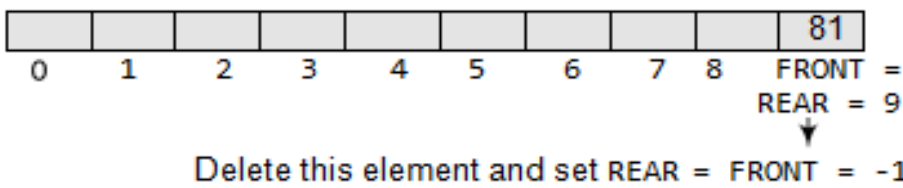
➤ To delete an element, we check for following three conditions:

1. If $FRONT = -1$, then there are no elements in the queue,

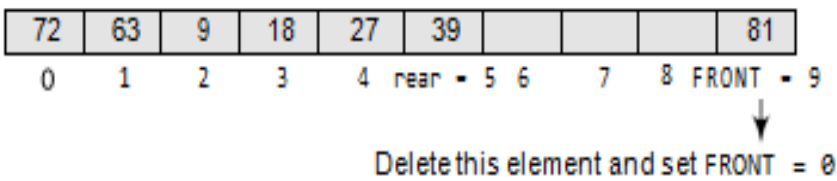
This condition is called as Underflow condition.



2. If the queue is not empty and $FRONT = REAR$, then after deleting the element at the front the queue becomes empty and so $FRONT$ and $REAR$ are set to -1 .



3. If the queue is not empty and $FRONT = MAX-1$, then after deleting the element at the front, $FRONT$ is set to 0.



```
Step 1: IF FRONT = -1
        Write "UNDERFLOW"
        Goto Step 4
    [END of IF]
Step 2: SET VAL = QUEUE[FRONT]
Step 3: IF FRONT = REAR
        SET FRONT = REAR = -1
    ELSE
        IF FRONT = MAX -1
            SET FRONT = 0
        ELSE
            SET FRONT = FRONT + 1
        [END of IF]
    [END OF IF]
Step 4: EXIT
```

Deque

Deque:

- A Dequeue is a list in which the elements can be inserted or deleted at either end.
- It is also known as a *head-tail linked list* because elements can be added to or removed from either the front (head) or the back (tail) end.



➤ Types of Dequeue:

1. **Input Restricted Dequeue :** In this dequeue insertions can be done only at one end but deletions can be done from both the ends.
2. **Output Restricted Dequeue :** In this dequeue deletions can be done only at one end but insertions can be done on both the ends.

Queue Applications

➤ Applications of Queue are:

1. Queues are widely used as waiting lists for a single shared resource like printer, disk, CPU.
2. Queues are used to transfer data asynchronously (data not necessarily received at same rate as sent) between two processes.
3. Queues are used as buffers on MP3 players and portable CD players, iPod playlist.
4. Queues are used in Playlist for jukebox to add songs to the end, play from the front of the list.
5. Queues are used in operating system for handling interrupts. When programming a real-time system that can be interrupted, for example, by a mouse click, it is necessary to process the interrupts immediately, before proceeding with the current job. If the interrupts have to be handled in the order of arrival, then a FIFO queue is the appropriate data structure.

UNIT - III

Trees – Terminology, Representation of Trees, Binary tree ADT, Properties of Binary Trees, Binary Tree Representations-array and linked representations, Binary Tree traversals, Threaded binary trees, Binary Heap-Properties, Max and Min Heap, Operations-Insertion and Deletion.

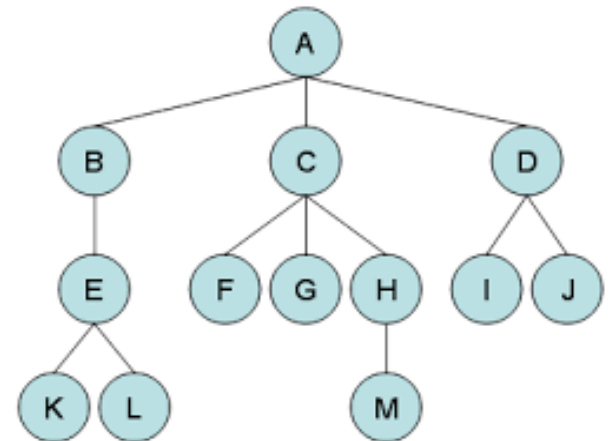
Search Trees-Binary Search tree, Tree traversals, AVL tree – operations, B-tree – operations, B+ trees, Red Black tree.

Tree:

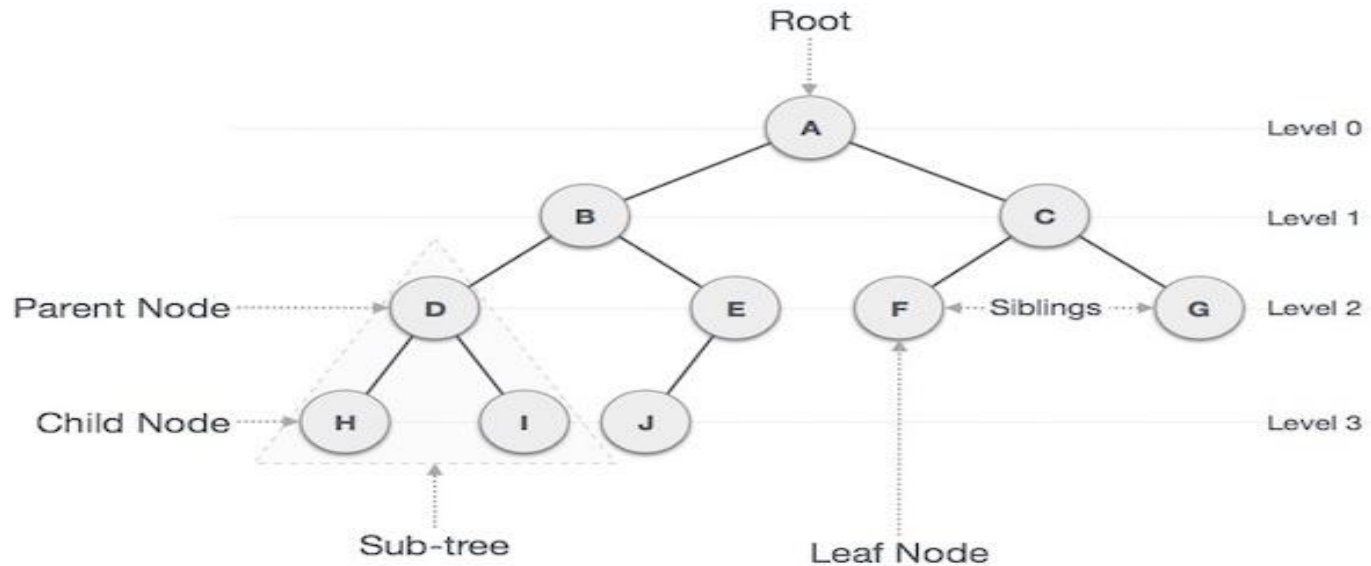
- A tree is a **non linear hierarchical** data structure, consists of **nodes** connected by **edges**.

Why Tree Data Structure?

- Other data structures such as arrays, linked list, stack, and queue are linear data structures that store data sequentially. In order to perform any operation in a linear data structure, the time complexity increases with the increase in the data size. But, it is not acceptable today's computational world.
- Different tree data structures allow quicker and easier access to the data as it is a non-linear data structure.



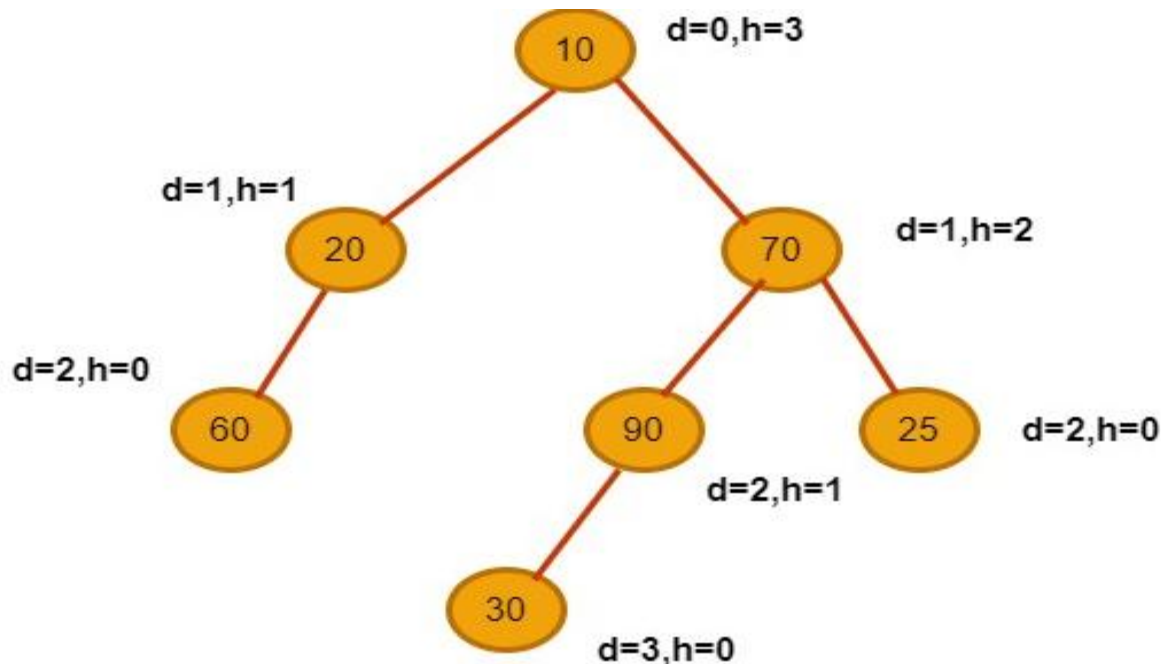
Tree Terminology



- **Node**-A node is an entity that contains a key or value and pointers/link to its child nodes.
- **Edge**-It is the link between any two nodes.
- **Root** -The node at the top/start of the tree is called root. There is only one root per tree.
- **Parent** - Any node except the root node has one edge upward to a node called parent.
- **Child** - The node below a given node connected by its edge downward is called its child node.
- **Leaf** - The node which does not have any child node is called the leaf node.
- **Subtree** -Subtree represents the descendants of a node.
- **Levels** -Level of a node represents the generation of a node. If the root node is at level 0, then its next child node is at level 1, its grandchild is at level 2, and so on.
- **Siblings**: if two or more nodes are having same parent in a tree, then they are called siblings.

Tree Terminology...

- **Height of a Node**-The height of a node is the number of edges from the node to the deepest leaf (ie. the longest path from the node to a leaf node).
- **Depth of a Node**-The depth of a node is the number of edges from the root to the node.
- **Height of a Tree**-The height of a Tree is the height of the root node or the depth of the deepest node or maximum of all nodes height or maximum of all nodes depth.
- In a tree, height of a tree is equal to depth of a tree.



Types of Trees

➤ Different types of tree are:

1. Binary Tree

- I. Strict Binary Tree
- II. Complete Binary Tree
- III. Perfect Binary Tree
- IV. Extended Binary Tree

2. Expression Tree

3. Threaded Binary Tree

4. Binary Heap

5. Search Trees

- I. Binary Search Tree(BST)
- II. AVL Tree
- III. Red Black Tree
- IV. B-Tree
- V. B+-Tree

Binary Tree

➤ A **Binary tree** is a tree which is either empty or each node can have maximum of two child nodes, i.e. each node can have 0, or 1 or 2 child nodes.

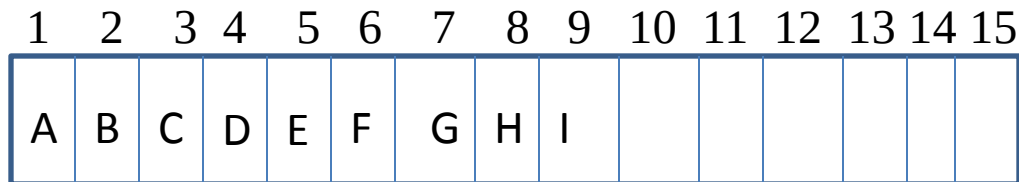
➤ **Binary Tree Representations:**

1. **Array Representation**

2. **Linked List Representation**

➤ **Array Representation of Binary Tree**

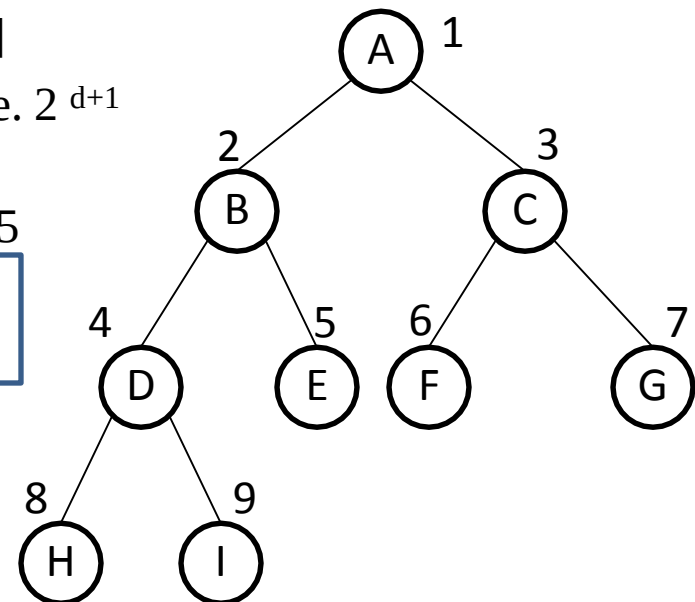
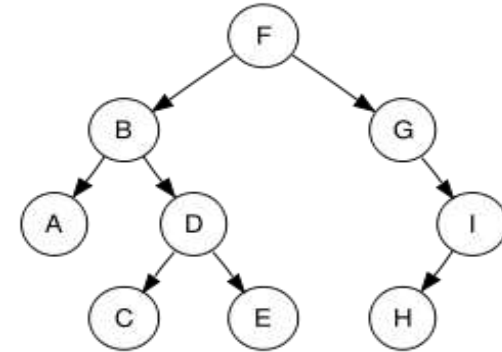
- Binary Tree is stored in a single array.
- Number are assigned to each node from leftmost to rightmost node.
- Root node always assign no. 1
- Left Child is placed at position $[2 * K]$ (k is position of root/parent)
- Right Child is placed at position $[2 * K + 1]$
- Size of array is Depends on Depth of tree i.e. 2^{d+1}



Note: If root is placed at 0th position then

L Child = $[2 * k + 1]$

R child = $[2 * K + 2]$



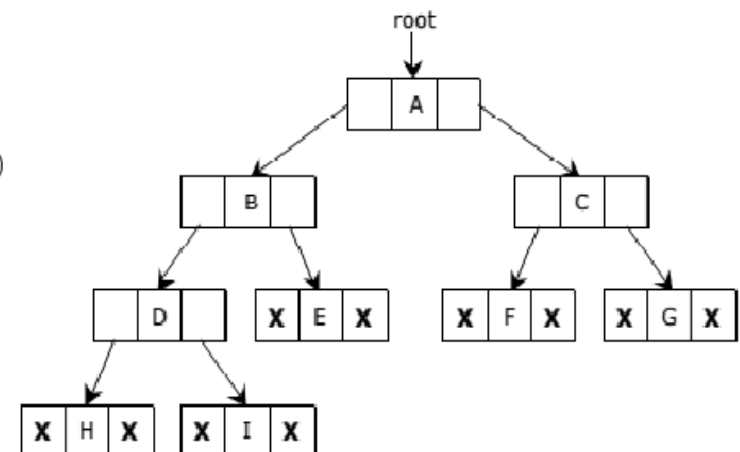
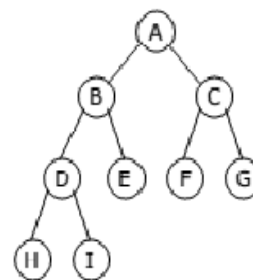
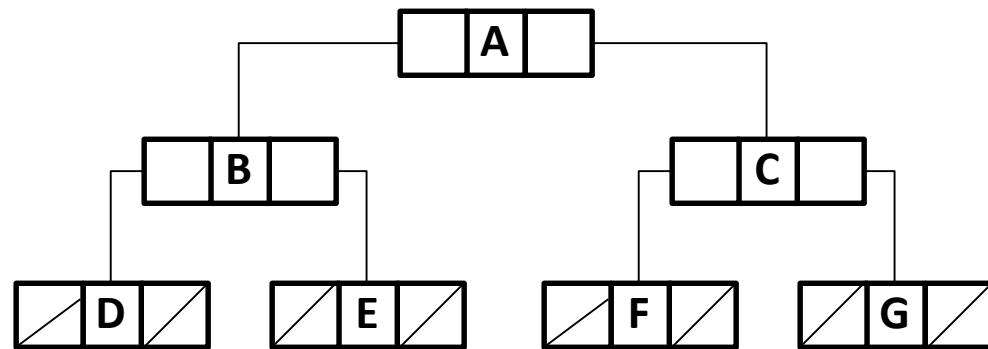
Binary Tree Representations...

➤ Linked List Representation of Binary Tree:

- Each node contains three fields –Left child address , Data and Right child address.
- If the node does not contain left /right child then address field contains NULL.
- **Node of a binary tree is declared as:**

```
struct node
{
    struct node*left;
    int data;
    struct node *right;
};
```

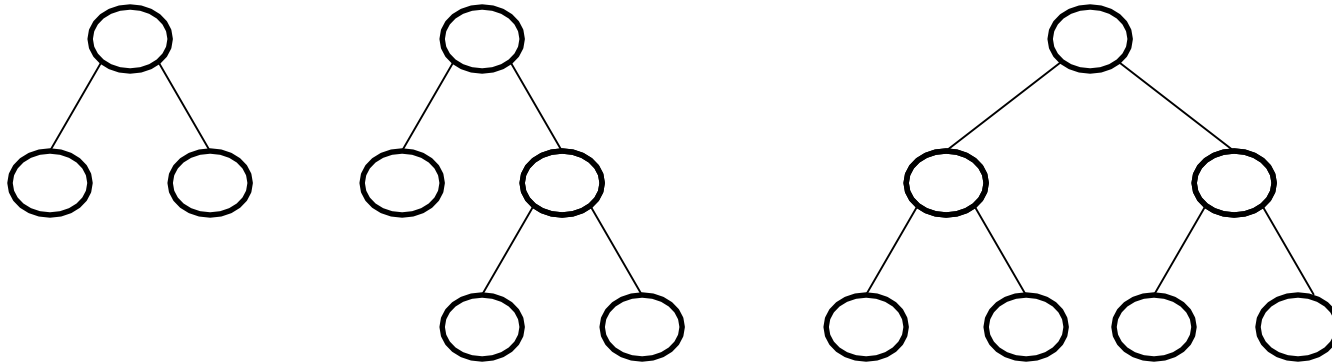
E.g.



Types of Binary Tree

1. Strict Binary Tree:

- Each node in a tree is either leaf or has exactly two children.



Strict Binary Tree

- Strictly binary tree with n Leaves always contains $2n-1$ nodes

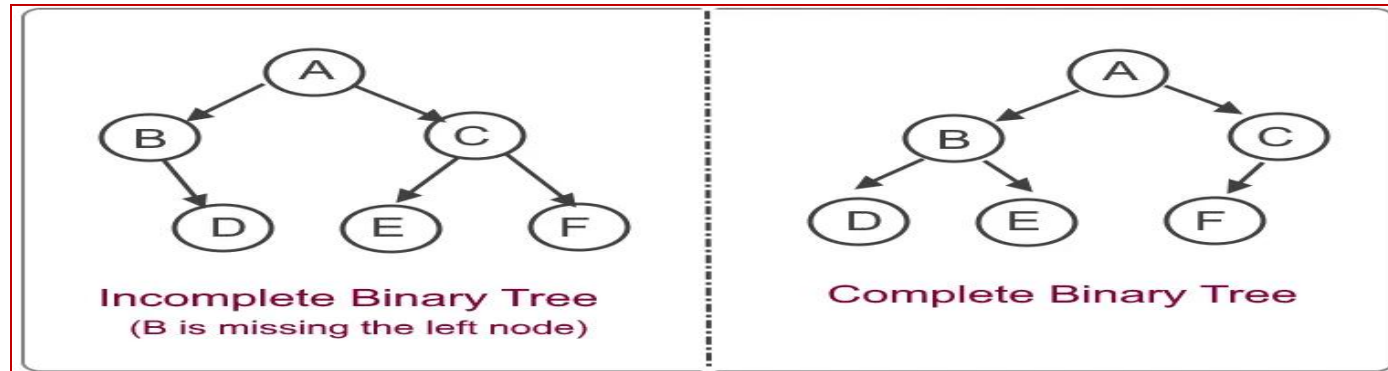
E.g.

- 2 Leaves in First Example having $2*2-1=3$ Nodes.
- 3 Leaves in Second Example having $3*2-1=5$ Nodes.
- 4 Leaves in Third Example having $4*2-1=7$ Nodes

Types of Binary Tree...

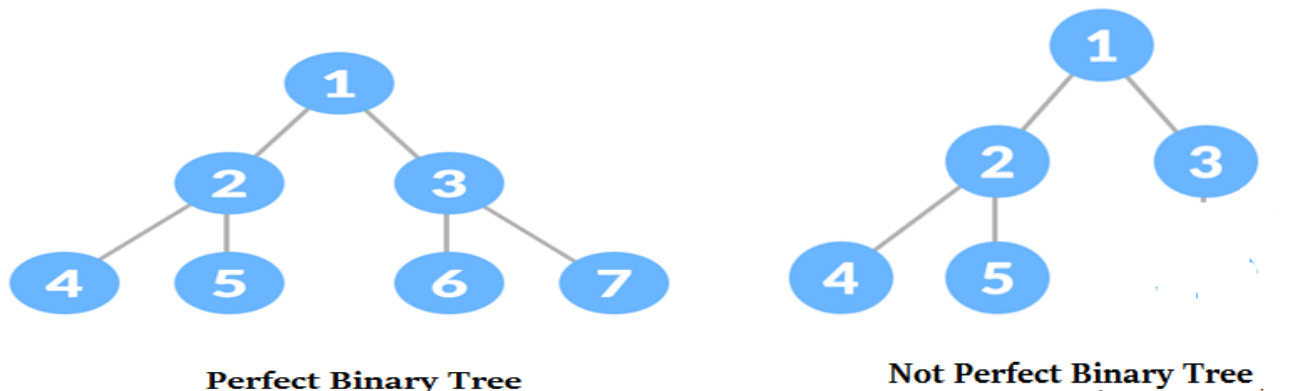
2.Complete Binary Tree:

- A complete binary tree is a binary tree in which all the levels are completely filled except possibly the lowest one, which is filled from the left.



3.Perfect Binary Tree:

- A **perfect binary tree** is a **binary tree** in which all interior nodes have two children and all leaves are at same level.



Types of Binary Tree...

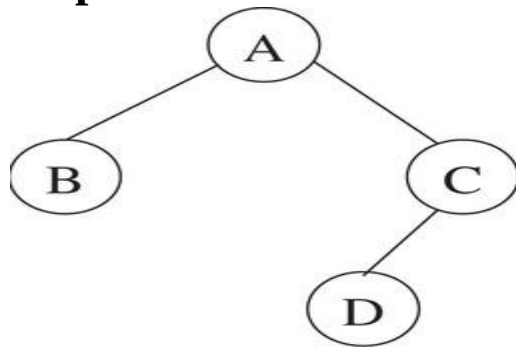
4.Extended Binary Tree:

➤ Extended binary tree is a type of binary tree in which all the null sub tree of the original tree are replaced with special nodes called external nodes whereas other nodes are called internal nodes.

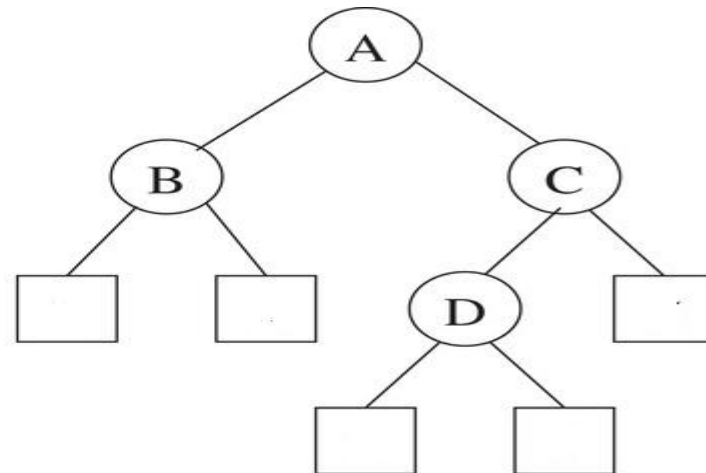
➤Properties of External binary tree

1. The nodes from the original tree are internal nodes and the special nodes are external nodes.
2. All external nodes are leaf nodes and the internal nodes are non-leaf nodes.
3. Every internal node has exactly two children and every external node is a leaf.

➤Example:



(a)



(b)

Binary Tree Traversals

Binary Tree Traversal

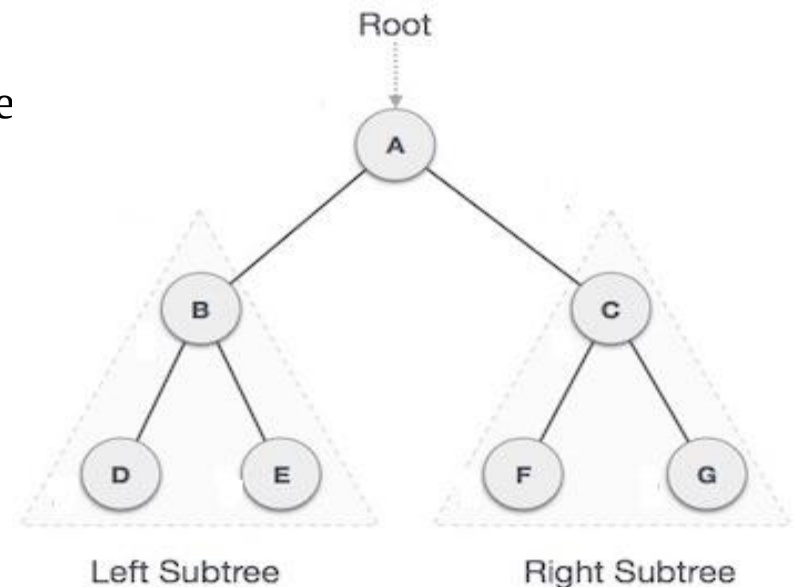
➤ Traversal is a process to visit all the nodes of a tree and print their values . Because, all nodes are connected via edges (links) we always start from the root node.

➤ Different types of tree traversals are:

1. In-order Traversal
2. Pre-order Traversal
3. Post-order Traversal

1. In-order Traversal (LDR)

1. First, visit all the nodes in the left sub tree
2. Then the root node.
3. Visit all the nodes in the right sub tree.

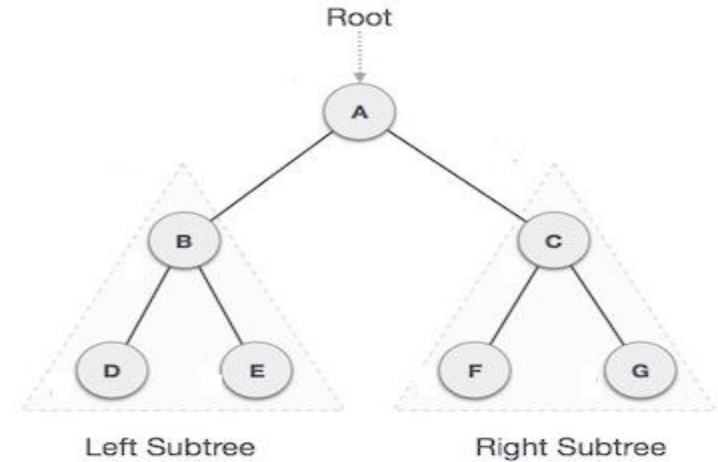


In-order: $D \rightarrow B \rightarrow E \rightarrow A \rightarrow F \rightarrow C \rightarrow G$

Binary Tree Traversals

2. Pre-order Traversal (DLR)

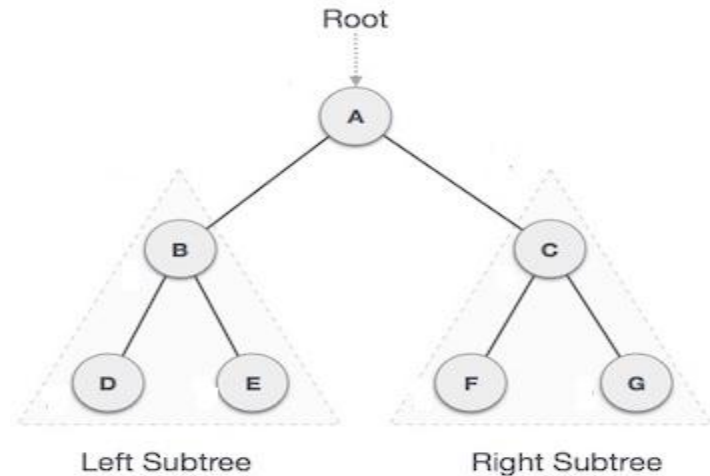
1. Then the root node.
2. First, visit all the nodes in the left sub tree.
3. Visit all the nodes in the right sub tree.



Pre -order: $A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$

3. Postorder traversal

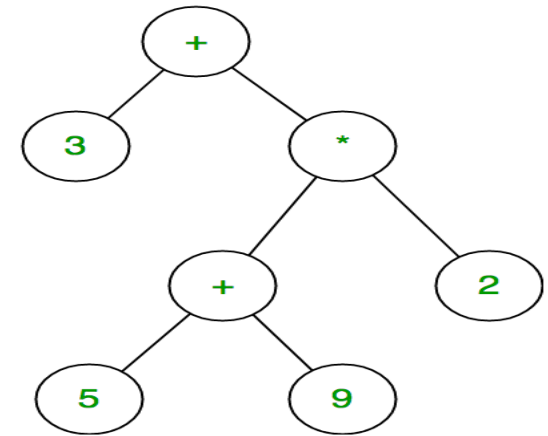
1. Visit all the nodes in the left subtree.
2. Visit all the nodes in the right subtree.
3. Visit the root node.



Post -order: $D \rightarrow E \rightarrow B \rightarrow F \rightarrow G \rightarrow C \rightarrow A$

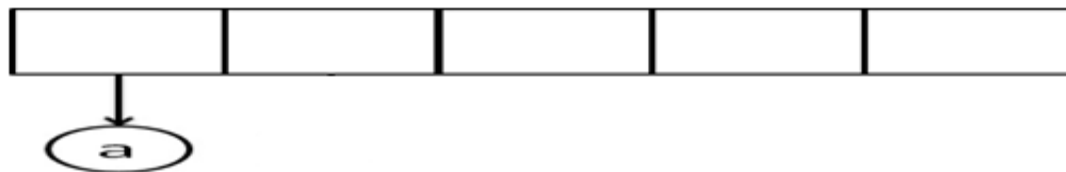
Expression Tree

- Expression tree is a binary tree in which each internal node corresponds to operator and each leaf node corresponds to operand.
- **Example:** Expression tree for $3 + ((5+9)*2)$
- **Steps for construction of Expression tree for a given expression:**
 1. Convert the given expression to post fix expression.
 2. Scan the postfix expression from left to right one character at a time.
 - i. If character is operand push that into stack.
 - ii. If character is operator pop two values from stack make them its child and push current node again.
 3. At the end only element of stack will be root of expression tree.



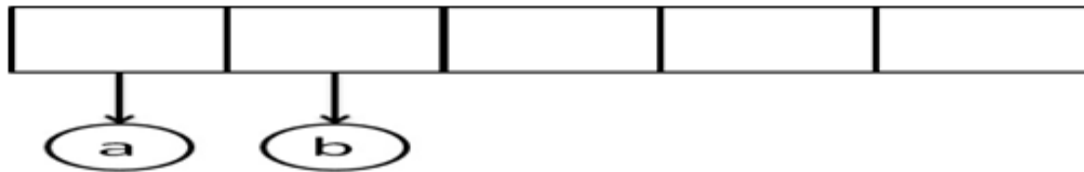
➤ **Problem:** Expression is $(a+b)*c$

1. The postfix is: $a\ b\ +\ c\ *$
2. The first symbol 'a' is operand, it is pushed onto the stack



Expression Tree...

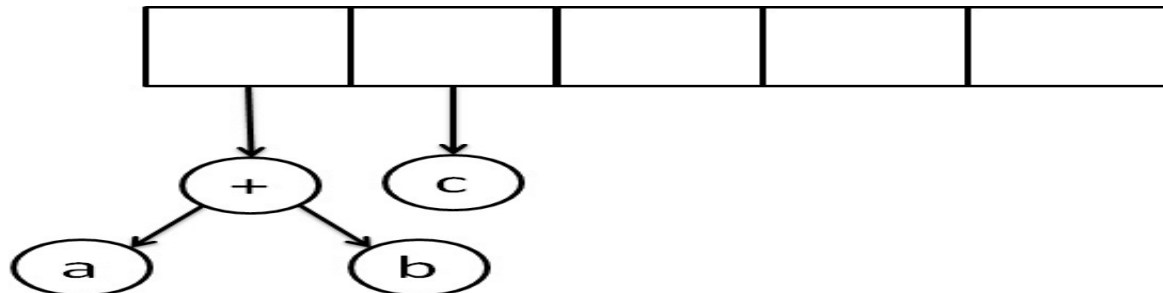
Next symbol 'b' is an operand , insert 'b' onto stack



Next, '+' symbol, so topmost two operands are popped , a new tree is formed and push a pointer to it onto the stack.

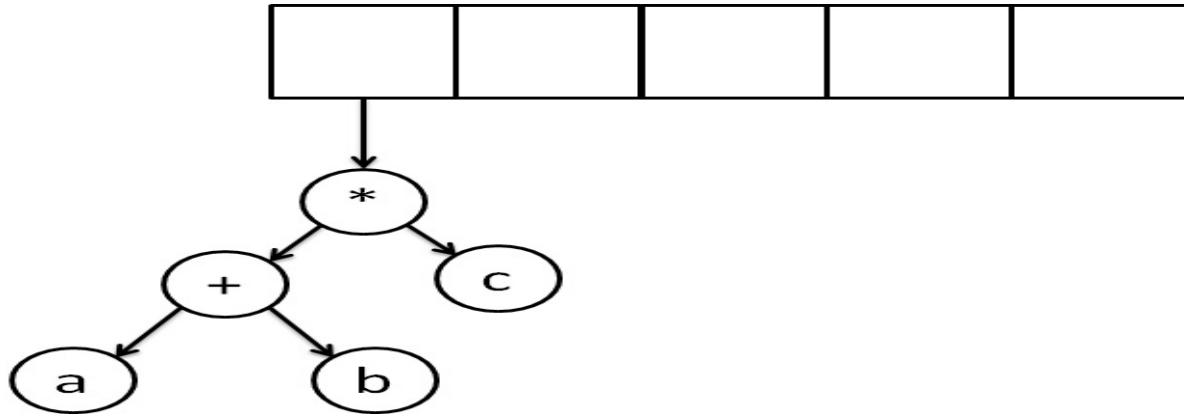


Next, 'c' is read, we create one node tree and push a pointer to it onto the stack.

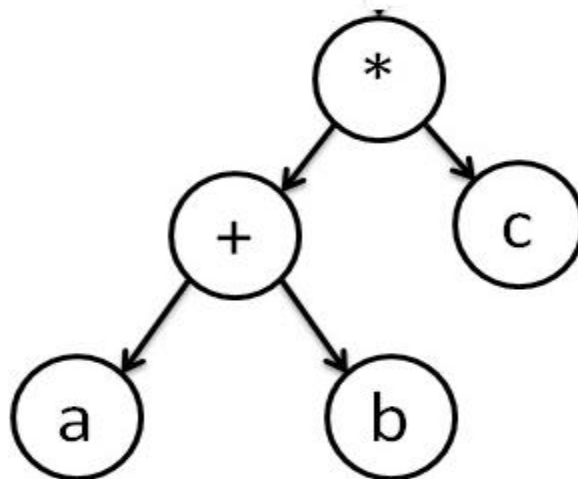


Expression Tree...

Finally, the last symbol is read ' * ', we pop topmost two tree pointers and form a new tree with ' * ' as root, and a pointer to the final tree is pushed on the stack.



Pop the only element from the stack , we get an expression tree



Threaded Binary Tree

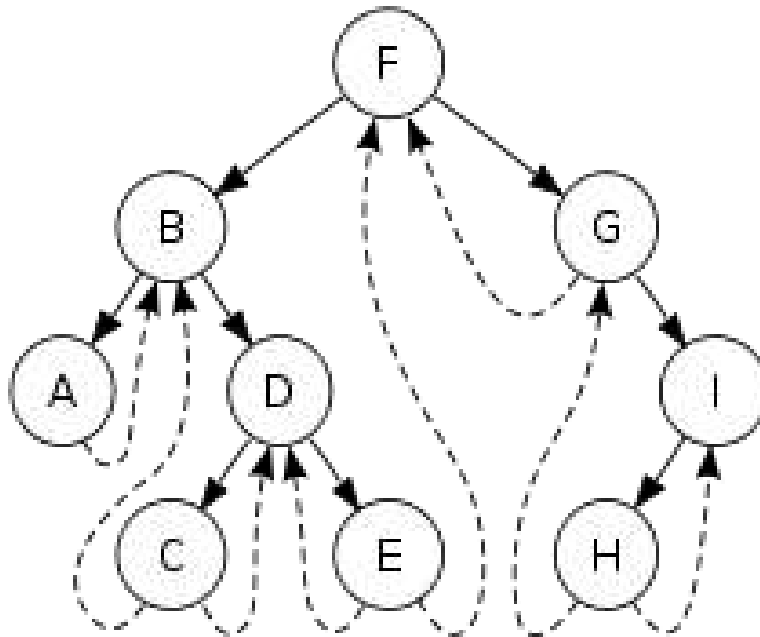
➤ Threaded Binary Tree:

Binary tree is said to be threaded by making all right child pointers that would normally be null point to the inorder successor of the node (if it exists), and all left child pointers that would normally be null point to the inorder predecessor of the node .

➤ Why do we need Threaded Binary Tree?

1. Binary trees have a lot of wasted space, the leaf nodes each have 2 null pointers. We can use these pointers to help us in inorder traversals.
2. Threaded binary tree makes the tree traversal faster since we do not need stack or recursion for traversal.

➤ Example:

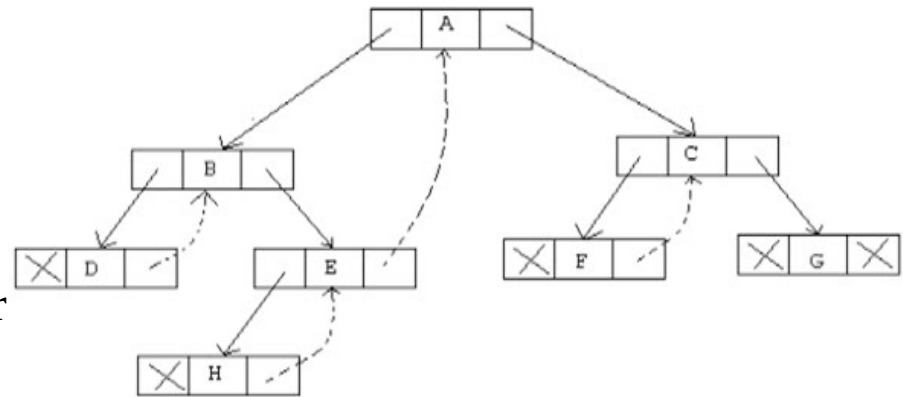


Threaded Binary Tree...

➤ Types of threaded binary tree

1. Right threaded binary tree

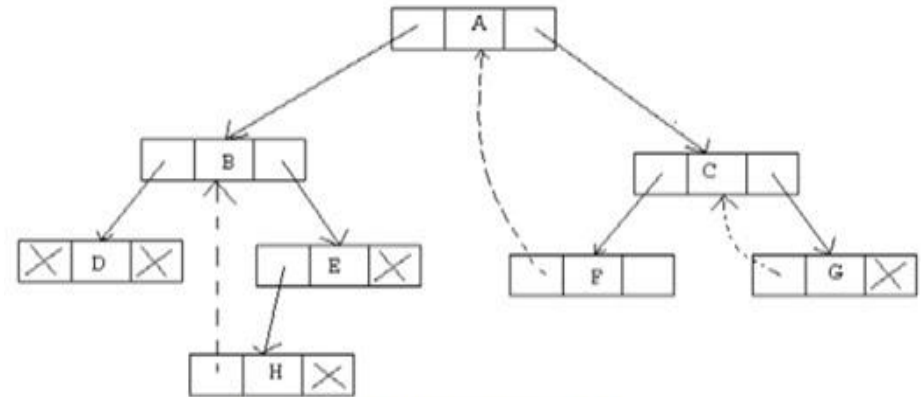
The right NULL pointer of the node is replaced by a thread to the inorder successor of that node is called as right threaded binary tree.



RIGHT THREADED BINARY TREE

2. Left threaded binary tree

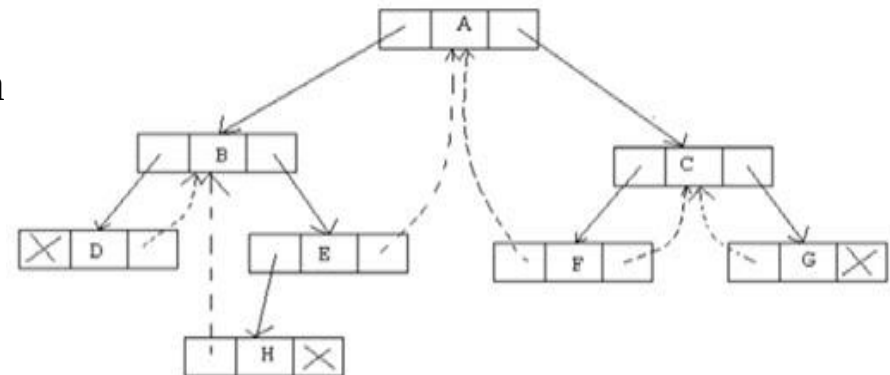
The left NULL pointer of the node is replaced by a thread to the inorder predecessor of that node is called as left threaded binary tree.



LEFT THREADED BINARY TREE

3. Fully threaded binary tree

Both left and right NULL pointers can be used to point to Inorder predecessor and successor of that node respectively, is called a fully threaded binary tree.



Fully Threaded Binary Tree

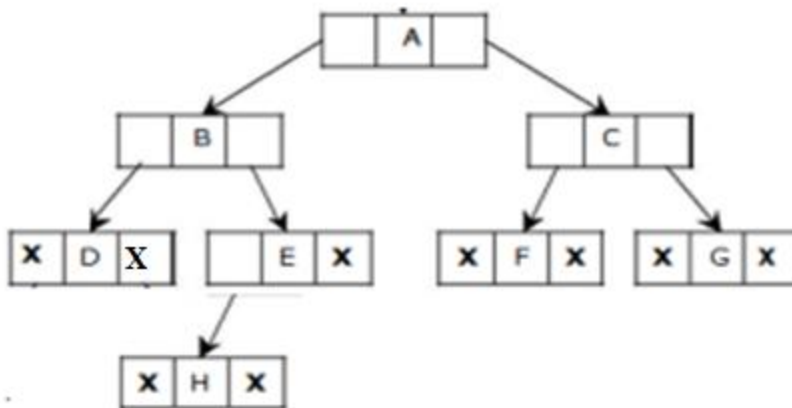
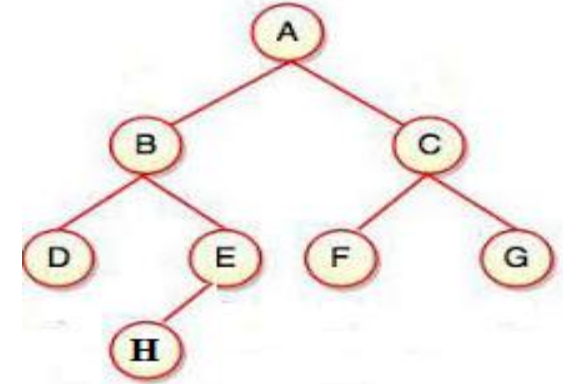
Threaded Binary Tree...

➤ Construct threaded binary tree for following binary tree

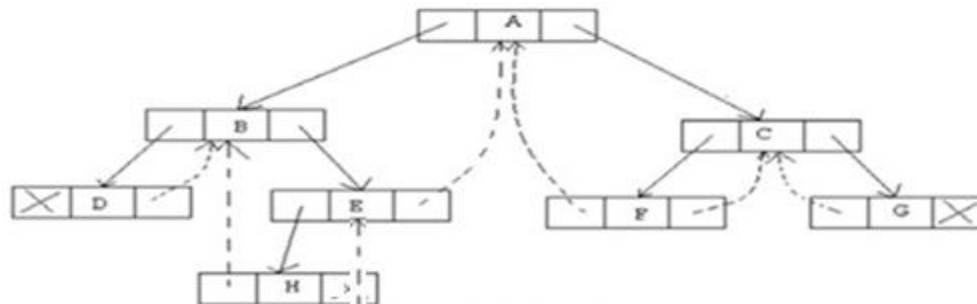
1. Write the inorder traversal for the given binary tree

Inorder is **D B H E A F C G**

2. Represent the binary tree using linked list representation



3. Link the left and right NULL pointers to inorder Successor or predecessor respectively as shown below



Binary Heap

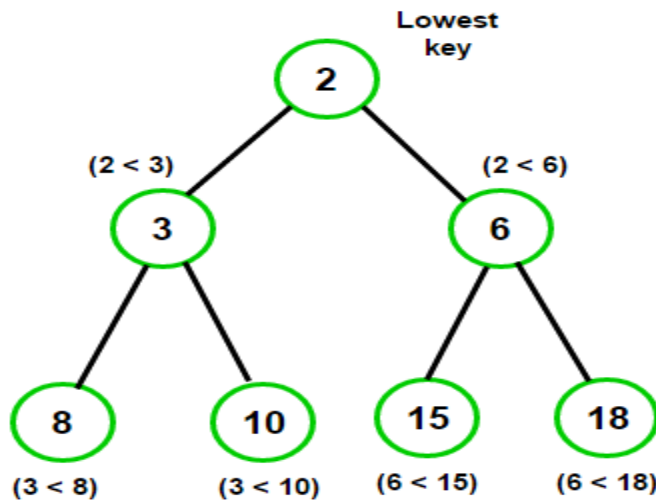
➤ A binary tree is said to be binary heap if it follows the following two properties:

1. Heap order property

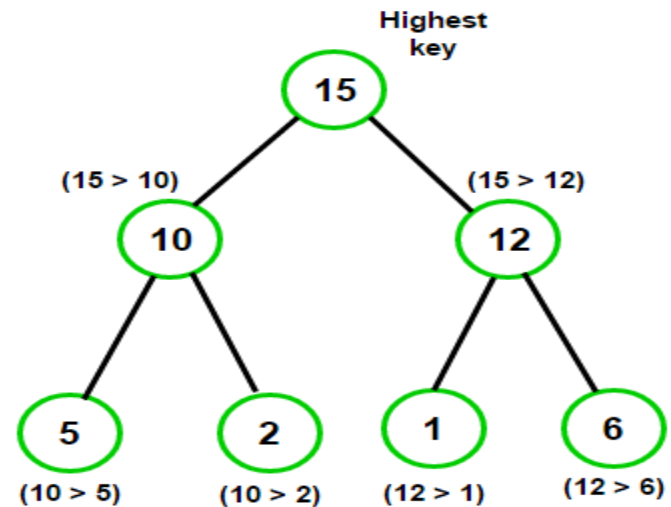
Every node is less than or equal to its children (Min Heap) or every node is greater than or equal to its children (Max Heap).

2. Structure property

The tree is completely filled except possibly the bottom level, which is filled from left to right.



Min Heap
(Parent key is less than or equal to $(\leq)$ the child key)



Max Heap
(Parent key is greater than or equal to $(\geq)$ the child key)

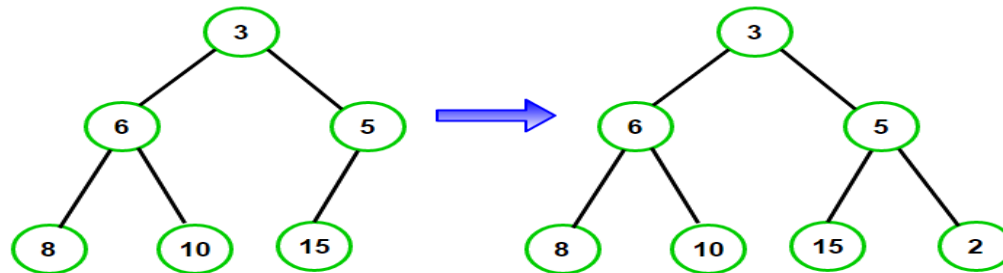
Binary Heap...

➤ Operations:

1. Insertion:

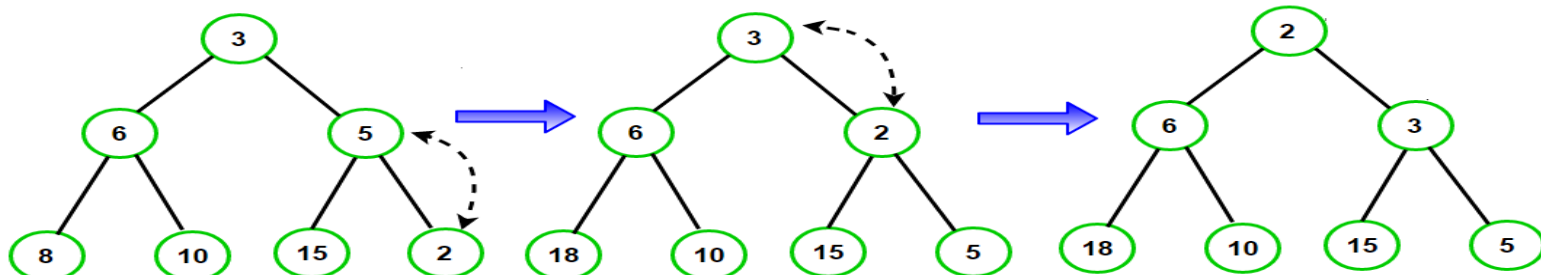
Steps:

1. First increase the heap size by 1, so that it can store the new element.
2. Insert the new element at the end of the Heap.
3. This newly inserted element may distort the properties of Heap for its parents. So, in order to keep the properties of Heap, **heapify** this newly inserted element following a bottom-up approach.



Add the new element 2 to the bottom level of the heap and call

Heapify-up(2)



Swap node 2 with its parent as heap property is violated

swap(5, 2)

Swap node 2 with its parent as heap property is still violated

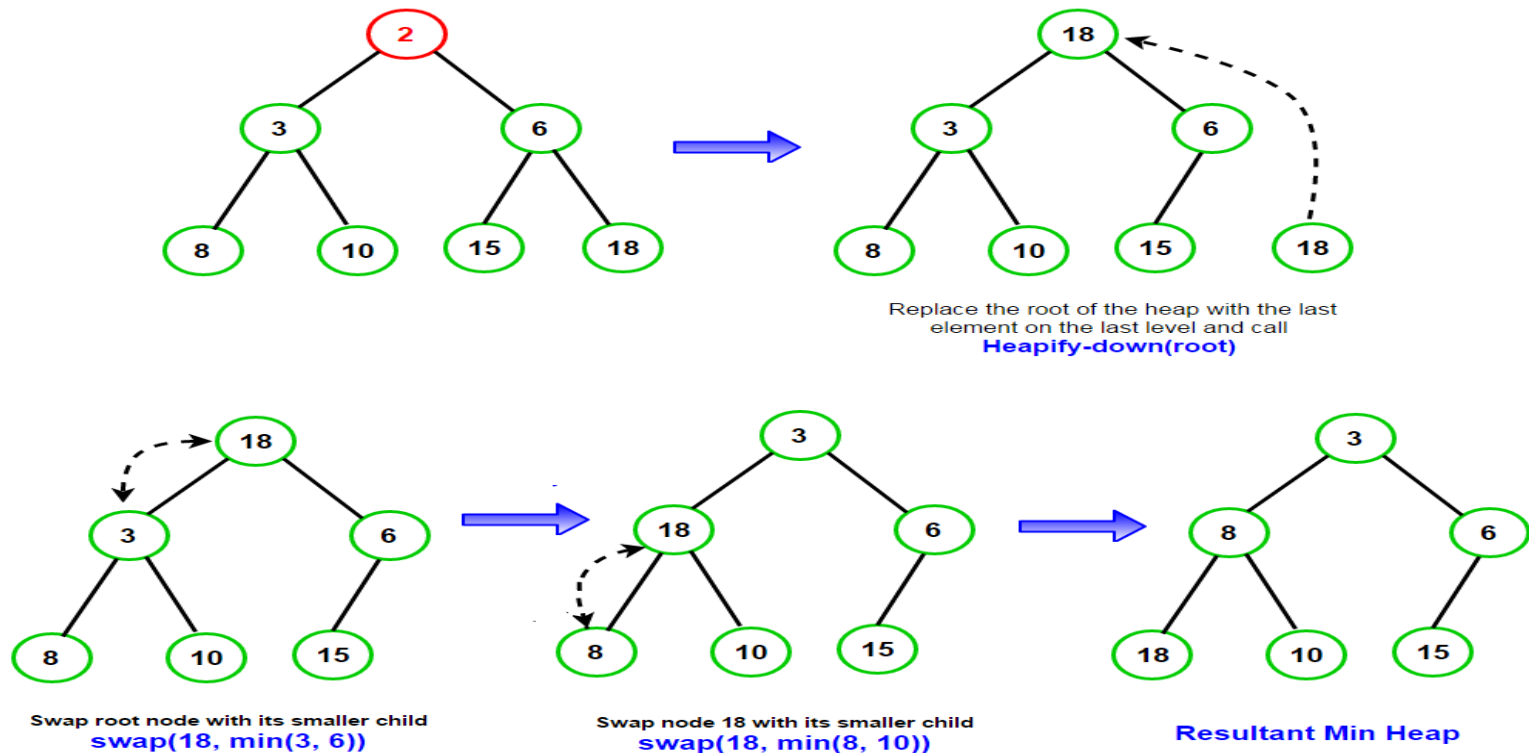
swap(3, 2)

Resultant Min Heap

Binary Heap...

2. Deletion:

1. Delete the root element
2. Replace the root by the last element.
3. Delete the last element from the Heap.
4. Since, the last element is now placed at the position of the root node. So, it may not follow the heap property. Therefore, **heapify** the last node placed at the position of root.

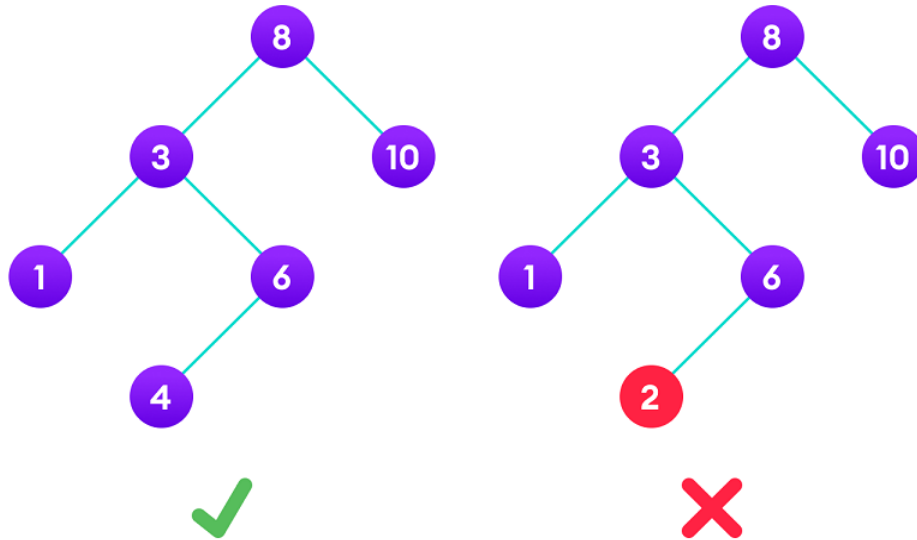


Binary Search Tree(BST)

➤ Binary Search Tree:

A binary search tree, also known as an ordered binary tree, in which all the nodes in the **left sub-tree** have a value less than that of the root node. Correspondingly, **all the nodes in the right sub-tree** have a value either equal to or greater than the root node. The same rule is applicable to every sub-tree in the tree.

➤ Example :



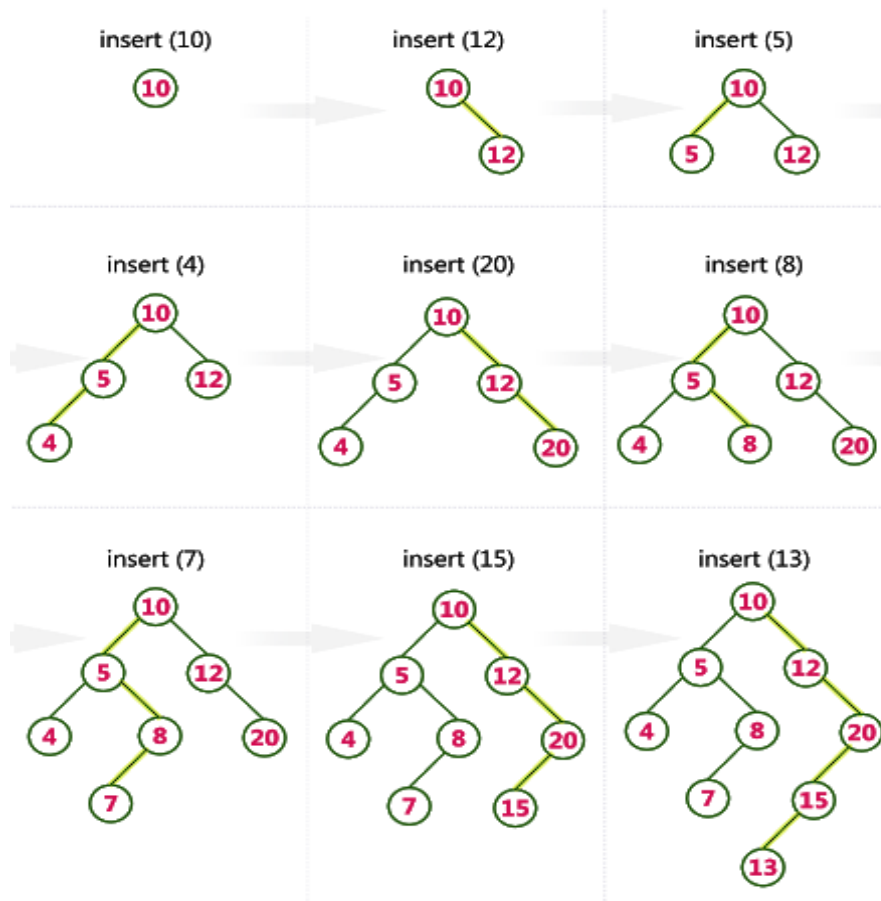
➤ Operations :

1. Insert: insert a node in the BST.
2. Search: Searches for a node in the BST.
3. Delete: deletes a node from the BST.

Binary Search Tree(BST)...

➤ Insert:

Inorder to insert an element, first locate its proper location. Start searching from the root node, then if the data is less than the key value, search for the empty location in the left sub tree and insert the data. Otherwise, search for the empty location in the right sub tree and insert the data.



Insert (TREE, VAL)

Step 1: IF TREE = NULL

Allocate memory for TREE

SET TREE -> DATA = VAL

SET TREE -> LEFT = TREE -> RIGHT = NULL

ELSE

IF VAL < TREE -> DATA

Insert(TREE -> LEFT, VAL)

ELSE

Insert(TREE -> RIGHT, VAL)

[END OF IF]

[END OF IF]

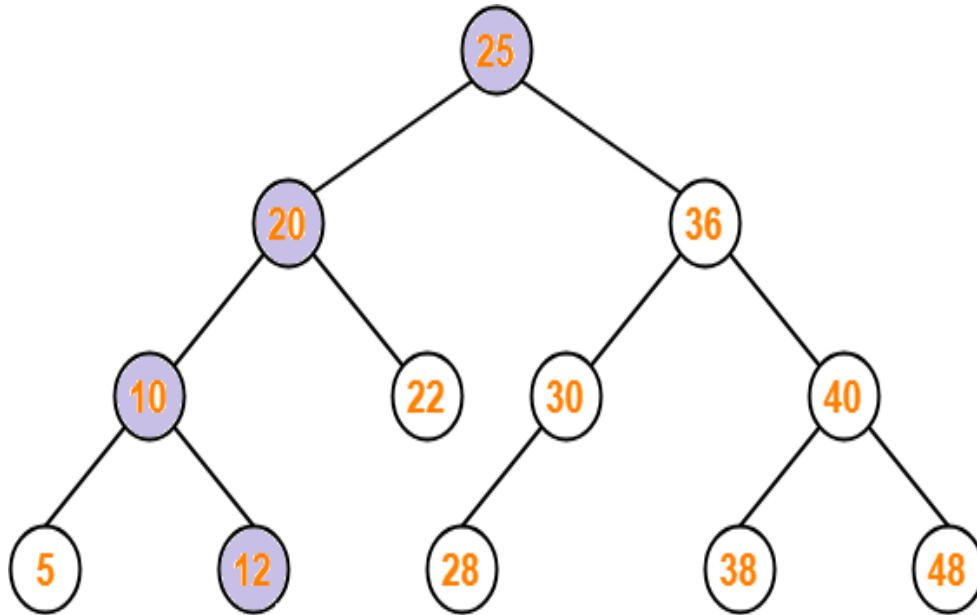
Step 2: END

Binary Search Tree(BST)...

➤ Search:

Inorder to search an element in BST, start from the root node. if the data is less than the key value, search for the element in the left sub tree. Otherwise, search for the element in the right sub tree. Follow the same procedure for each node.

➤ Example: Search for element 12



```
searchElement (TREE, VAL)
```

```
Step 1: IF TREE -> DATA = VAL OR TREE = NULL
```

```
    Return TREE
```

```
ELSE
```

```
    IF VAL < TREE -> DATA
```

```
        Return searchElement(TREE -> LEFT, VAL)
```

```
    ELSE
```

```
        Return searchElement(TREE -> RIGHT, VAL)
```

```
    [END OF IF]
```

```
    [END OF IF]
```

```
Step 2: END
```


Binary Search Tree(BST)...

➤ Delete:

Deleting a node from Binary search tree includes following three cases:

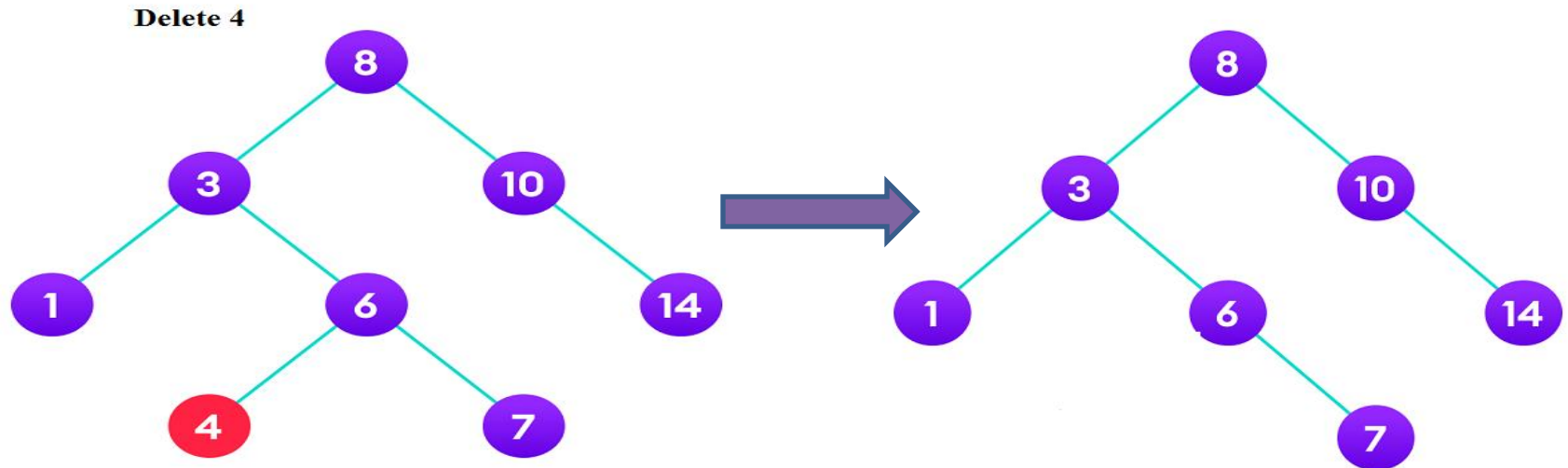
Case 1: Deleting a Leaf node (A node with no children)

Case 2: Deleting a node with one child

Case 3: Deleting a node with two children

➤ Case 1: Deleting a Leaf node (A node with no children)

Search for that node in BST, If the node to be deleted is the leaf node then simply delete the node from the tree .



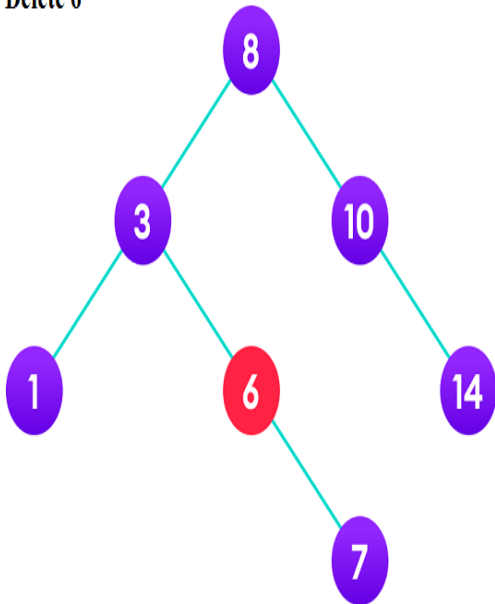
Binary Search Tree(BST)...

➤Case 2: Deleting a node with one child

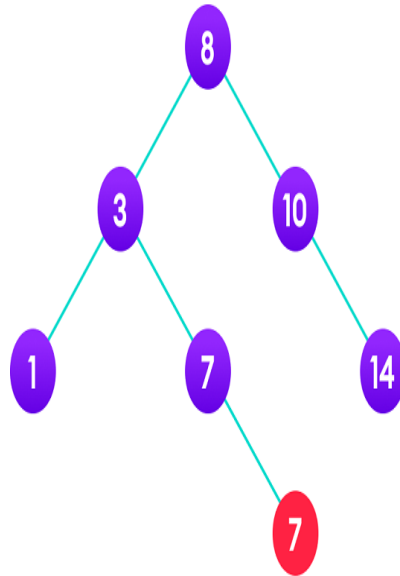
In the second case, the node to be deleted lies has a single child node. In such a case follow the steps below:

1. Replace that node with its child node.
2. Remove the child node from its original position.

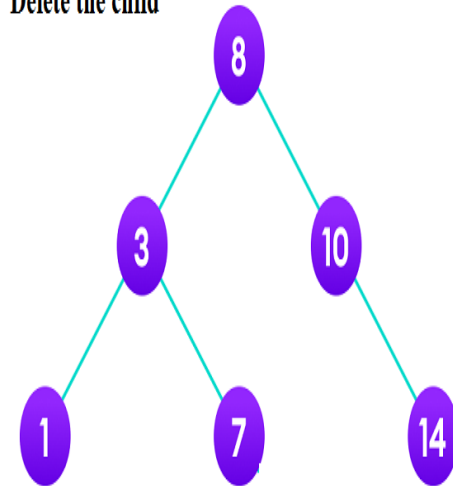
Delete 6



Replace Node 6 with its child node 7



Delete the child

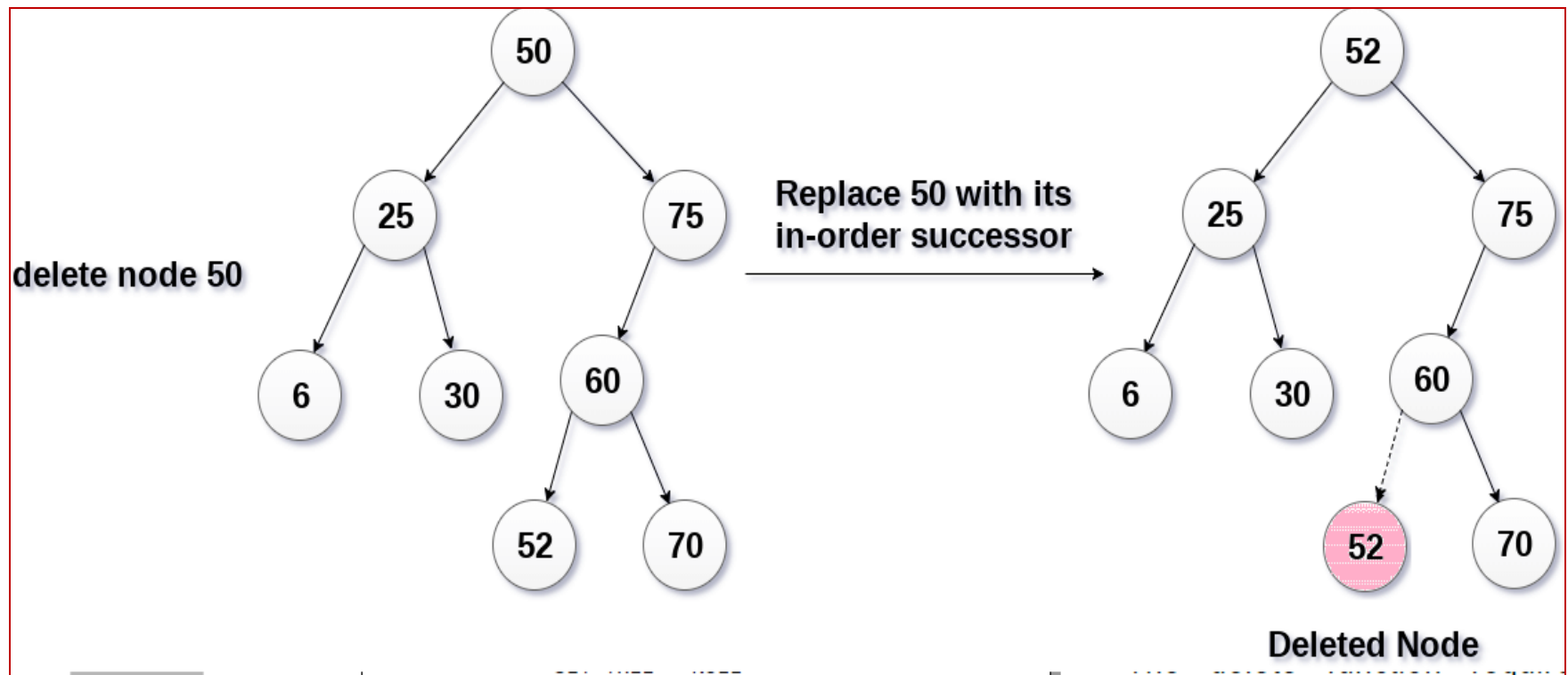


Binary Search Tree(BST)...

➤Case 3: Deleting a node with two children

In the third case, the node to be deleted has two children. In such a case follow the steps below:

1. Replace the node's value with its in-order predecessor (largest value in the left sub-tree) or in-order successor (smallest value in the right sub-tree).
2. The in-order predecessor or the successor can then be deleted using case1 or case2.



Binary Search Tree(BST)...

Deletion:

Delete (TREE, VAL)

Step 1: IF TREE = NULL

Write "VAL not found in the tree"

ELSE IF VAL < TREE -> DATA

Delete(TREE->LEFT, VAL)

ELSE IF VAL > TREE -> DATA

Delete(TREE -> RIGHT, VAL)

ELSE IF TREE -> LEFT AND TREE -> RIGHT

SET TEMP = findLargestNode(TREE -> LEFT)

SET TREE -> DATA = TEMP -> DATA

Delete(TREE -> LEFT, TEMP -> DATA)

ELSE

SET TEMP = TREE

IF TREE -> LEFT = NULL AND TREE -> RIGHT = NULL

SET TREE = NULL

ELSE IF TREE -> LEFT != NULL

SET TREE = TREE -> LEFT

ELSE

SET TREE = TREE -> RIGHT

[END OF IF]

FREE TEMP

[END OF IF]

Step 2: END

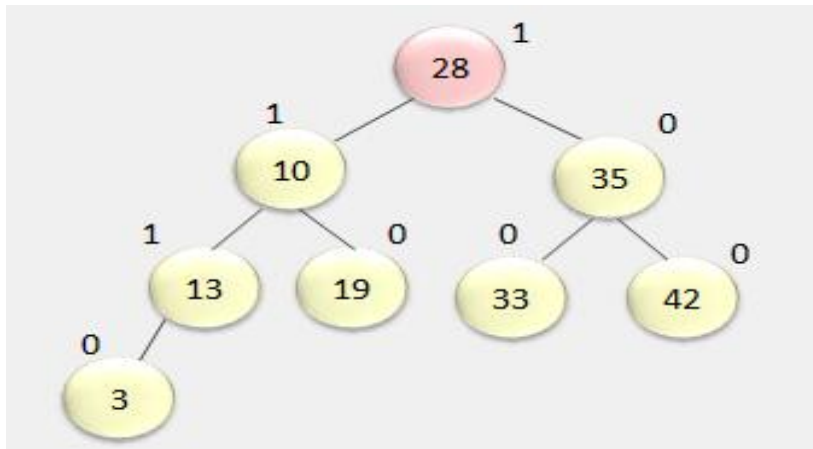
AVL Tree

AVL Tree:

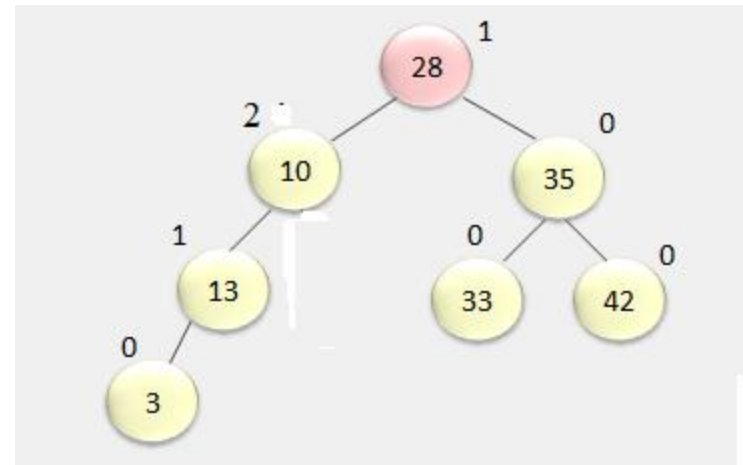
- It is a self-balancing binary search tree invented by G.M. Adelson-Velsky and E.M. Landis in 1962.
- The structure of an AVL tree is the same as that of a binary search tree, in addition to this every node in this tree is associated with balance factor.
- The balance factor of a node is calculated by subtracting the height of its right sub-tree from the height of its left sub-tree.

$$\text{Balance factor} = \text{Height (left sub-tree)} - \text{Height (right sub-tree)}$$

- A binary search tree in which every node has a balance factor of **-1, 0, or 1** is said to be height balanced. A node with any other balance factor is considered to be unbalanced and requires rebalancing of the tree.



AVL Tree

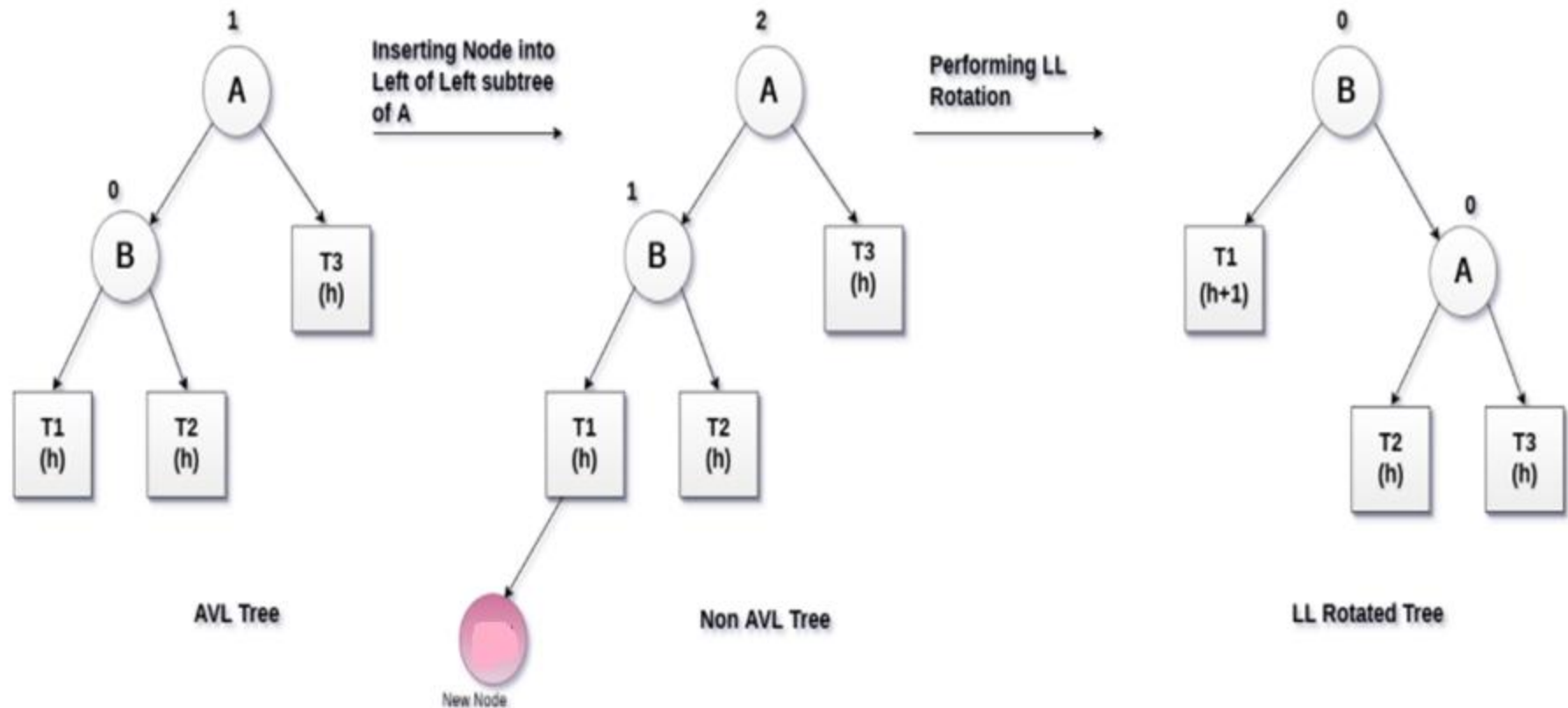


Not AVL Tree

AVL Tree...

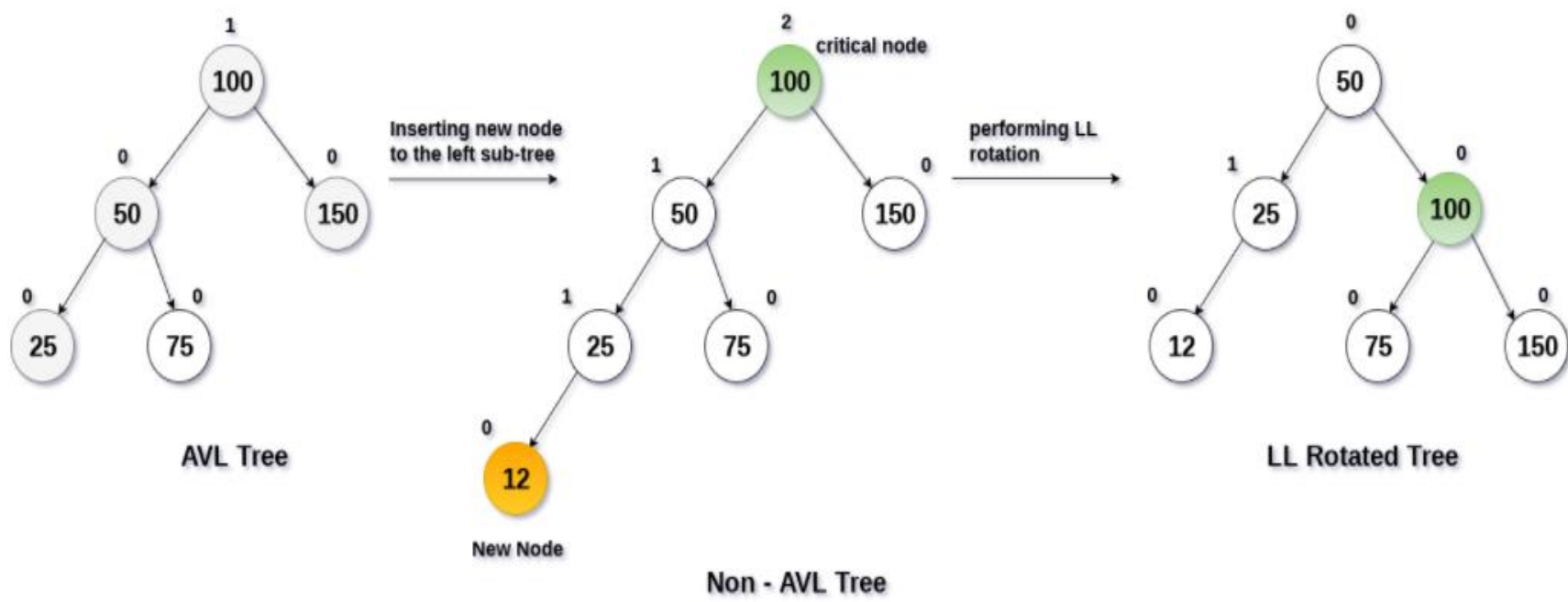
- The Unbalanced node in the AVL tree is called critical node.
- In order to balance the unbalanced tree/node we have to make rotations .Four types of rotations are:

1. LL rotation: The new node is inserted in the left sub-tree of the left sub-tree of the critical node.



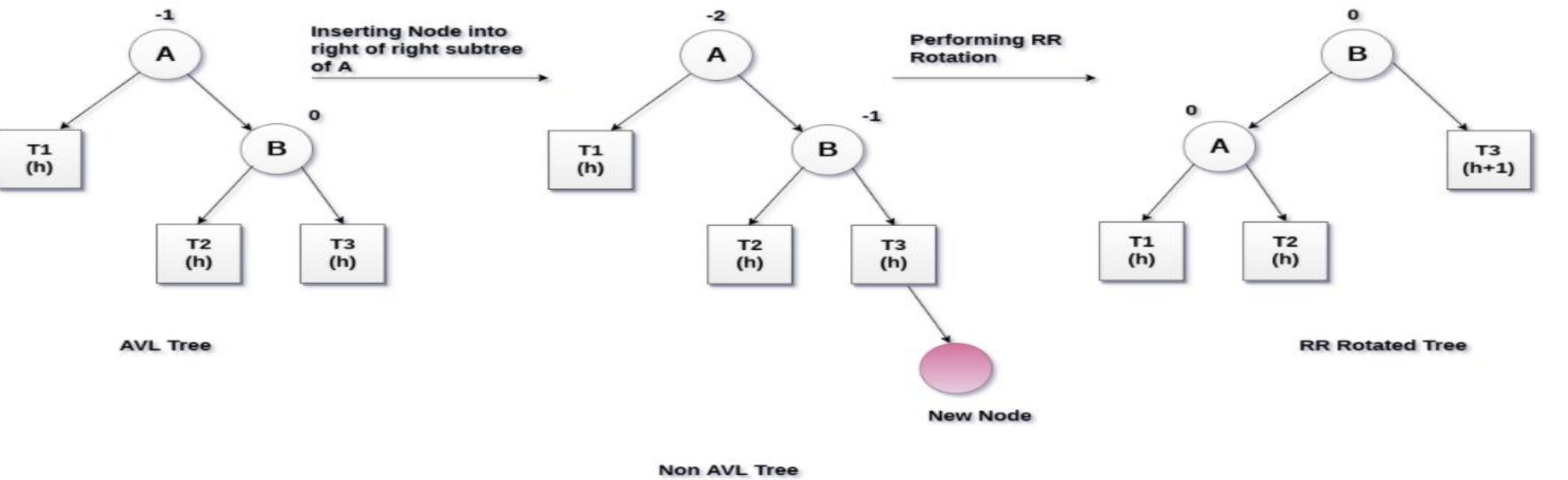
AVL Tree...

LL Rotation Example:

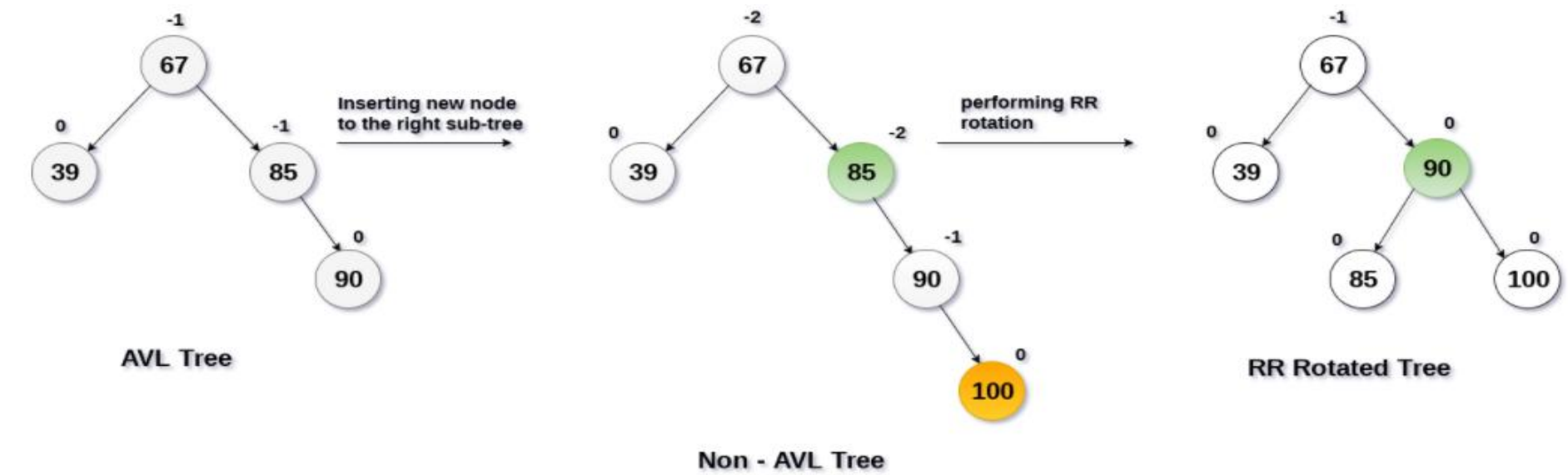


2.RR Rotation: The new node is inserted in the right sub-tree of the right sub-tree of the critical node.

AVL Tree...

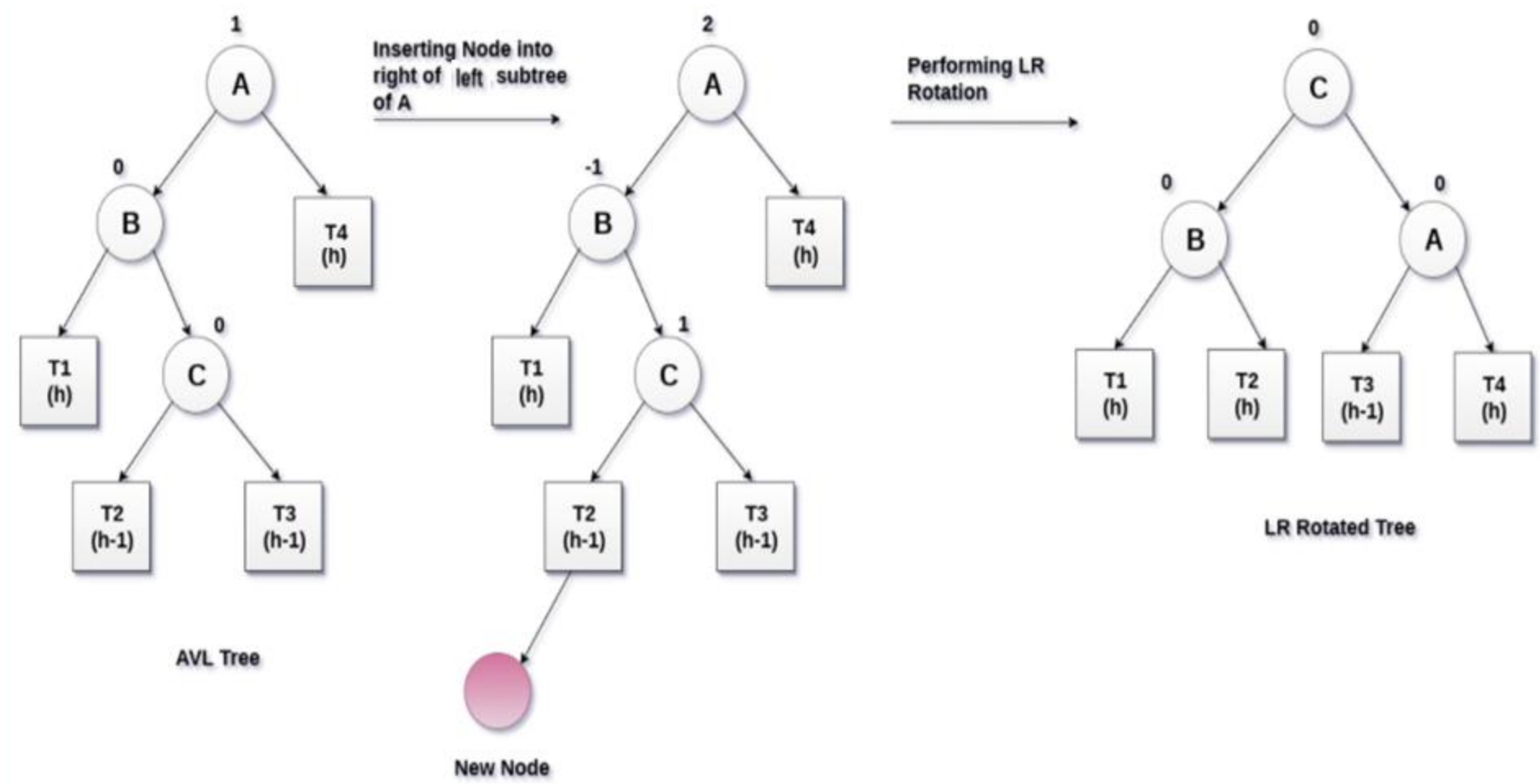


RR Rotation Example



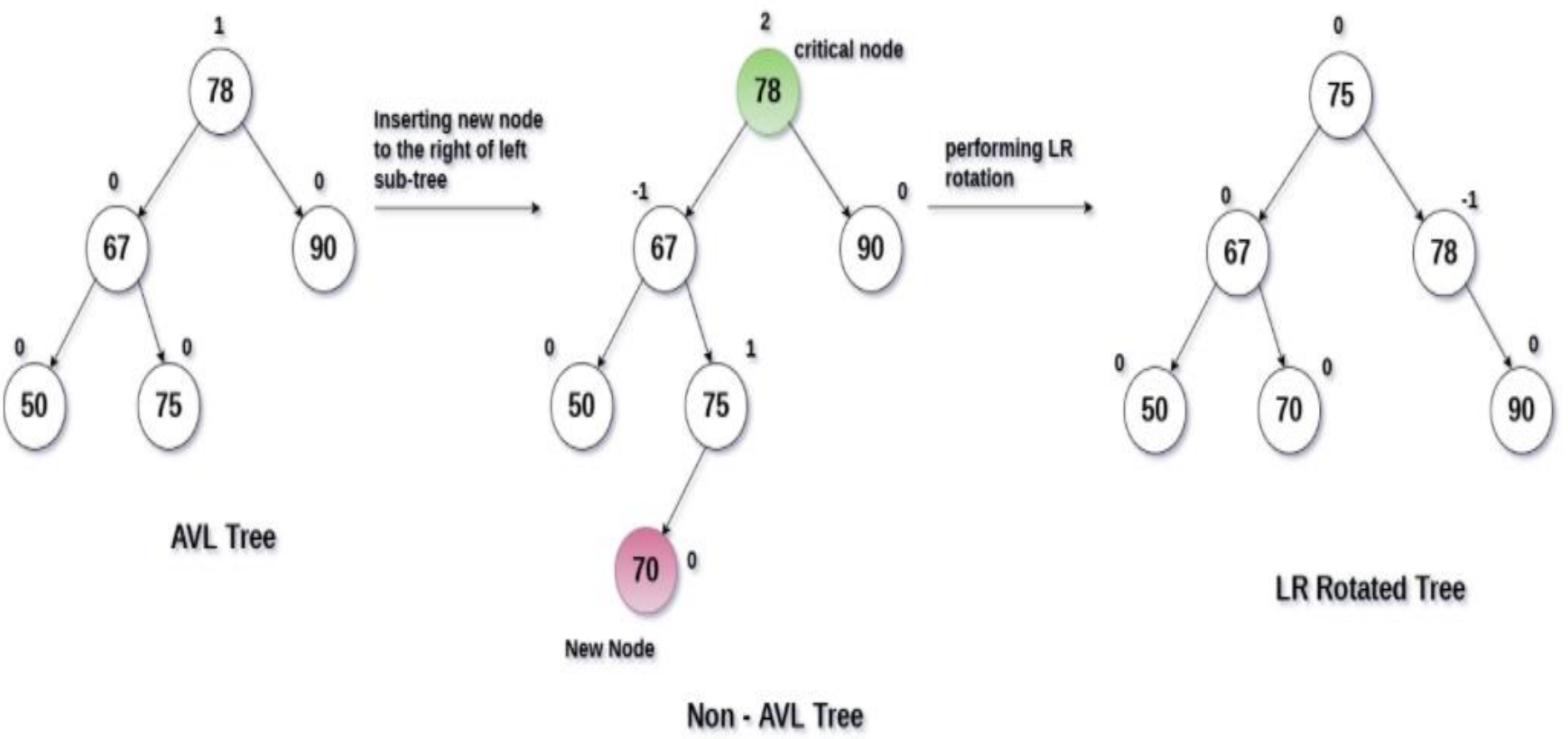
AVL Tree...

3.LR Rotation :The new node is inserted in the right sub-tree of the left sub-tree of the critical node.



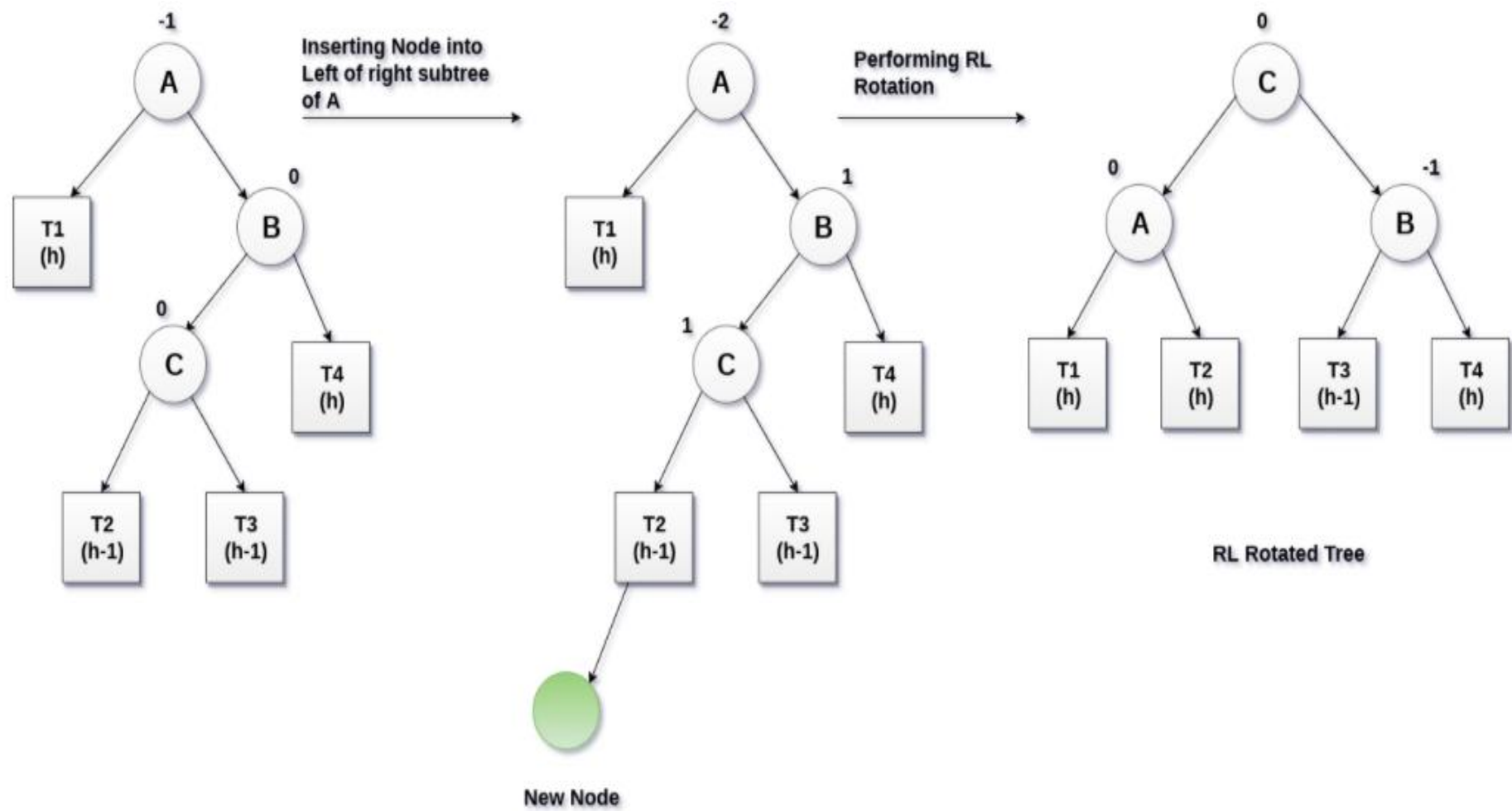
AVL Tree...

LR Rotation Example



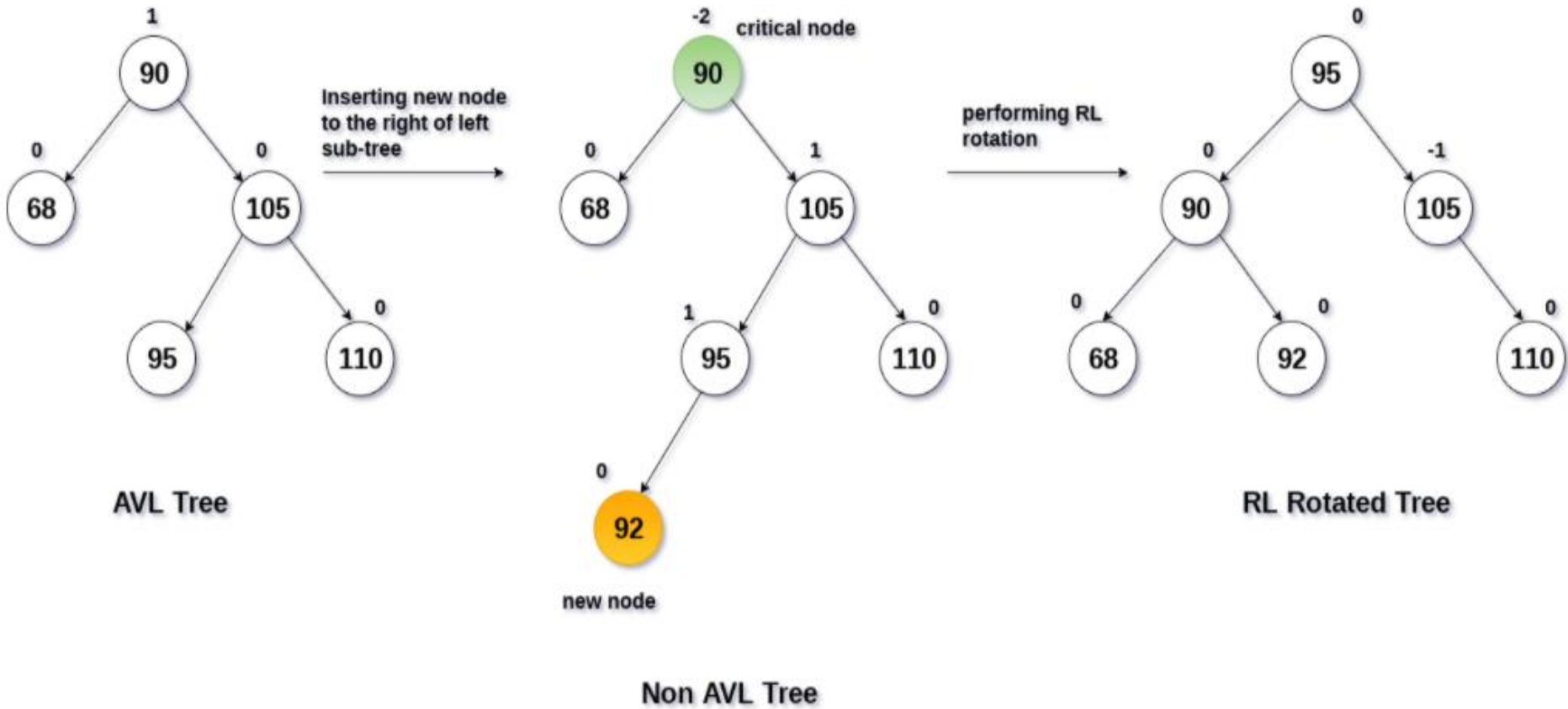
AVL Tree...

4.RL Rotation: The new node is inserted in the left sub-tree of the right sub-tree of the critical node.



AVL Tree...

RL Rotation Example



AVL Tree...

Operations:

1. **Insertion**
2. **Deletion**
3. **Searching** (Search operation in AVL is same as BST)

1. Insertion:

- Insertion in an AVL tree is done in the same way as it is done in a binary search tree.
- In the AVL tree, the new node is always inserted as the leaf node.
- After insertion the balance factor of each node has to be updated. If any unbalance node is found then rotation has to be performed to balance the tree.

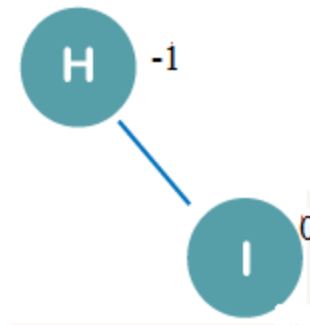
Example:

Construct an AVL tree for the elements: H, I, J, B, A, E, C, F, D, G, K, L

Insert H



Insert I

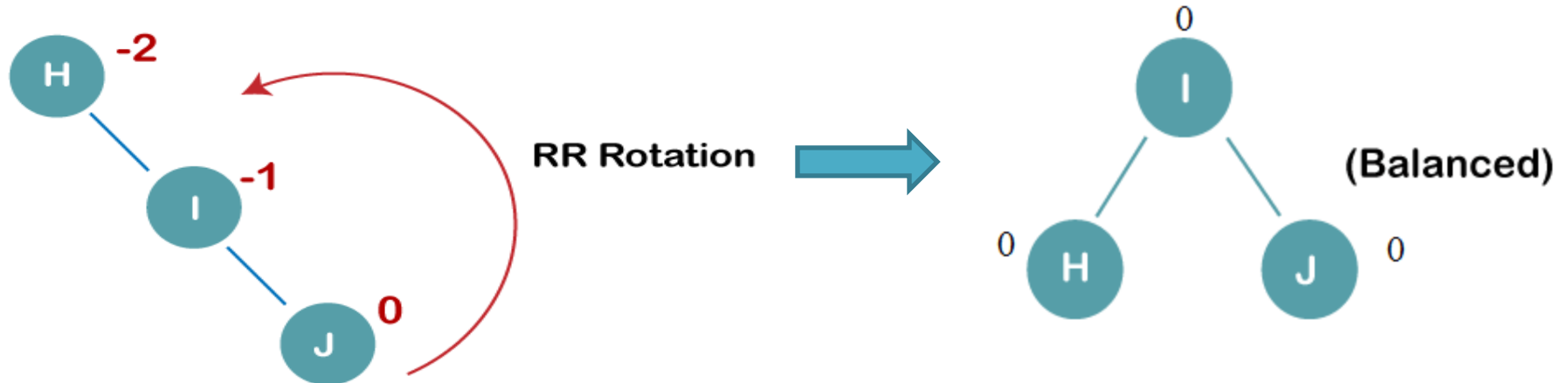


AVL Tree...

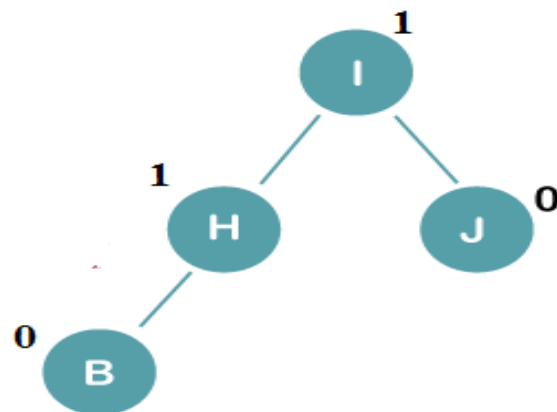
Example...

Elements: H, I, J, B, A, E, C, F, D, G, K

Insert J



Insert B

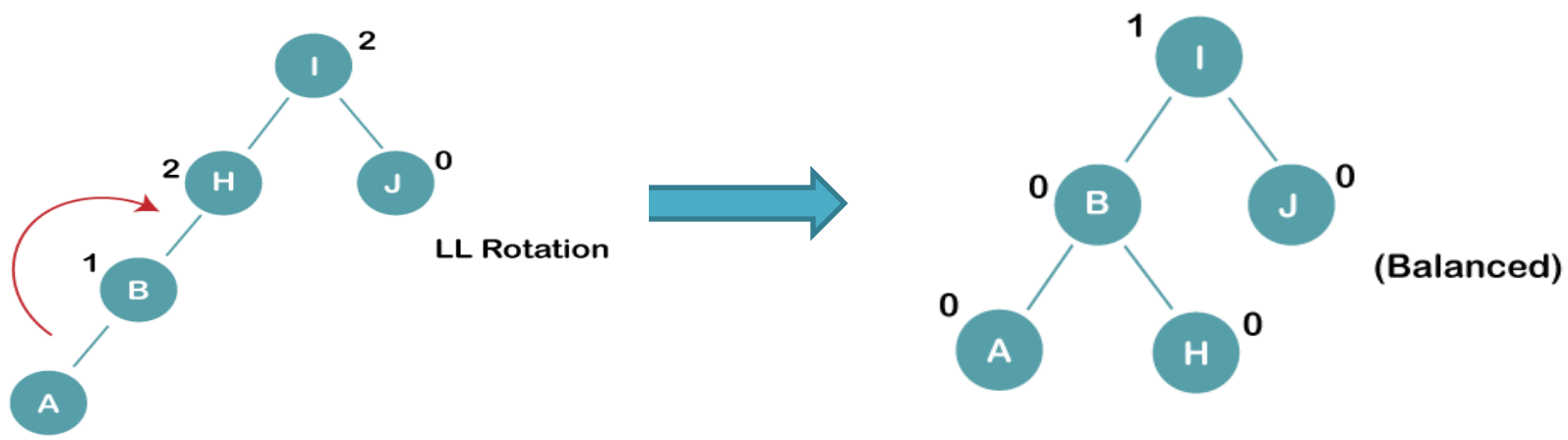


AVL Tree...

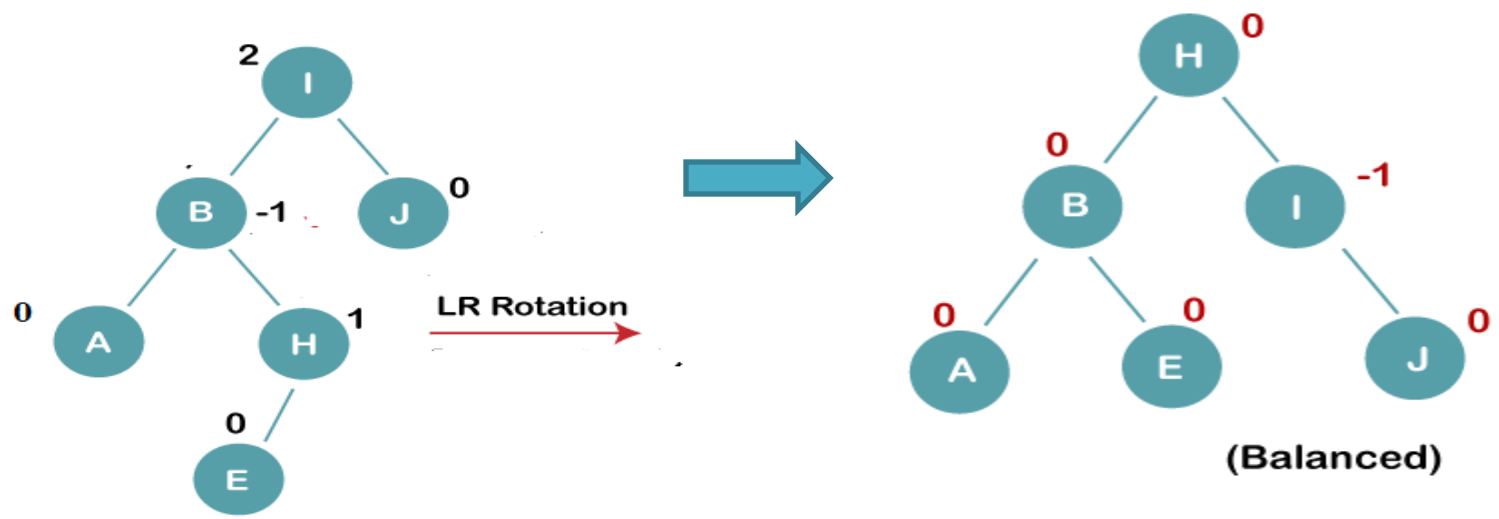
Example...

Elements: H, I, J, B, A, E, C, F, D, G, K

Insert A



Insert E

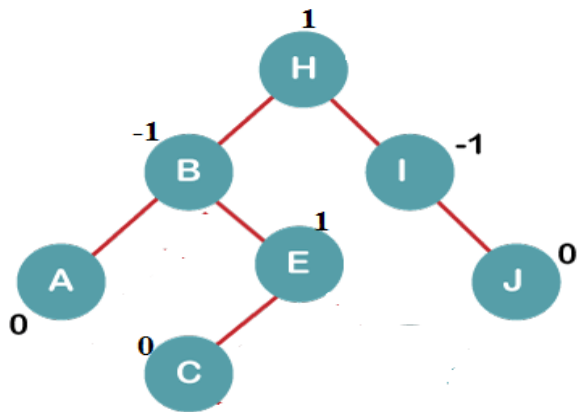


AVL Tree...

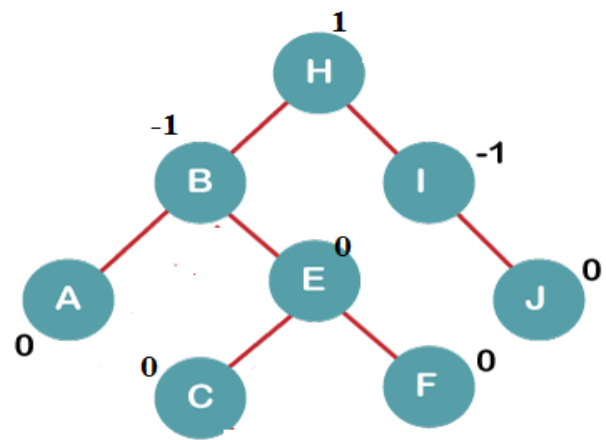
Example...

Elements: H, I, J, B, A, E, C, F, D, G, K

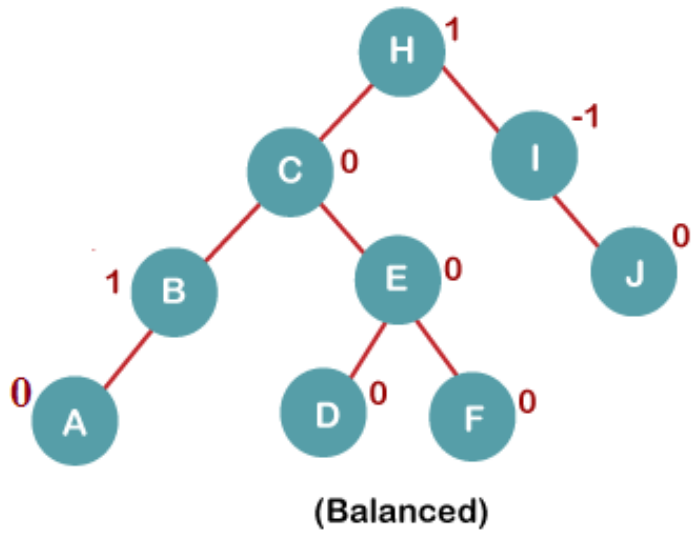
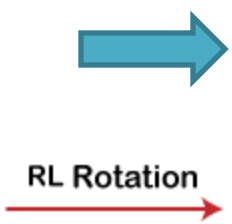
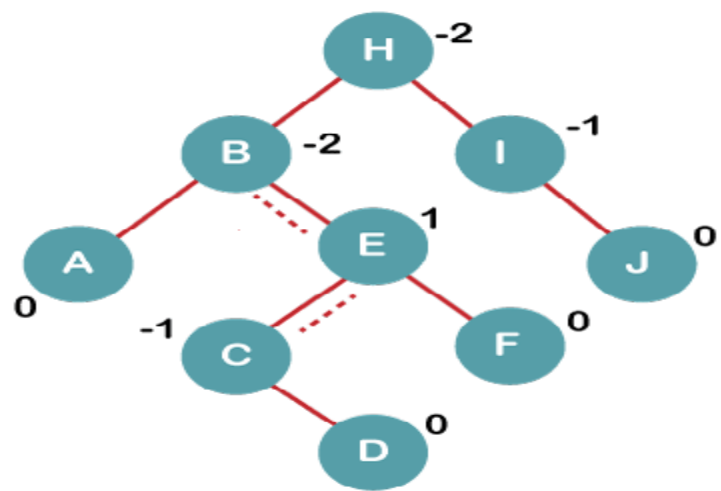
Insert C



Insert F



Insert D

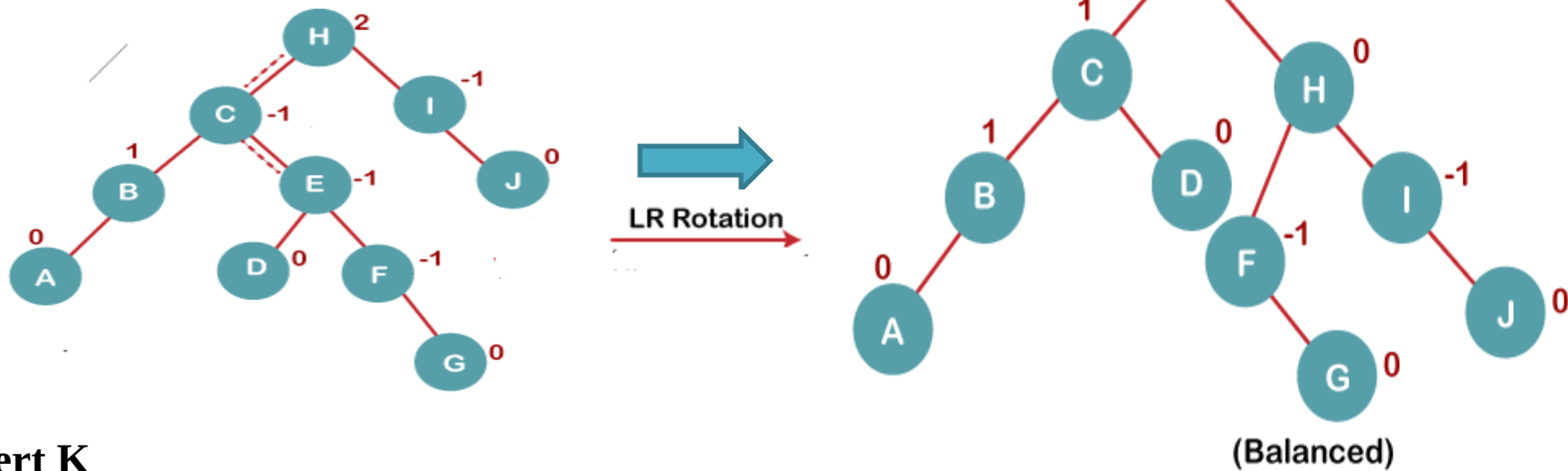


AVL Tree...

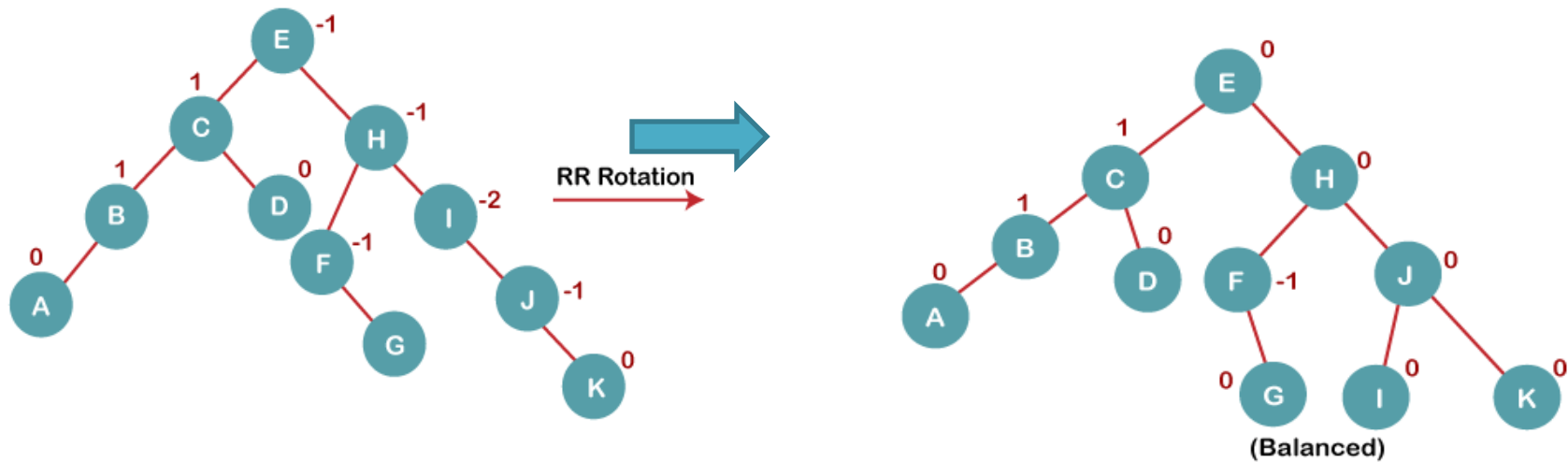
Example...

Elements: H, I, J, B, A, E, C, F, D, G, K

Insert G



Insert K



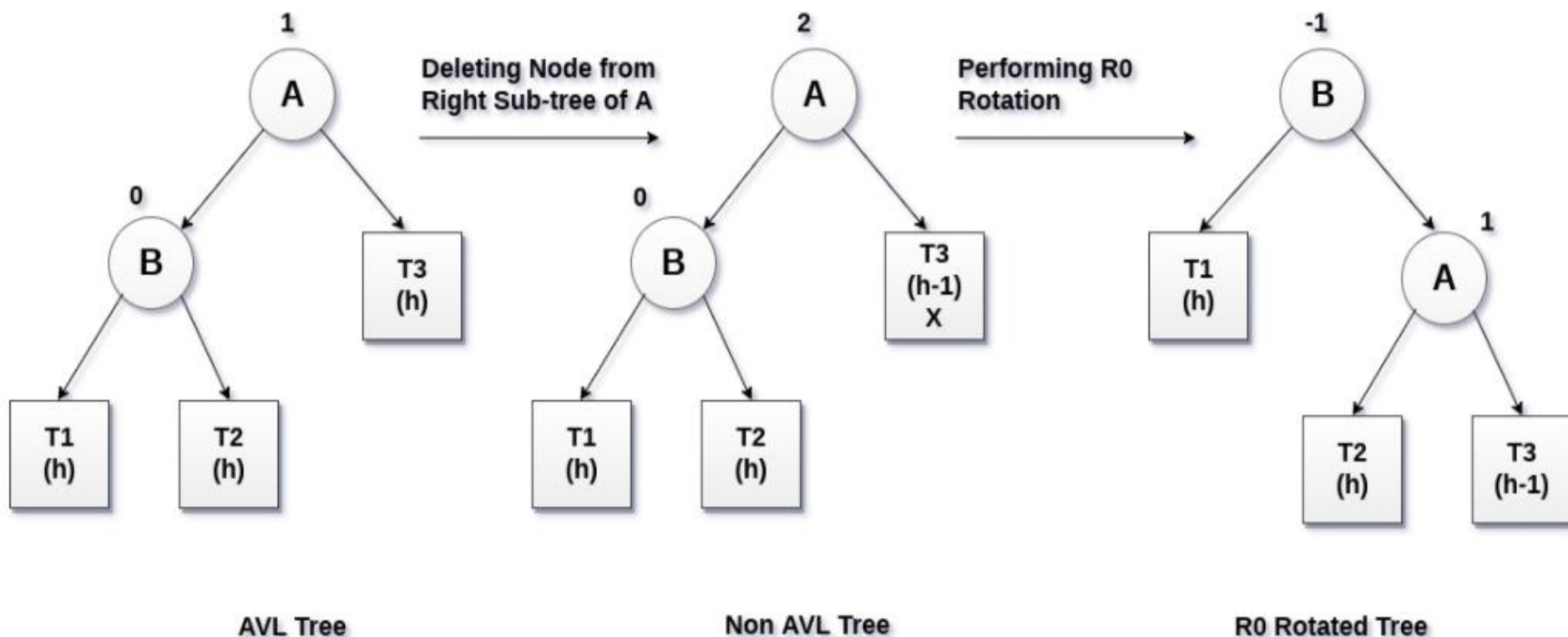
AVL Tree...

2. Deletion:

➤ Deleting a node from an AVL tree is similar to that in a binary search tree. Deletion may disturb the balance factor of an AVL tree and therefore the tree needs to be rebalanced in order to maintain the AVLness. For this purpose, we need to perform rotations. The two types of rotations are L rotation and R rotation. (L rotations are mirror images of R Rotations)

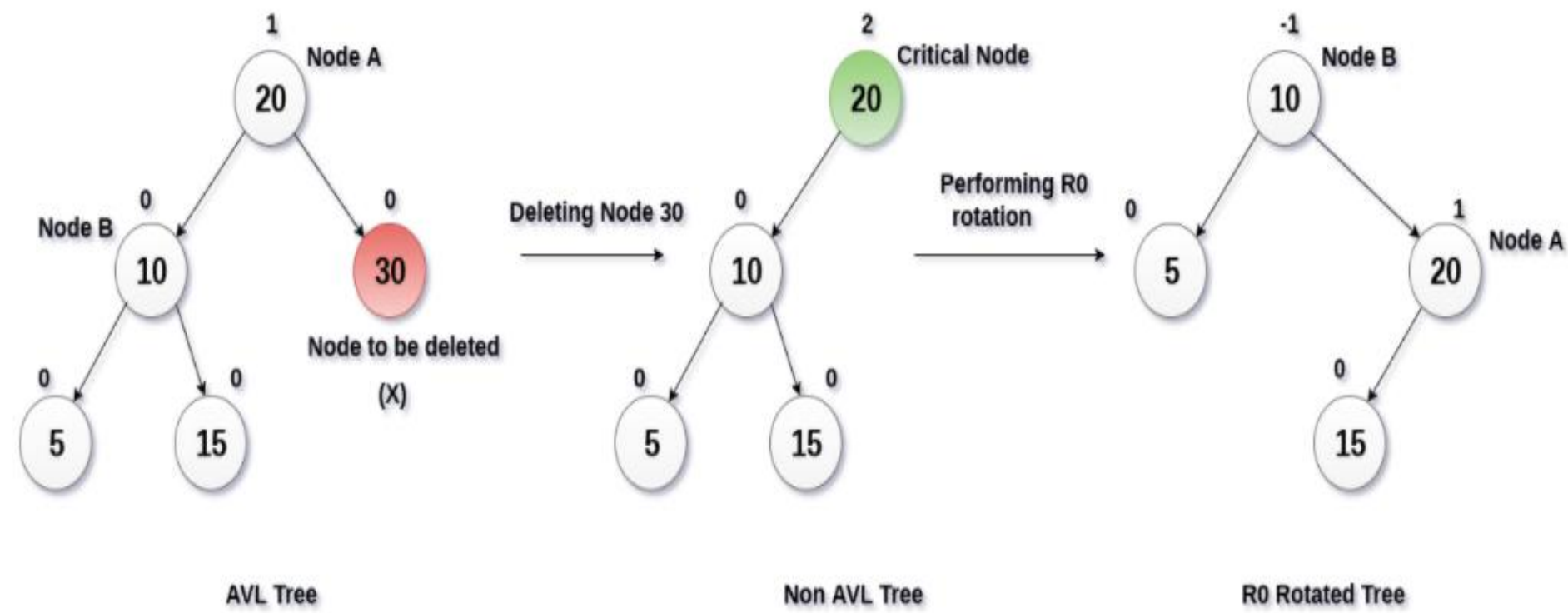
➤ **R rotations are**

1. R0 rotation (Node B has balance factor 0)



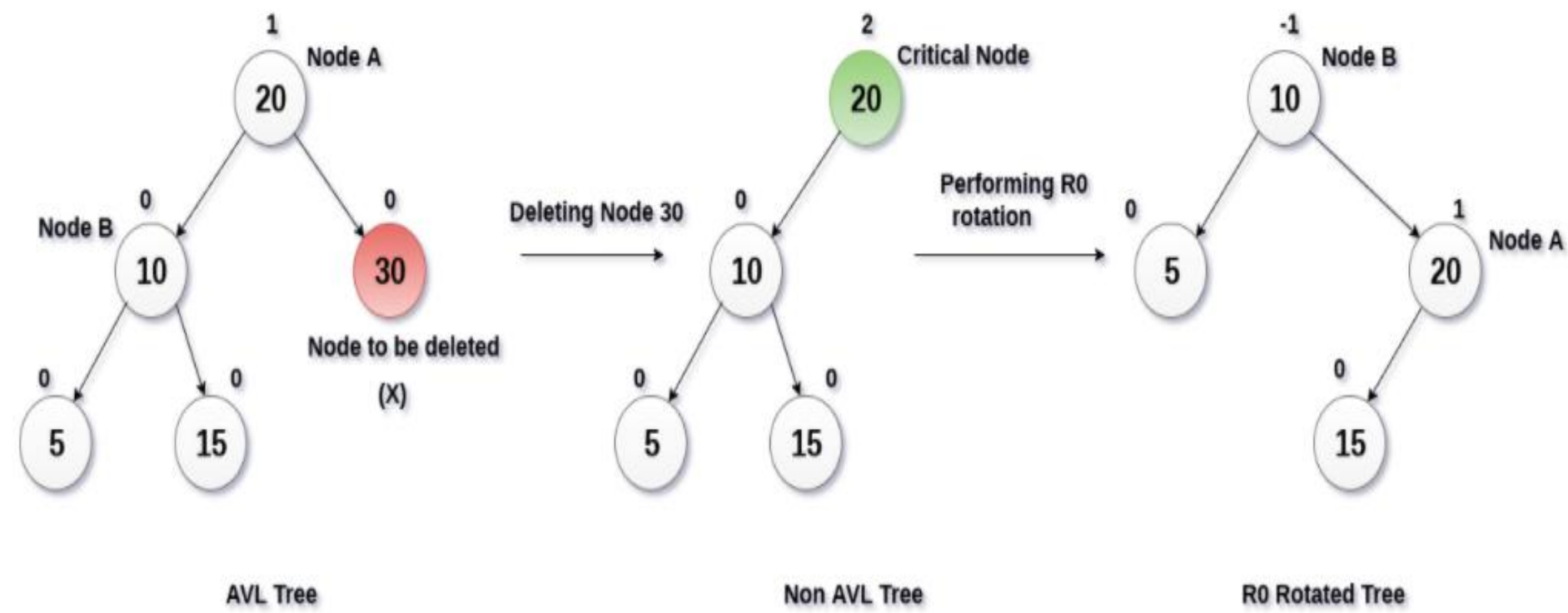
AVL Tree...

R0 Rotation Example



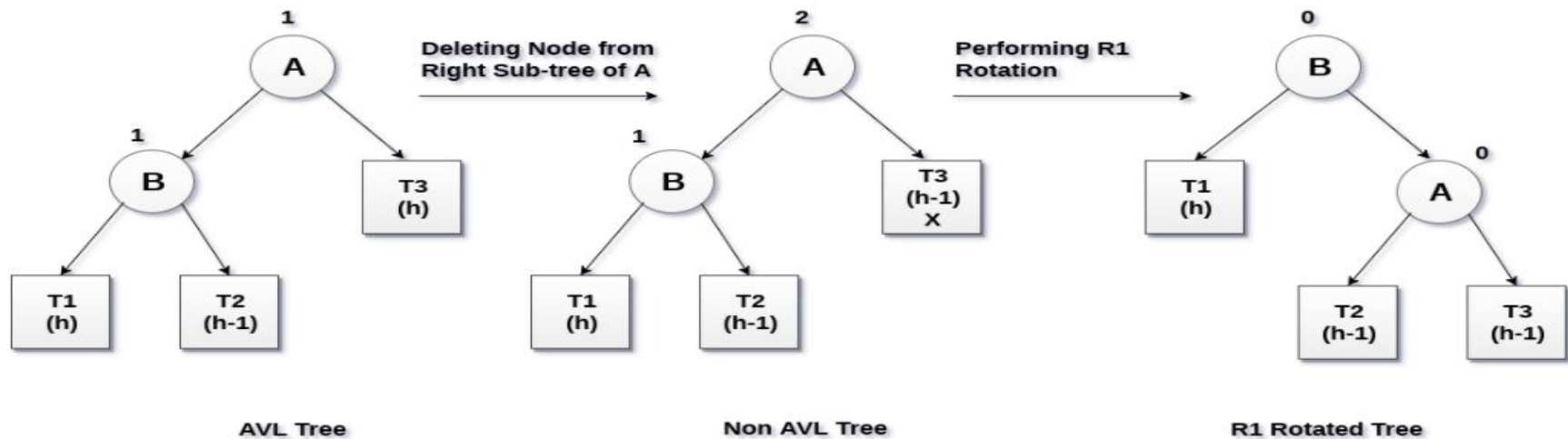
AVL Tree...

R0 Rotation Example

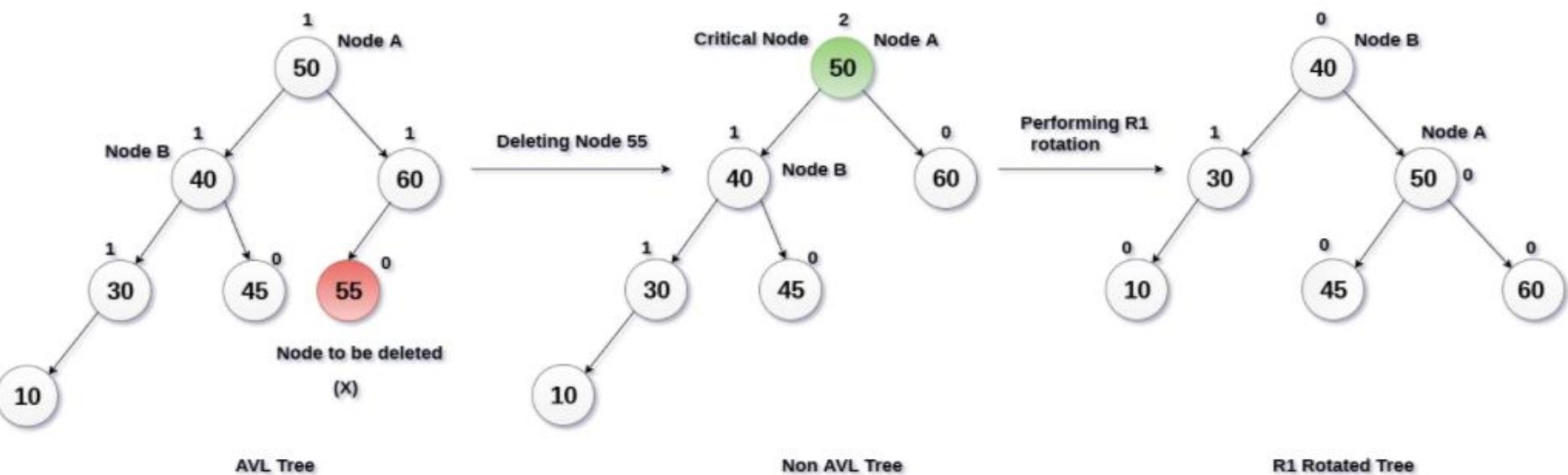


AVL Tree...

2. R1 Rotation (Node B has balance factor 1)

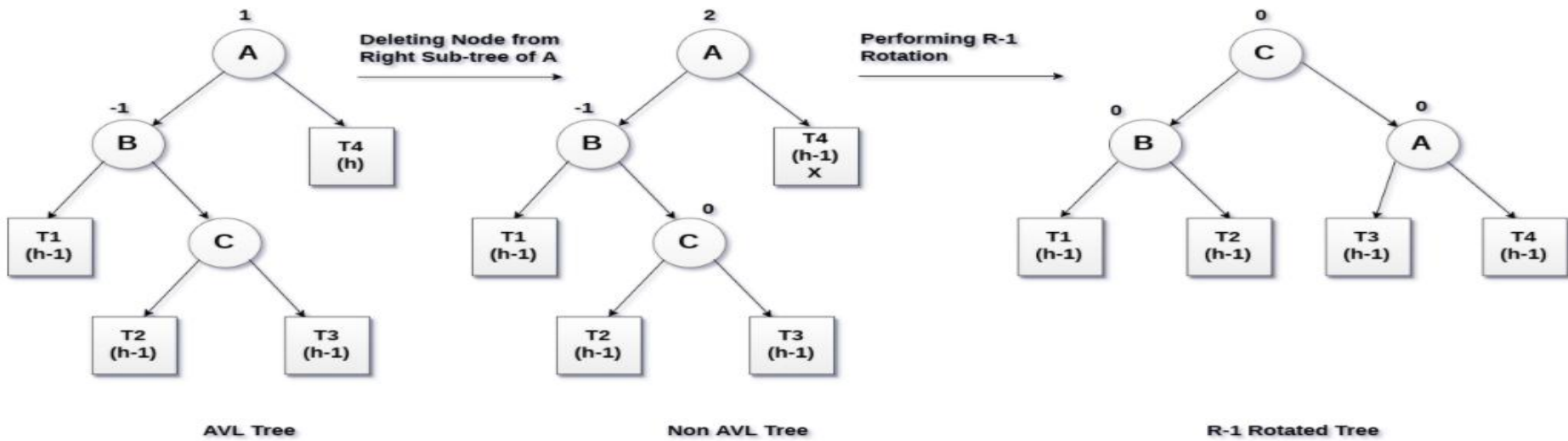


Example

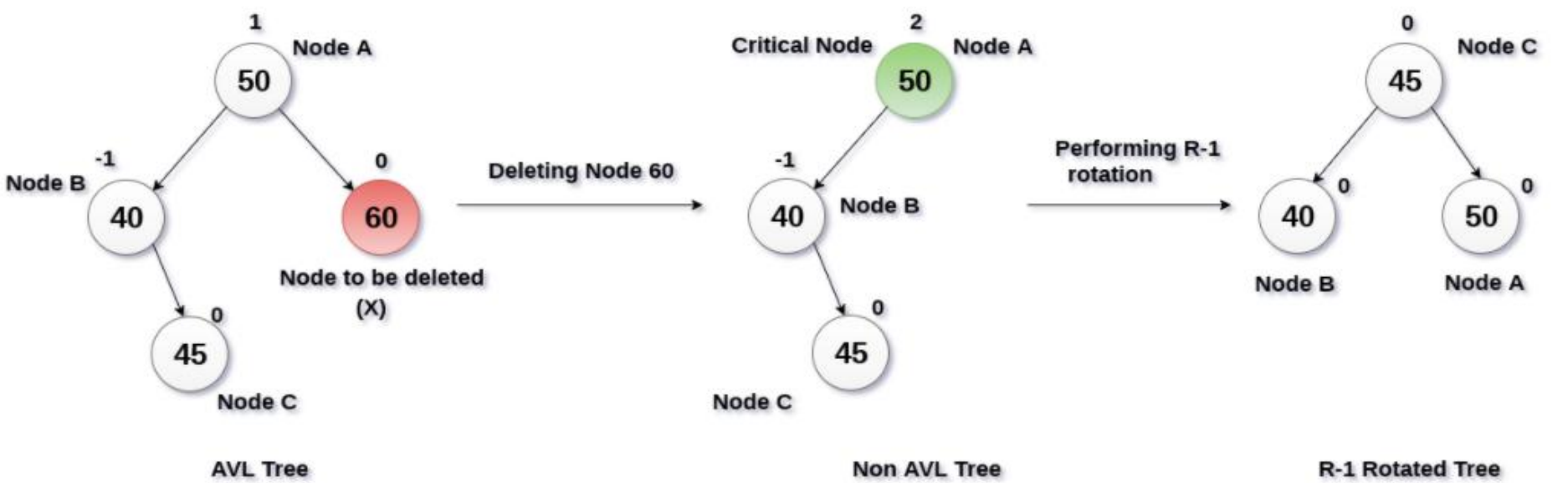


AVL Tree...

2. R-1 Rotation (Node B has balance factor -1)



Example:



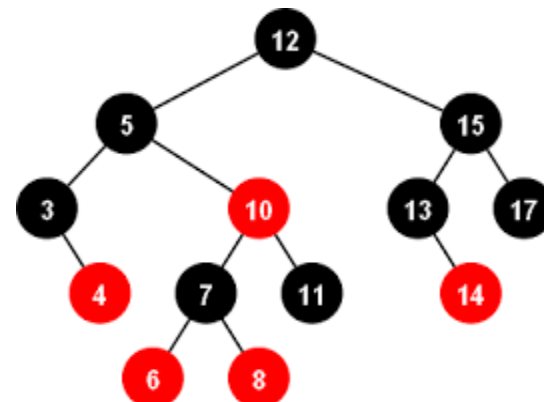
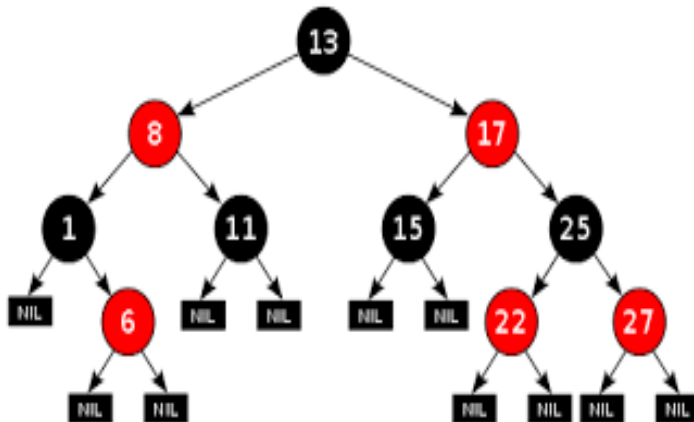
Red Black Tree

Red Black Tree:

- It is a self balanced Binary Search Tree in which every node is colored either RED or BLACK.
- In Red Black Tree, the color of a node is decided based on the properties of Red-Black Tree. Every Red Black Tree has the following properties.

Properties of Red Black Tree

1. Red - Black Tree must be a Binary Search Tree.
2. The ROOT node must be colored BLACK.
3. The children of Red colored node must be colored BLACK. (There should not be two consecutive RED nodes).
4. In all the paths of the tree, there should be same number of BLACK colored nodes.
5. Every new node must be inserted with RED color.
6. Every leaf (e.i. NULL node) must be colored BLACK.

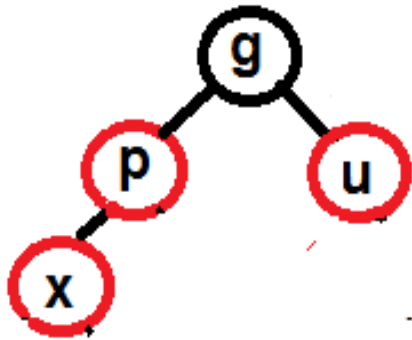


Red Black Tree...

Operations:

1.Insertion:

- Insertion in red black tree is same as BST.
- The color of inserted node is Red.



- In above Red Black tree g- Grand parent of node x, p is parent of x and u is uncle of x, where x is newly inserted node.
- In Red-Black tree, we use two tools to do balancing.
 - 1) Recoloring
 - 2) Rotation
- If uncle is red, we do recoloring.
- If uncle is black, we do rotations and/or recoloring.
- Color of a NULL node is considered as BLACK.

Red Black Tree...

Insertion Algorithm:

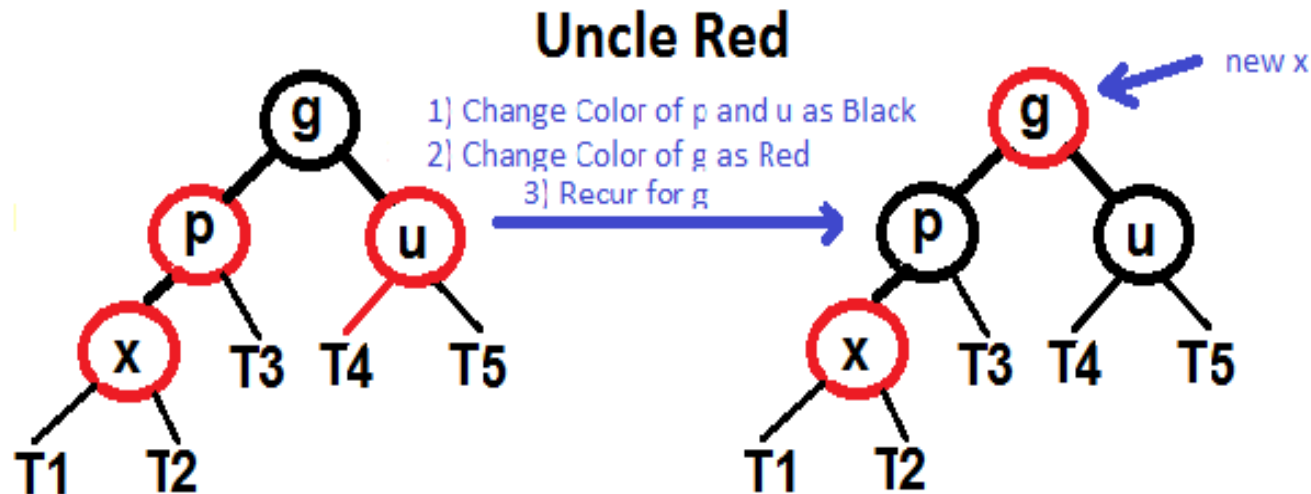
Let x be the newly inserted node, the color of newly inserted nodes as **RED**.

Case 1: If x is root, change color of x as **BLACK**



Case 2: If x 's uncle is **RED** (**Red Uncle Condition**)

1. Change color of parent and uncle as **BLACK**.
2. color of grand parent as **RED**.
3. Change $x = x$'s grandparent, repeat steps 2 and 3 for new x .



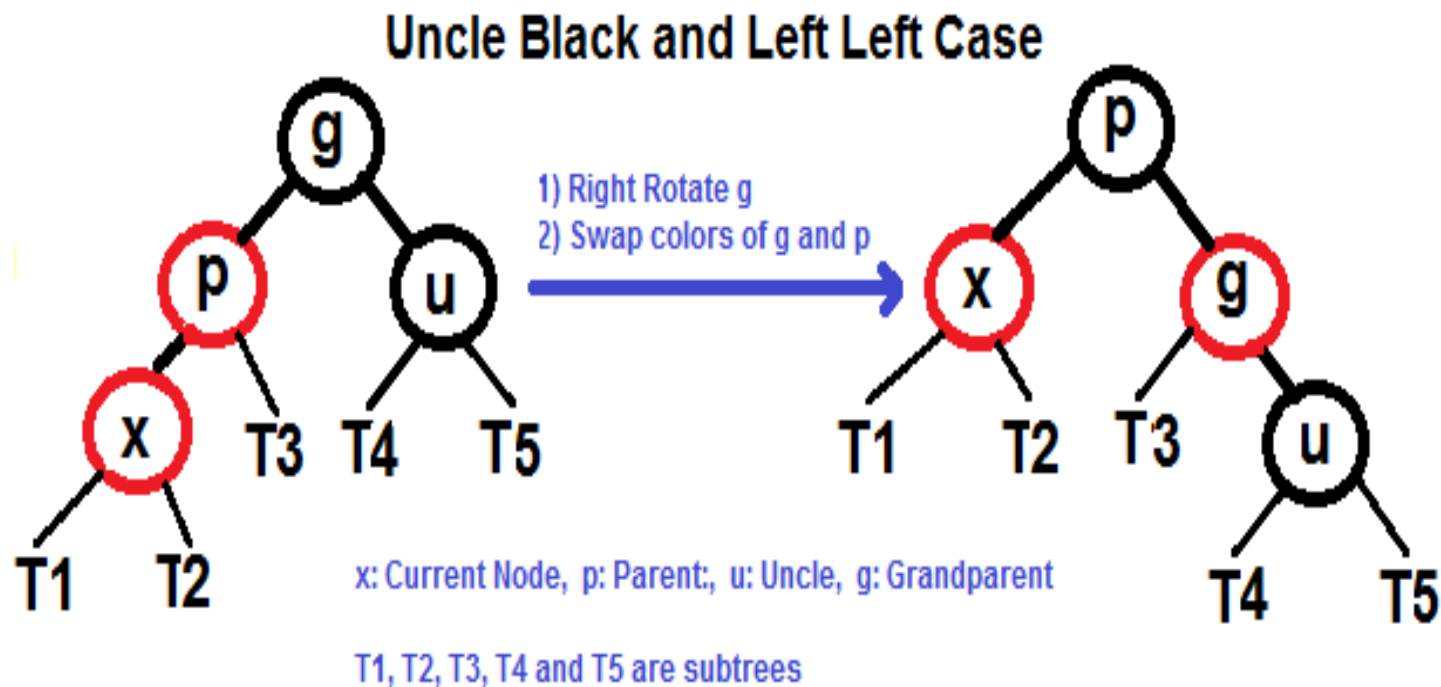
x : Current Node, p : Parent, u : Uncle, g : Grandparent

$T1$, $T2$, $T3$, $T4$ and $T5$ are subtrees

Red Black Tree...

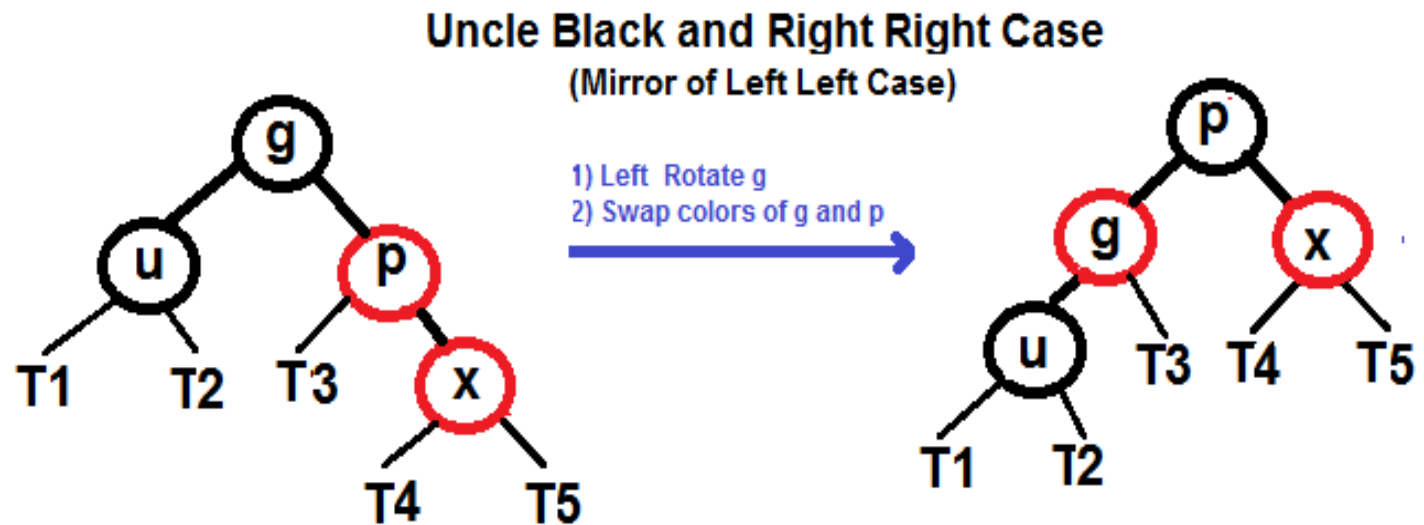
Case 3: If x's uncle is BLACK or no uncle, The possible four configurations are

1. Left Left Case (p is left child of g and x is left child of p).
 - i. Right Rotate Grand parent g
 - ii. Swap colors of g and p



Red Black Tree...

2. Right Right Case (p is right child of g and x is right child of p).
 - i. Left Rotate Grand parent g
 - ii. Swap colors of g and p



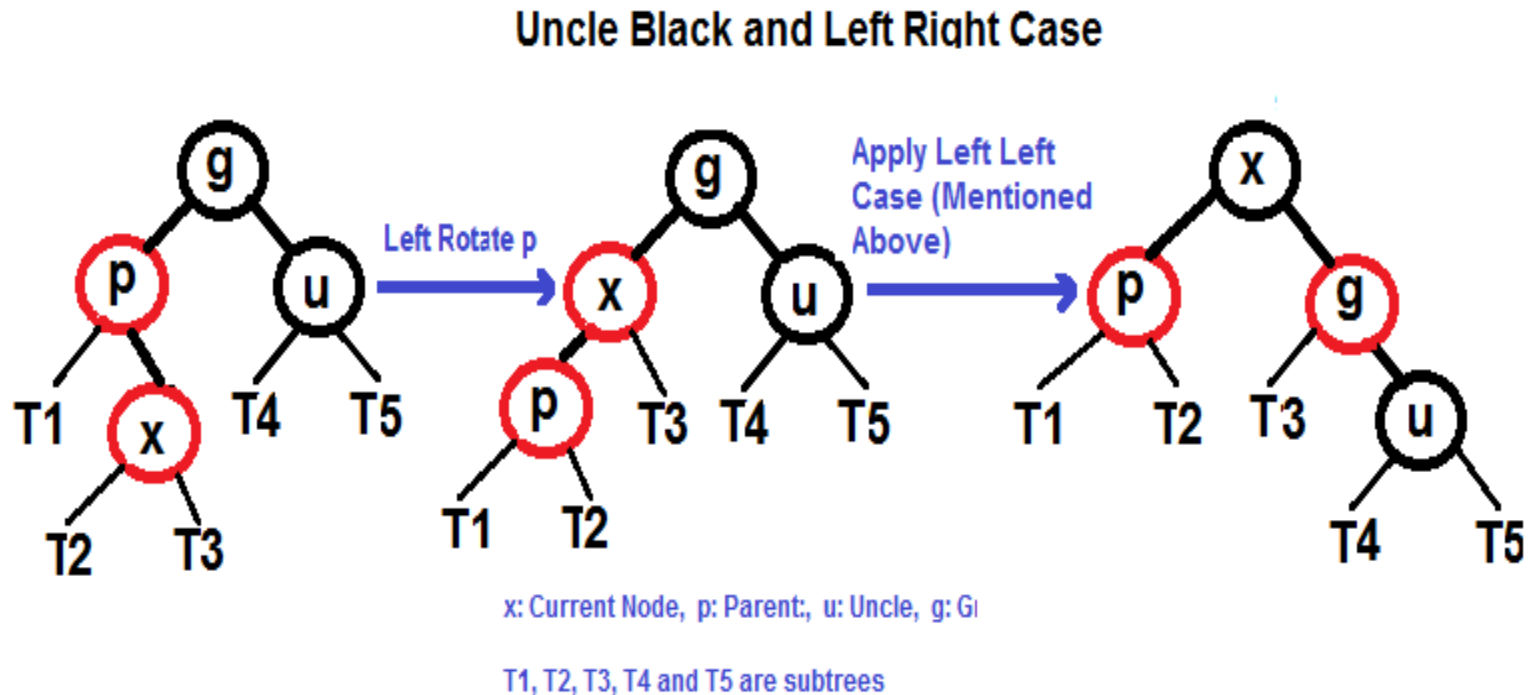
x: Current Node, p: Parent, u: Uncle, g: Grandparent

T1, T2, T3, T4 and T5 are subtrees

Red Black Tree...

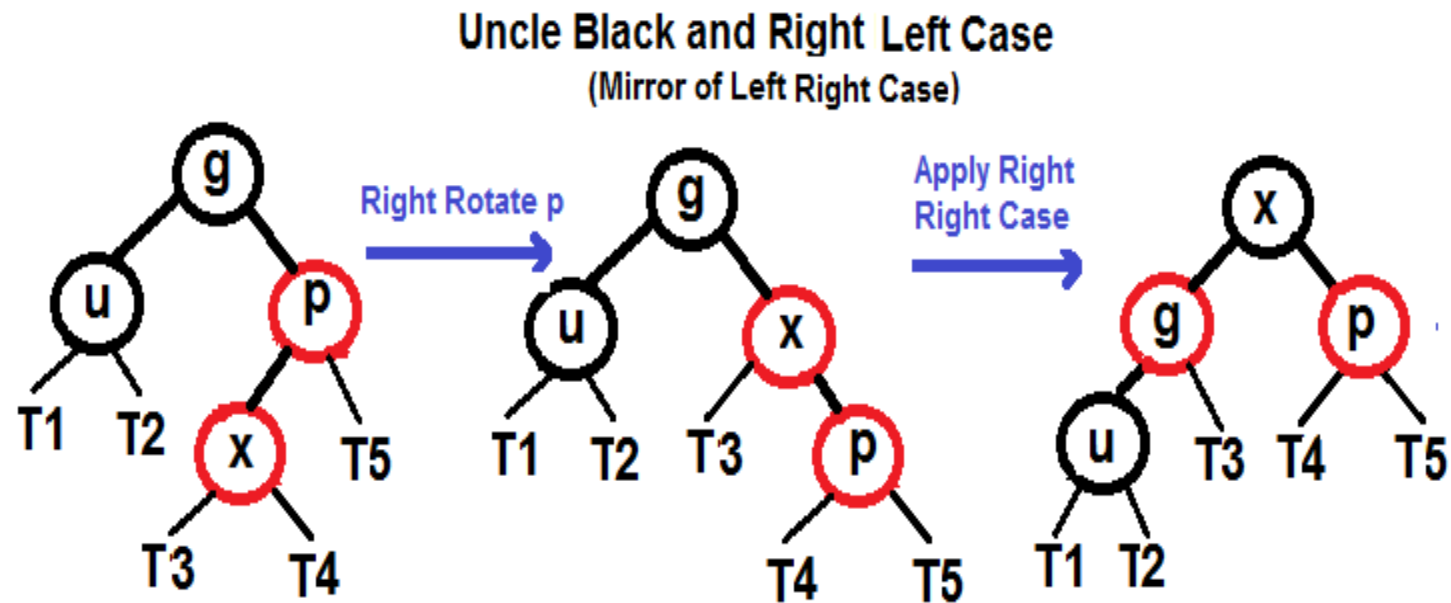
3. Left Right Case (p is left child of g and x is right child of p).

- i. Left Rotate p
- ii. Apply Left-Left Case



Red Black Tree...

4. Right Left Case (p is right child of g and x is left child of p).
 - i. Right Rotate p
 - ii. Apply Right-Right Case



x: Current Node, p: Parent, u: Uncle, g: Grandparent

T1, T2, T3, T4 and T5 are subtrees

Red Black Tree...

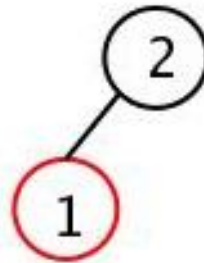
Problem: Insert the following elements into Red black tree

2, 1, 4, 5, 9, 3, 6, 7

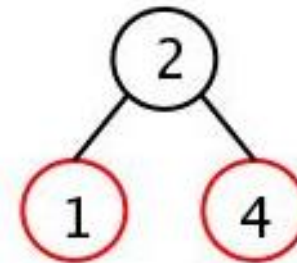
Insert(2)



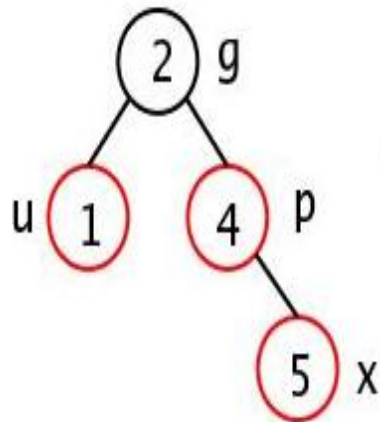
Insert(1)



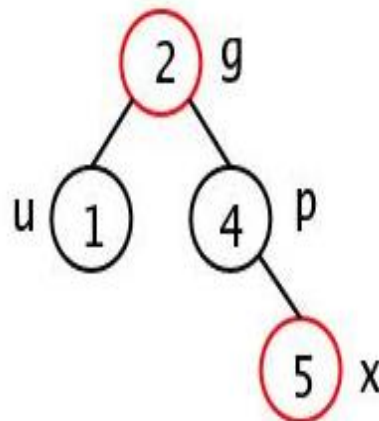
Insert(4)



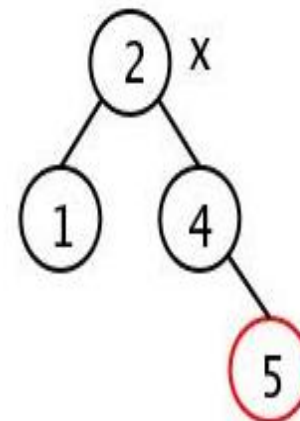
Insert(5)



case 2:



case 1:

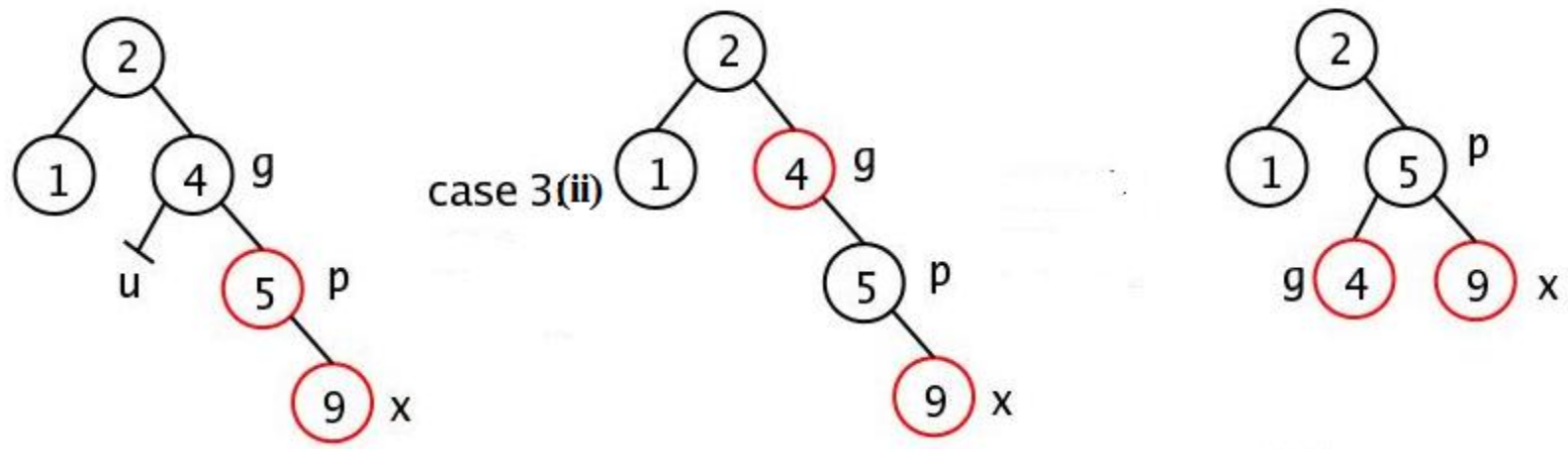


Red Black Tree...

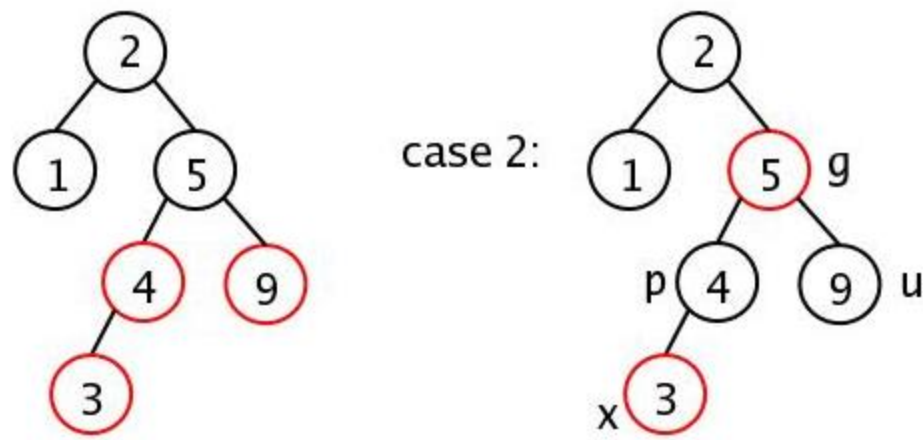
Problem: Insert the following elements into Red black tree

2, 1, 4, 5, 9, 3, 6, 7

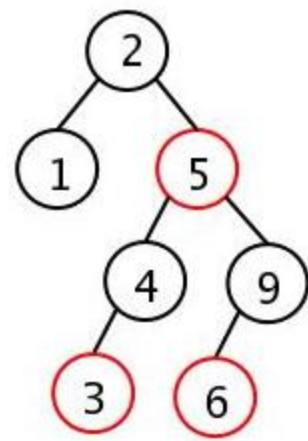
Insert(9)



Insert(3)



Insert(6)

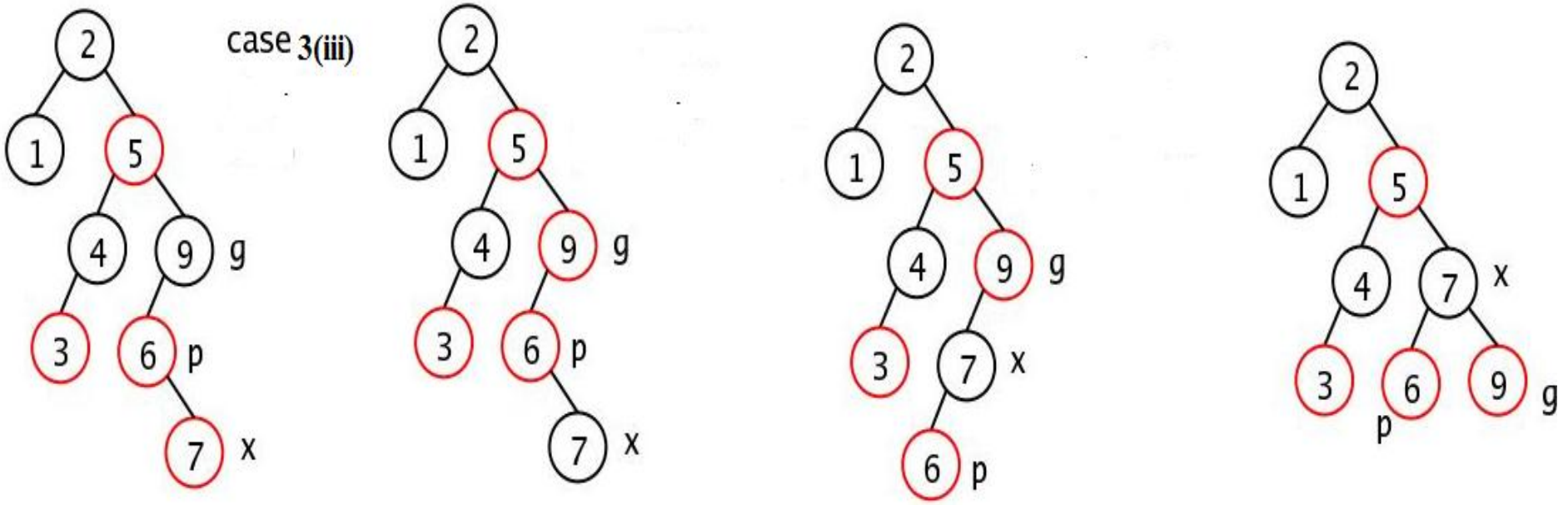


Red Black Tree...

Problem: Insert the following elements into Red black tree

2, 1, 4, 5, 9, 3, 6, 7

Insert(7)



Red Black Tree...

Deletion:

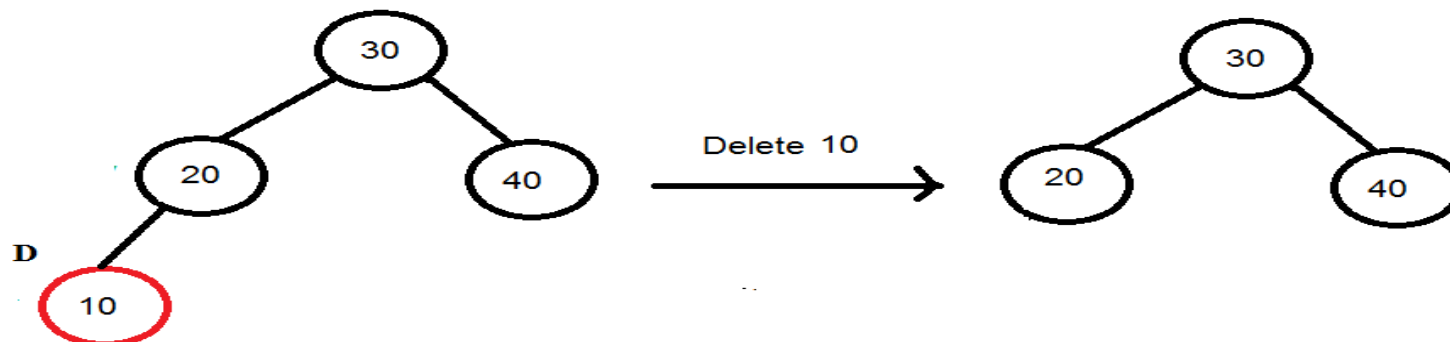
Important points to be remembered during deletion:

1. Red Black tree follows BST Deletion
2. In this only last nodes(leave nodes) are deleted ,Nodes with children are replaced.
3. In case node to be replaced has two children then replace it with closest predecessor from left sub tree.
4. In case node to be replaced has one child then replace it with its child.
5. In case node to be replaced has no child then replace it with NULL node.
6. If black node replaces black node then it will become double black.
7. If red node replaces black node then it will become single black.

Deletion Cases:

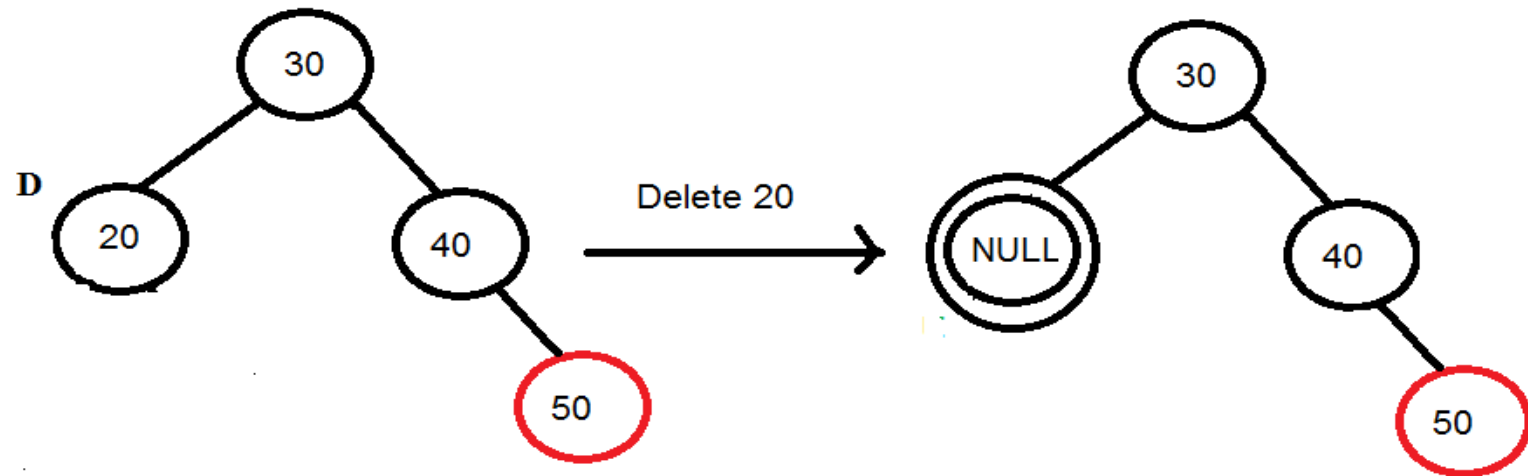
Cases 1:

- i) if deleted node D is RED-Delete it directly and do nothing.



Red Black Tree...

ii) if deleted node D is not RED-then Double Black DB Exists.



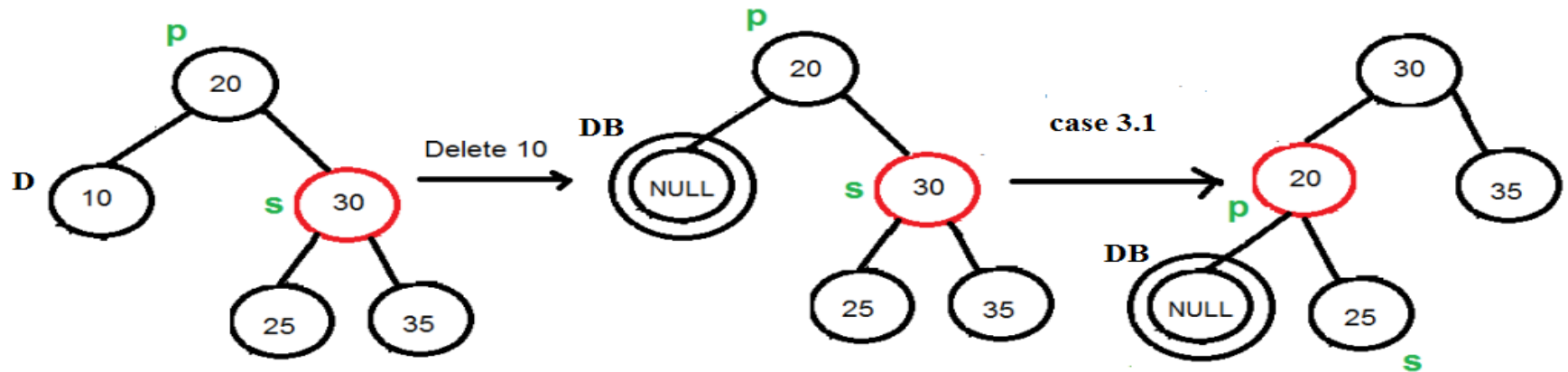
Case 2: If DB is a root node –then remove DB directly and make it single black.

Case 3:

3.1 if DB's sibling is red

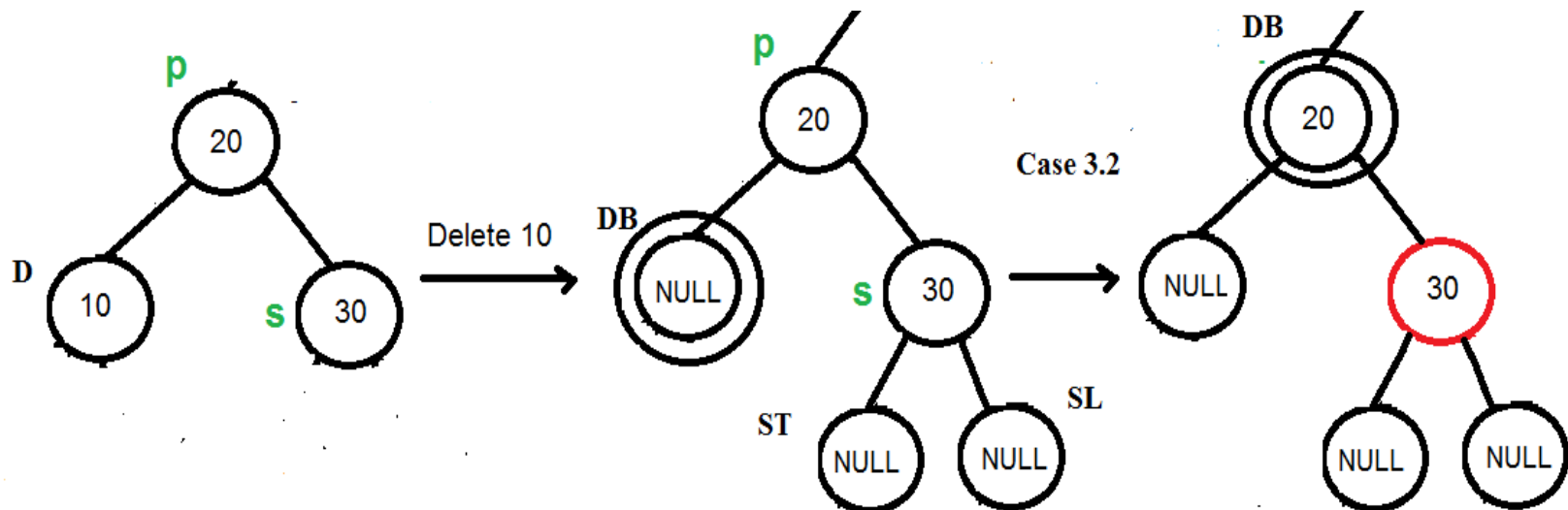
- i. Rotate P in DB direction
- ii. Swap colors of P and S
- iii. Reapply cases if needed.

Red Black Tree...



3.2 if DB's sibling is black and both children are black (SL and ST are black)

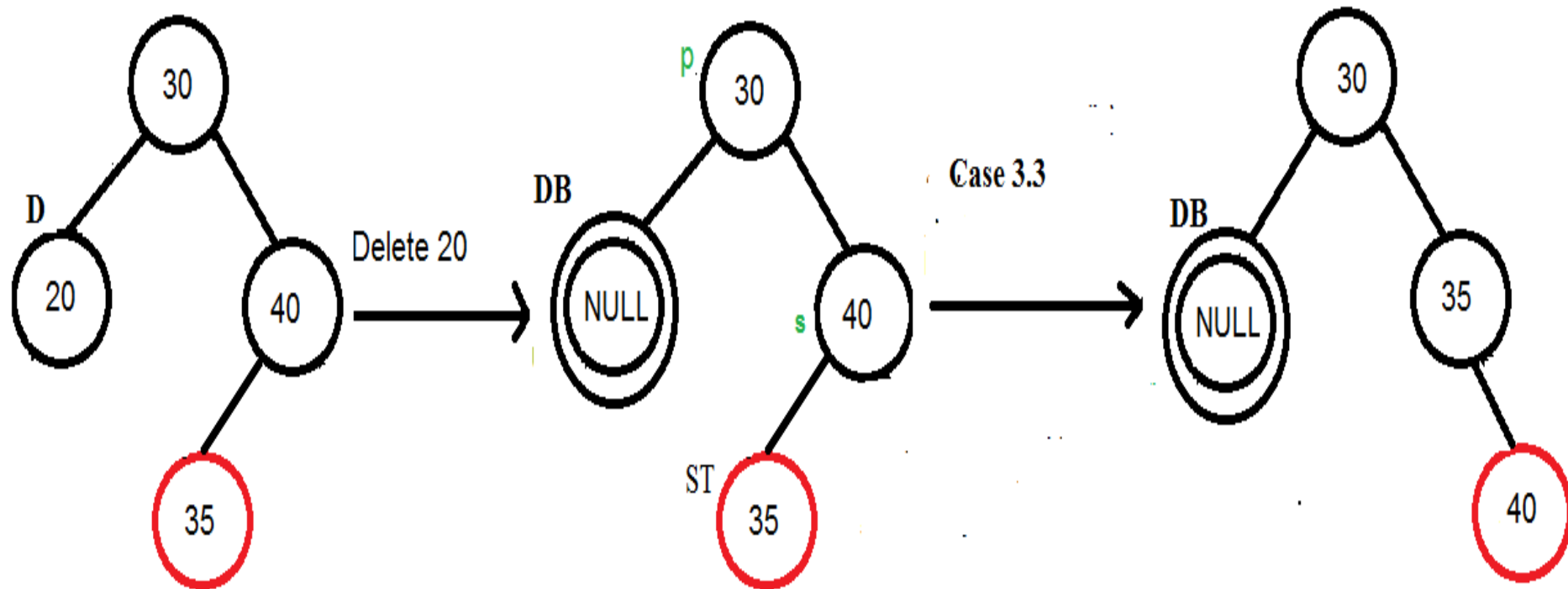
- Remove DB
- Add black to P: if P is black make DB, if P is red make single black.
- Make S as RED
- DB still exist, follow other cases.



Red Black Tree...

3.3 if DB's sibling is black ,inline child SL is black and ST is RED

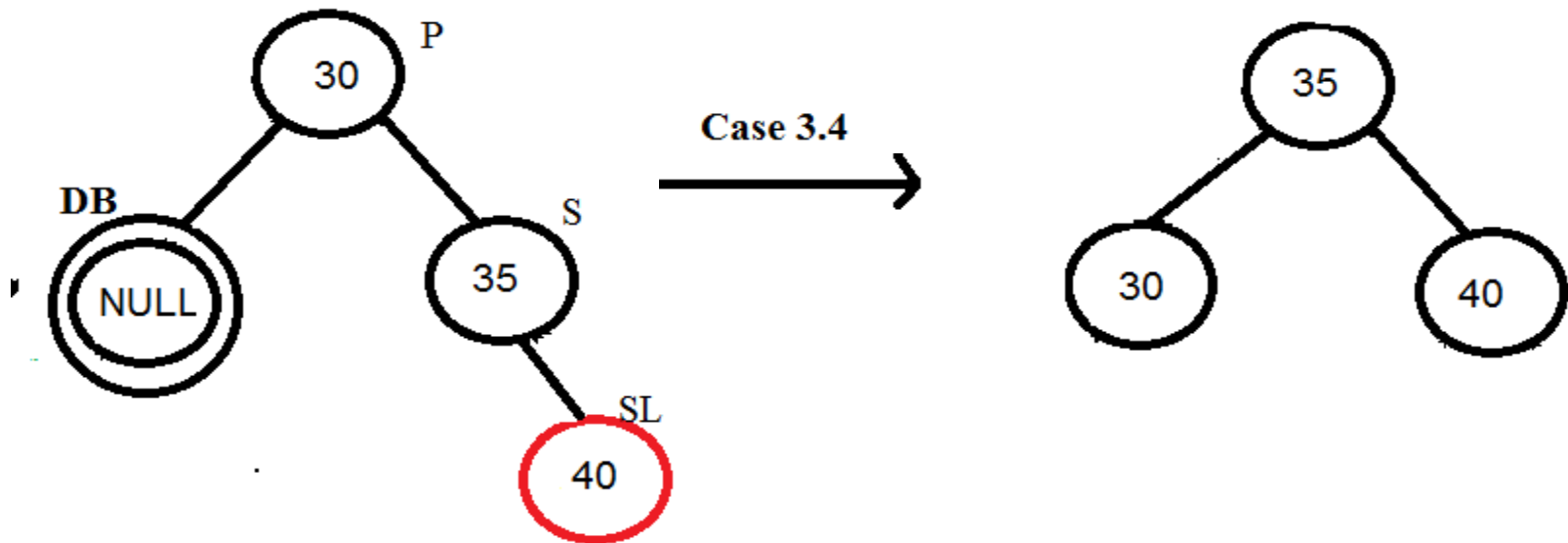
- Swap colors of S and ST
- Rotate S in opposite direction to DB
- Follow case 3.4



Red Black Tree...

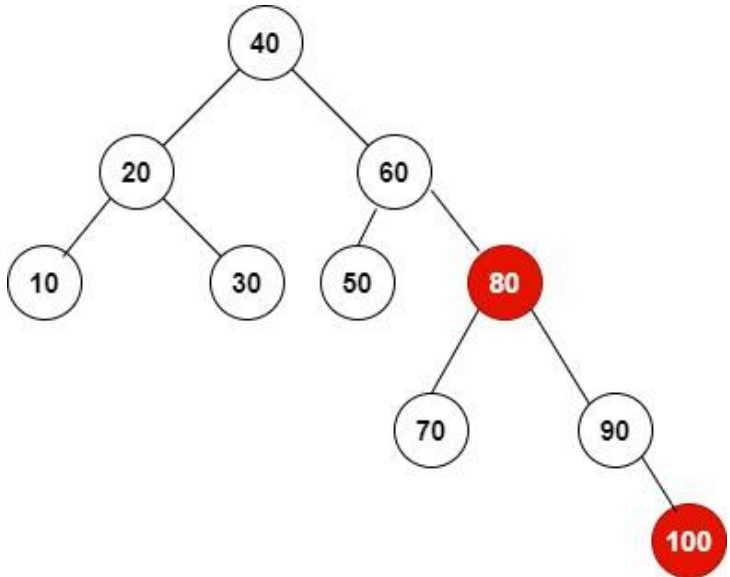
3.4 if DB's sibling is black ,inline child SL is RED

- i. Swap colors of P and S
- ii. Rotate P in same direction to DB
- iii. Remove BD
- iv. Change color of SL to Black

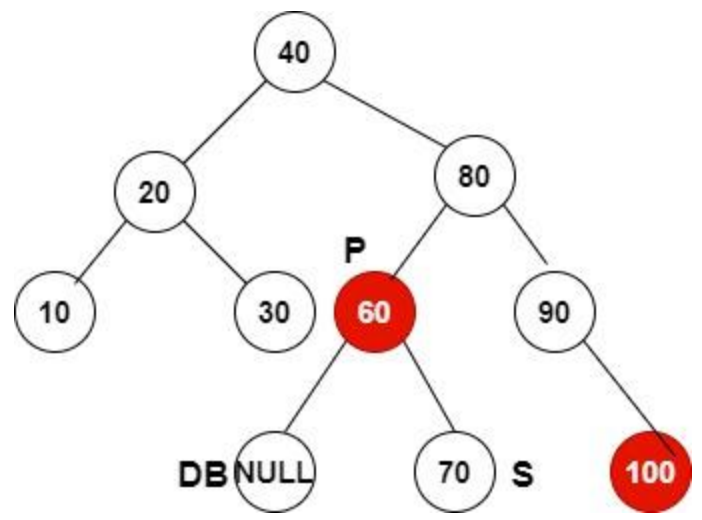
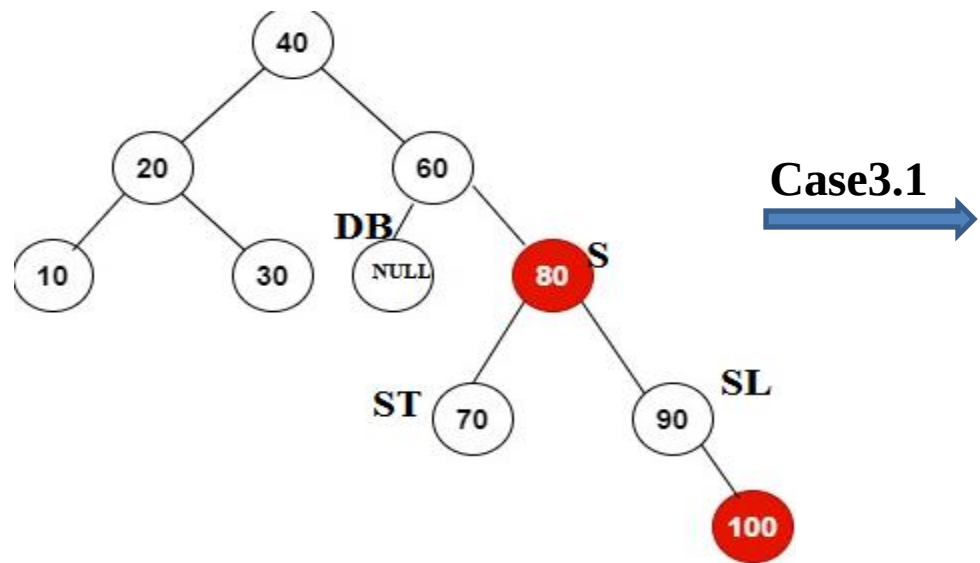


Red Black Tree...

Deletion Example: Delete 50,20,100,90,40,60,10,30,70,80 from the following RB Tree

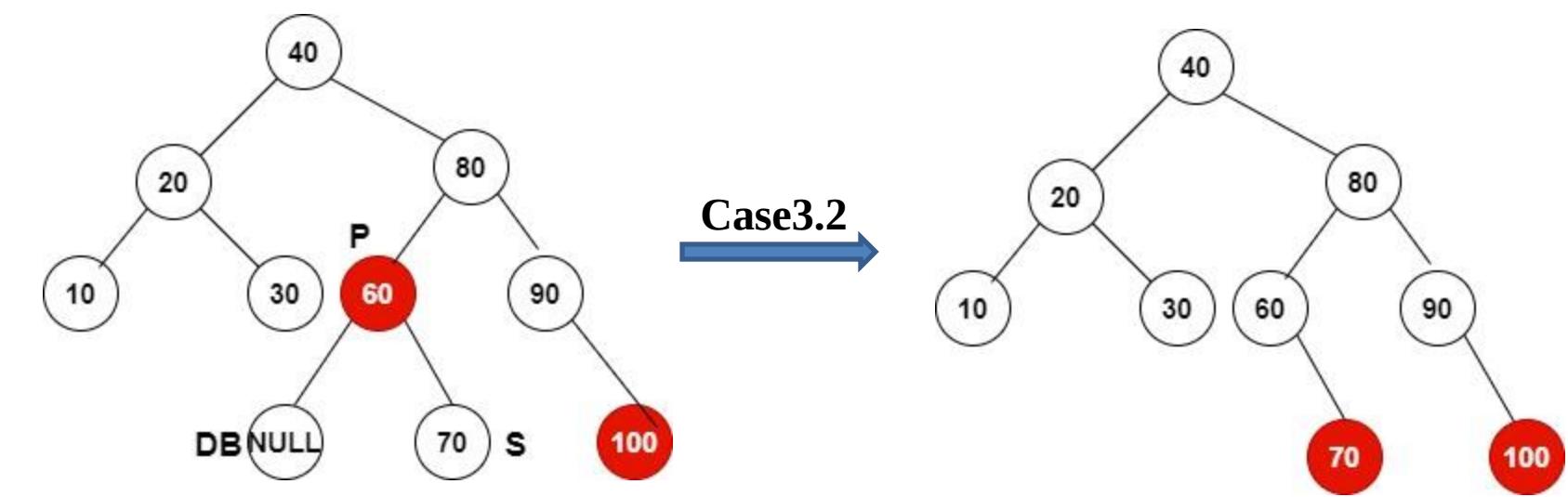


Delete 50

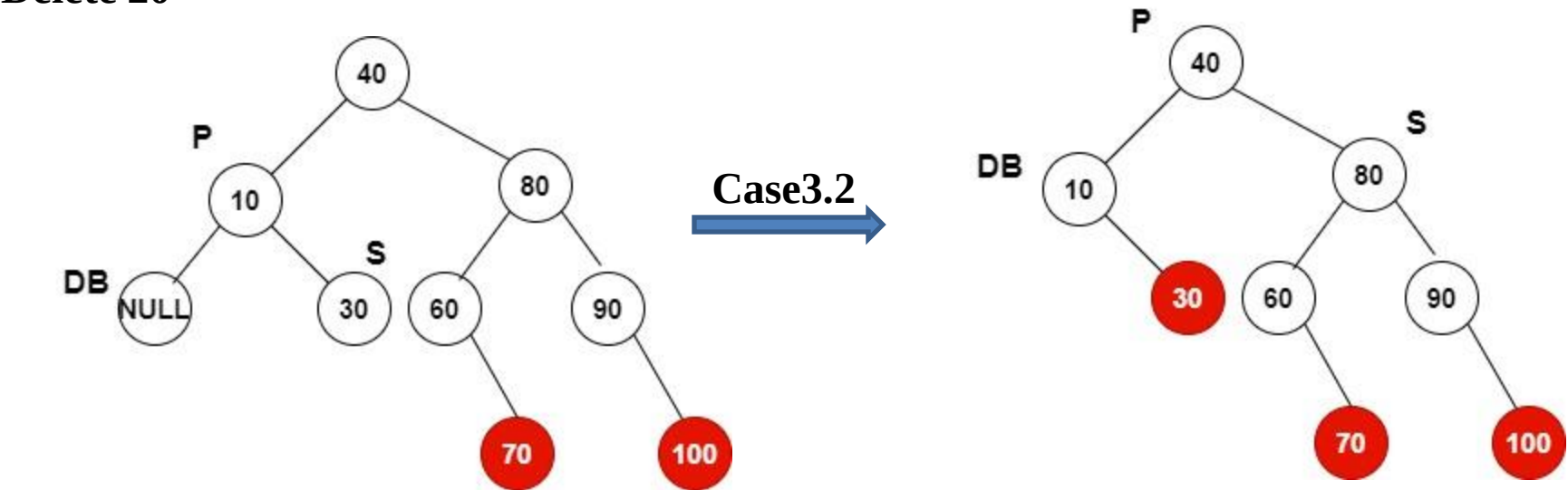


Red Black Tree...

Deletion Example: Delete 50,20,100,90 from the following RB Tree

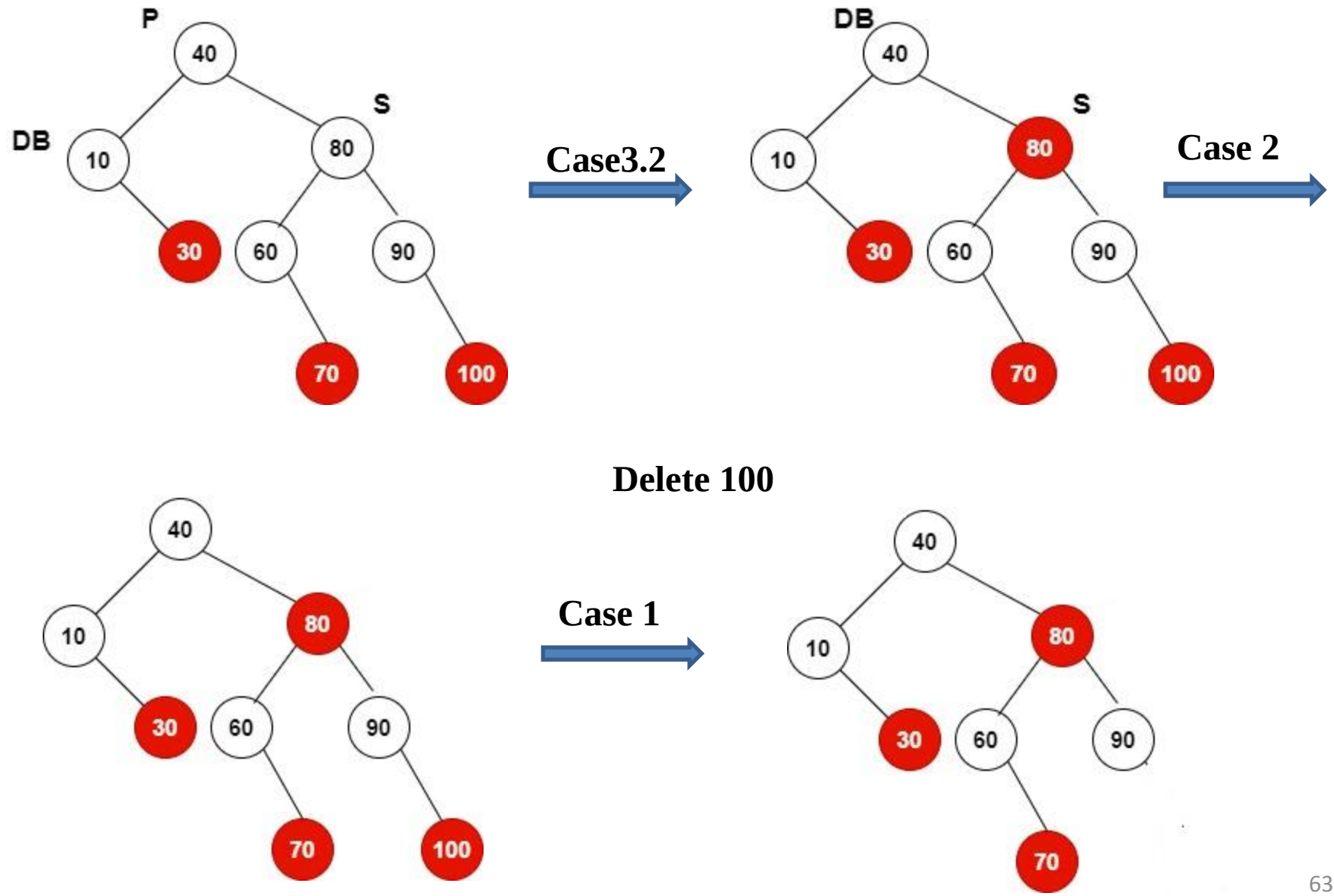


Delete 20



Red Black Tree...

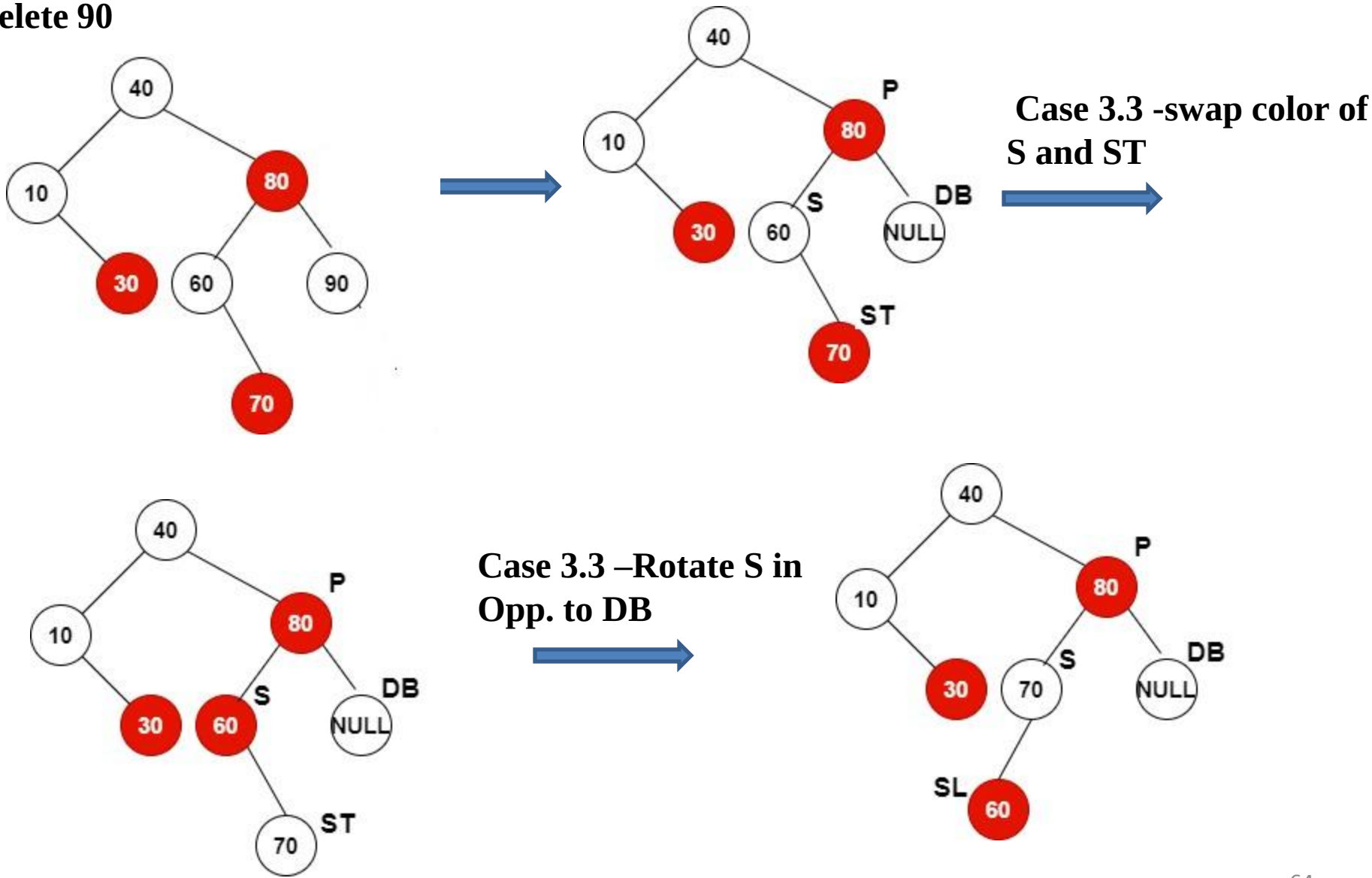
Deletion Example: Delete 50,20,100,90 from the following RB Tree



Red Black Tree...

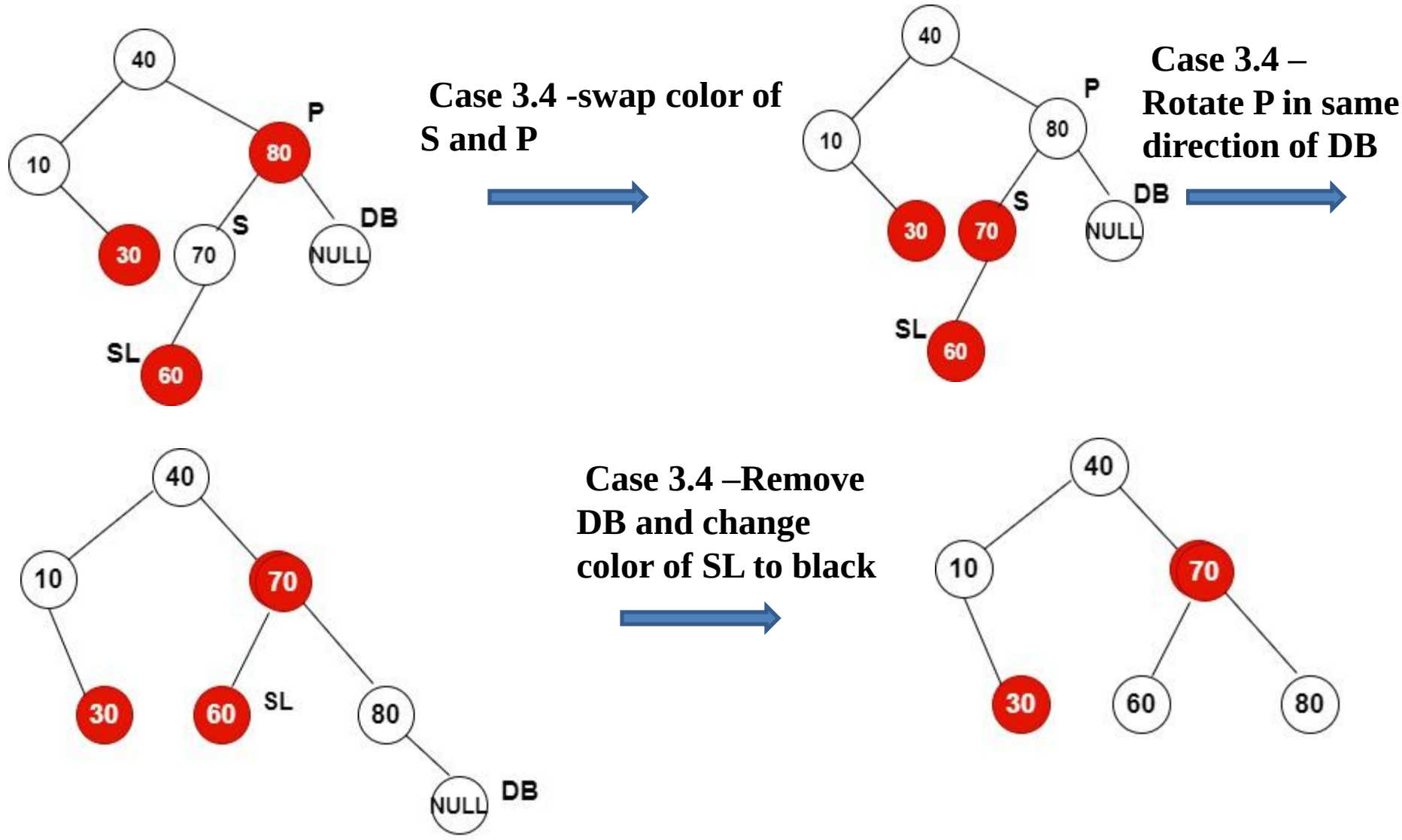
Deletion Example: Delete 50,20,100,90 from the following RB Tree

Delete 90



Red Black Tree...

Deletion Example: Delete 50,20,100,90 from the following RB Tree

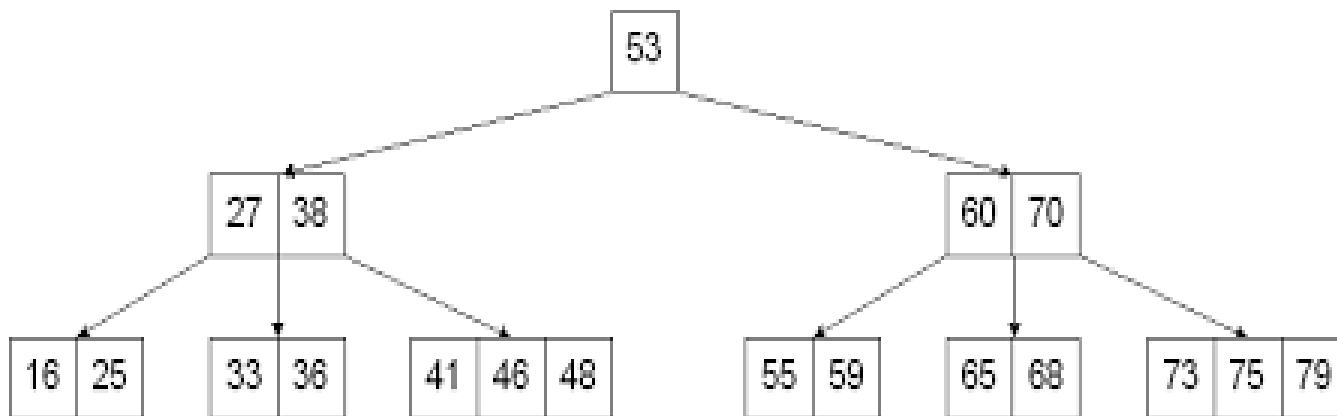


B Tree

- B-Tree is a M-way search tree that can be widely used for disk access.
- One of the main reason of using B tree is its capability to store large number of keys in a single node by keeping the height of the tree relatively small.

Properties of B-Tree:

1. B-Tree of order M will have atmost M children.
2. Every node in a B-tree except root and the leaf nodes has at least $\lceil M/2 \rceil$ children.
3. All leaves are at the same level.
4. All nodes may contain maximum $M - 1$ keys and minimum of $\lceil M/2 \rceil - 1$ keys.
5. All keys of a node are sorted in increasing order. The child between two keys k_1 and k_2 contains all keys in the range from k_1 and k_2 .
6. The root has at least two children if it is not a leaf node.



B Tree...

Operations :

➤ **Searching:** Searching in B-tree is similar to BST

➤ **Insertion:**

In a B-Tree, a new element must be added only at the leaf node. That means, the new key value is always attached to the leaf node only. The insertion operation is performed as follows...

Step 1 - Check whether tree is Empty.

Step 2 - If tree is **Empty**, then create a new node with new key value and insert it into the tree as a root node.

Step 3 - If tree is **Not Empty**, then find the suitable leaf node to which the new key value is added using Binary Search Tree logic.

Step 4 - If that leaf node has empty position, add the new key value to that leaf node in ascending order of key value within the node.

Step 5 - If that leaf node is already full, **split** that leaf node by sending middle value to its parent node. Repeat the same until the sending value is fixed into a node.

Step 6 - If the splitting is performed at root node then the middle value becomes new root node for the tree and the height of the tree is increased by one.

B Tree...

➤ Insert the following elements into a B-tree of order 3

1,2,3,4,5,6,7,8,9,10

Max no. of keys = $M-1 = 3-1=2$

Min no. of keys = $\lceil M/2 \rceil - 1 = 3/2 - 1 = 2 - 1 = 1$

Min no. of children = $\lceil M/2 \rceil = 3/2 = 2$

Insert 1



Insert 2



Insert 3



B Tree...

➤ Insert the following elements into a B-tree of order 3

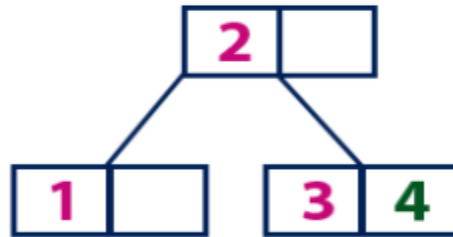
1,2,3,4,5,6,7,8,9,10

Max no. of keys = $M-1 = 3-1=2$

Min no. of keys = $\lceil M/2 \rceil - 1 = 3/2 - 1 = 2 - 1 = 1$

Min no. of children = $\lceil M/2 \rceil = 3/2 = 2$

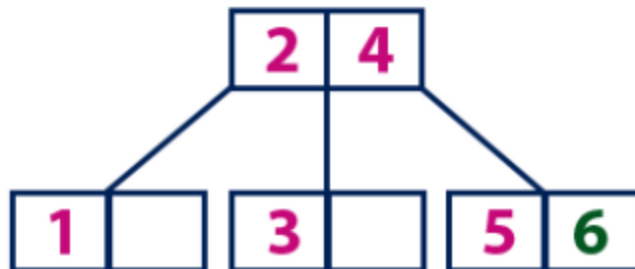
Insert 4



Insert 5



Insert 6



B- Tree...

➤ Insert the following elements into a B-tree of order 3

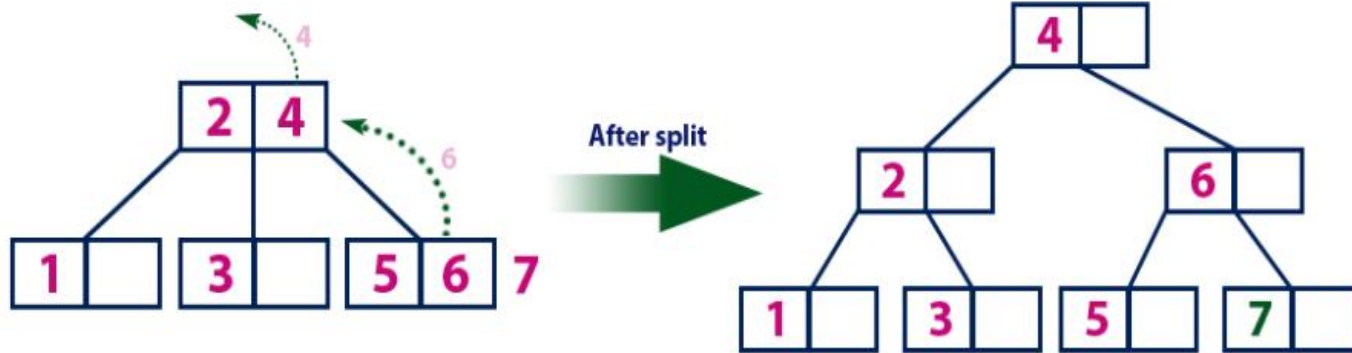
1,2,3,4,5,6,7,8,9,10

Max no. of keys = $M-1 = 3-1=2$

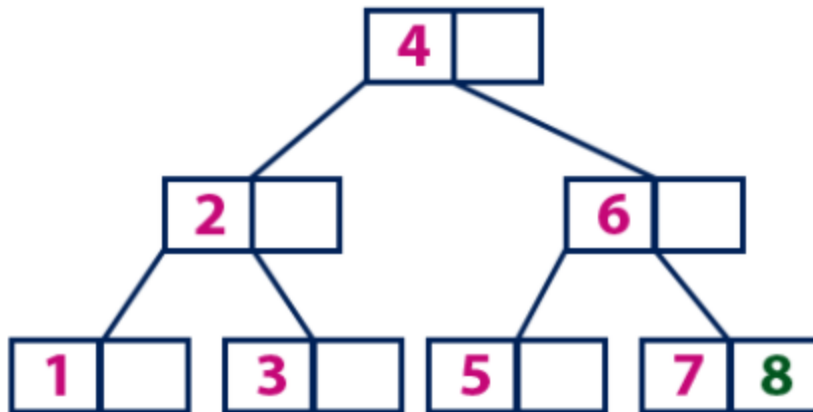
Min no. of keys = $\lceil M/2 \rceil - 1 = 3/2 - 1 = 2 - 1 = 1$

Min no. of children = $\lceil M/2 \rceil = 3/2 = 2$

Insert 7



Insert 8



B Tree...

➤ Insert the following elements into a B-tree of order 3

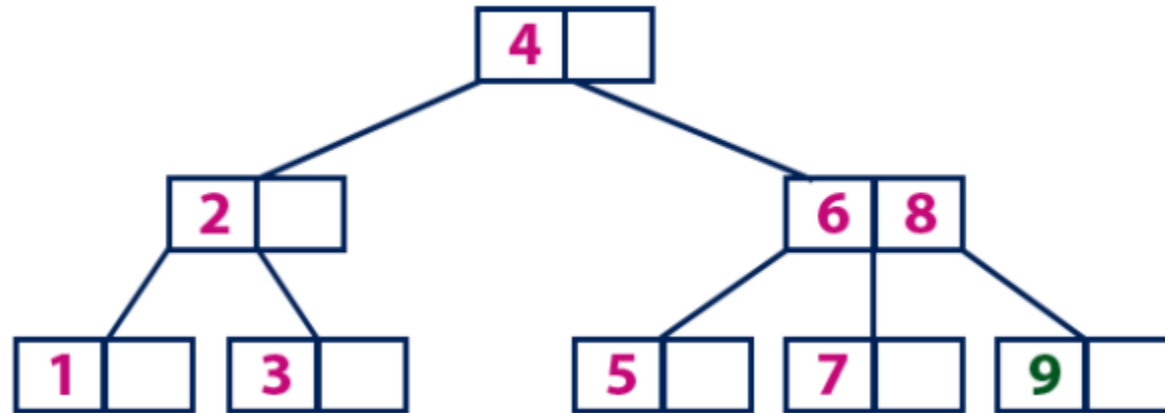
1,2,3,4,5,6,7,8,9,10

Max no. of keys = $M-1 = 3-1=2$

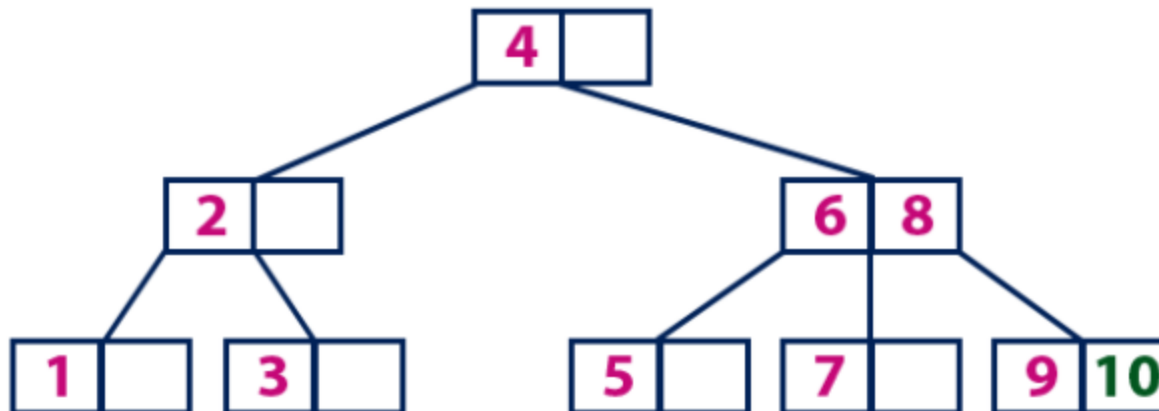
Min no. of keys = $\lceil M/2 \rceil - 1 = 3/2 - 1 = 2 - 1 = 1$

Min no. of children = $\lceil M/2 \rceil = 3/2 = 2$

Insert 9



Insert 10



B Tree...

➤ Insert the following elements into a B-tree of order 3

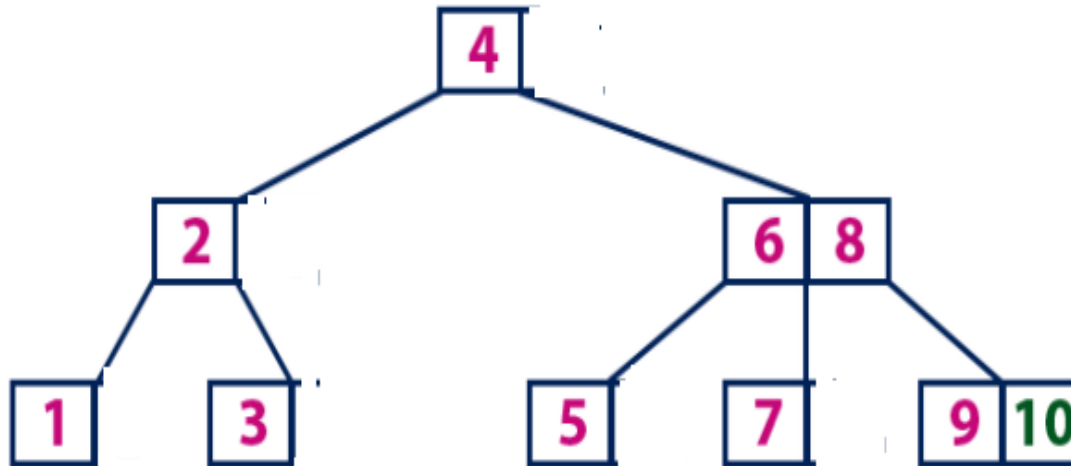
1,2,3,4,5,6,7,8,9,10

Max no. of keys = $M-1 = 3-1=2$

Min no. of keys = $\lceil M/2 \rceil - 1 = 3/2 - 1 = 2 - 1 = 1$

Min no. of children = $\lceil M/2 \rceil = 3/2 = 2$

Finally, B-tree is



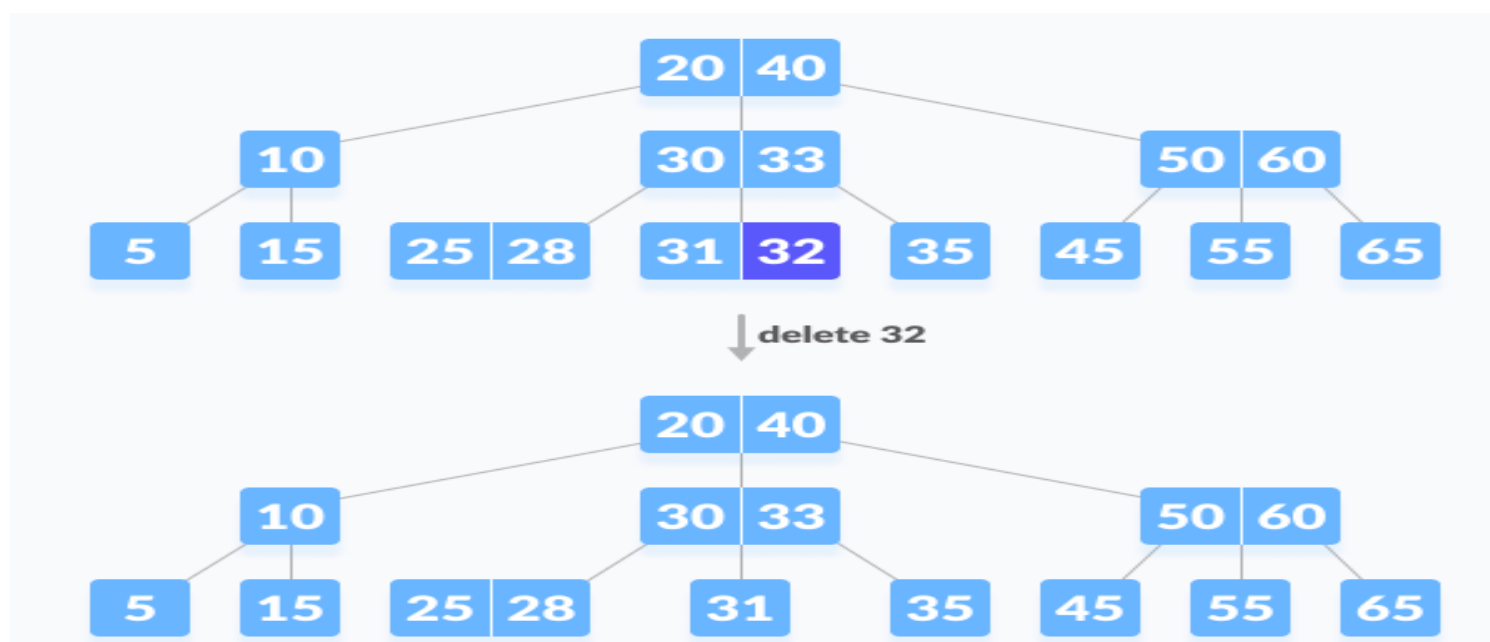
B Tree...

➤ Deletion:

- Deleting an element on a B-tree consists of three main events: **searching** the node where the key to be deleted exists, **deleting** the key and **balancing** the tree if required.
- While deleting a tree, a condition called **underflow** may occur. Underflow occurs when a node contains less than the minimum number of keys it should hold.
- There are three main cases for deletion operation in a B tree.

Case I: The key to be deleted lies in the leaf. There are two cases for it.

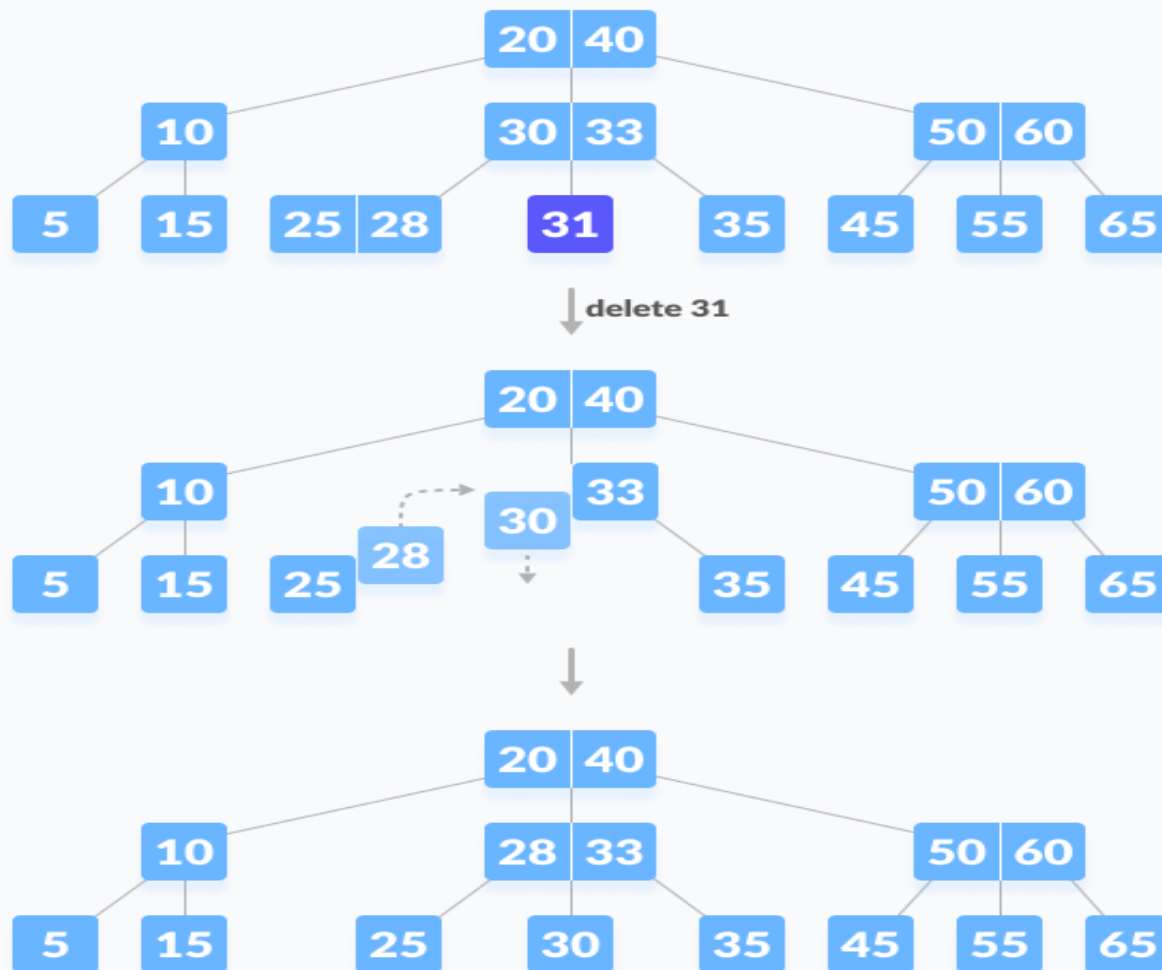
i) The deletion of the key does not violate the property of the minimum number of keys.



B Tree...

ii) The deletion of the key violates the property of the minimum number of keys.

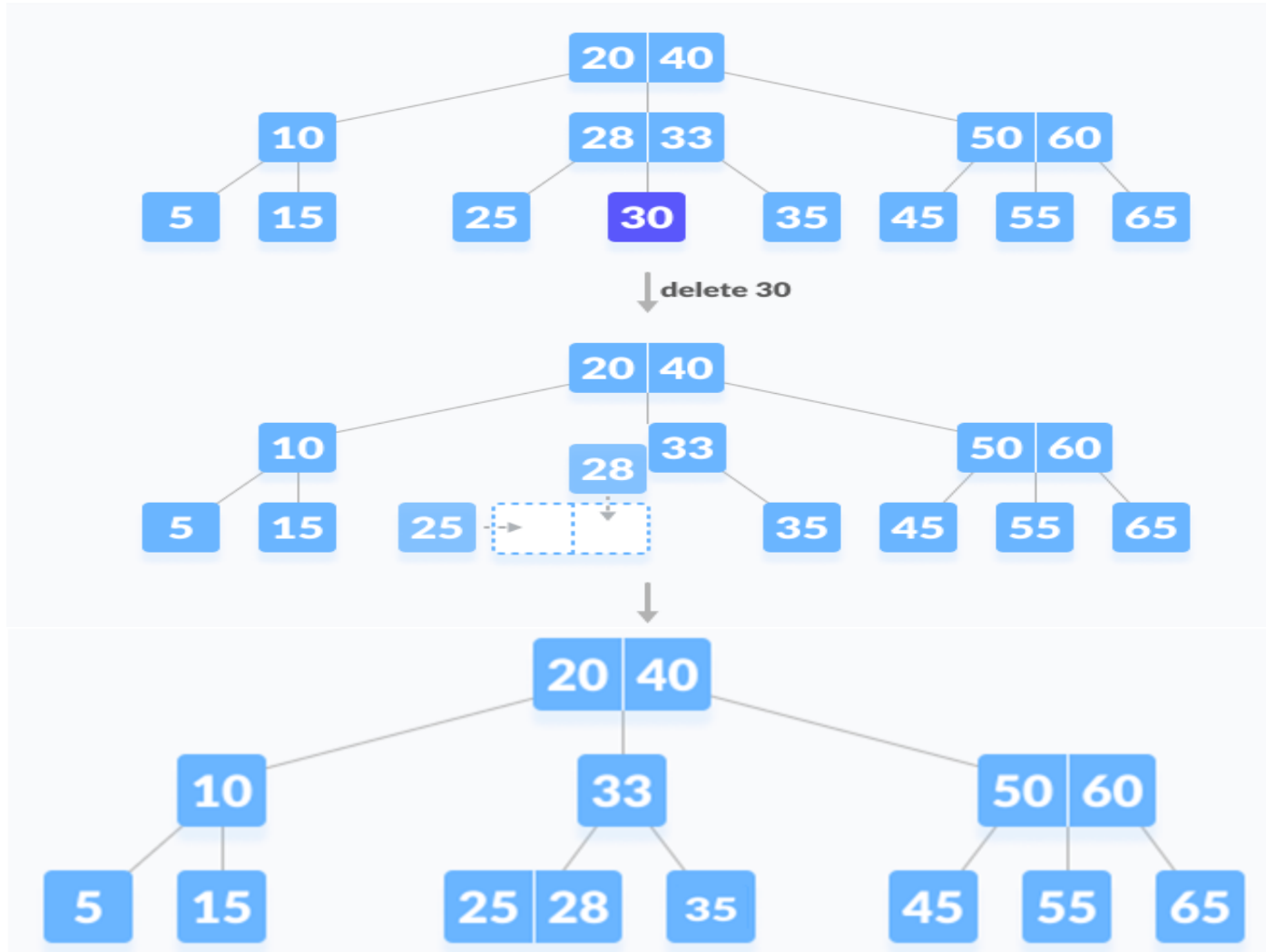
In this case, we borrow a key from its immediate neighbouring sibling node in the order of left to right. First, visit the immediate left sibling. If the left sibling node has more than a minimum number of keys, then borrow a key from this node. Else, check to borrow from the immediate right sibling node



B Tree...

iii) If both the immediate sibling nodes already have a minimum number of keys:

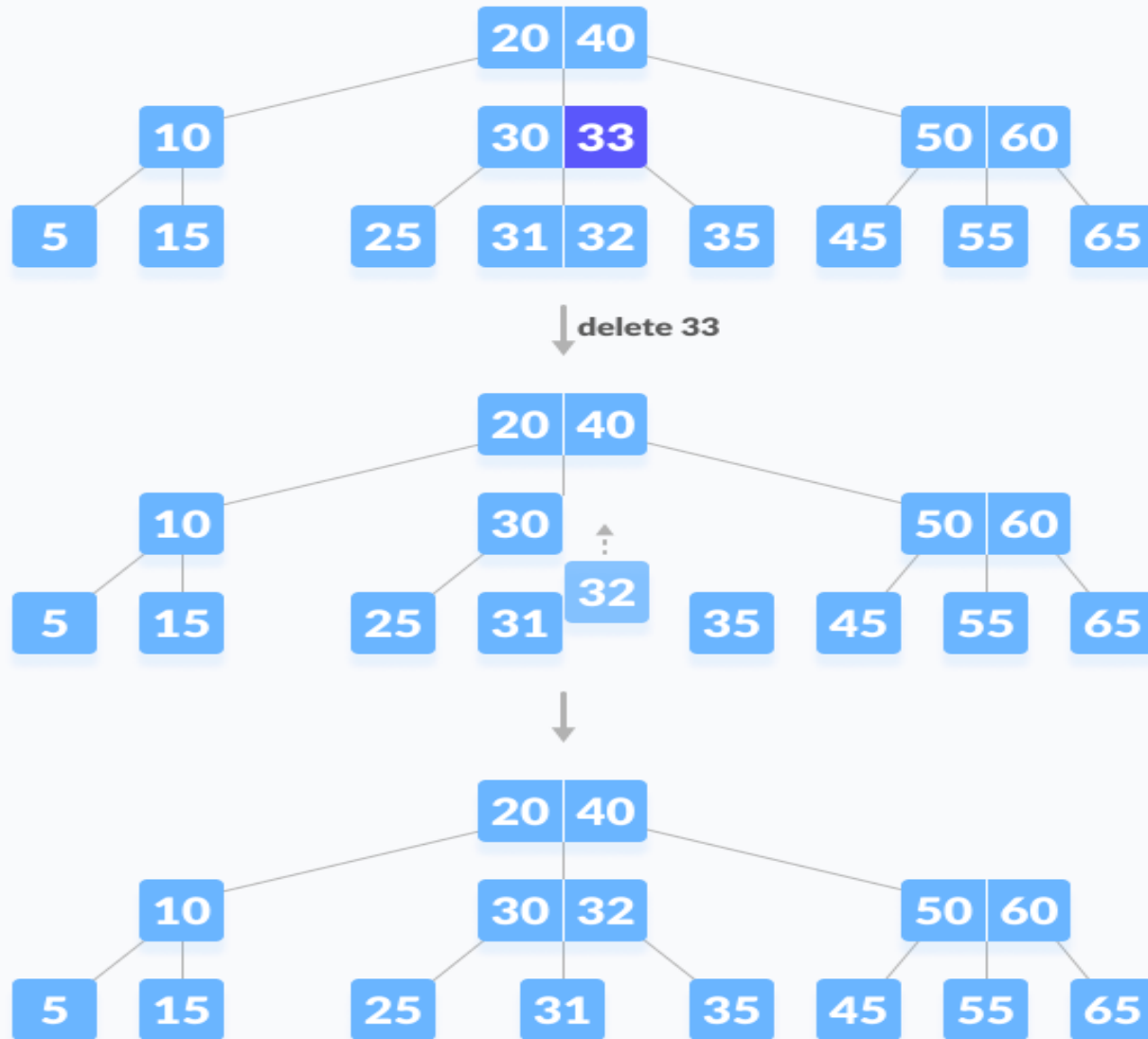
then merge the node with either the left sibling node or the right sibling node. This merging is done through the parent node.



B Tree...

Case II :If the key to be deleted lies in the internal node, the following cases occur.

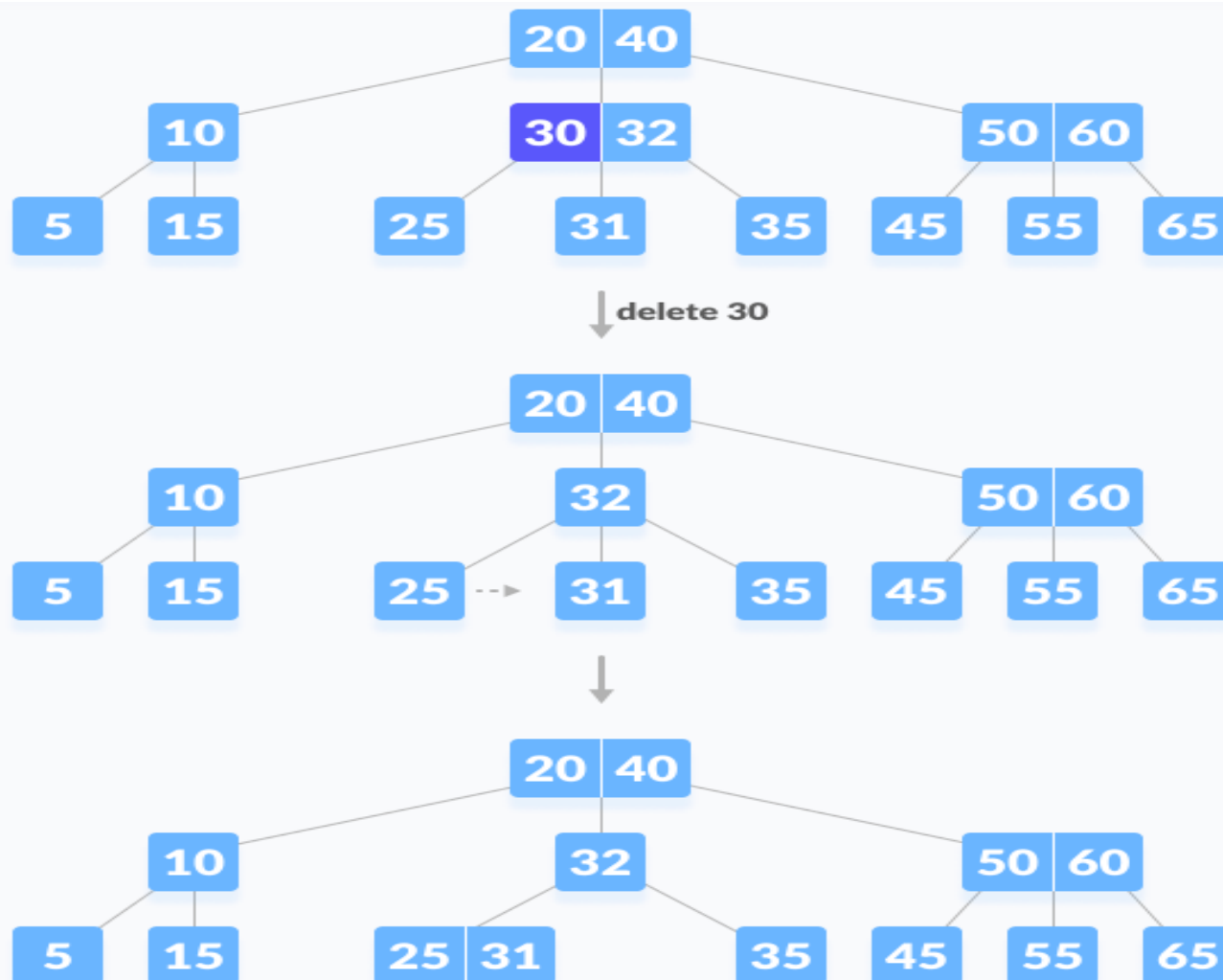
- i) The internal node, which is deleted, is replaced by an Inorder predecessor if the left child has more than the minimum number of keys.



B Tree...

Case II :If the key to be deleted lies in the internal node, the following cases occur.

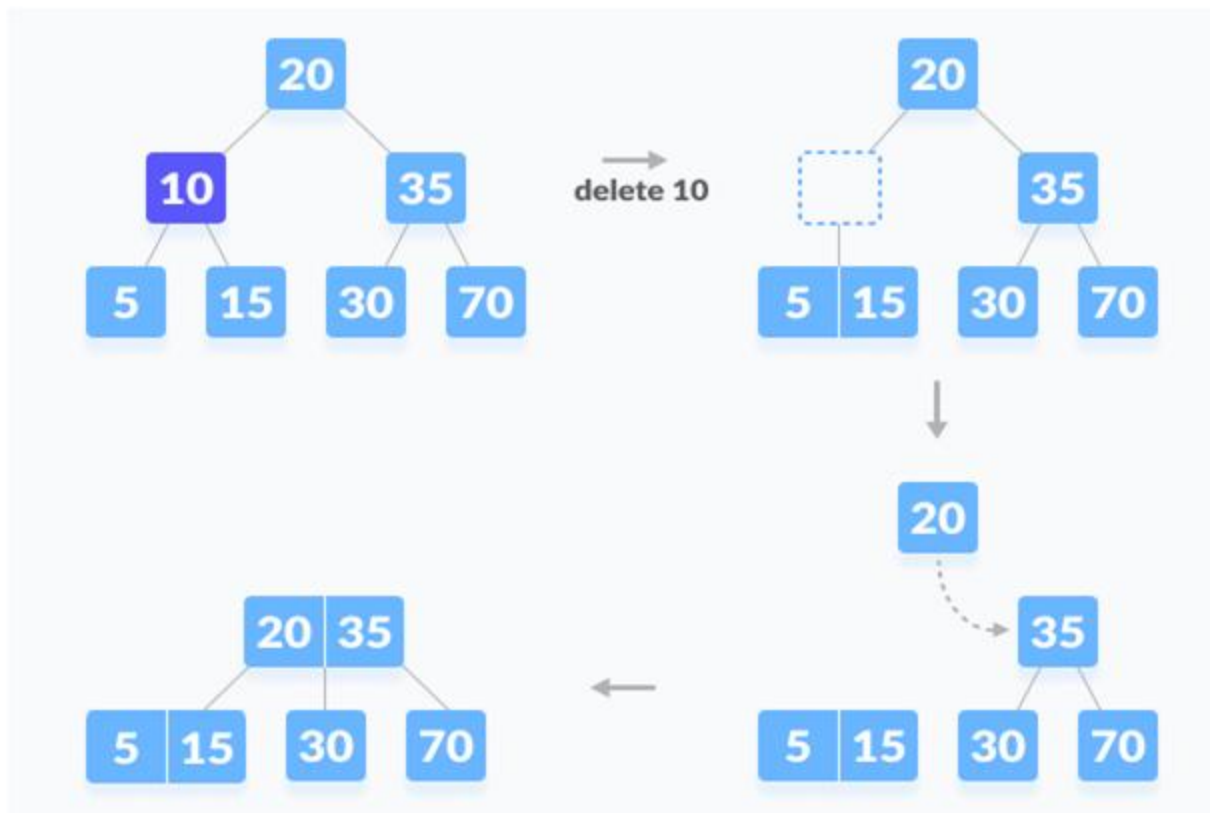
- ii) If either child has exactly a minimum number of keys then, merge the left and the right children. After merging if the parent node has less than the minimum number of keys then, look for the siblings as in Case I.



B Tree...

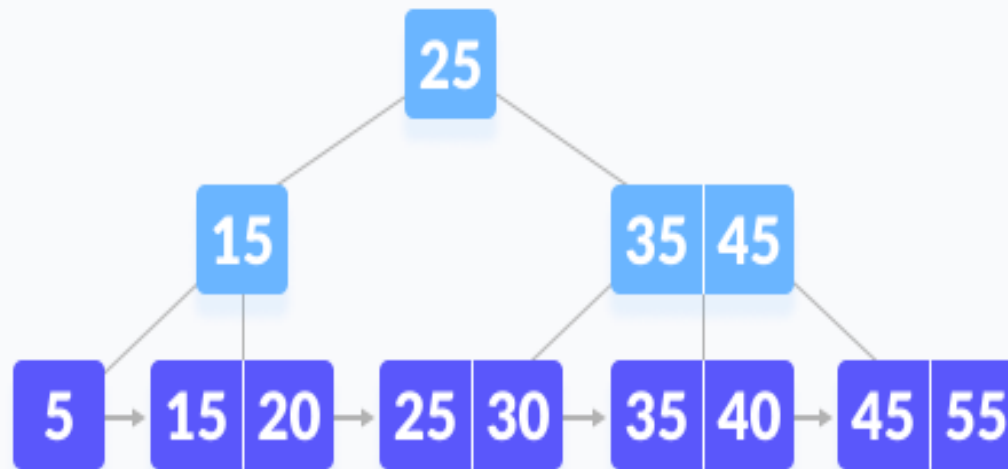
Case III : In this case, the height of the tree shrinks. If the target key lies in an internal node, and the deletion of the key leads to a fewer number of keys in the node (i.e. less than the minimum required), then look for the inorder predecessor and the inorder successor. If both the children contain a minimum number of keys then, borrowing cannot take place. This leads to Case II(ii) i.e. merging the children.

Again, look for the sibling to borrow a key. But, if the sibling also has only a minimum number of keys then, merge the node with the sibling along with the parent. Arrange the children accordingly (increasing order).



B+ Tree...

- B+ Tree is an extension of B Tree which allows efficient insertion, deletion and search operations.
- In B Tree, Keys and records both can be stored in the internal as well as leaf nodes. Whereas, in B+ tree, records (data) can only be stored on the leaf nodes while internal nodes can only store the key values. The internal nodes of B+ tree are often called index nodes.
- The leaf nodes of a B+ tree are linked together in the form of a singly linked lists to make the search queries more efficient.



B+ Tree...

Operations:

➤ Insertion:

Case I

If the leaf is not full, insert the key into the leaf node in increasing order.

Case II

1. If the leaf is full, insert the key into the leaf node in increasing order and balance the tree in the following way.
2. Break the node at $m/2$ th position.
3. Add $m/2$ th key to the parent node as well.
4. If the parent node is already full, follow steps 2 to 3.

Example:

Construct B+ Tree of order 3 for the elements 5, 15, 25, 35, 45.

Max no. of keys $= M - 1 = 3 - 1 = 2$

Min no. of keys $= \lceil M/2 \rceil - 1 = \lceil 3/2 \rceil - 1 = 2 - 1 = 1$

Min no. of children $= \lceil M/2 \rceil = \lceil 3/2 \rceil = 2$

B+ Tree...

Construct B+ Tree of order 3 for the elements 5, 15, 25, 35, 45.

Max no. of keys = $M-1 = 3-1=2$

Min no. of keys = $\lceil M/2 \rceil - 1 = 3/2 - 1 = 2 - 1 = 1$

Min no. of children = $\lceil M/2 \rceil = 3/2 = 2$

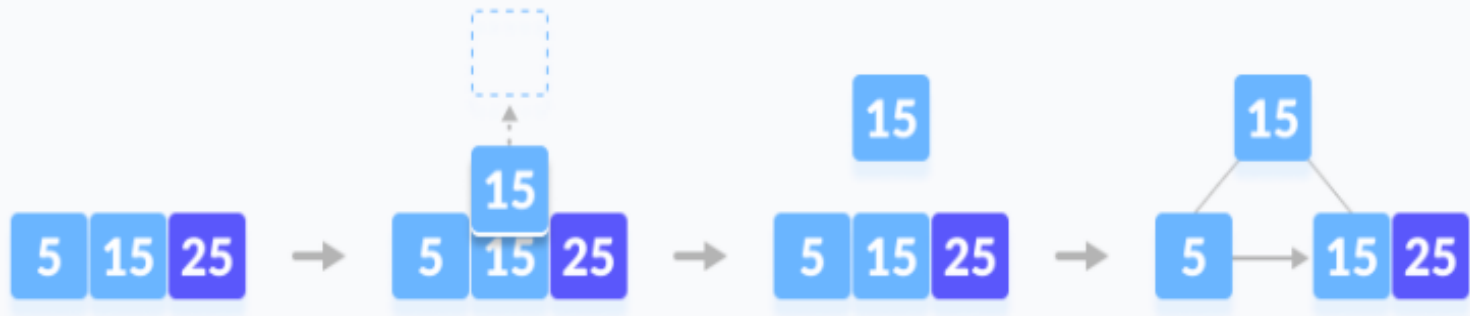
Insert 5



Insert 15



Insert 25



B+ Tree...

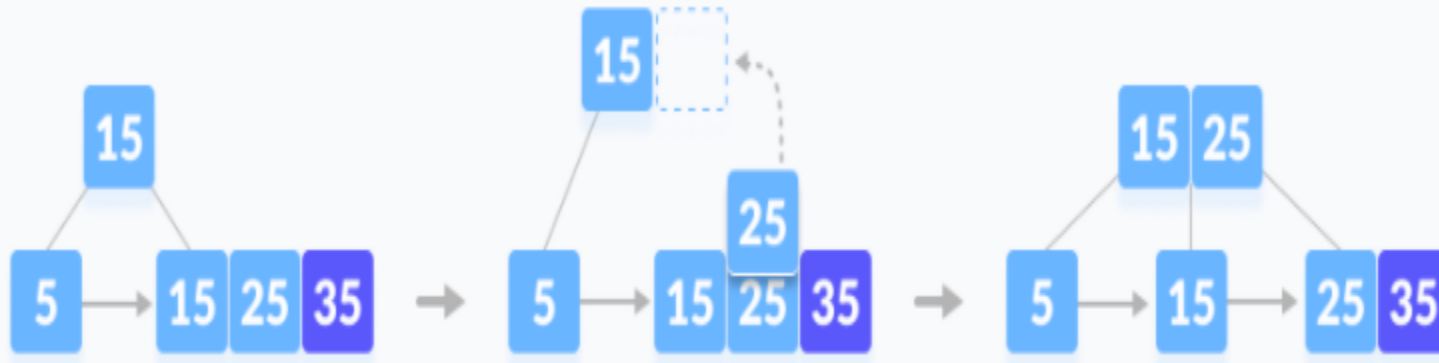
Construct B+ Tree of order 3 for the elements 5, 15, 25, 35, 45.

Max no. of keys = $M-1 = 3-1=2$

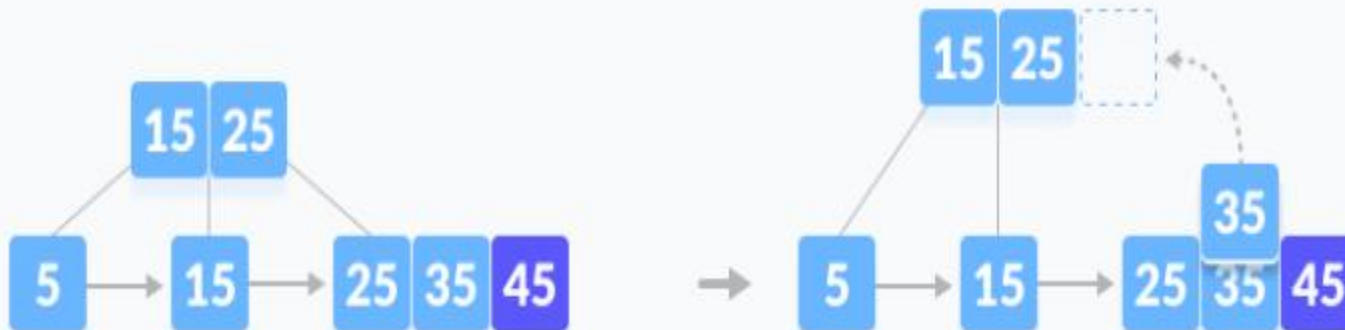
Min no. of keys = $\lceil M/2 \rceil - 1 = \lceil 3/2 \rceil - 1 = 2 - 1 = 1$

Min no. of children = $\lceil M/2 \rceil = \lceil 3/2 \rceil = 2$

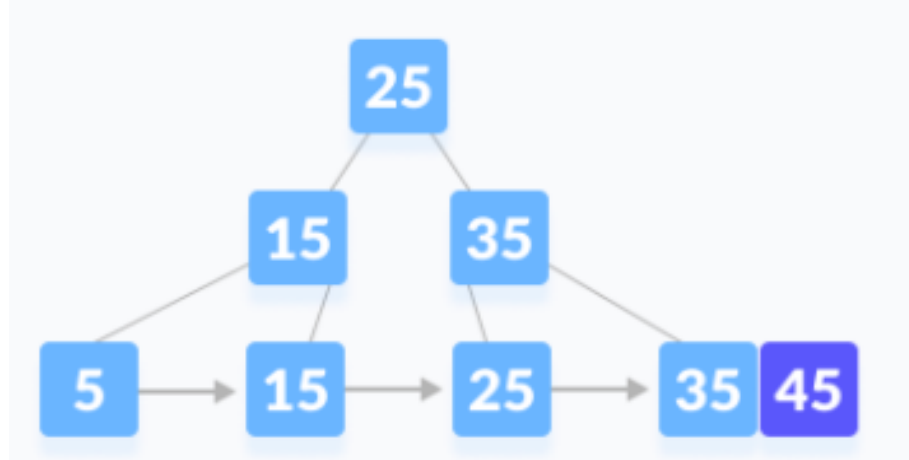
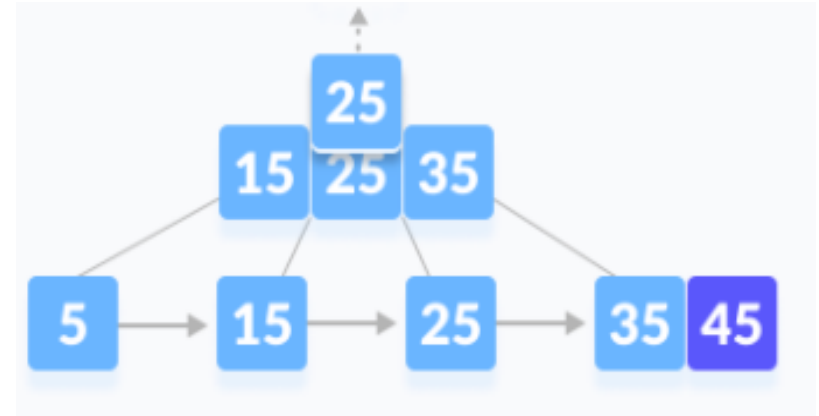
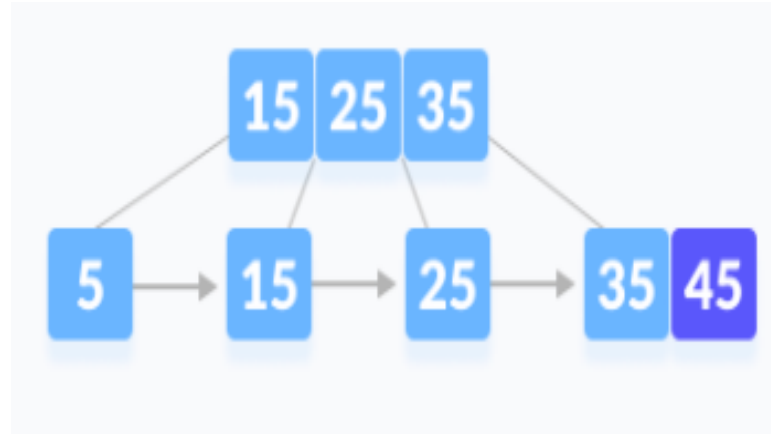
Insert 35



Insert 45



B+ Tree...



B+ Tree...

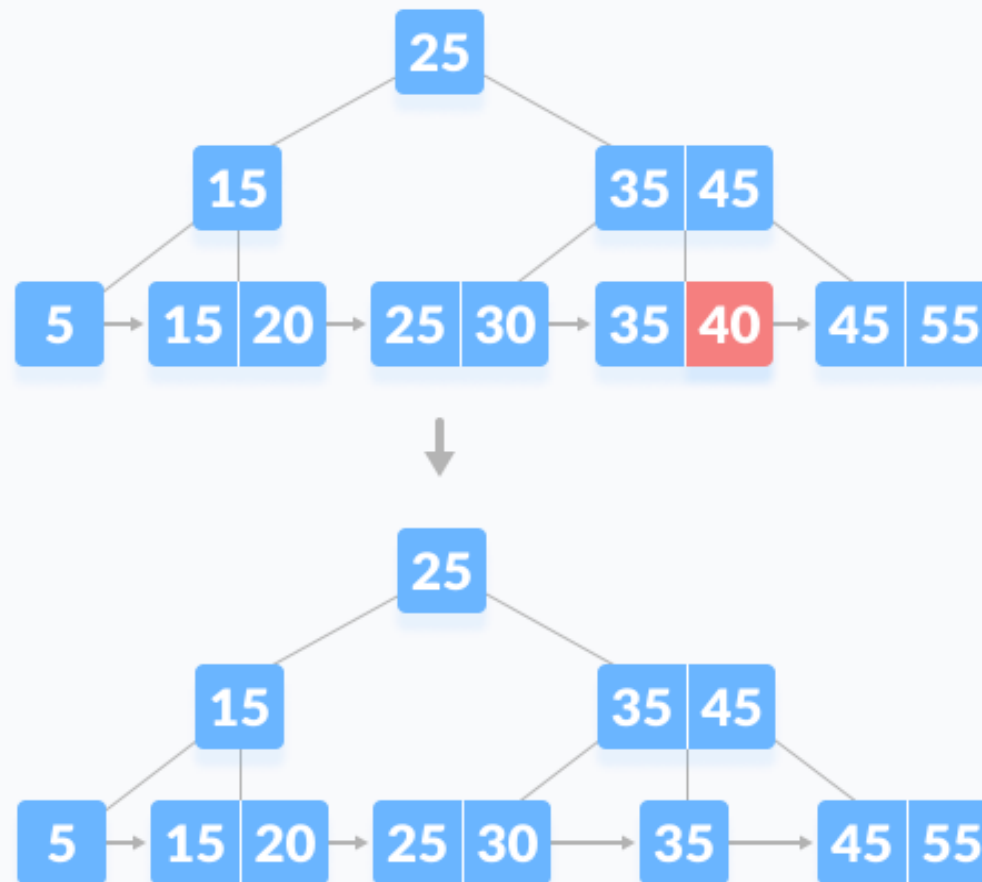
➤ Deletion: (B+ Tree of order 3)

Case I: The key to be deleted is present only at the leaf node not in the indexes (or internal nodes).

There are two cases for it:

i) There is more than the minimum number of keys in the node. Simply delete the key.

Delete 40:



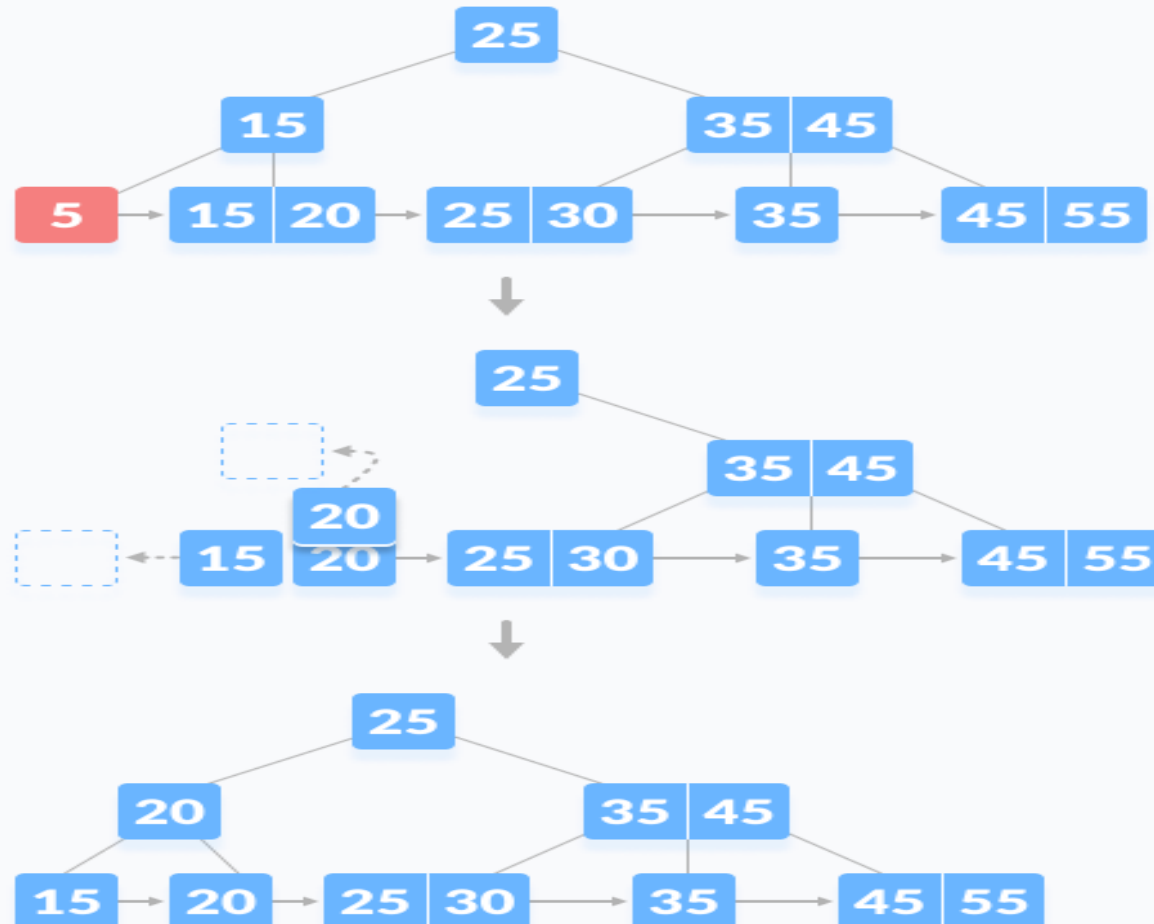
B+ Tree...

➤ Deletion: (B+ Tree of order 3)

Case I: The key to be deleted is present only at the leaf node not in the indexes (or internal nodes).

ii) There is an exact minimum number of keys in the node. Delete the key and borrow a key from the immediate sibling. Minimum from right child through parent.

Delete 5:



B+ Tree...

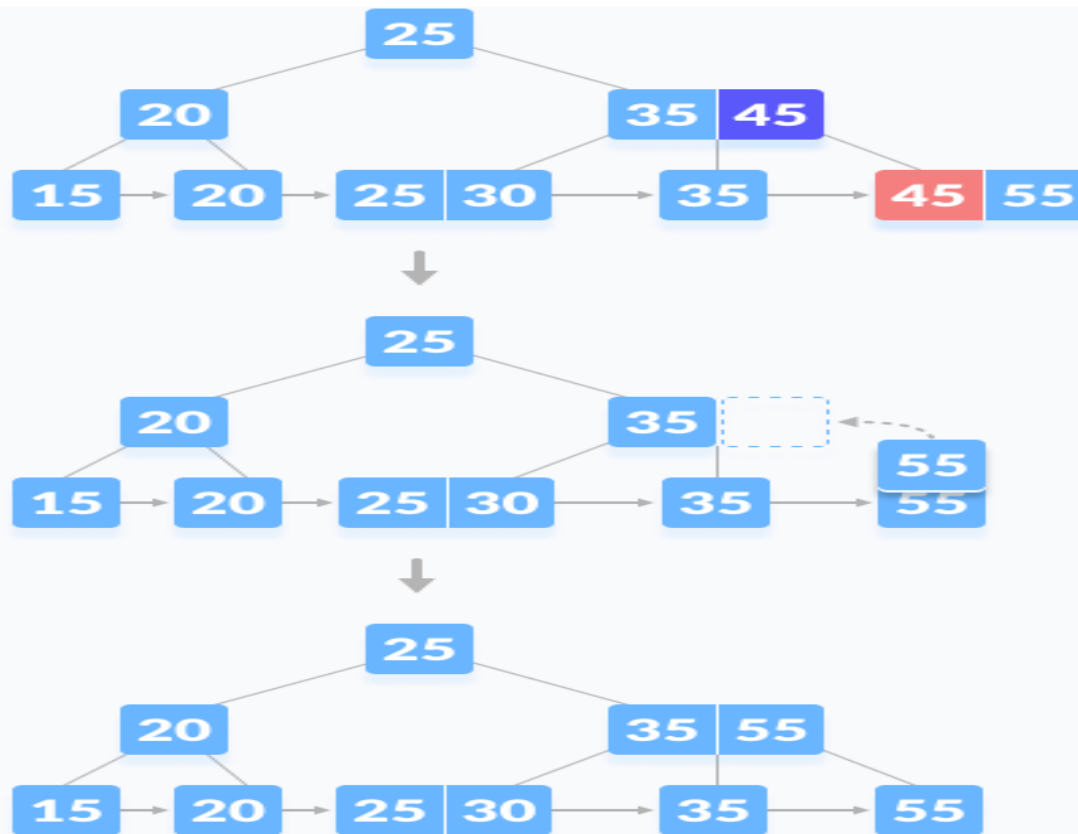
➤ Deletion: (B+ Tree of order 3)

Case 2: The key to be deleted is present in the internal nodes and leaf node. Then we have to remove them from the internal nodes and leaf node.

There are the following cases for this situation:

- If there is more than the minimum number of keys in the node, simply delete the key from the leaf node and delete the key from the internal node as well. Fill the empty space in the internal node with the Inorder successor.

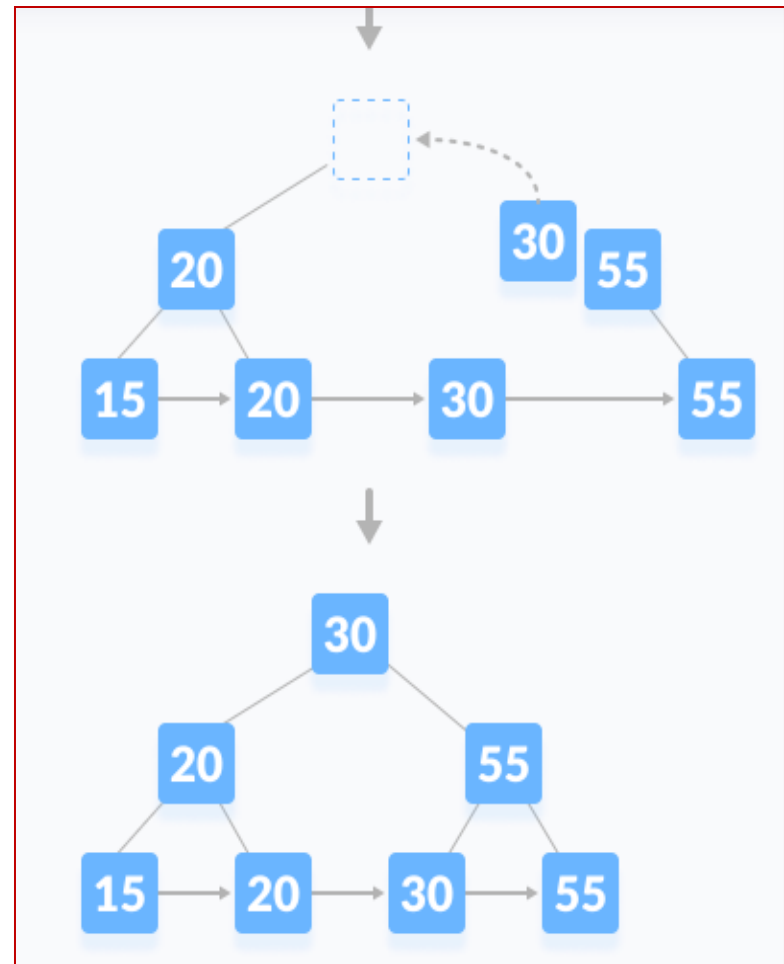
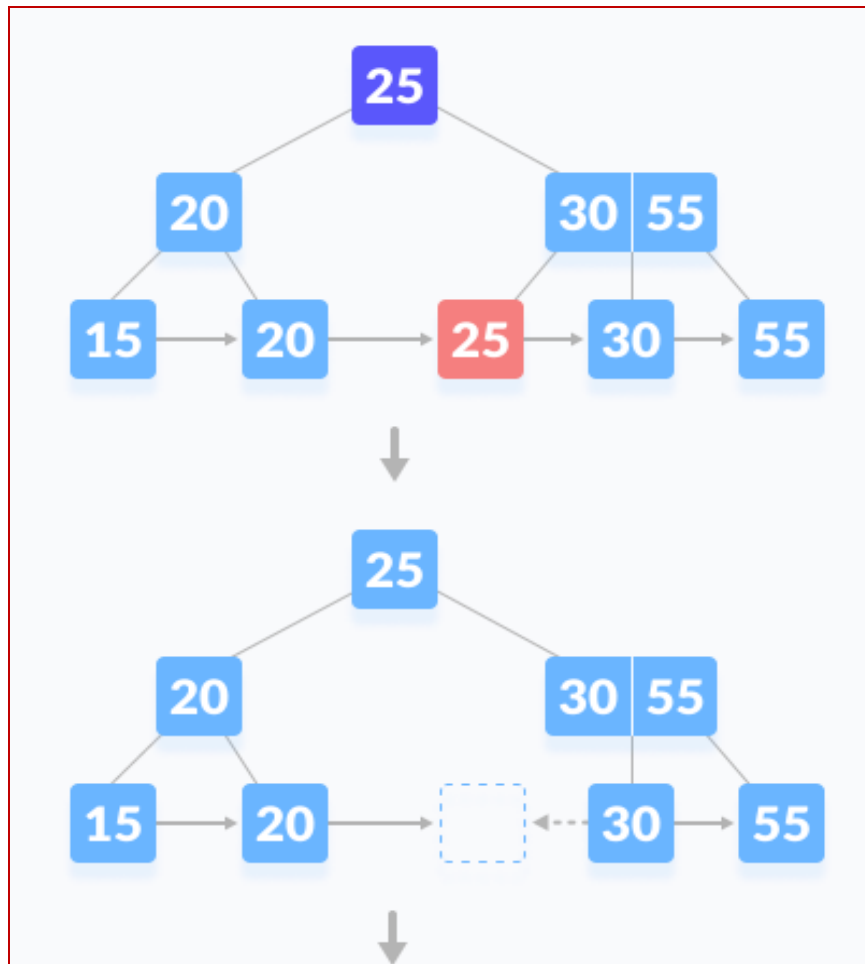
Delete 45:



B+ Tree...

iii) This case is similar to Case 2(i) but here, empty space is generated above the immediate parent node. After deleting the key, merge the empty space with its sibling. Fill the empty space in the grandparent node with the inorder successor.

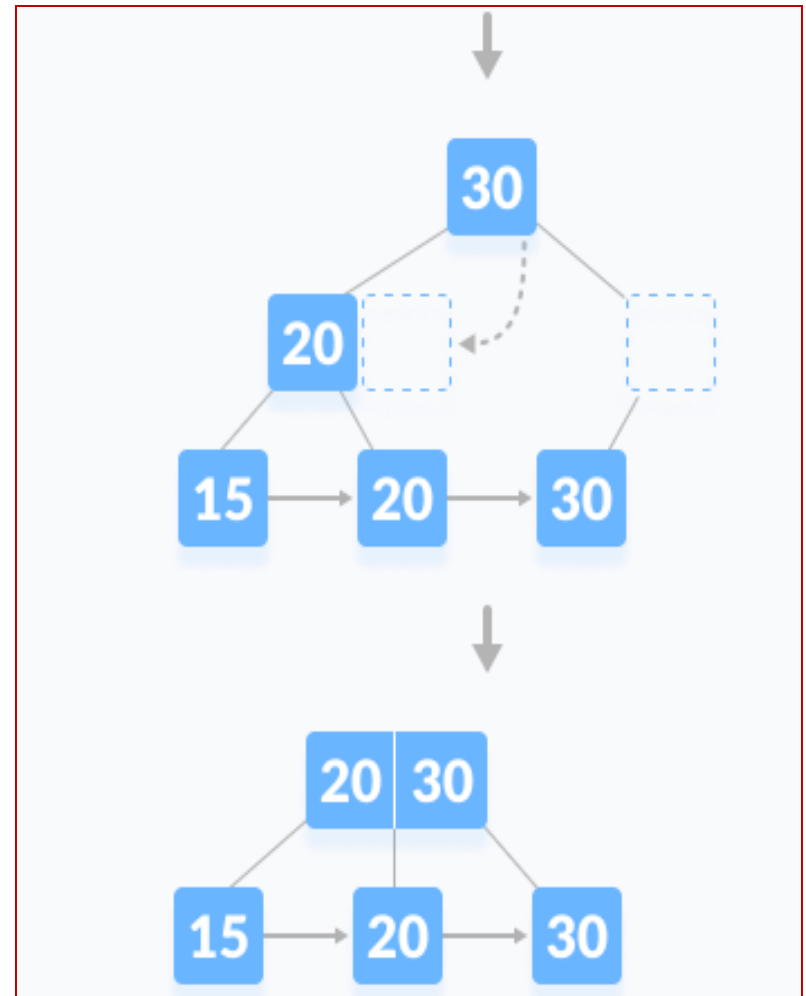
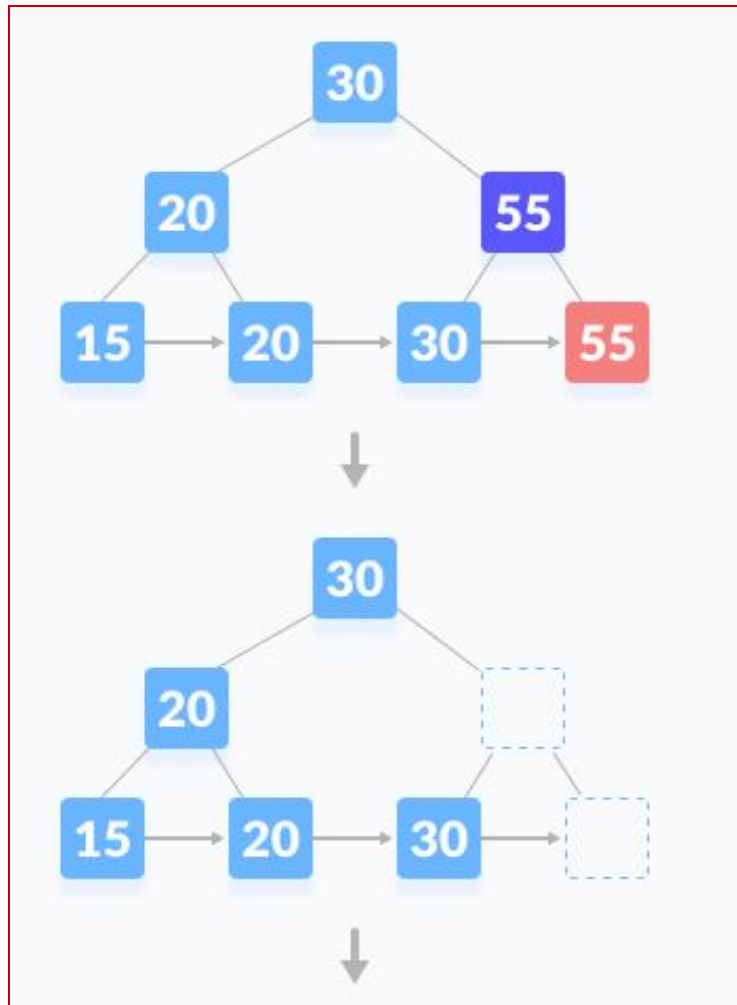
Delete 25:



B+ Tree...

Case 3: In this case, the height of the tree gets shrunk. It is a little complicated. Deleting 55 from the tree below leads to this condition. It can be understood in the illustrations below.

Delete 55:



Comparison between B-Tree and B+ Tree

S.NO	B tree	B+ tree
1.	All internal and leaf nodes have data pointers	Only leaf nodes have data pointers
2.	Since all keys are not available at leaf, search often takes more time.	All keys are at leaf nodes, hence search is faster and accurate..
3.	No duplicate of keys is maintained in the tree.	Duplicate of keys are maintained and all nodes are present at leaf.
4.	Insertion takes more time and it is not predictable sometimes.	Insertion is easier and the results are always the same.
5.	Deletion of internal node is very complex and tree has to undergo lot of transformations.	Deletion of any node is easy because all node are found at leaf.
6.	Leaf nodes are not stored as structural linked list.	Leaf nodes are stored as structural linked list.
7.	No redundant search keys are present..	Redundant search keys may be present..

UNIT - IV

Graphs-Terminology, sequential and linked representation, graph traversals : Depth First Search & Breadth First Search implementation. Spanning trees, Prim's and Kruskal's method.

Searching and Sorting - Linear Search, Binary Search, Insertion Sort, Selection Sort, Merge Sort, Quick sort, Heap Sort.

Graph:

➤ A graph G is defined as an ordered set (V, E) , where $V(G)$ represents the set of vertices and $E(G)$ represents the edges that connect these vertices.

The figure shows a graph with

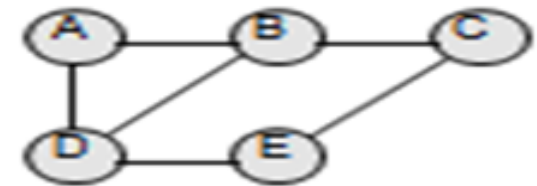
$V(G) = \{A, B, C, D \text{ and } E\}$

and $E(G) = \{(A, B), (B, C), (A, D), (B, D), (D, E), (C, E)\}$.

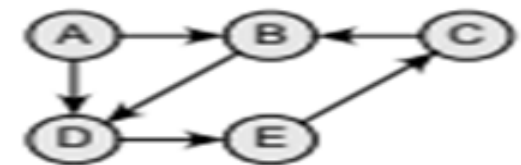
➤ A graph can be directed or undirected.

➤ In an **undirected graph**, edges do not have any direction associated with them. That is, if an edge is drawn between nodes then the nodes can be traversed from A to B as well as from B to A.

➤ In a **directed graph**, edges form an ordered pair. If there is an edge from A to B, then there is a path from A to B but not from B to A.



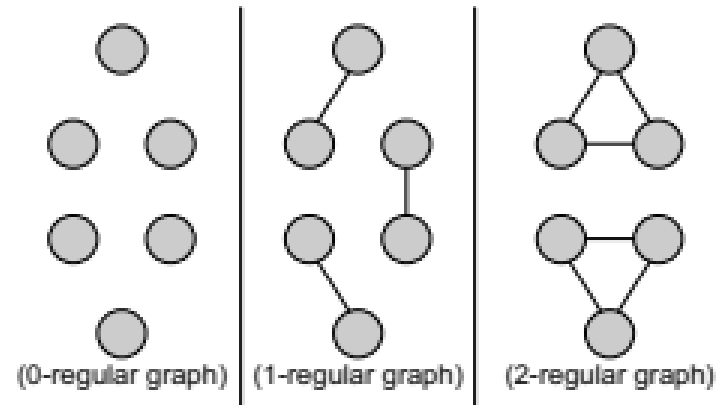
Undirected Graph



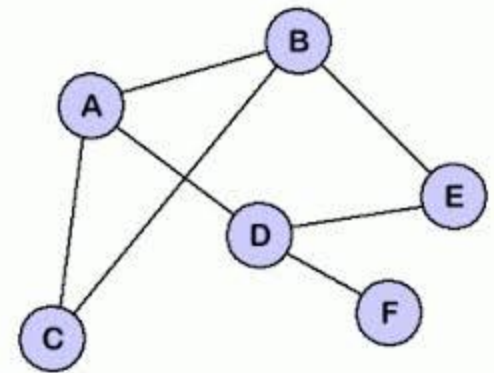
Directed Graph

Graph Terminology

- **Degree of a node:** Degree of a node u , $\deg(u)$, is the total number of edges containing the node u . If $\deg(u) = 0$, it means that u does not belong to any edge and such a node is known as an isolated node.
- **Regular graph :** It is a graph where each vertex has the same number of neighbours. That is, every node has the same degree. A regular graph with vertices of degree k is called a k -regular graph or a regular graph of degree k .
- **Path:** A path P written as $P = \{v_0, v_1, v_2, \dots, v_n\}$, of length n from a node u to v is defined as a sequence of $(n+1)$ nodes.
- **Closed path:** A path P is known as a closed path if the edge has the same end-points. That is, if $v_0 = v_n$.
- **Simple path** A path P is known as a simple path if all the nodes in the path are distinct with an exception that v_0 may be equal to v_n . If $v_0 = v_n$, then the path is called a closed simple path.
- **Cycle** A path in which the first and the last vertices are same. A simple cycle has no repeated edges or vertices (except the first and last vertices).

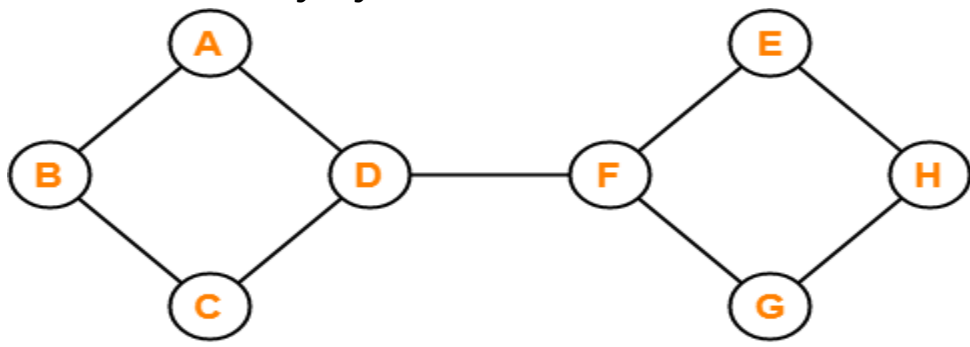


Regular graphs



Graph Terminology...

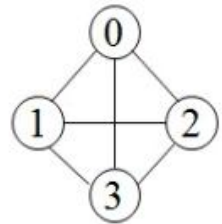
➤ **Connected graph:** A graph is said to be connected if for any two vertices (u, v) in V there is a path from u to v. That is to say that there are no isolated nodes in a connected graph. A connected graph that does not have any cycle is called a tree. Therefore, a tree is treated as a special graph.



Example of Connected Graph

➤ **Complete graph:** A graph G is said to be complete if all its nodes are fully connected.

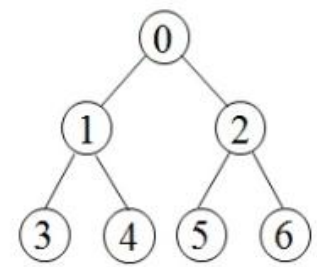
That is, there is a path from one node to every other node in the graph. A complete graph has $n(n-1)/2$ edges, where n is the number of nodes in G.



G₁

complete graph

No.of edges = $n(n-1)/2$
= $4*3/2$
= $12/2$
= 6



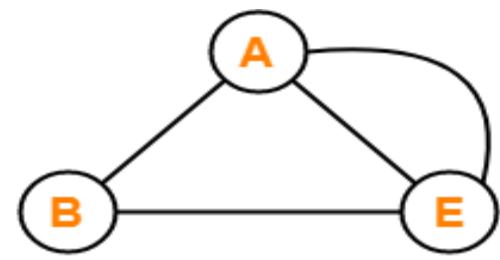
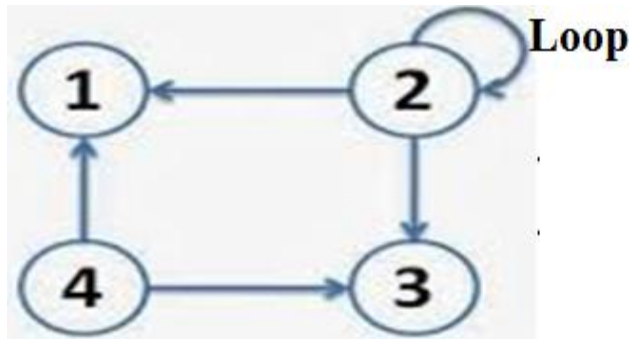
G₂

incomplete graph

No.of edges = $n(n-1)/2$
= $7*6/2$
= $42/2$
= 21

Graph Terminology...

- **Loop:** An edge that has identical end-points is called a loop. That is, $e = (u, u)$. Example: $e=(2,2)$
- **Size of a graph :** The size of a graph is the total number of edges in it.
- **Multiple edges:** Distinct edges which connect the same end-points are called multiple edges. That is, $e = (u, v)$ and $e' = (u, v)$ are known as multiple edges of G .
- **Multi-graph:** A graph with multiple edges and/or loops is called a multi-graph.

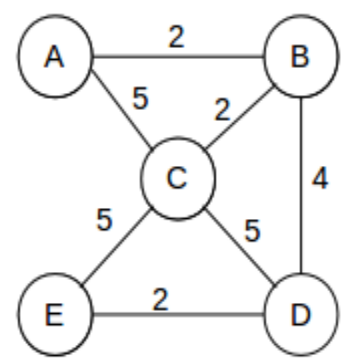


Example of Multi Graph

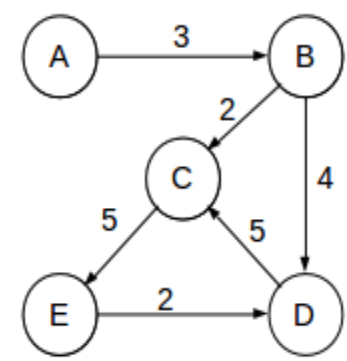
- **Labelled graph or weighted graph:**
A graph is said to be labelled if every edge in the graph is assigned some data.
In a weighted graph, the edges of the graph are assigned some weight or length.

Two types of weighted graphs are:

- i) Undirected Weighted Graph
- ii) Directed Weighted Graph



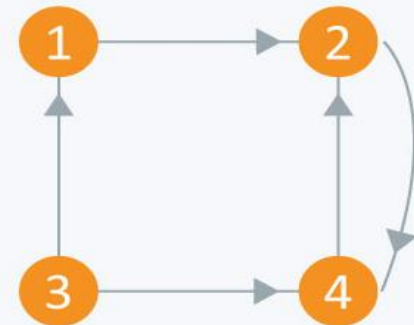
Undirected Weighted Graph



Directed Weighted Graph

Graph Terminology...

- **Out-degree of a node** :The out-degree of a node u , written as $\text{outdeg}(u)$, is the number of edges that originate at u .
- **In-degree of a node**: The in-degree of a node u , written as $\text{indeg}(u)$, is the number of edges that terminate at u .
- **Degree of a node** :The degree of a node, written as $\text{deg}(u)$, is equal to the sum of in-degree and out-degree of that node. Therefore, $\text{deg}(u) = \text{indeg}(u) + \text{outdeg}(u)$.
- **Directed Graphs** :
A directed graph G , also known as a digraph, is a graph in which every edge has a direction assigned to it. An edge of a directed graph is given as an ordered pair (u, v) of nodes in G . For an edge (u, v) ,
 - The edge begins at u and terminates at v .
 - u is known as the origin or initial point of e .
 - v is known as the destination or terminal point of e .
 - u is the predecessor of v .
 - v is the successor of u .
 - Nodes u and v are adjacent to each other.



Directed Graph

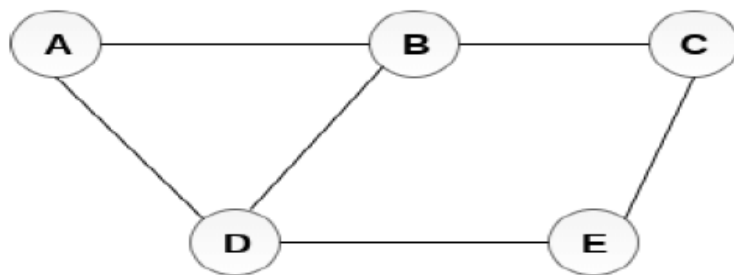
Graph Representations

Graphs are represented in two ways:

- i) Adjacency Matrix
- ii) Adjacency List

Adjacency Matrix Representation:

- Adjacency matrix representation is also called as sequential representation.
- In adjacency matrix, the rows and columns are represented by the graph vertices.
- A graph having n vertices, adjacency matrix will have a dimension $n \times n$.
- An entry M_{ij} in the adjacency matrix representation of an undirected graph G will be 1 if there exists an edge between V_i and V_j otherwise 0.



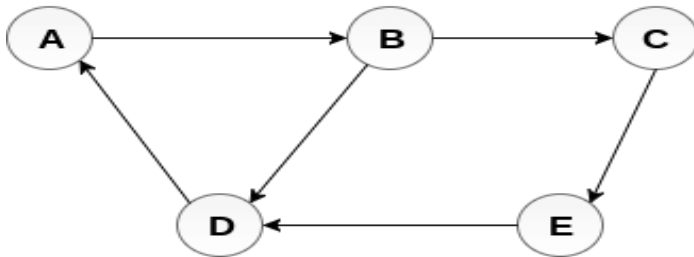
Undirected Graph

	A	B	C	D	E
A	0	1	0	1	0
B	1	0	1	1	0
C	0	1	0	0	1
D	1	1	0	0	1
E	0	0	1	1	0

Adjacency Matrix

Adjacency Matrix Representations...

In directed graph, an entry M_{ij} will be 1 only when there is an edge directed from V_i to V_j otherwise 0.

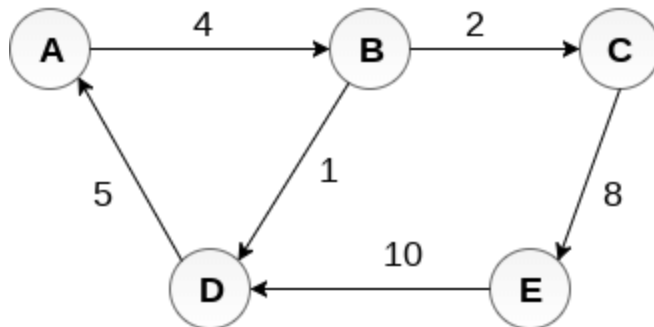


Directed Graph

	A	B	C	D	E
A	0	1	0	0	0
B	0	0	1	1	0
C	0	0	0	0	1
D	1	0	0	0	0
E	0	0	0	1	0

Adjacency Matrix

If the graph is a weighted ,then Non- zero entries of the adjacency matrix are represented by the weight of respective edges.



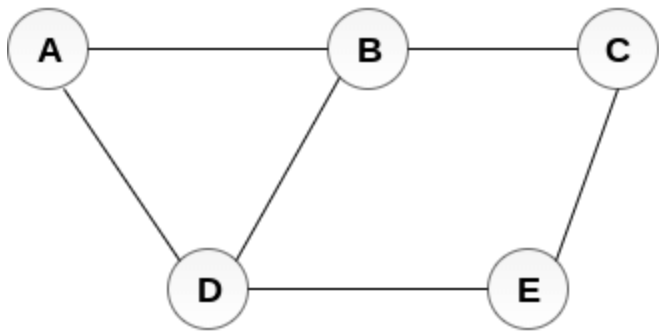
Weighted Directed Graph

	A	B	C	D	E
A	0	4	0	0	0
B	0	0	2	1	0
C	0	0	0	0	8
D	5	0	0	0	0
E	0	0	0	10	0

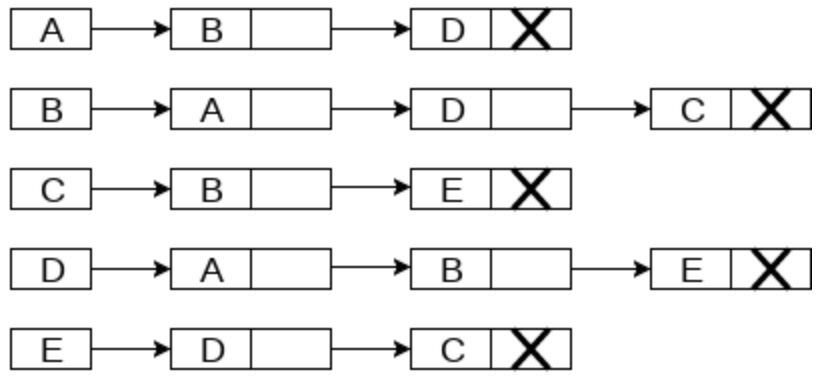
Adjacency Matrix

Adjacency List Representation

- An adjacency list is another way in which graphs can be represented in the computer's memory.
- An adjacency list is maintained for each node present in the graph which stores the node value and a pointer to the next adjacent node to the respective node. If all the adjacent nodes are traversed then store the NULL in the pointer field of last node of the list.
- For a directed graph, the sum of the lengths of all adjacency lists is equal to the number of edges in G.
- For an undirected graph, the sum of the lengths of all adjacency lists is equal to twice the number of edges in G because an edge (u, v) means an edge from node u to v as well as an edge from v to u.

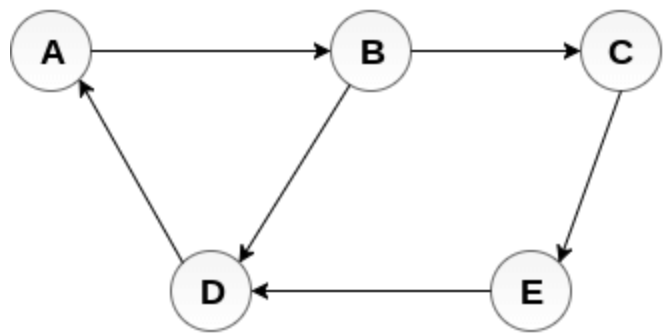


Undirected Graph

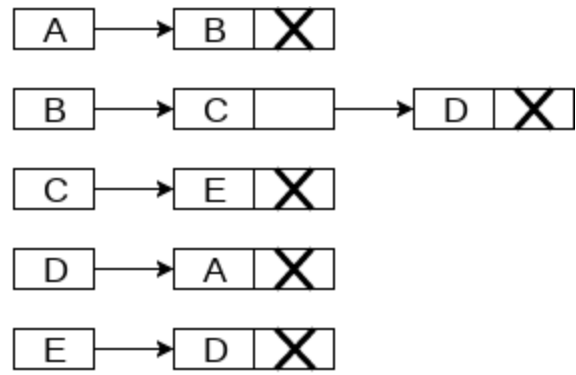


Adjacency List

Adjacency List Representation...

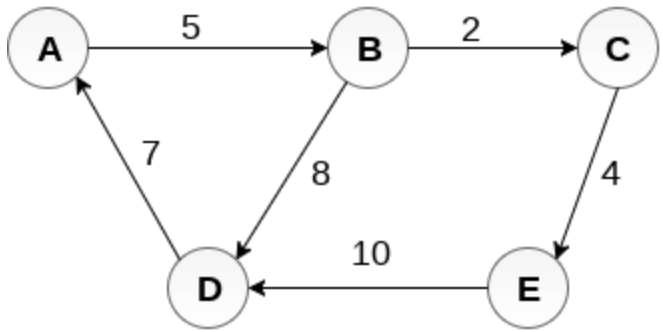


Directed Graph

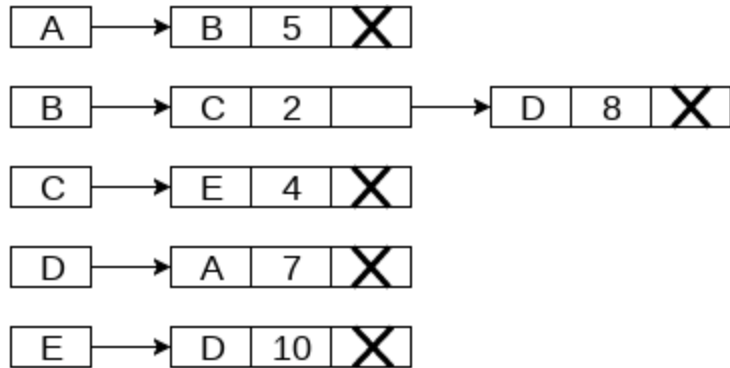


Adjacency List

- Adjacency lists can also be modified to store weighted graphs.
- In the case of weighted directed graph, each node contains an extra field that is called the weight of the node. The adjacency list representation of a directed graph is shown in the following figure.



Weighted Directed Graph



Adjacency List

Graph Traversals

There are two standard methods of graph traversal:

1. Breadth-first search
2. Depth-first search

Value of status and its significance :

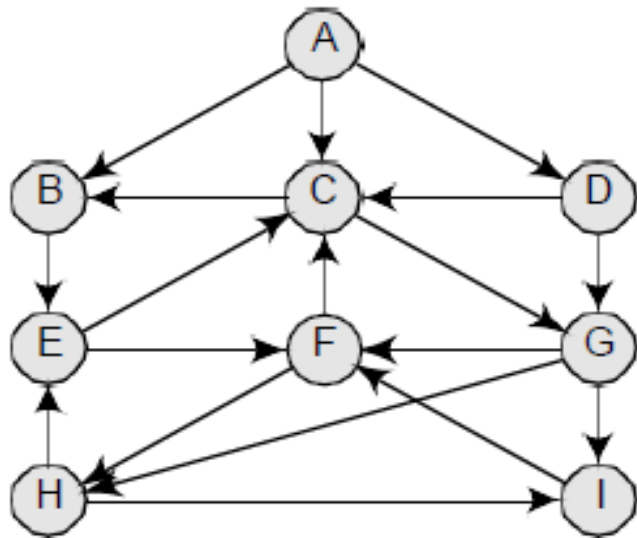
Status	State of the node	Description
1	Ready	The initial state of the node N
2	Waiting	Node N is placed on the queue or stack and waiting to be processed
3	Processed	Node N has been completely processed

Breadth-First Search :

- Breadth first search is a graph traversal algorithm that starts traversing the graph from root node and explores all the neighbouring nodes. Then, it selects the nearest node and explore all the unexplored nodes. The algorithm follows the same process for each of the nearest node until it finds the goal.
- BFS uses the Queue data structure that will hold the nodes that are waiting for further processing and a variable STATUS to represent the current state of the node.

Breadth-First Search...

Example:



Adjacency lists

A: B, C, D

B: E

C: B, G

D: C, G

E: C, F

F: C, H

G: F, H, I

H: E, I

I: F

Step 1: SET STATUS = 1 (ready state)
for each node in G

Step 2: Enqueue the starting node A
and set its STATUS = 2
(waiting state)

Step 3: Repeat Steps 4 and 5 until
QUEUE is empty

Step 4: Dequeue a node N. Process it
and set its STATUS = 3
(processed state).

Step 5: Enqueue all the neighbours of
N that are in the ready state
(whose STATUS = 1) and set
their STATUS = 2
(waiting state)
[END OF LOOP]

Step 6: EXIT

Breadth-First Search...

During the execution of the algorithm, we use two arrays:

- **QUEUE** and **ORIG**. While **QUEUE** is used to hold the nodes that have to be processed, **ORIG** is used to keep track of the origin of each edge.
- Initially, **FRONT** = **REAR** = -1. The algorithm for this is as follows:

(a) Add A to **QUEUE** and add **NULL** to **ORIG**.

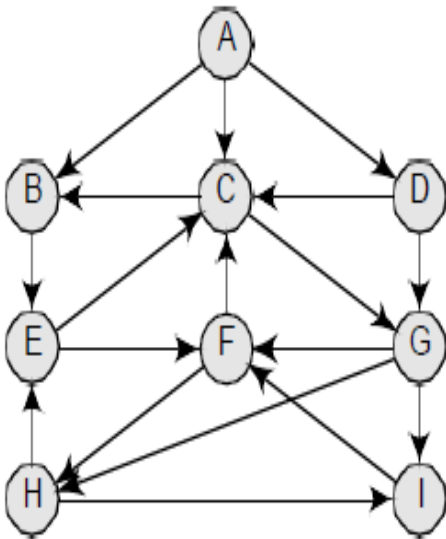
FRONT = 0	QUEUE = A
REAR = 0	ORIG \0

(b) Dequeue a node by setting **FRONT** = **FRONT** + 1 (remove the **FRONT** element of **QUEUE**) and enqueue the neighbours of A. Also, add A as the **ORIG** of its neighbours.

FRONT = 1	QUEUE = A	B	C	D
REAR = 3	ORIG \0	A	A	A

(c) Dequeue a node by setting **FRONT** = **FRONT** + 1 and enqueue the neighbours of B. Also, add B as the **ORIG** of its neighbours.

FRONT = 2	QUEUE = A	B	C	D	E
REAR = 4	ORIG \0	A	A	A	B



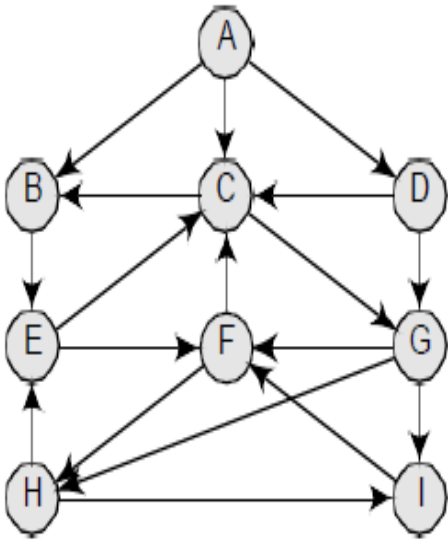
Breadth-First Search...

(d) Dequeue a node by setting $FRONT = FRONT + 1$ and enqueue the neighbours of C. Also, add C as the ORIG of its neighbours. Note that C has two neighbours B and G. Since B has already been added to the queue and it is not in the Ready state, we will not add B and only add G.

FRONT = 3	QUEUE = A	B	C	D	E	G
REAR = 5	ORIG \0	A	A	A	B	C

(e) Dequeue a node by setting $FRONT = FRONT + 1$ and enqueue the neighbours of D. Also, add D as the ORIG of its neighbours . Note that D has two neighbours C and G. Since both of them have already been added to the queue and they are not in the Ready state, we will not add them again.

FRONT = 4	QUEUE = A	B	C	D	E	G
REAR = 5	ORIG \0	A	A	A	B	C

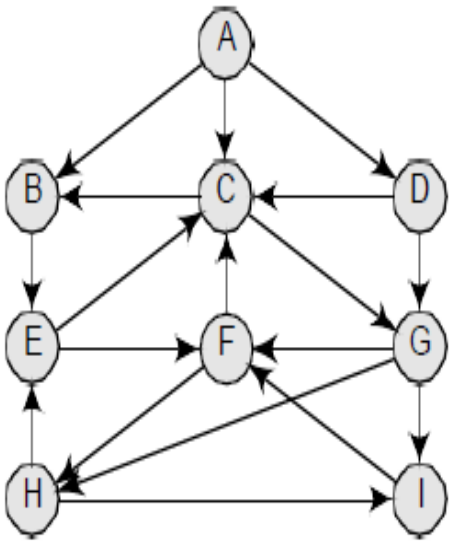


(f) Dequeue a node by setting $FRONT = FRONT + 1$ and enqueue the neighbours of E. Also, add E as the ORIG of its neighbours. Note that E has two neighbours C and F. Since C has already been added to the queue and it is not in the Ready state, we will not add C and add only F.

Breadth-First Search...

FRONT = 5	QUEUE = A	B	C	D	E	G	F
REAR = 6	ORIG \0	A	A	A	B	C	E

(g) Dequeue a node by setting $FRONT = FRONT + 1$ and enqueue the neighbours of G. Also, add G as the ORIG of its neighbours. Note that G has three neighbours F, H, and I.



FRONT = 6	QUEUE = A	B	C	D	E	G	F	H	I
REAR = 9	ORIG \0	A	A	A	B	C	E	G	G

Since F has already been added to the queue, we will only add H and I. As I is our final destination, we stop the execution of this algorithm as soon as it is encountered and added to the QUEUE. Now, backtrack from I using ORIG to find the minimum path P. Thus, we have P as **A -> C -> G -> I**.

Depth-first Search

- The depth-first search algorithm progresses by expanding the starting node of G and then going deeper and deeper until the goal node is found, or until a node that has no children is encountered.
- When a dead-end is reached, the algorithm backtracks, returning to the most recent node that has not been completely explored.
- Depth first search uses stack data structure.

```
Step 1: SET STATUS = 1 (ready state) for each node in G
Step 2: Push the starting node A on the stack and set
        its STATUS = 2 (waiting state)
Step 3: Repeat Steps 4 and 5 until STACK is empty
Step 4: Pop the top node N. Process it and set its
        STATUS = 3 (processed state)
Step 5: Push on the stack all the neighbours of N that
        are in the ready state (whose STATUS = 1) and
        set their STATUS = 2 (waiting state)
        [END OF LOOP]
Step 6: EXIT
```

Depth-first Search...

(a) Push H onto the stack.

STACK: H

(b) Pop and print the top element of the STACK, that is, H. Push all the neighbours of H onto the stack that are in the ready state. The STACK now becomes

PRINT: H STACK: E, I

(c) Pop and print the top element of the STACK, that is, I. Push all the neighbours of I onto the stack that are in the ready state. The STACK now becomes

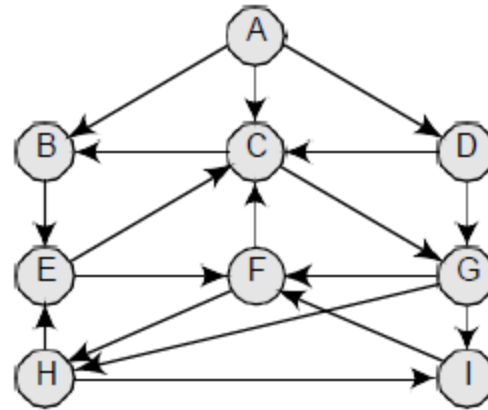
PRINT: I STACK: E, F

(d) Pop and print the top element of the STACK, that is, F. Push all the neighbours of F onto the stack that are in the ready state. (Note F has two neighbours, C and H. But only C will be added, as H is not in the ready state.) The STACK now becomes

PRINT: F STACK: E, C

(e) Pop and print the top element of the STACK, that is, C. Push all the neighbours of C onto the stack that are in the ready state. The STACK now becomes

PRINT: C STACK: E, B, G



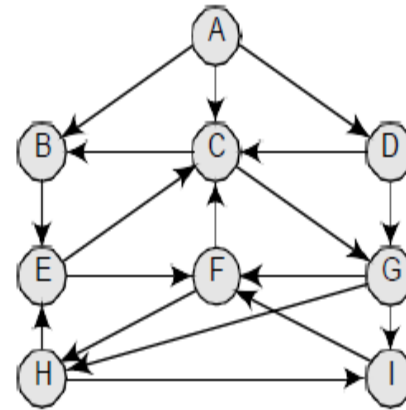
Adjacency lists

A: B, C, D
B: E
C: B, G
D: C, G
E: C, F
F: C, H
G: F, H, I
H: E, I
I: F

Depth-first Search...

(f) Pop and print the top element of the STACK, that is, G.
Push all the neighbours of G onto the stack that are in the ready state. Since there are no neighbours of G that are in the ready state, no push operation is performed.
The STACK now becomes

PRINT: G **STACK: E, B**



Adjacency lists	
A:	B, C, D
B:	E
C:	B, G
D:	C, G
E:	C, F
F:	C, H
G:	F, H, I
H:	E, I
I:	F

(g) Pop and print the top element of the STACK, that is, B. Push all the neighbours of B onto the stack that are in the ready state. Since there are no neighbours of B that are in the ready state, no push operation is performed. The STACK now becomes

PRINT: B **STACK: E**

(h) Pop and print the top element of the STACK, that is, E. Push all the neighbours of E onto the stack that are in the ready state. Since there are no neighbours of E that are in the ready state, no push operation is performed. The STACK now becomes empty.

PRINT: E **STACK:**

Since the STACK is now empty, the depth-first search of G starting at node H is complete and the nodes which were printed are:

H, I, F, C, G, B, E

These are the nodes which are reachable from the node H.

Applications of BFS & DFS

Applications of Breadth-First Search :

Breadth-first search can be used to solve many problems such as:

- Finding all connected components in a graph G .
- Finding all nodes within an individual connected component.
- Finding the shortest path between two nodes, u and v , of an unweighted graph.
- Finding the shortest path between two nodes, u and v , of a weighted graph.

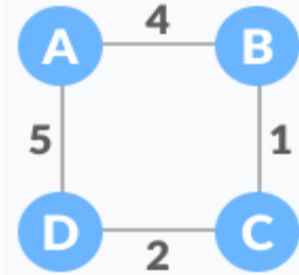
Applications of Depth-First Search Algorithm

Depth-first search is useful for:

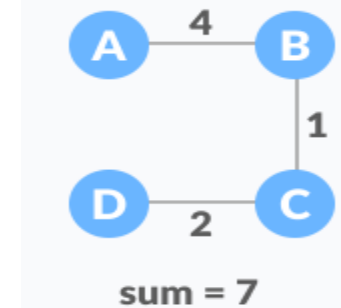
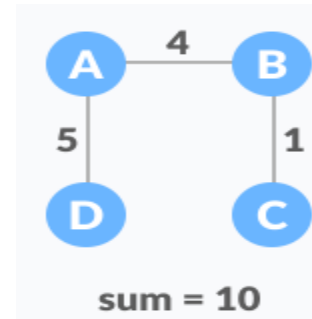
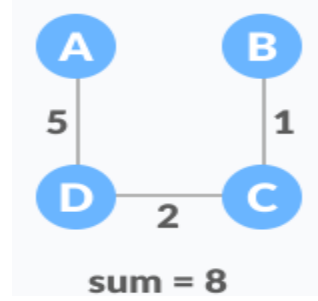
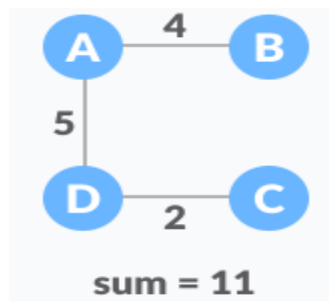
- Finding a path between two specified nodes, u and v , of an unweighted graph.
- Finding a path between two specified nodes, u and v , of a weighted graph.
- Finding whether a graph is connected or not.
- Computing the spanning tree of a connected graph.

Minimum Spanning Tree

- A **spanning tree** is a subset of Graph G, which has all the vertices covered with minimum possible number of edges.
- It does not have cycles and it cannot be disconnected.
- The total number of spanning trees with n vertices that can be created from a complete graph is equal to $n^{(n-2)}$.



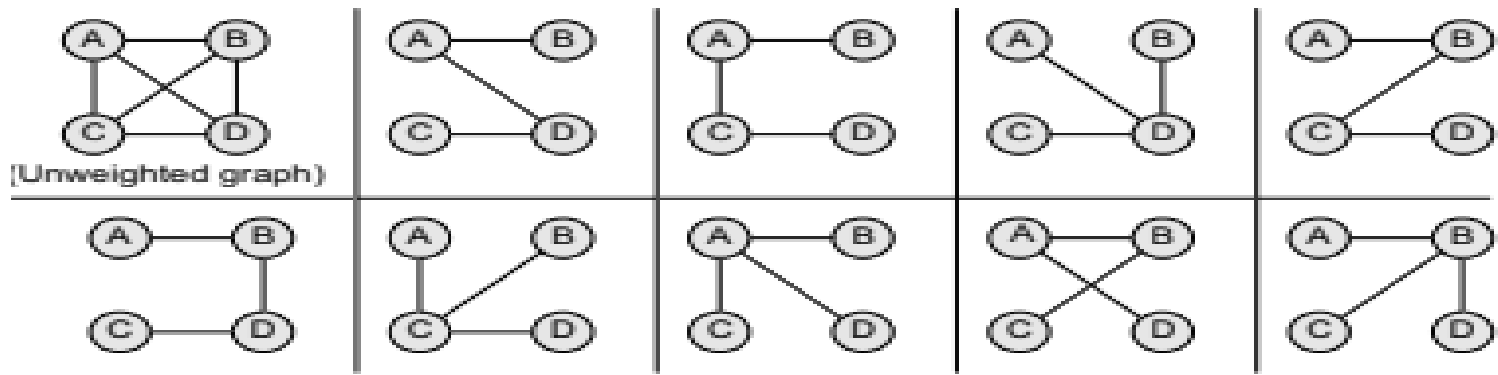
Possible Spanning trees are:



- A **minimum spanning tree** (MST) is defined as a spanning tree with weight less than or equal to the weight of every other spanning trees.
- In other words, a minimum spanning tree is a spanning tree that has weights associated with its edges, and the total weight of the tree (the sum of the weights of its edges) is at a minimum.

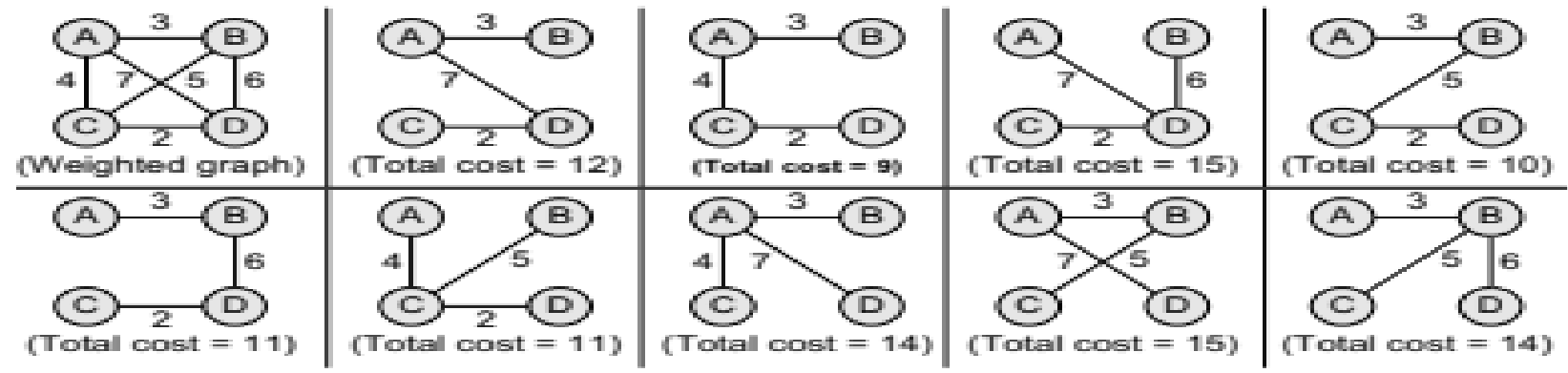
Minimum Spanning Tree...

Consider an **unweighted graph G** given below. From G, we can draw many distinct spanning trees. Eight of them are given here. For an unweighted graph, every spanning tree is a minimum spanning tree.



Unweighted graph and its spanning trees

Consider a **weighted graph G**. From G, we can draw three distinct spanning trees. But only a single minimum spanning tree can be obtained, that is, the one that has the minimum weight (cost) associated with it of all the spanning trees given in, the one that is highlighted is called the minimum spanning tree, as it has the lowest cost associated with it.



Weighted graph and its spanning trees

Minimum Spanning Trees...

Applications of MST:

- MSTs are widely used for designing networks. For instance, people separated by varying distances wish to be connected together through a telephone network. A minimum spanning tree is used to determine the least costly paths with no cycles in this network, thereby providing a connection that has the minimum cost involved.
- MSTs are used to find airline routes. While the vertices in the graph denote cities, edges represent the routes between these cities. No doubt, more the distance between the cities, higher will be the amount charged. Therefore, MSTs are used to optimize airline routes by finding the least costly path with no cycles.
- MSTs are also used to find the cheapest way to connect terminals, such as cities, electronic components or computers via roads, airlines, railways, wires or telephone lines.
- MSTs are applied in routing algorithms for finding the most efficient path.

Algorithms for finding Minimum Spanning Trees:

- i) Kruskal's Algorithm
- ii) Prim's Algorithm

Kruskal's Algorithm

- Kruskal's algorithm is used to find the minimum spanning tree for a connected weighted graph.
- In this algorithm, we use a priority queue in which edges that have minimum weight takes a priority over any other edge in the graph.
- The algorithm aims to find a subset of the edges that forms a tree that includes every vertex, the total weight of all the edges in the tree is minimized.
- When the Kruskal's algorithm terminates, the forest has only one component and forms a minimum spanning tree of the graph.

Kruskal's Algorithm:

Step 1: Create a forest in such a way that each graph is a separate tree.

Step 2: Create a priority queue Q that contains all the edges of the graph.

Step 3: Repeat Steps 4 and 5 while Q is NOT EMPTY

Step 4: Remove an edge from Q

Step 5: IF the edge obtained in Step 4 connects two different trees, then Add it to the forest
(for combining two trees into one tree).

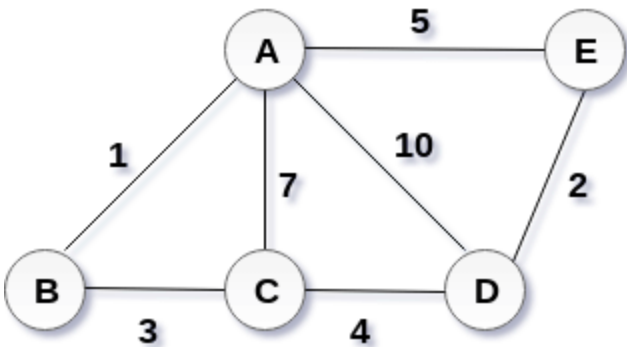
ELSE

Discard the edge

Step 6: END

Kruskal's Algorithm...

Example : Apply Kruskal's algorithm on the graph



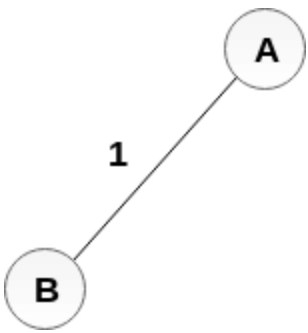
The weight of the edges given as :

Edge	AE	AD	AC	AB	BC	CD	DE
Weight	5	10	7	1	3	4	2

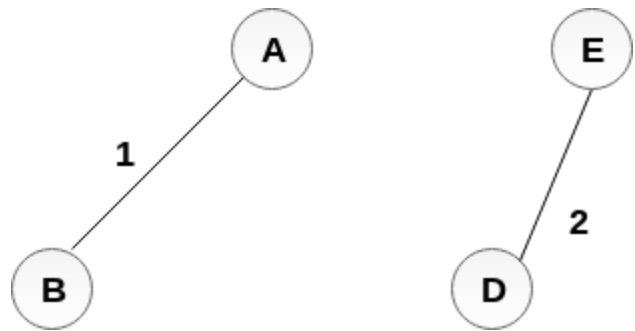
Sort the edges according to their weights:

Edge	AB	DE	BC	CD	AE	AC	AD
Weight	1	2	3	4	5	7	10

Add AB to the MST:

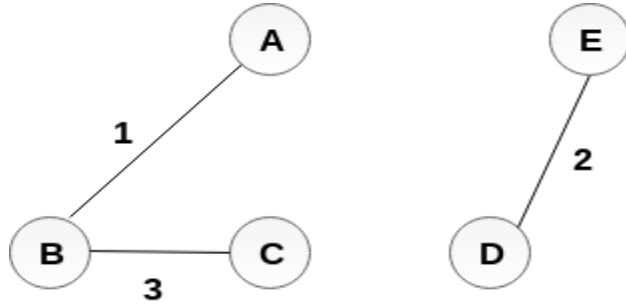


Add DE to MST:

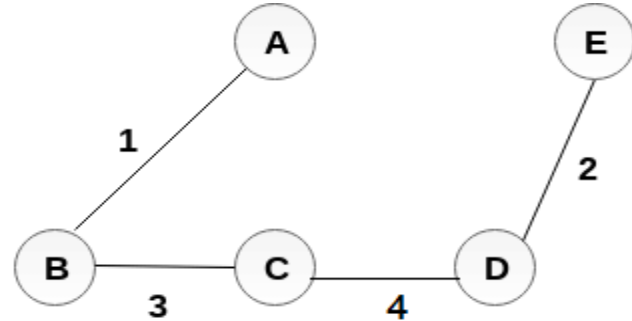


Kruskal's Algorithm...

Add BC to the MST:



Add CD to MST:



The next step is to add AE, but we can't add that as it will cause a cycle.
The next edge to be added is AC, but it can't be added as it will cause a cycle.
The next edge to be added is AD, but it can't be added as it will contain a cycle.

The cost of MST = 1 + 2 + 3 + 4 = 10

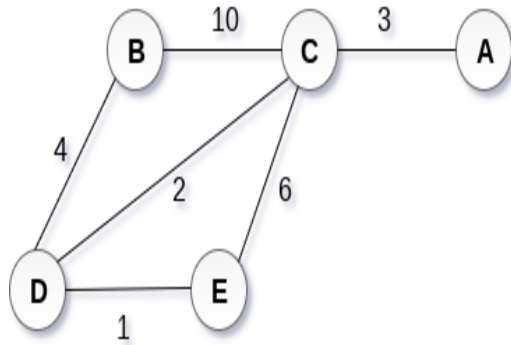
Prim's Algorithm

- Prim's algorithm is a greedy algorithm that is used to form a minimum spanning tree for a connected weighted undirected graph.
- This algorithm maintains three sets of vertices which can be given as below:
 1. **Tree vertices** : Vertices that are a part of the minimum spanning tree T.
 2. **Fringe vertices** : Vertices that are currently not a part of T, but are adjacent to some tree vertex.
 3. **Unseen vertices** : Vertices that are neither tree vertices nor fringe vertices fall under this category.
- The steps involved in the Prim's algorithm are:

```
Step 1: Select a starting vertex
Step 2: Repeat Steps 3 and 4 until there are fringe vertices
Step 3:   Select an edge e connecting the tree vertex and
          fringe vertex that has minimum weight
Step 4:   Add the selected edge and the vertex to the
          minimum spanning tree T
          [END OF LOOP]
Step 5: EXIT
```

Prim's Algorithm...

Example : Construct a minimum spanning tree of the graph given using Prim's algorithm.



Step 1 : Choose a starting vertex B.

Step 2 : Add the vertices that are adjacent to B. the edges that connecting the vertices are shown by dotted lines.

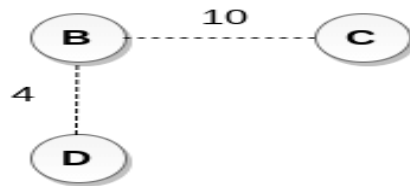
Step 3 : Choose the edge with the minimum weight among all. i.e. BD and add it to MST. Add the adjacent vertices of D i.e. C and E.

Step 4 : Choose the edge with the minimum weight among all. In this case, the edges DE and CD are such edges. Add them to MST and explore the adjacent of C i.e. E and A.

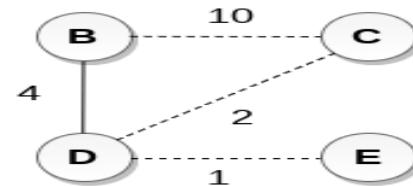
Step 5 : Choose the edge with the minimum weight i.e. CA. We can't choose CE as it would cause cycle in the graph.



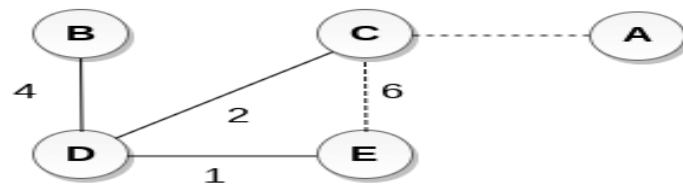
Step 1



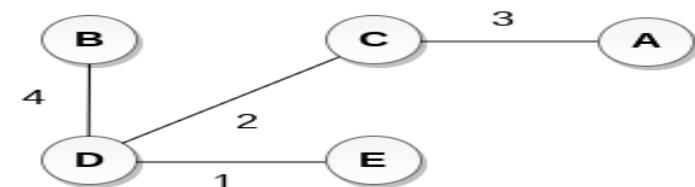
Step 2



Step 3



Step 4



Step 5

$$\text{cost(MST)} = 4 + 2 + 1 + 3 = 10 \text{ units.}$$

Searching

- Searching means to find whether a particular value is present in an array or not.
- If the value is present in the array, then searching is said to be successful and the searching process gives the location of that value in the array.
- If the value is not present in the array, the searching process displays an appropriate message and in this case searching is said to be unsuccessful.
- There are two popular methods for searching the array elements:
 - i) Linear Search
 - ii) Binary Search.

Linear Search:

- Linear search, also called as sequential search, is a very simple method used for searching an array for a particular value.
- It works by comparing the value to be searched with every element of the array one by one in a sequence until a match is found.
- Linear search is mostly used to search an unordered list of elements.

For example :

```
int A[] = {10, 8, 2, 7, 3, 4, 9, 1, 6, 5};
```

The value to be searched is **VAL = 7**, then searching means to find whether the value '7' is present in the array or not. If yes, then it returns the position of its occurrence.

Here, POS = 3 (index starting from 0).

Linear Search...

Linear Search Algorithm:

```
LINEAR_SEARCH(A, N, VAL)
Step 1: [INITIALIZE] SET POS = -1
Step 2: [INITIALIZE] SET I = 1
Step 3:   Repeat Step 4 while I<=N
Step 4:   IF A[I] = VAL
           SET POS = I
           PRINT POS
           Go to Step 6
           [END OF IF]
           SET I = I + 1
           [END OF LOOP]
Step 5: IF POS = -1
       PRINT "VALUE IS NOT PRESENT
       IN THE ARRAY"
       [END OF IF]
Step 6: EXIT
```

- The best case of linear search is when VAL is equal to the first element of the array. In this case, only one comparison will be made.
- The worst case will happen when either VAL is not present in the array or it is equal to the last element of the array. In both the cases, n comparisons will have to be made.

Binary Search

- Binary search is a searching algorithm that works efficiently with a sorted list.
- In this algorithm, BEG and END are the beginning and ending positions of the list that we are looking to search for the element. MID is calculated as $(BEG + END)/2$.
- if VAL is not equal to A[MID], then the values of BEG, END, and MID will be changed depending on whether VAL is smaller or greater than A[MID].
 - (a) If $VAL < A[MID]$, then VAL will be present in the left segment of the array. So, the value of END will be changed as $END = MID - 1$.
 - (b) If $VAL > A[MID]$, then VAL will be present in the right segment of the array. So, the value of BEG will be changed as $BEG = MID + 1$.
- The algorithm will terminate when $A[MID] = VAL$. When the algorithm ends, we will set $POS = MID$. POS is the position at which the value is present in the array.
- Finally, if VAL is not present in the array, then eventually, END will be less than BEG. When this happens, the algorithm will terminate and the search will be unsuccessful.

```
BINARY_SEARCH(A, lower_bound, upper_bound, VAL)
Step 1: [INITIALIZE] SET BEG = lower_bound
           END = upper_bound, POS = - 1
Step 2: Repeat Steps 3 and 4 while BEG <= END
Step 3:   SET MID = (BEG + END)/2
Step 4:   IF A[MID] = VAL
           SET POS = MID
           PRINT POS
           Go to Step 6
         ELSE IF A[MID] > VAL
           SET END = MID - 1
         ELSE
           SET BEG = MID + 1
         [END OF IF]
       [END OF LOOP]
Step 5: IF POS = -1
       PRINT "VALUE IS NOT PRESENT IN THE ARRAY"
       [END OF IF]
Step 6: EXIT
```

Binary Search

BINARY_SEARCH(A, lower_bound, upper_bound, VAL)

Step 1: [INITIALIZE] SET BEG = lower_bound

 END = upper_bound, POS = - 1

Step 2: Repeat Steps 3 and 4 while BEG <= END

Step 3: SET MID = (BEG + END)/2

Step 4: IF A[MID] = VAL

 SET POS = MID

 PRINT POS

 Go to Step 6

 ELSE IF A[MID] > VAL

 SET END = MID - 1

 ELSE

 SET BEG = MID + 1

 [END OF IF]

 [END OF LOOP]

Step 5: IF POS = -1

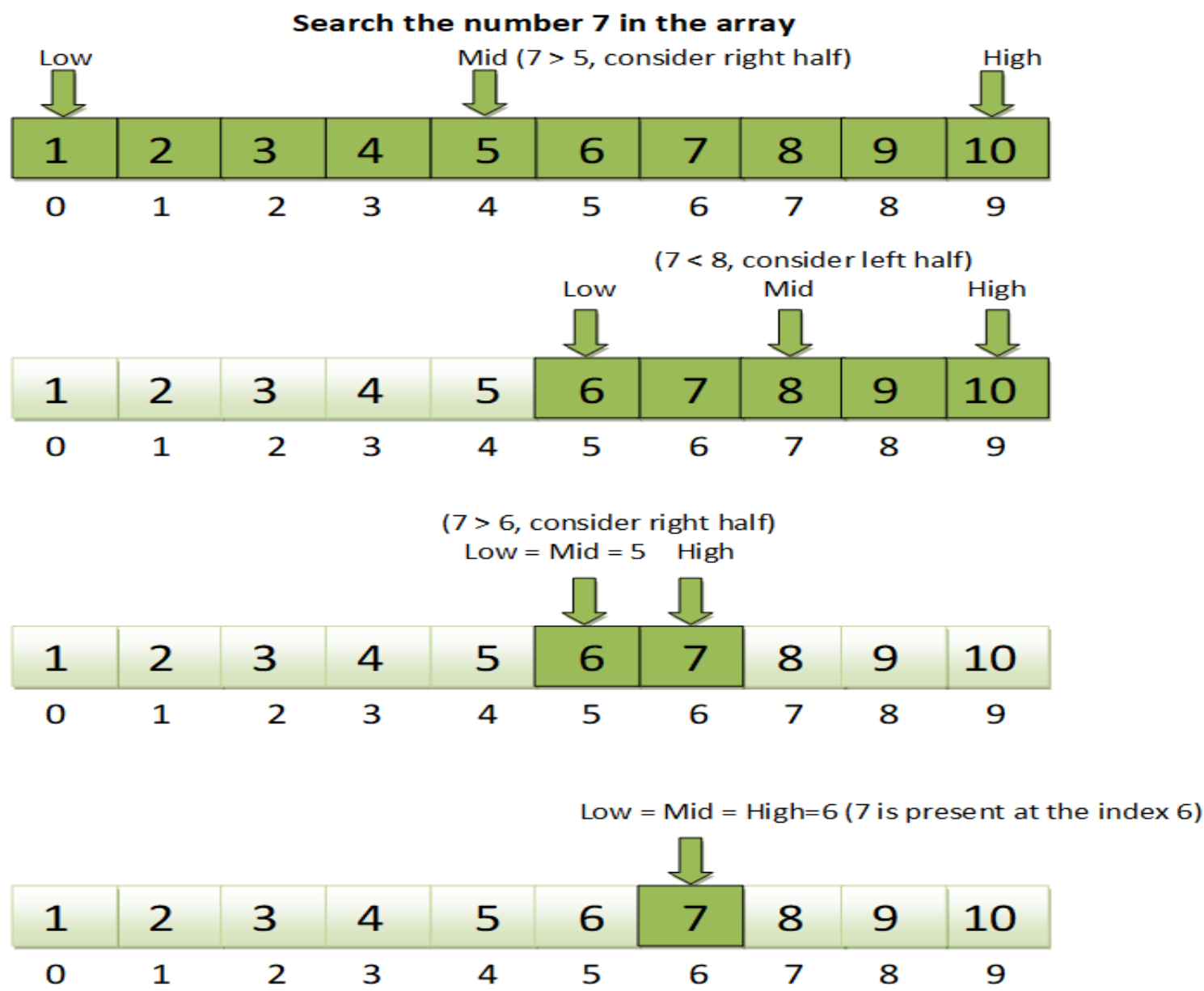
 PRINT "VALUE IS NOT PRESENT IN THE ARRAY"

 [END OF IF]

Step 6: EXIT

Binary Search...

Binary Search



Sorting

- Sorting means arranging the elements of an array so that they are placed in some relevant order which may be either ascending or descending.
- For example, if we have an array that is declared and initialized as

```
int A[] = {21, 34, 11, 9, 1, 0, 22};
```

Then the sorted array (ascending order) can be given as:

```
A[] = {0, 1, 9, 11, 21, 22, 34};
```
- There are two types of sorting:
 1. **Internal sorting** : which deals with sorting the data stored in the computer's memory
 2. **External sorting** : which deals with sorting the data stored in files. External sorting is applied when there is voluminous data that cannot be stored in the memory.

Bubble Sort:

- Bubble sort is a very simple method that sorts the array elements by repeatedly moving the largest element to the highest index position of the array segment.
- Here consecutive adjacent pairs of elements in the array are compared with each other.
- If the element at the lower index is greater than the element at the higher index, the two elements are interchanged so that the smaller element is placed before the bigger one.
- This process will continue till the list of unsorted elements are over.

Bubble Sort ...

Ex: $A[] = \{30, 52, 29, 87, 63, 27, 19, 54\}$

Pass1:

(a) Compare 30 and 52. Since $30 < 52$, no swapping is done.

(b) Compare 52 and 29. Since $52 > 29$, swapping is done.

30, 29, 52, 87, 63, 27, 19, 54

(c) Compare 52 and 87. Since $52 < 87$, no swapping is done.

(d) Compare 87 and 63. Since $87 > 63$, swapping is done.

30, 29, 52, 63, 87, 27, 19, 54

(e) Compare 87 and 27. Since $87 > 27$, swapping is done.

30, 29, 52, 63, 27, 87, 19, 54

(f) Compare 87 and 19. Since $87 > 19$, swapping is done.

30, 29, 52, 63, 27, 19, 87, 54

(g) Compare 87 and 54. Since $87 > 54$, swapping is done.

30, 29, 52, 63, 27, 19, 54, 87

After the end of the first pass, the largest element is placed at the highest index of the array.

BUBBLE_SORT(A, N)

Step 1: Repeat Step 2 For $i = 0$ to $N-1$

Step 2: Repeat For $j = 0$ to $N - i$

Step 3: IF $A[j] > A[j + 1]$

SWAP $A[j]$ and $A[j+1]$

[END OF INNER LOOP]

[END OF OUTER LOOP]

Step 4: EXIT

Bubble Sort...

Pass 2:

(a) Compare 30 and 29. Since $30 > 29$, swapping is done.

29, 30, 52, 63, 27, 19, 54, 87

(b) Compare 30 and 52. Since $30 < 52$, no swapping is done.

(c) Compare 52 and 63. Since $52 < 63$, no swapping is done.

(d) Compare 63 and 27. Since $63 > 27$, swapping is done.

29, 30, 52, 27, 63, 19, 54, 87

(e) Compare 63 and 19. Since $63 > 19$, swapping is done.

29, 30, 52, 27, 19, 63, 54, 87

(f) Compare 63 and 54. Since $63 > 54$, swapping is done.

29, 30, 52, 27, 19, 54, 63, 87

After the end of the second pass, the second largest element is placed at the second highest index of the array.

Pass 3:

(a) Compare 29 and 30. Since $29 < 30$, no swapping is done.

(b) Compare 30 and 52. Since $30 < 52$, no swapping is done.

(c) Compare 52 and 27. Since $52 > 27$, swapping is done.

29, 30, 27, 52, 19, 54, 63, 87

(d) Compare 52 and 19. Since $52 > 19$, swapping is done.

29, 30, 27, 19, 52, 54, 63, 87

Bubble Sort ...

(e) Compare 52 and 54. Since $52 < 54$, no swapping is done.

Observe that after the end of the third pass, the third largest element is placed at the third highest index of the array. All the other elements are still unsorted.

Pass 4:

(a) Compare 29 and 30. Since $29 < 30$, no swapping is done.

(b) Compare 30 and 27. Since $30 > 27$, swapping is done.

29, 27, 30, 19, 52, 54, 63, 87

(c) Compare 30 and 19. Since $30 > 19$, swapping is done.

29, 27, 19, 30, 52, 54, 63, 87

(d) Compare 30 and 52. Since $30 < 52$, no swapping is done.

After the end of the fourth pass, the fourth largest element is placed at the fourth highest index of the array.

Pass 5:

(a) Compare 29 and 27. Since $29 > 27$, swapping is done.

27, 29, 19, 30, 52, 54, 63, 87

(b) Compare 29 and 19. Since $29 > 19$, swapping is done. 27, 19, 29, 30, 52, 54, 63, 87

(c) Compare 29 and 30. Since $29 < 30$, no swapping is done

After the end of the fifth pass, the fifth largest element is placed at the fifth highest index of the array.

Bubble Sort ...

Pass 6:

(a) Compare 27 and 19. Since $27 > 19$, swapping is done.

19, 27, 29, 30, 52, 54, 63, 87

(b) Compare 27 and 29. Since $27 < 29$, no swapping is done.

After the end of the sixth pass, the sixth largest element is placed at the sixth largest index of the array.

Pass 7:

(a) Compare 19 and 27. Since $19 < 27$, no swapping is done.

Observe that the entire list is sorted now.

19, 27, 29, 30, 52, 54, 63, 87

Insertion Sort

Insertion sort works as follows:

- The array of values to be sorted is divided into two sets. One that stores sorted values and another that contains unsorted values.
- The sorting algorithm will proceed until there are no elements in the unsorted set.
- Suppose there are n elements in the array. Initially, the element with index 0 (assuming $LB = 0$) is in the sorted set. Rest of the elements are in the unsorted set.
- The first element of the unsorted partition has array index 1 (if $LB = 0$).
- During each iteration of the algorithm, the first element in the unsorted set is picked up and inserted into the correct position in the sorted set.

INSERTION-SORT (ARR, N)

Step 1: Repeat Steps 2 to 5 for $K = 1$ to $N - 1$

Step 2: SET TEMP = ARR[K]

Step 3: SET $J = K - 1$

Step 4: Repeat while TEMP $\leq$ ARR[J]
 SET ARR[J + 1] = ARR[J]
 SET $J = J - 1$

 [END OF INNER LOOP]

Step 5: SET ARR[J + 1] = TEMP

 [END OF LOOP]

Step 6: EXIT

Insertion sort...

Example : Sort the elements using Insertion sort

50,10,30,80,70,20,60,40

PASS	SORTED LIST	UNSORTED LIST	TEST
1	<u>50</u>	10 30 80 70 20 60 40	50 > 10
2	10 <u>50</u>	30 80 70 20 60 40	50 > 30
	<u>10</u> 50	30 80 70 20 60 40	10 < 30
3	10 30 <u>50</u>	80 70 20 60 40	50 < 80
4	10 30 50 <u>80</u>	70 20 60 40	80 > 70
	10 30 <u>50</u> 80	70 20 60 40	50 < 70
5	10 30 50 70 <u>80</u>	20 60 40	80 > 20
	10 30 50 <u>70</u> 80	20 60 40	70 > 20
	10 30 <u>50</u> 70 80	20 60 40	50 > 20
	10 <u>30</u> 50 70 80	20 60 40	30 > 20
	<u>10</u> 30 50 70 80	20 60 40	10 < 20
6	10 20 30 50 70 <u>80</u>	60 40	80 > 60
	10 20 30 50 <u>70</u> 80	60 40	70 > 60
	10 20 30 <u>50</u> 70 80	60 40	50 < 60
7	10 20 30 50 60 70 <u>80</u>	40	80 > 40
	10 20 30 50 60 <u>70</u> 80	40	70 > 40
	10 20 30 50 <u>60</u> 70 80	40	60 > 40
	10 20 30 <u>50</u> 60 70 80	40	50 > 40
	10 20 <u>30</u> 50 60 70 80	40	30 < 40
	10 20 30 40 50 60 70 80	<empty>	

Selection sort

Selection sort works as follows:

- First find the smallest value in the array and place it in the first position. Then, find the second smallest value in the array and place it in the second position. Repeat this procedure until the entire array is sorted.
- In Pass 1, find the position POS of the smallest value in the array and then swap $ARR[POS]$ and $ARR[0]$. Thus, $ARR[0]$ is sorted.
- In Pass 2, find the position POS of the smallest value in sub-array of $N-1$ elements. Swap $ARR[POS]$ with $ARR[1]$. Now, $ARR[0]$ and $ARR[1]$ is sorted.
- In Pass $N-1$, find the position POS of the smaller of the elements $ARR[N-2]$ and $ARR[N-1]$. Swap $ARR[POS]$ and $ARR[N-2]$ so that $ARR[0], ARR[1], \dots, ARR[N-1]$ is sorted.

SMALLEST (ARR, K, N, POS)

```
Step 1: [INITIALIZE] SET SMALL = ARR[K]
Step 2: [INITIALIZE] SET POS = K
Step 3: Repeat for J = K+1 to N-1
        IF SMALL > ARR[J]
            SET SMALL = ARR[J]
            SET POS = J
        [END OF IF]
    [END OF LOOP]
Step 4: RETURN POS
```

SELECTION SORT(ARR, N)

```
Step 1: Repeat Steps 2 and 3 for K = 1
        to N-1
Step 2:     CALL SMALLEST(ARR, K, N, POS)
Step 3:     SWAP A[K] with ARR[POS]
        [END OF LOOP]
Step 4: EXIT
```

Selection Sort...

Example : Perform selection sort on the following elements 39,9,81,45,90,27,72,18

ARR[0]	ARR[1]	ARR[2]	ARR[3]	ARR[4]	ARR[5]	ARR[6]	ARR[7]
39	9	81	45	90	27	72	18

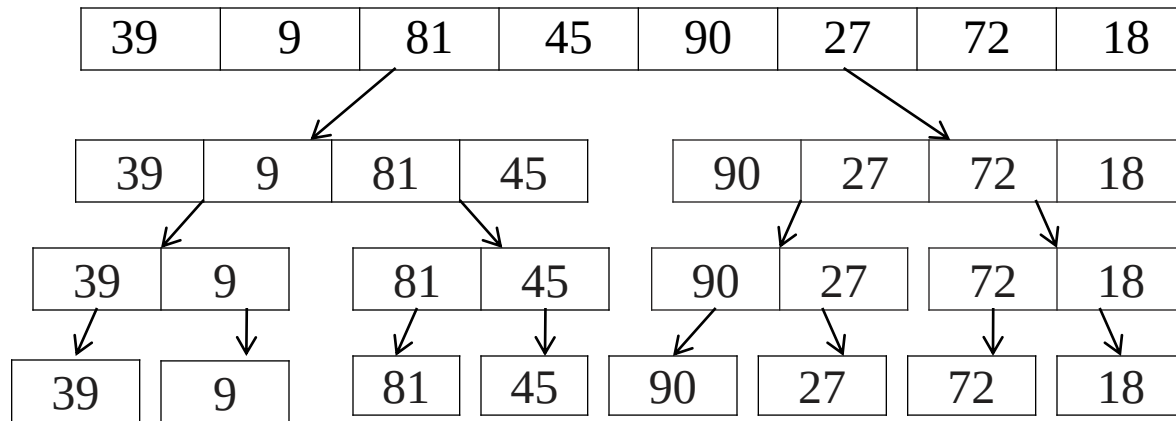
PASS	POS	ARR[0]	ARR[1]	ARR[2]	ARR[3]	ARR[4]	ARR[5]	ARR[6]	ARR[7]
1	1	9	39	81	45	90	27	72	18
2	7	9	18	81	45	90	27	72	39
3	5	9	18	27	45	90	81	72	39
4	7	9	18	27	39	90	81	72	45
5	7	9	18	27	39	45	81	72	90
6	6	9	18	27	39	45	72	81	90
7	6	9	18	27	39	45	72	81	90

Merge Sort

- Merge sort is a sorting algorithm that uses the divide, conquer, and combine algorithmic paradigm.
- **Divide** means partitioning the n -element array to be sorted into two sub-arrays of $n/2$ elements. If A is an array containing zero or one element, then it is already sorted. However, if there are more elements in the array, divide A into two sub-arrays, A_1 and A_2 , each containing about half of the elements of A .
- **Conquer** means sorting the two sub-arrays recursively using merge sort.
- **Combine** means merging the two sorted sub-arrays of size $n/2$ to produce the sorted array of n elements.

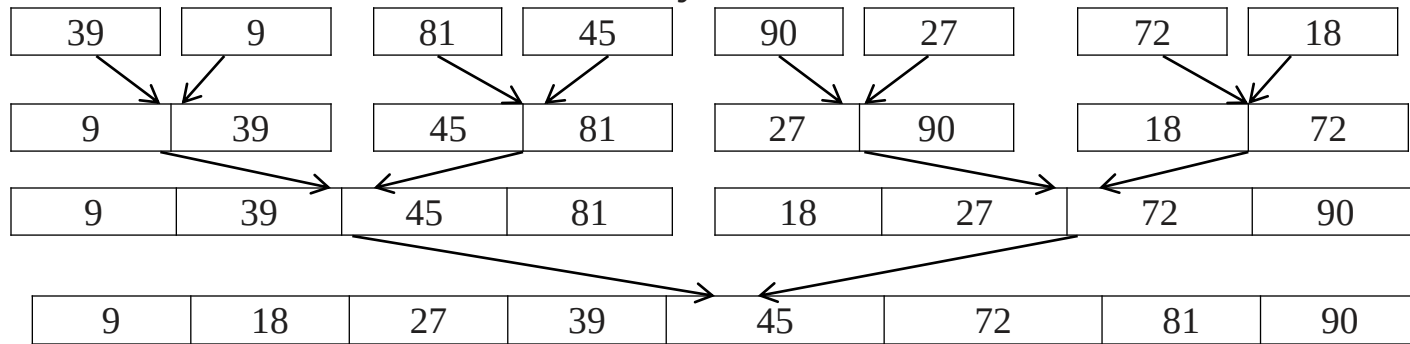
Example: 39,9,81,45,90,27,72,18

- Divide and Conquer the array



Merge Sort...

➤ Combine the elements to form the sorted array:



➤ The merge sort algorithm recursively divides the list into smaller lists, the merge algorithm conquers the list to sort the elements in individual lists. Finally, the smaller lists are merged to form one list.

Merge Sort Algorithm:

```
MERGE_SORT(ARR, BEG, END)
```

```
Step 1: IF BEG < END
```

```
    SET MID = (BEG + END)/2
```

```
    CALL MERGE_SORT (ARR, BEG, MID)
```

```
    CALL MERGE_SORT (ARR, MID + 1, END)
```

```
    MERGE (ARR, BEG, MID, END)
```

```
    [END OF IF]
```

```
Step 2: END
```

Merge Sort...

Combine the elements to form the sorted array:

MERGE (ARR, BEG, MID, END)

Step 1: [INITIALIZE] SET I = BEG, J = MID + 1, INDEX = 0

Step 2: Repeat while (I <= MID) AND (J <= END)

 IF ARR[I] < ARR[J]

 SET TEMP[INDEX] = ARR[I]

 SET I = I + 1

 ELSE

 SET TEMP[INDEX] = ARR[J]

 SET J = J + 1

 [END OF IF]

 SET INDEX = INDEX + 1

 [END OF LOOP]

Step 3: [Copy the remaining elements of right sub-array, if any]

 IF I > MID

 Repeat while J <= END

 SET TEMP[INDEX] = ARR[J]

 SET INDEX = INDEX + 1, SET J = J + 1

 [END OF LOOP]

 [Copy the remaining elements of left sub-array, if any]

 ELSE

 Repeat while I <= MID

 SET TEMP[INDEX] = ARR[I]

 SET INDEX = INDEX + 1, SET I = I + 1

 [END OF LOOP]

 [END OF IF]

Step 4: [Copy the contents of TEMP back to ARR] SET K=0

Step 5: Repeat while K < INDEX

 SET ARR[K] = TEMP[K]

 SET K = K + 1

 [END OF LOOP]

Step 6: END

Merge Sort...

9	39	45	81	18	27	72	90
BEG, I			MID	J			END

TEMP							
9							
INDEX							

9	39	45	81	18	27	72	90
BEG	I		MID	J			END

TEMP							
9	18						
INDEX							

9	39	45	81	18	27	72	90
BEG	I		MID		J		END

9	18	27					
INDEX							

9	39	45	81	18	27	72	90
BEG	I		MID			J	END

9	18	27	39				
INDEX							

9	39	45	81	18	27	72	90
BEG		I	MID			J	END

9	18	27	39	45			
INDEX							

9	39	45	81	18	27	72	90
BEG			I, MID			J	END

9	18	27	39	45	72		
INDEX							

9	39	45	81	18	27	72	90
BEG			I, MID			J	END

9	18	27	39	45	72	81	
INDEX							

9	39	45	81	18	27	72	90
BEG			MID	I		J	END

9	18	27	39	45	72	81	90
INDEX							

Quick Sort...

Technique

Quick sort works as follows:

1. Set the index of the first element in the array to loc and left variables. Also, set the index of the last element of the array to the right variable.

That is, $loc = 0$, $left = 0$, and $right = n-1$ (where n is the number of elements in the array)

2. Start from the element pointed by $right$ and scan the array from right to left, comparing each element on the way with the element pointed by the variable loc .

That is, $a[loc]$ should be less than $a[right]$.

- (a) If that is the case, then simply continue comparing until $right$ becomes equal to loc . Once $right = loc$, it means the pivot has been placed in its correct position.

- (b) However, if at any point, we have $a[loc] > a[right]$, then interchange the two values and jump to Step 3.

- (c) Set $loc = right$

3. Start from the element pointed by $left$ and scan the array from left to right, comparing each element on the way with the element pointed by loc .

That is, $a[loc]$ should be greater than $a[left]$.

- (a) If that is the case, then simply continue comparing until $left$ becomes equal to loc . Once $left = loc$, it means the pivot has been placed in its correct position.

- (b) However, if at any point, we have $a[loc] < a[left]$, then interchange the two values and jump to Step 2.

- (c) Set $loc = left$.

Quick Sort...

27	10	36	18	25	45
----	----	----	----	----	----

We choose the first element as the pivot.
Set $loc = 0$, $left = 0$ and $right = 5$.

27	10	36	18	25	45
----	----	----	----	----	----

loc $right$
 $left$

Scan from right to left. Since $a[loc] < a[right]$, decrease the value of $right$.

27	10	36	18	25	45
----	----	----	----	----	----

loc $right$
 $left$

Start scanning from left to right. Since $a[loc] > a[left]$, increment the value of $left$.

25	10	36	18	27	45
----	----	----	----	----	----

$right$
 $left$ loc

Since $a[loc] > a[right]$, interchange the two values and set $loc = right$.

25	10	36	18	27	45
----	----	----	----	----	----

$left$ $right$
 loc

Since $a[loc] < a[left]$, interchange the values and set $loc = left$.

25	10	27	18	36	45
----	----	----	----	----	----

$left$ $right$
 loc

Scan from right to left. Since $a[loc] < a[right]$, decrement the value of $right$.

25	10	27	18	36	45
----	----	----	----	----	----

$left$ $right$
 loc

Since $a[loc] > a[right]$, interchange the two values and set $loc = right$.

25	10	18	27	36	45
----	----	----	----	----	----

$left$ $right$
 loc

Start scanning from left to right. Since $a[loc] > a[left]$, increment the value of $left$.

25	10	18	27	36	45
----	----	----	----	----	----

$right$
 loc
 $left$

Quick Sort...

```
Step 1: [INITIALIZE] SET LEFT = BEG, RIGHT = END, LOC = BEG, FLAG = 0
Step 2: Repeat Steps 3 to 6 while FLAG = 0
Step 3: Repeat while ARR[LOC] <= ARR[RIGHT] AND LOC != RIGHT
        SET RIGHT = RIGHT - 1
    [END OF LOOP]
Step 4: IF LOC = RIGHT
        SET FLAG = 1
    ELSE IF ARR[LOC] > ARR[RIGHT]
        SWAP ARR[LOC] with ARR[RIGHT]
        SET LOC = RIGHT
    [END OF IF]
Step 5: IF FLAG = 0
        Repeat while ARR[LOC] >= ARR[LEFT] AND LOC != LEFT
        SET LEFT = LEFT + 1
    [END OF LOOP]
Step 6: IF LOC = LEFT
        SET FLAG = 1
    ELSE IF ARR[LOC] < ARR[LEFT]
        SWAP ARR[LOC] with ARR[LEFT]
        SET LOC = LEFT
    [END OF IF]
[END OF IF]
Step 7: [END OF LOOP]
Step 8: END
```

QUICK_SORT (ARR, BEG, END)

```
Step 1: IF (BEG < END)
        CALL PARTITION (ARR, BEG, END, LOC)
        CALL QUICKSORT(ARR, BEG, LOC - 1)
        CALL QUICKSORT(ARR, LOC + 1, END)
    [END OF IF]
Step 2: END
```

UNIT - V

Hashing-Hash table, Hash table representations, hash functions, collision resolution techniques- separate chaining, open addressing-linear probing, quadratic probing, double hashing, Re hashing, Extendible hashing,

Pattern matching : Introduction, Brute force, the Boyer –Moore algorithm, Knuth-Morris-Pratt algorithm.

Hashing: Hashing is a technique to convert a range of key values into a range of indexes of an array. Hashing techniques is implemented using hash function and hash table.

Hash Table: It is data structure which contains index and associated data.

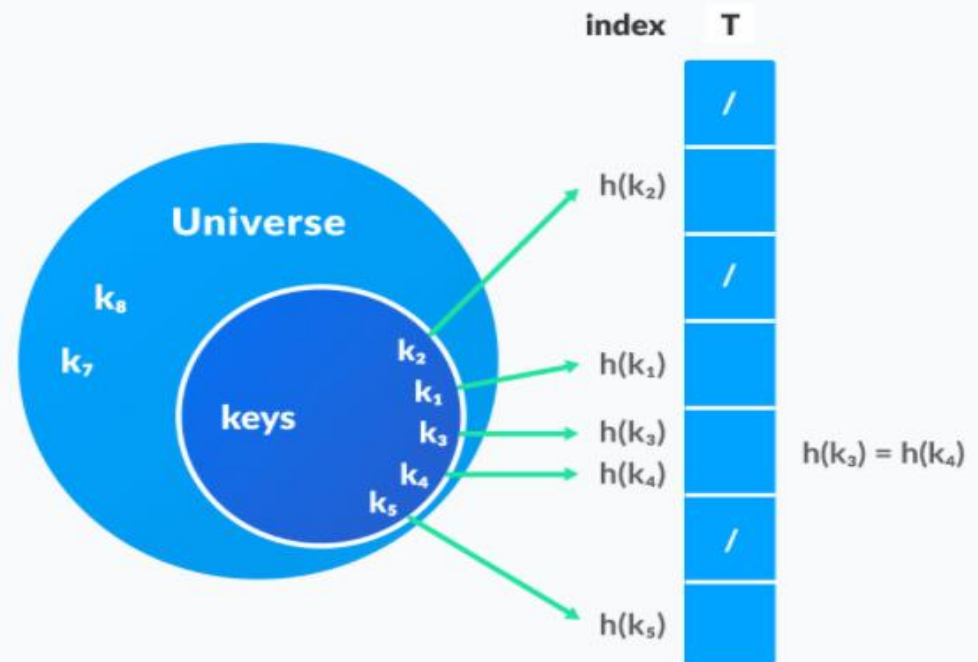
Access of data becomes very fast if we know the index of the desired data.

Hash Function: It is a function which is used to map the key to a hash value.

It is represented as $h(x)$.

Collision: If the same index or hash value is produced by the hash function for multiple keys then, conflict arises.

This situation is called collision.



Hashing...

Types of hash functions:

➤ Division method

It is the most simple method of hashing an integer x . This method divides x by m and then uses the remainder obtained as hash value. In this case, the hash function can be given as

$$h(x) = x \bmod m.$$

It requires only a single division operation, therefore this method works very fast.

Example:

calculate the hash values of keys 1234 and 5462, where $m=97$.

$$h(1234) = 1234 \% 97 = 70, \quad h(5642) = 5642 \% 97 = 16$$

➤ Multiplication method

The steps involved in the multiplication method are as follows:

Step 1: choose a constant a such that $0 < a < 1$.

Step 2: multiply the key k by a .

Step 3: extract the fractional part of ka .

Step 4: multiply the result of step 3 by the size of hash table (m).

Hence, the hash function can be given as:

$h(k) = m (ka \bmod 1)$ where, $(ka \bmod 1)$ gives the fractional part of ka and m is the total number of indices in the hash table.

Hash Function Types...

Example:

Given a hash table of size 1000, map the key 12345 to an appropriate location in the hash table.

we will use $a = 0.618033$, $m = 1000$, and $k = 12345$

$$h(12345) = 1000 (12345 * 0.618033 \bmod 1)$$

$$= 1000 (7629.617385 \bmod 1)$$

$$= 1000 (0.617385)$$

$$= 617.385$$

$$= 617$$

➤Mid Square Method:

The mid-square method is a good hash function which works in two steps:

Step 1: square the value of the key. That is, find k^2 .

Step 2: extract the middle r digits of the result obtained in step 1.

In the mid-square method, the same r digits must be chosen from all the keys. Therefore, the hash function can be given as:

$H(k) = s$, Where s is obtained by selecting r digits from k^2 .

.

Hash Function Types...

Example:

calculate the hash value for keys 1234 and 5642 using the mid-square method.

The hash table has 100 memory locations.

Note that the hash table has 100 memory locations whose indices vary from 0 to 99.

This means that only two digits are needed to map the key to a location in the hash table, so $r = 2$.

When $k = 1234$, $k^2 = 1522756$, $h(1234) = 27$

When $k = 5642$, $k^2 = 31832164$, $h(5642) = 21$

Observe that the 3rd and 4th digits starting from the right are chosen.

Folding method :

The folding method works in the following two steps:

Step 1: divide the key value into a number of parts. That is, divide k into parts $k_1, k_2, \dots, k_n$, where each part has the same number of digits except the last part which may have lesser digits than the other parts.

Step 2: add the individual parts. That is, obtain the sum of $k_1 + k_2 + k_3 + \dots + k_n$. This hash value produced by ignoring the last carry, if any.

Hash FunctionTypes...

Example:

Given a hash table of 100 locations, calculate the hash value using folding method for keys 5678, 321, and 34567.

Since there are 100 memory locations to address, we will break the key into parts where each part (except the last) will contain two digits. The hash values can be obtained as shown below:

Key	5678	321	34567
Parts	56 and 78	32 and 1	34, 56 and 7
Sum	134	33	97
Hash value	34 (ignore the last carry)	33	97

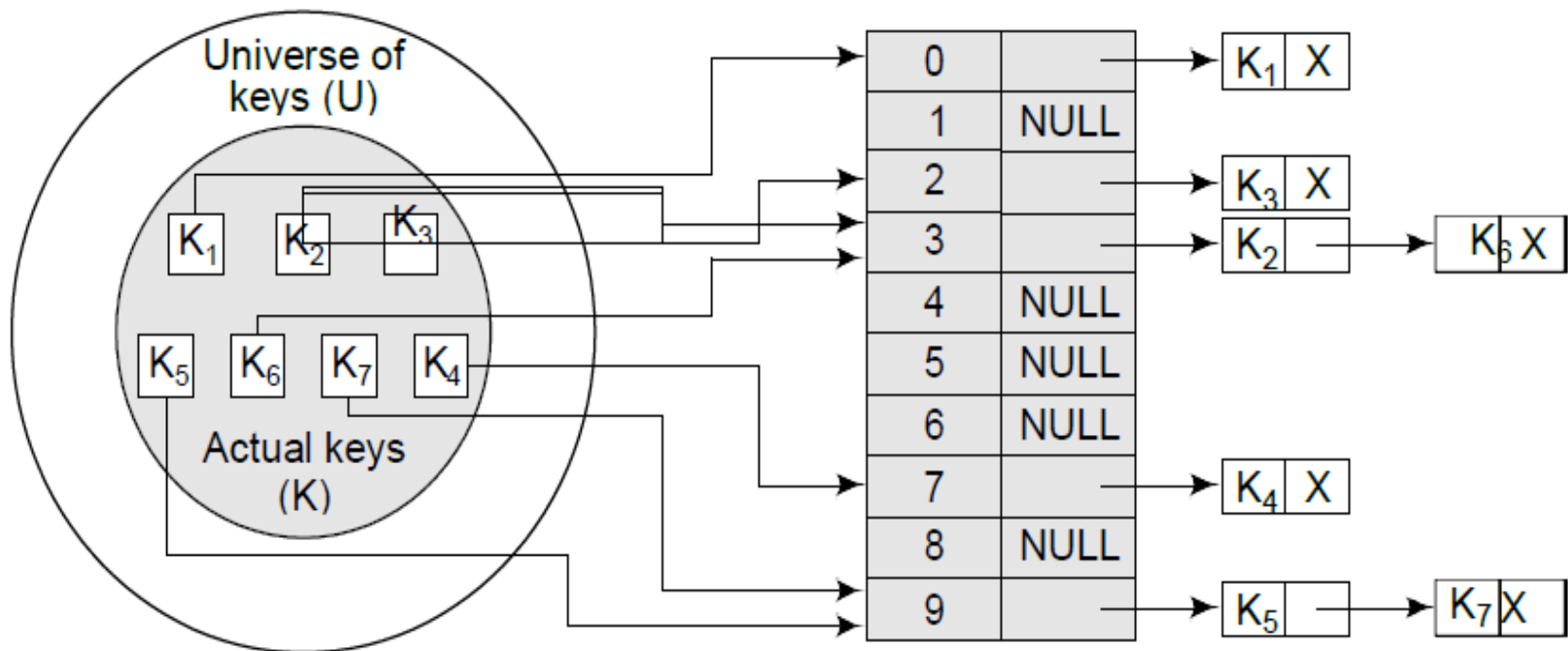
Collision Resolution Techniques

The collision resolution techniques are :

1. Separate chaining
2. Open addressing

Separate chaining:

- In this technique, if a hash function produces the same index for multiple elements, these elements are stored in the same index by using a linked list.
- if no element is hashed to a particular index then it will contain NULL.



Collision Resolution Techniques...

Insert the keys 7, 24, 18, 52, 36, 54, 11, and 23 in a chained hash table of 9 memory locations. Use hash function $h(k) = k \bmod m$.

In this case, $m=9$.

0	NULL
1	NULL
2	NULL
3	NULL
4	NULL
5	NULL
6	NULL
7	NULL
8	NULL

Insert 7

Key = 7
 $H(k) = 7 \bmod 9$
 $= 7$

0	NULL
1	NULL
2	NULL
3	NULL
4	NULL
5	NULL
6	NULL
7	→ [7 X]
8	NULL

Insert 24

Key = 24
 $H(k) = 24 \bmod 9$
 $= 6$

0	NULL
1	NULL
2	NULL
3	NULL
4	NULL
5	NULL
6	→ [24 X]
7	→ [7 X]
8	NULL

Insert 18

key = 18
 $H(k) = 18 \bmod 9 = 0$

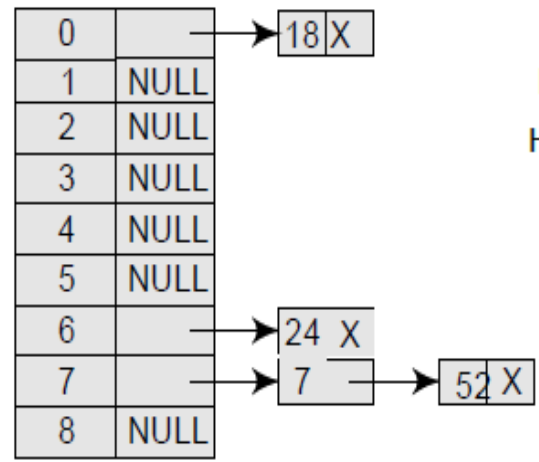
0	→ [18 X]
1	NULL
2	NULL
3	NULL
4	NULL
5	NULL
6	→ [24 X]
7	→ [7 X]
8	NULL

Collision Resolution Techniques...

Insert the keys 7, 24, 18, 52, 36, 54, 11, and 23 in a chained hash table of 9 memory locations. Use hash function $h(k) = k \bmod m$.

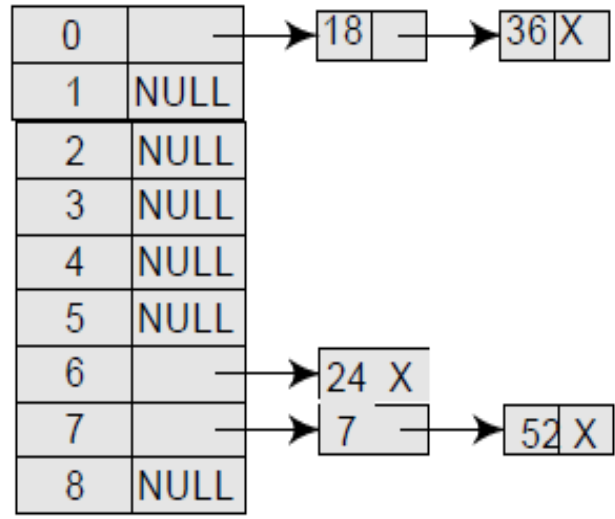
Insert 52

key = 52
 $h(k) = 52 \bmod 9 = 7$

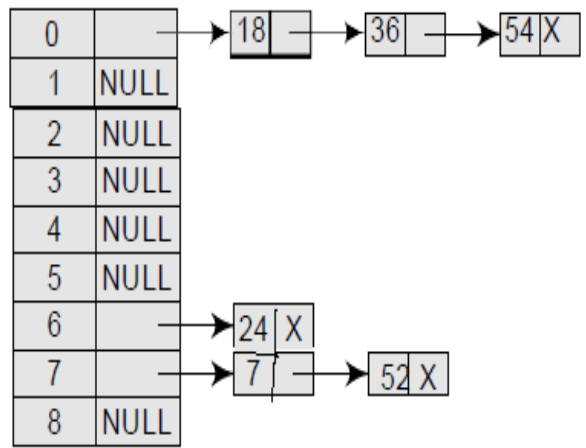


Insert 36

key = 36
 $H(k) = 36 \bmod 9 = 0$

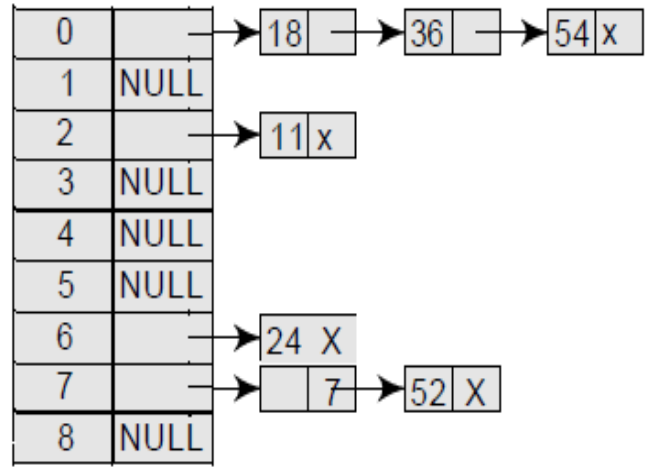


Insert 54



Insert 11

key = 11
 $H(k) = 11 \bmod 9 = 2$

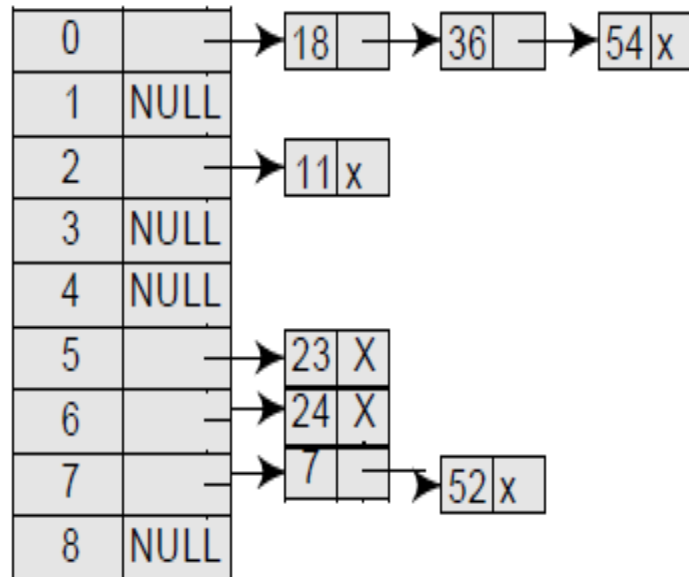


Collision Resolution Techniques...

Insert the keys 7, 24, 18, 52, 36, 54, 11, and 23 in a chained hash table of 9 memory locations. Use hash function $h(k) = k \bmod m$.

Insert 23

key = 23
 $h(k) = 23 \bmod 9 = 5$



Collision Resolution Techniques...

Open Addressing:

- In this technique, the hash table contains two types of values: sentinel values (e.g., -1) and data values. The presence of a sentinel value indicates that the location contains no data value at present but can be used to hold a value.
- When a key is mapped to a particular memory location, then the value it holds is checked. If it contains a sentinel value, then the location is free and the data value can be stored in it.
- If the location already has some data value stored in it, then other slots are examined systematically in the forward direction to find a free slot. If even a single free location is not found, then we have an **overflow condition**.
- The process of examining memory locations in the hash table is called **probing**.
- Open addressing technique can be implemented using:
 1. **Linear probing**
 2. **Quadratic probing**
 3. **Double hashing**
 4. **Rehashing.**

Collision Resolution Techniques...

Linear probing:

➤ The simplest approach to resolve a collision is linear probing. In this technique, if a value is already stored at a location generated by $h(k)$, then the following hash function is used to resolve the collision:

$$H(k, i) = [h(k) + i] \bmod m$$

Where m is the size of the hash table, $h(k) = (k \bmod m)$, and i is the probe number that varies from 0 to $m-1$.

Example: Consider a hash table of size 10. Using linear probing, insert the keys 72, 27, 36, 24, 63, 81, 92 into the table.

Let $h(k) = k \bmod m$, $m = 10$, Initially, the hash table can be given as:

0	1	2	3	4	5	6	7	8	9
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

Step 1 key = 72

$$\begin{aligned} H(72, 0) &= (72 \bmod 10 + 0) \bmod 10 \\ &= (2) \bmod 10 \\ &= 2 \end{aligned}$$

Since $t[2]$ is vacant, insert key 72 at this location.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	-1	-1	-1

Linear Probing...

Example: Consider a hash table of size 10. Using linear probing, insert the keys 72, 27, 36, 24, 63, 81, 92 into the table.

Let $h(k) = k \bmod m$, $m = 10$

Step 2 key = 27

$$\begin{aligned} H(27, 0) &= (27 \bmod 10 + 0) \bmod 10 \\ &= (7) \bmod 10 \\ &= 7 \end{aligned}$$

Since $t[7]$ is vacant, insert key 27 at this location.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	27	-1	-1

Step 3 key = 36

$$\begin{aligned} H(36, 0) &= (36 \bmod 10 + 0) \bmod 10 \\ &= (6) \bmod 10 \\ &= 6 \end{aligned}$$

Since $t[6]$ is vacant, insert key 36 at this location.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	36	27	-1	-1

Linear Probing...

Example: Consider a hash table of size 10. Using linear probing, insert the keys 72, 27, 36, 24, 63, 81, 92 into the table.

Let $h(k) = k \bmod m$, $m = 10$

Step 4 key = 24

$$\begin{aligned} H(24, 0) &= (24 \bmod 10 + 0) \bmod 10 \\ &= (4) \bmod 10 \\ &= 4 \end{aligned}$$

Since $t[4]$ is vacant, insert key 24 at this location.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	24	-1	36	27	-1	-1

Step 5 key = 63

$$\begin{aligned} H(63, 0) &= (63 \bmod 10 + 0) \bmod 10 \\ &= (3) \bmod 10 \\ &= 3 \end{aligned}$$

Since $t[3]$ is vacant, insert key 63 at this location.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	63	24	-1	36	27	-1	-1

Linear Probing...

Example: Consider a hash table of size 10. Using linear probing, insert the keys 72, 27, 36, 24, 63, 81, 92 into the table.

Let $h(k) = k \bmod m$, $m = 10$

Step 6 key = 81

$$\begin{aligned} H(81, 0) &= (81 \bmod 10 + 0) \bmod 10 \\ &= (1) \bmod 10 \\ &= 1 \end{aligned}$$

Since $t[1]$ is vacant, insert key 81 at this location.

0	1	2	3	4	5	6	7	8	9
0	81	72	63	24	-1	36	27	-1	-1

Step 7 key = 92

$$\begin{aligned} H(92, 0) &= (92 \bmod 10 + 0) \bmod 10 \\ &= (2) \bmod 10 \\ &= 2 \end{aligned}$$

Now $t[2]$ is occupied, so we cannot store the key 92 in $t[2]$. Therefore, try again for the next location. Thus probe, $i = 1$, this time.

Key = 92

$$\begin{aligned} H(92, 1) &= (92 \bmod 10 + 1) \bmod 10 \\ &= (2 + 1) \bmod 10 \\ &= 3 \end{aligned}$$

Linear Probing...

Example: Consider a hash table of size 10. Using linear probing, insert the keys 72, 27, 36, 24, 63, 81, 92 into the table.

Let $h(k) = k \bmod m$, $m = 10$

Now $t[3]$ is occupied, so we cannot store the key 92 in $t[3]$. Therefore, try again for the next location. Thus probe, $i = 2$, this time.

Key = 92

$$\begin{aligned} H(92, 2) &= (92 \bmod 10 + 2) \bmod 10 \\ &= (2 + 2) \bmod 10 \\ &= 4 \end{aligned}$$

Now $t[4]$ is occupied, so we cannot store the key 92 in $t[4]$. Therefore, try again for the next location. Thus probe, $i = 3$, this time.

Key = 92

$$\begin{aligned} H(92, 3) &= (92 \bmod 10 + 3) \bmod 10 \\ &= (2 + 3) \bmod 10 \\ &= 5 \end{aligned}$$

Since $t[5]$ is vacant, insert key 92 at this location.

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	92	36	27	-1	-1

Linear Probing...

Advantages:

- Easy to compute.

Disadvantages:

- Table must be big enough to get a free cell.
- Time to get a free cell may be quite large.
- Primary Clustering: Any key that hashes into the cluster will require several attempts to resolve the collision.

Quadratic probing

Quadratic probing:

- In this technique, if a value is already stored at a location generated by $h(k)$, then the following hash function is used to resolve the collision:

$$H(k, i) = [h(k) + c_1 i + c_2 i^2] \bmod m$$

Where,

m is the size of the hash table,

$h(k) = (k \bmod m)$,

i is the probe number that varies from 0 to $M-1$, and

c_1 and c_2 are constants.

- Quadratic probing eliminates the primary clustering phenomenon of linear probing because instead of doing a linear search, it does a quadratic search.

Example: Consider a hash table of size 10. Using quadratic probing, insert the keys 72, 27, 36, 24, 63, 81, and 101 into the table. Take $c_1 = 1$ and $c_2 = 3$.

Let $h(k) = k \bmod m$, $m = 10$

Initially, the hash table can be given as:

0	1	2	3	4	5	6	7	8	9
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

Quadratic probing...

Example: Consider a hash table of size 10. Using quadratic probing, insert the keys 72, 27, 36, 24, 63, 81, and 101 into the table. Take $c_1 = 1$ and $c_2 = 3$.

We have,

$$H(k, i) = [h(k) + c_1 i + c_2 i^2] \bmod m$$

Step 1 key = 72

$$\begin{aligned} H(72, 0) &= [72 \bmod 10 + 1 * 0 + 3 * 0] \bmod 10 \\ &= [72 \bmod 10] \bmod 10 \\ &= 2 \bmod 10 \\ &= 2 \end{aligned}$$

Since $t[2]$ is vacant, insert the key 72 in $t[2]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	-1	-1	-1

Step 2 key = 27

$$\begin{aligned} H(27, 0) &= [27 \bmod 10 + 1 * 0 + 3 * 0] \bmod 10 \\ &= [27 \bmod 10] \bmod 10 \\ &= 7 \bmod 10 \\ &= 7 \end{aligned}$$

Since $t[7]$ is vacant, insert the key 27 in $t[7]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	27	-1	-1

Quadratic probing...

Example: Consider a hash table of size 10. Using quadratic probing, insert the keys 72, 27, 36, 24, 63, 81, and 101 into the table. Take $c = 1$ and $c = 3$.

Step 3 key = 36

$$\begin{aligned} H(36, 0) &= [36 \bmod 10 + 1 * 0 + 3 * 0] \bmod 10 \\ &= [36 \bmod 10] \bmod 10 \\ &= 6 \bmod 10 \\ &= 6 \end{aligned}$$

Since $t[6]$ is vacant, insert the key 36 in $t[6]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	36	27	-1	-1

Step 4 key = 24

$$\begin{aligned} H(24, 0) &= [24 \bmod 10 + 1 * 0 + 3 * 0] \bmod 10 \\ &= [24 \bmod 10] \bmod 10 \\ &= 4 \bmod 10 \\ &= 4 \end{aligned}$$

Since $t[4]$ is vacant, insert the key 24 in $t[4]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	24	-1	36	27	-1	-1

Quadratic probing...

Example: Consider a hash table of size 10. Using quadratic probing, insert the keys 72, 27, 36, 24, 63, 81, and 101 into the table. Take $c = 1$ and $c = 3$.

Step 5 key = 63

$$\begin{aligned} H(63, 0) &= [63 \bmod 10 + 1 * 0 + 3 * 0] \bmod 10 \\ &= [63 \bmod 10] \bmod 10 \\ &= 3 \bmod 10 \\ &= 3 \end{aligned}$$

Since $t[3]$ is vacant, insert the key 63 in $t[3]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	63	24	-1	36	27	-1	-1

Step 6 key = 81

$$\begin{aligned} H(81, 0) &= [81 \bmod 10 + 1 * 0 + 3 * 0] \bmod 10 \\ &= [81 \bmod 10] \bmod 10 \\ &= 81 \bmod 10 \\ &= 1 \end{aligned}$$

Since $t[1]$ is vacant, insert the key 81 in $t[1]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	-1	36	27	-1	-1

Quadratic probing...

Example: Consider a hash table of size 10. Using quadratic probing, insert the keys 72, 27, 36, 24, 63, 81, and 101 into the table. Take $c = 1$ and $c = 3$.

Step 7 key = 101

$$\begin{aligned} H(101, 0) &= [101 \bmod 10 + 1 * 0 + 3 * 0] \bmod 10 \\ &= [101 \bmod 10 + 0] \bmod 10 \\ &= 1 \bmod 10 \\ &= 1 \end{aligned}$$

Since $t[1]$ is already occupied, the key 101 cannot be stored in $t[1]$. Therefore, try again for next location. Thus probe, $i = 1$, this time.

Key = 101

$$\begin{aligned} H(101, 0) &= [101 \bmod 10 + 1 * 1 + 3 * 1] \bmod 10 \\ &= [101 \bmod 10 + 1 + 3] \bmod 10 \\ &= [101 \bmod 10 + 4] \bmod 10 \\ &= [1 + 4] \bmod 10 \\ &= 5 \bmod 10 \\ &= 5 \end{aligned}$$

Since $t[5]$ is vacant, insert the key 101 in $t[5]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	101	36	27	-1	-1

Quadratic probing...

Advantage:

- Eliminates Primary Clustering problem.

Disadvantage:

- Secondary Clustering: Elements that hash to the same position will probe the same alternative cells.

Double Hashing:

- In double hashing, we use two hash functions rather than a single function. The hash function in the case of double hashing can be given as:

$$H(k, i) = [h_1(k) + i h_2(k)] \bmod m$$

- Where m is the size of the hash table, $h_1(k)$ and $h_2(k)$ are two hash functions given as $h_1(k) = k \bmod m$ and $h_2(k) = k \bmod m'$, i is the probe number that varies from 0 to $m-1$, and m' is chosen to be less than m . We can choose $m' = m-1$ or $m-2$.

Advantage:

- Double hashing minimizes repeated collisions and the effects of clustering. That is, double hashing is free from problems associated with primary clustering as well as secondary clustering.

Disadvantage:

- Implementation is difficult as it uses two hash functions.

Double Hashing...

Example : consider a hash table of size = 10. Using double hashing, insert the keys 72, 27, 36, 24, 63, 81, 92 into the table. Take $h_1 = (k \bmod 10)$ and $h_2 = (k \bmod 8)$.

Let $m = 10$

Initially, the hash table can be given as:

0	1	2	3	4	5	6	7	8	9
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

We have,

$$H(k, i) = [h_1(k) + ih_2(k)] \bmod m$$

Step 1 key = 72

$$\begin{aligned} H(72, 0) &= [72 \bmod 10 + (0 * 72 \bmod 8)] \bmod 10 \\ &= [2 + (0 * 0)] \bmod 10 \\ &= 2 \bmod 10 \\ &= 2 \end{aligned}$$

Since $t[2]$ is vacant, insert the key 72 in $t[2]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	-1	-1	-1

Step 2 key = 27

$$\begin{aligned} H(27, 0) &= [27 \bmod 10 + (0 * 27 \bmod 8)] \bmod 10 \\ &= [7 + (0 * 3)] \bmod 10 \\ &= 7 \bmod 10 \\ &= 7 \end{aligned}$$

Since $t[7]$ is vacant, insert the key 27 in $t[7]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	27	-1	-1

Double Hashing...

Example : consider a hash table of size = 10. Using double hashing, insert the keys 72, 27, 36, 24, 63, 81, 92 into the table. Take $h_1 = (k \bmod 10)$ and $h_2 = (k \bmod 8)$.

Step 3

key = 36

$$\begin{aligned} H(36, 0) &= [36 \bmod 10 + (0 * 36 \bmod 8)] \bmod 10 \\ &= [6 + (0 * 4)] \bmod 10 \\ &= 6 \bmod 10 \\ &= 6 \end{aligned}$$

Since $t[6]$ is vacant, insert the key 36 in $t[6]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	36	27	-1	-1

Step 4

key = 24

$$\begin{aligned} H(24, 0) &= [24 \bmod 10 + (0 * 24 \bmod 8)] \bmod 10 \\ &= [4 + (0 * 0)] \bmod 10 \\ &= 4 \bmod 10 \\ &= 4 \end{aligned}$$

Since $t[4]$ is vacant, insert the key 24 in $t[4]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	24	-1	36	27	-1	-1

Double Hashing...

Example : consider a hash table of size = 10. Using double hashing, insert the keys 72, 27, 36, 24, 63, 81, 92 into the table. Take $h_1 = (k \bmod 10)$ and $h_2 = (k \bmod 8)$.

Step 5 key = 63

$$\begin{aligned} H(63, 0) &= [63 \bmod 10 + (0 * 63 \bmod 8)] \bmod 10 \\ &= [3 + (0 * 7)] \bmod 10 \\ &= 3 \bmod 10 \\ &= 3 \end{aligned}$$

Since $t[3]$ is vacant, insert the key 63 in $t[3]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	63	24	-1	36	27	-1	-1

Step 6 key = 81

$$\begin{aligned} H(81, 0) &= [81 \bmod 10 + (0 * 81 \bmod 8)] \bmod 10 \\ &= [1 + (0 * 1)] \bmod 10 \\ &= 1 \bmod 10 \\ &= 1 \end{aligned}$$

Since $t[1]$ is vacant, insert the key 81 in $t[1]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	-1	36	27	-1	-1

Double Hashing...

Example : consider a hash table of size = 10. Using double hashing, insert the keys 72, 27, 36, 24, 63, 81, 92 into the table. Take $h_1 = (k \bmod 10)$ and $h_2 = (k \bmod 8)$.

Step 7

key = 92

$$\begin{aligned} H(92, 0) &= [92 \bmod 10 + (0 * 92 \bmod 8)] \bmod 10 \\ &= [2 + (0 * 4)] \bmod 10 \\ &= 2 \bmod 10 \\ &= 2 \end{aligned}$$

Now $t[2]$ is occupied, so we cannot store the key 92 in $t[2]$. Therefore, try again for the next location. Thus probe, $i = 1$, this time.

Key = 92

$$\begin{aligned} H(92, 1) &= [92 \bmod 10 + (1 * 92 \bmod 8)] \bmod 10 \\ &= [2 + (1 * 4)] \bmod 10 \\ &= (2 + 4) \bmod 10 \\ &= 6 \bmod 10 \\ &= 6 \end{aligned}$$

Now $t[6]$ is occupied, so we cannot store the key 92 in $t[6]$. Therefore, try again for the next location. Thus probe, $i = 2$, this time.

Key = 92

$$\begin{aligned} H(92, 2) &= [92 \bmod 10 + (2 * 92 \bmod 8)] \bmod 10 \\ &= [2 + (2 * 4)] \bmod 10 \\ &= [2 + 8] \bmod 10 \\ &= 10 \bmod 10 \\ &= 0 \end{aligned}$$

Since $t[0]$ is vacant, insert the key 92 in $t[0]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
92	81	72	63	24	-1	36	27	-1	-1

Rehashing

Rehashing :

- When the hash table becomes nearly full, the number of collisions increases, thereby degrading the performance of insertion and search operations. In such cases, a better option is to create a new hash table with size double of the original hash table.
- All the entries in the original hash table will then have to be moved to the new hash table. This is done by taking each entry, computing its new hash value, and then inserting it in the new hash table. Though rehashing seems to be a simple process, it is quite expensive and must therefore not be done frequently.

Example : Consider the hash table of size 5 given below. The hash function used is $h(x) = x \% 5$. Rehash the entries into to a new hash table.

0	1	2	3	4
	26	31	43	17

Note that the new hash table is of 10 locations, double the size of the original table.

0	1	2	3	4	5	6	7	8	9

Now, rehash the key values from the old hash table into the new one using hash function— $h(x)$
 $= x \% 10$.

0	1	2	3	4	5	6	7	8	9
	31		43			26	17		

Extendible Hashing

Extendible Hashing :

- It is a dynamic hashing method in which directories and buckets are used to hash data.
- It is an aggressively flexible method in which the hash function also experiences dynamic changes.

Terminology used in Extendible hashing:

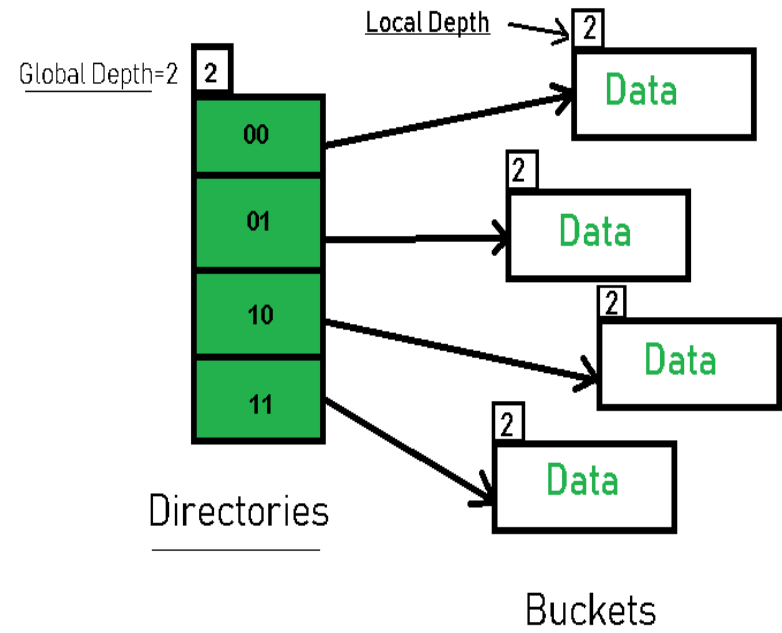
Directories: The directories store addresses of the buckets in pointers. An id is assigned to each directory which may change each time when Directory Expansion takes place.

Buckets: The buckets are used to hash the actual data.

Global Depth: It is associated with the Directories. They denote the number of bits which are used by the hash function to categorize the keys.

Global Depth = Number of bits in directory id.

Local Depth: It is the same as that of Global Depth except for the fact that Local Depth is associated with the buckets and not the directories. Local depth in accordance with the global depth is used to decide the action that to be performed in case an overflow occurs. Local Depth is always less than or equal to the Global Depth.



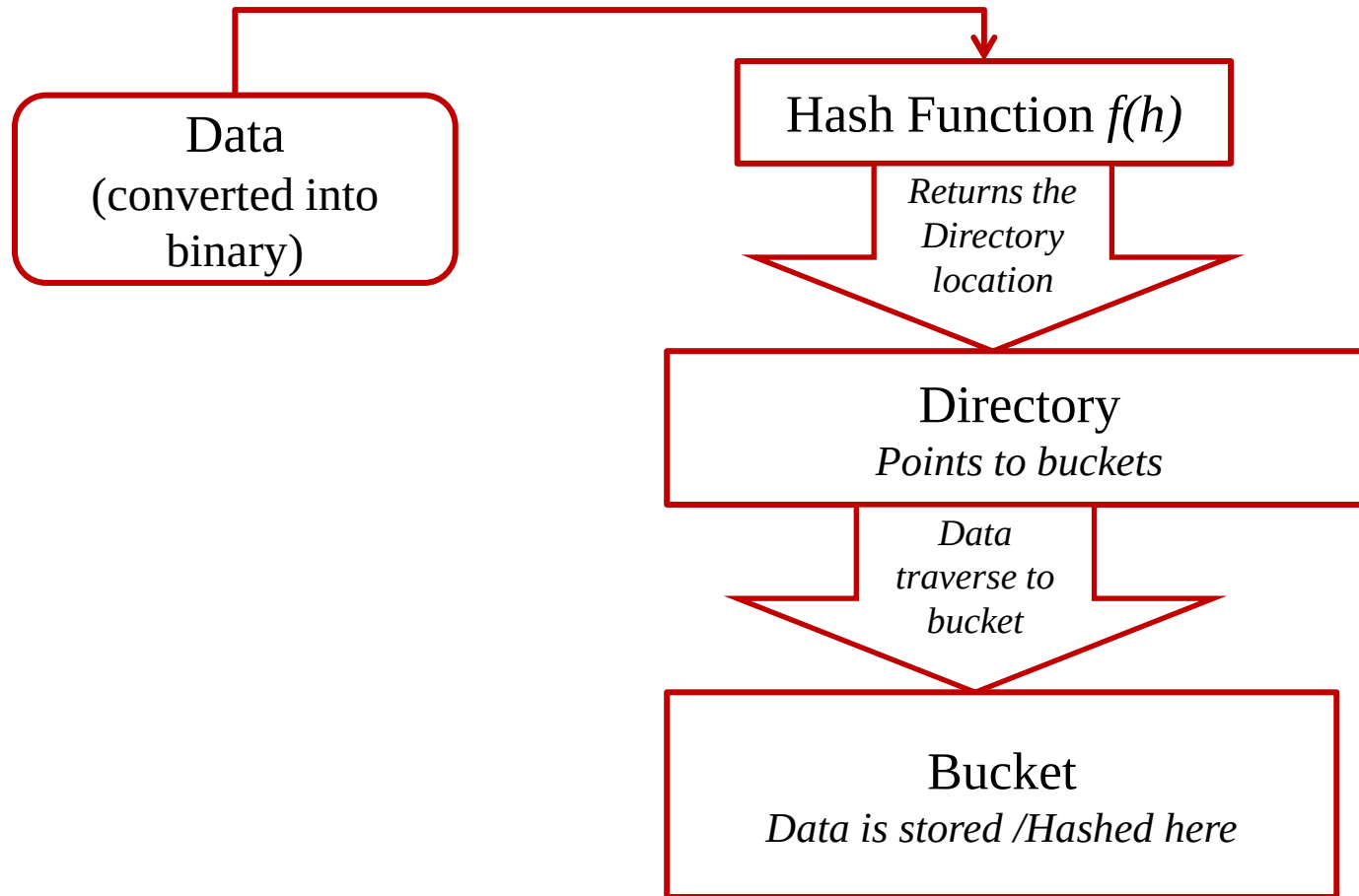
Extendible Hashing

Extendible Hashing...

Bucket Splitting: When the number of elements in a bucket exceeds a particular size, then the bucket is split into two parts.

Directory Expansion: Directory Expansion Takes place when a bucket overflows. Directory Expansion is performed when the local depth of the overflowing bucket is equal to the global depth.

Basic Working of Extendible Hashing:



Extendible Hashing...

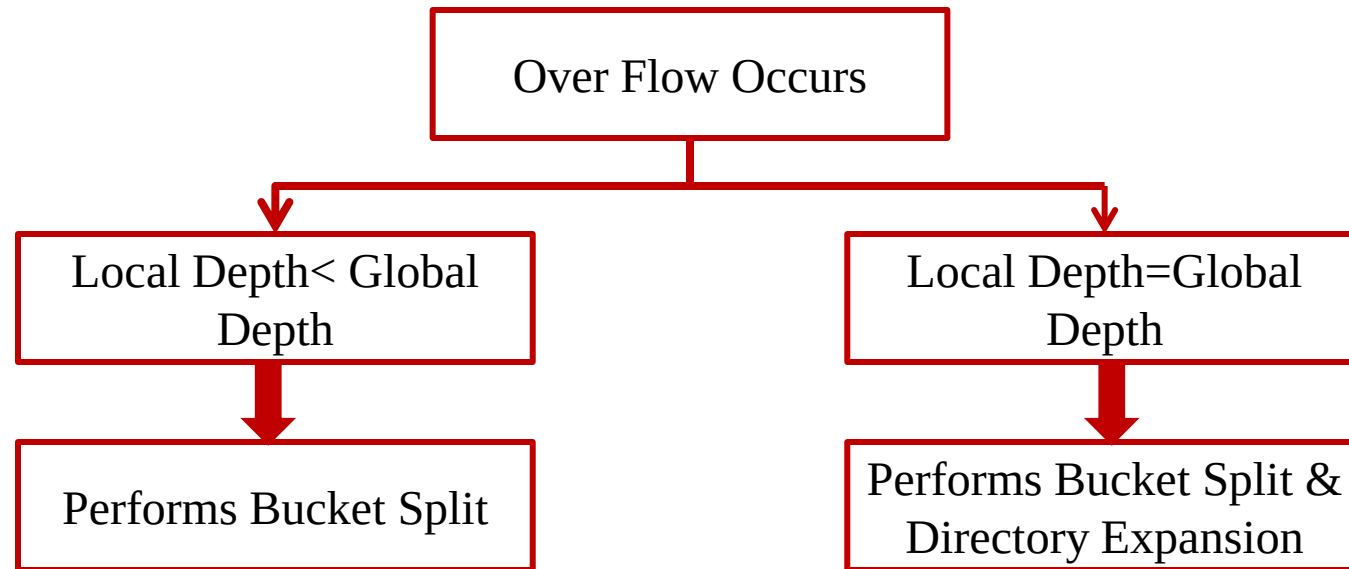
Tackling Over Flow Condition during Data Insertion:

- Many times, while inserting data in the buckets, it might happen that the Bucket overflows. In such cases, we need to follow an appropriate procedure to avoid mishandling of data.
- First, Check if the local depth is less than or equal to the global depth. Then choose one of the cases below.

Case1: If the local depth of the overflowing Bucket is equal to the global depth, then Directory Expansion, as well as Bucket Split, needs to be performed. Then increment the global depth and the local depth value by 1. And, assign appropriate pointers.

Directory expansion will double the number of directories present in the hash structure.

Case2: In case the local depth is less than the global depth, then only Bucket Split takes place. Then increment only the local depth value by 1. And, assign appropriate pointers.

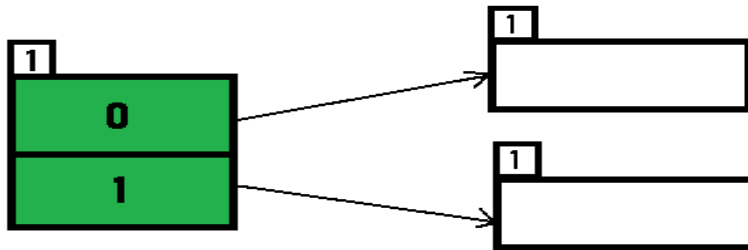


Extendible Hashing...

Example : Hash the following elements: 16,4,6,22,24,10,31,7,9,20,26 using Extendible hashing where bucket size is 3 and Hash Function: Suppose the global depth is X ,then the Hash Function returns X LSBs.

➤ Initially, the global-depth and local-depth is always 1.

Thus, the hashing frame looks like this:

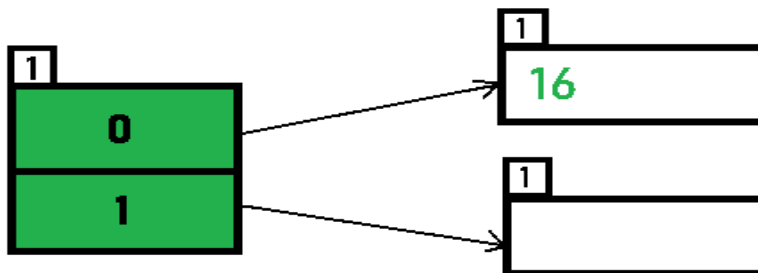


➤ **Insert 16:**

The binary format of 16 is 10000 and global-depth is 1.

The hash function returns 1 LSB of 1000**0** which is 0.

Hence, 16 is mapped to the directory with id=0.



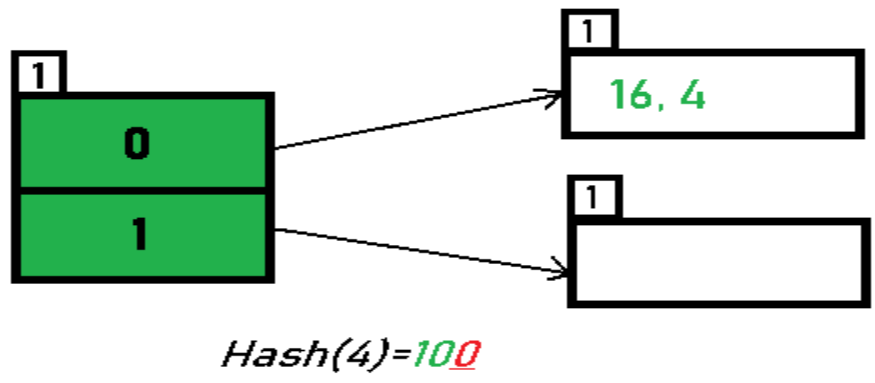
*Hash(16) = 1000**0***

Key	Binary Representation
16	10000
4	00100
6	00110
22	10110
24	11000
10	01010
31	11111
7	00111
9	01001
20	10100
26	01101

Extendible Hashing...

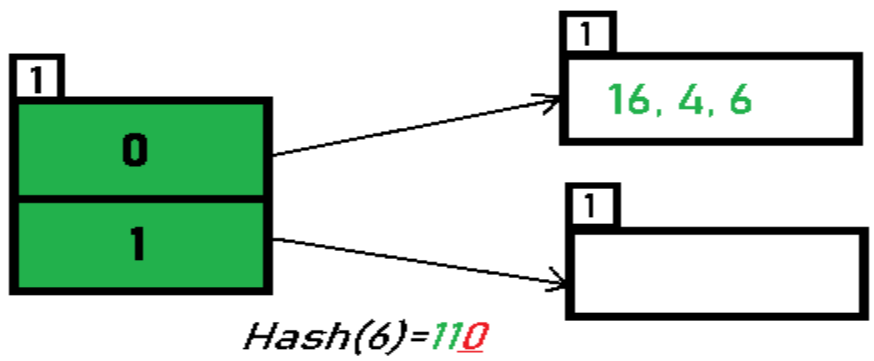
➤Insert 4:

The binary format of 4 is 00100 and global-depth is 1.
The hash function returns 1 LSB of 00100 which is 0.
Hence, 4 is mapped to the directory with id=0.



➤Insert 6:

Hash(6)=00110

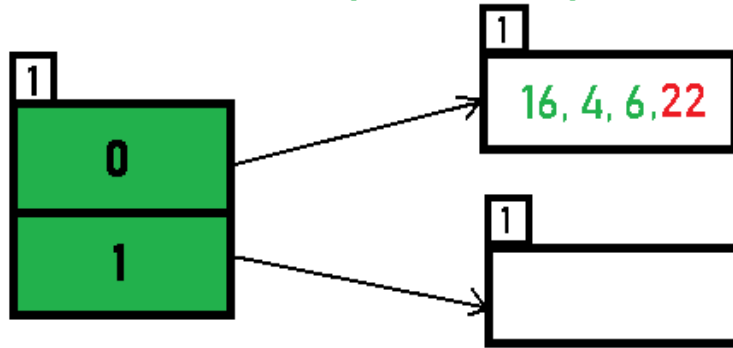


Key	Binary Representation
16	10000
4	00100
6	00110
22	10110
24	11000
10	01010
31	11111
7	00111
9	01001
20	10100
26	01101

Extendible Hashing...

➤ Insert 22

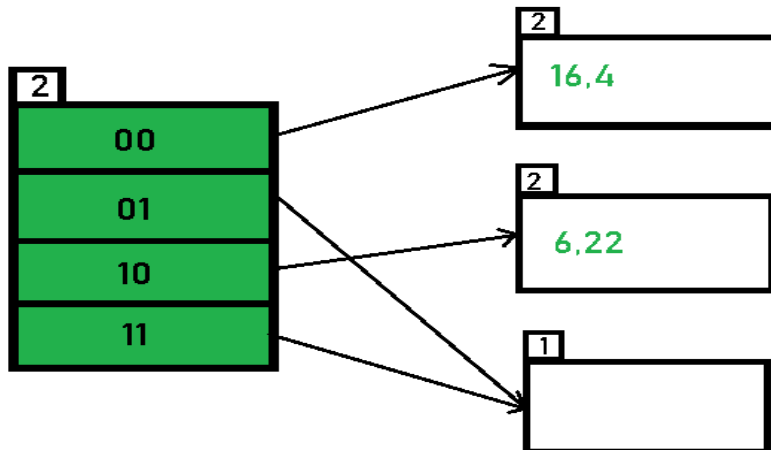
Hash(22)=10110, The bucket pointed by directory 0 is already full. Hence, Over Flow occurs.



Hash(22)=10110

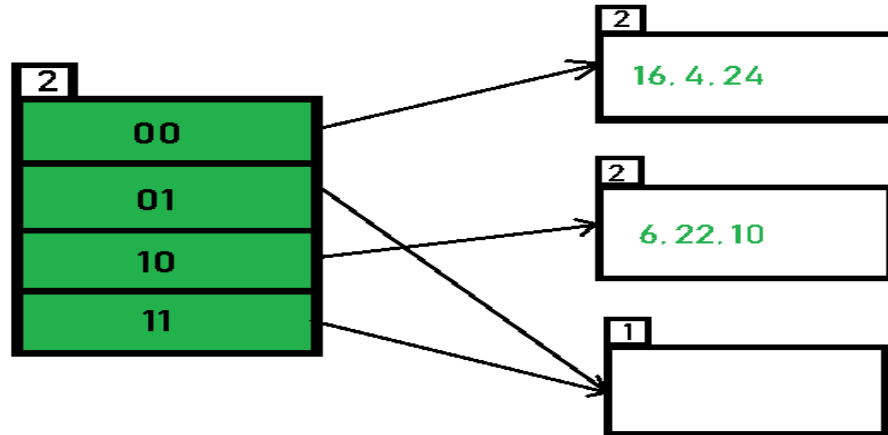
Since Local Depth = Global Depth, the bucket splits and directory expansion takes place. Also, rehashing of numbers present in the overflowing bucket takes place after the split. And, since the global depth is incremented by 1, now, the global depth is 2. Hence, 16, 4, 6, 22 are now rehashed w.r.t 2 LSBs. [16(10000), 4(100), 6(110), 22(10110)]

After Bucket Split and Directory Expansion



Extendible Hashing...

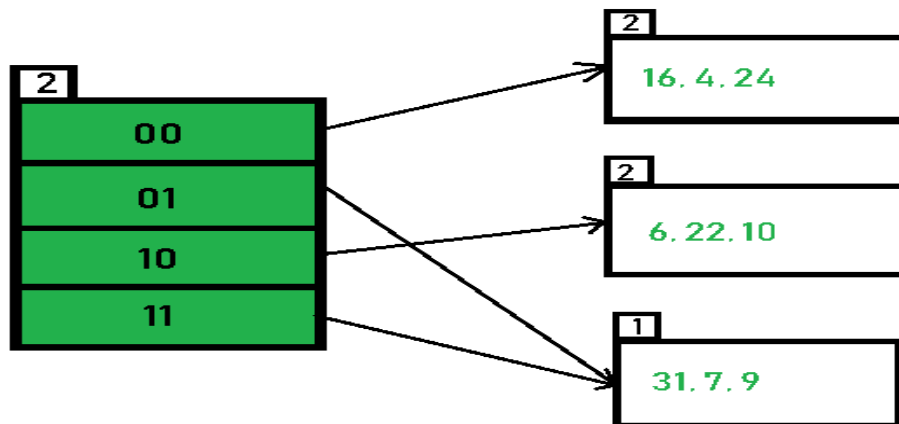
- **Inserting 24 and 10:** 24(110**00**) and 10 (10**10**) can be hashed based on directories with id 00 and 10



*Hash(24) = 110**00***

*Hash(10) = 10**10***

- **Inserting 31, 7, 9:** All of these elements[31(111**11**), 7(1**11**), 9(10**01**)] have either 01 or 11 in their LSBs. Hence, they are mapped on the bucket pointed out by 01 and 11.



*Hash(31) = 111**11***

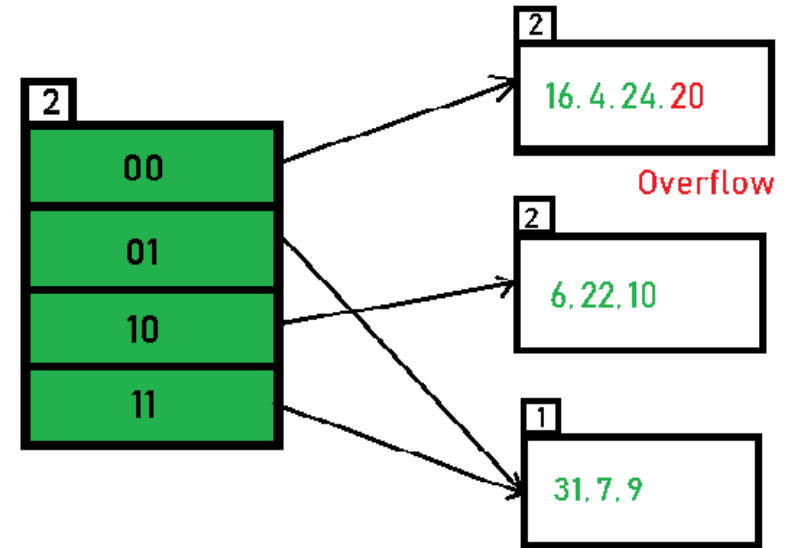
*Hash(7) = 1**11***

*Hash(9) = 10**01***

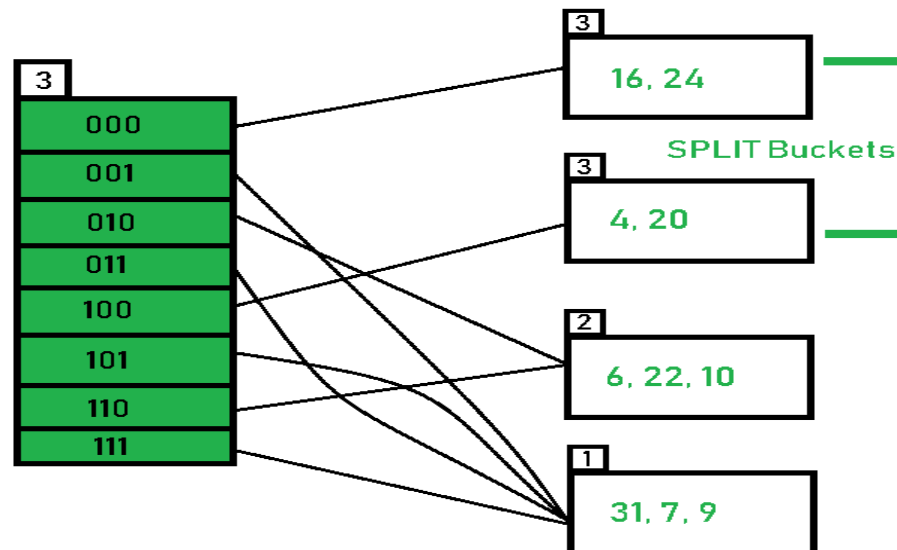
Extendible Hashing...

- **Inserting 20:** Insertion of data element 20 (101**00**) will again cause the overflow problem.
- 20 is inserted in bucket pointed out by 00. since the **local depth of the bucket = global-depth**, directory expansion (doubling) takes place along with bucket splitting. Elements present in overflowing bucket are rehashed with the new global depth..

Overflow, Local Depth=Global Depth

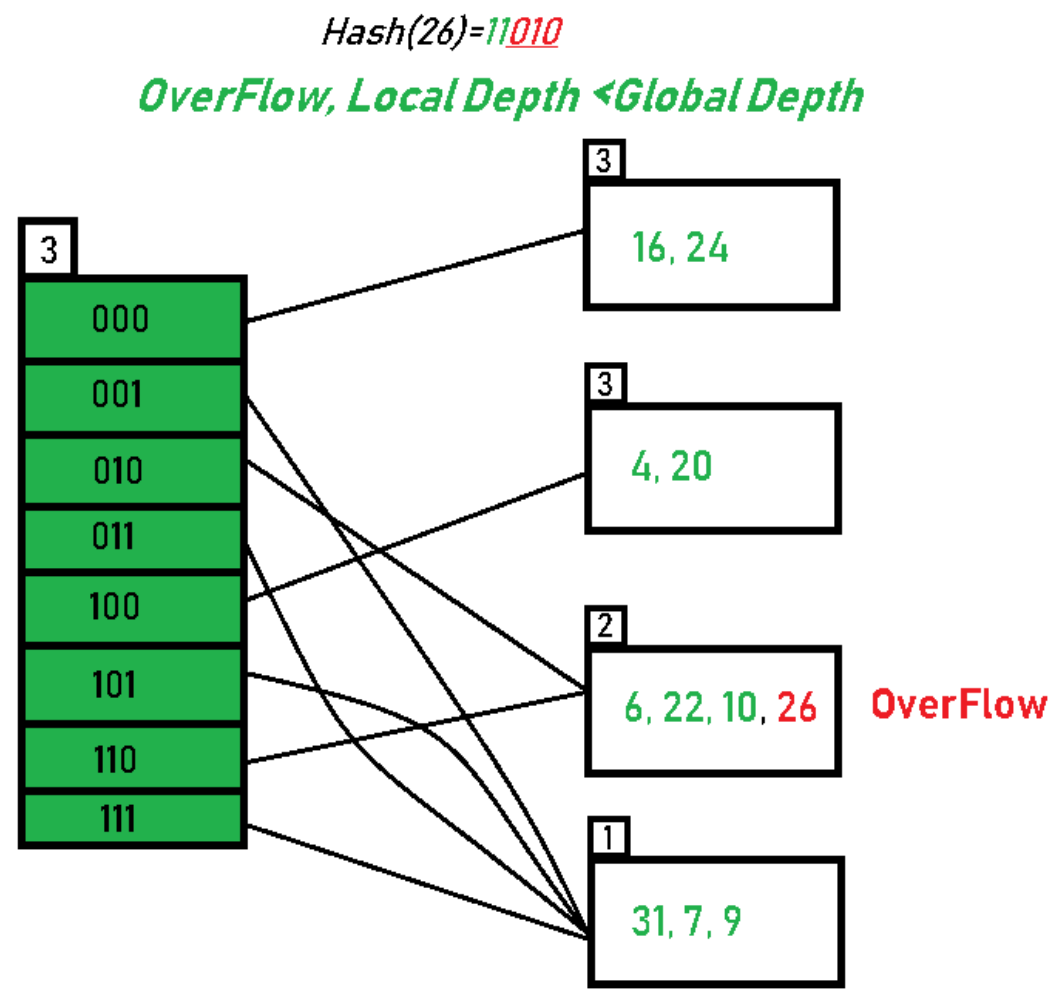


*Hash(20)=101**00***



Extendible Hashing...

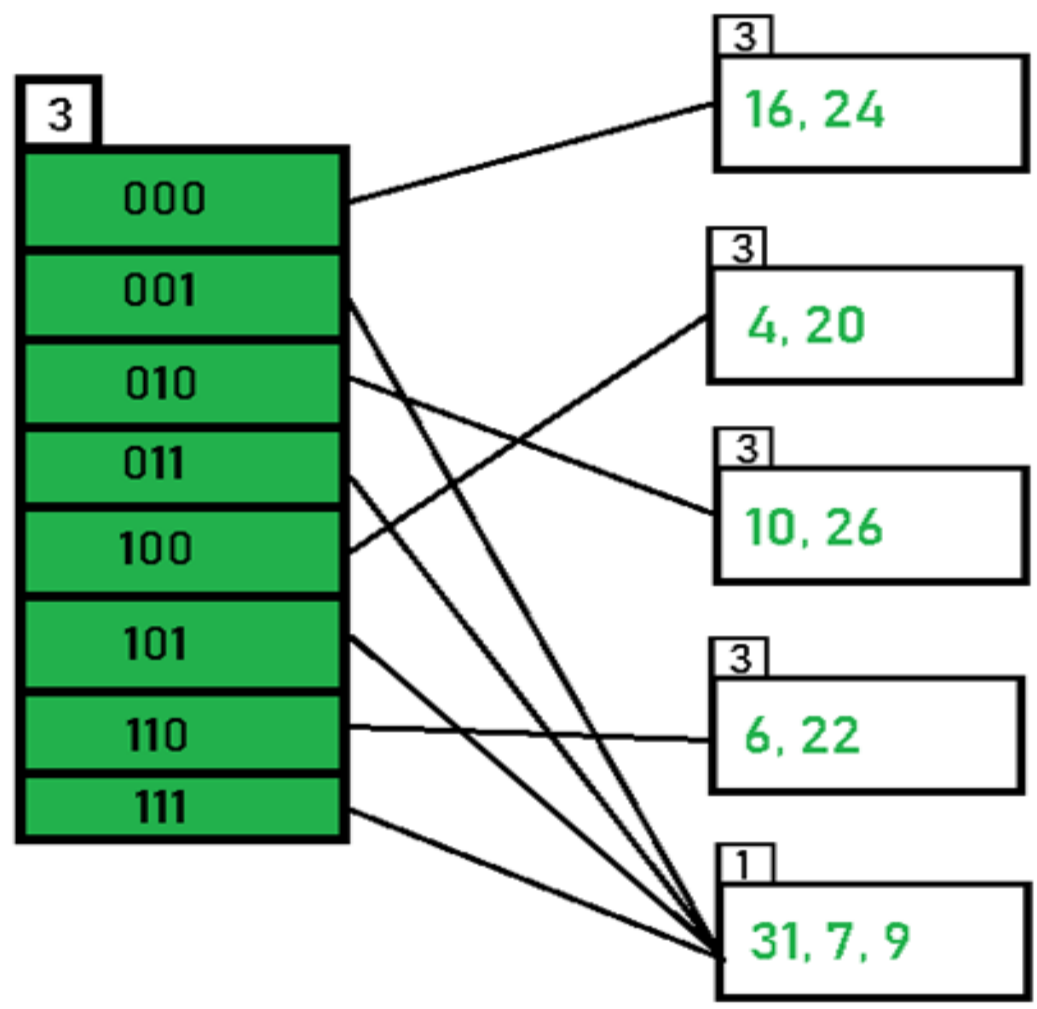
➤ **Inserting 26:** Global depth is 3. Hence, 3 LSBs of 26(11**010**) are considered. Therefore 26 best fits in the bucket pointed out by directory 010.



Key	Binary Representation
16	10000
4	00100
6	00110
22	10110
24	11000
10	01010
31	11111
7	00111
9	01001
20	10100
26	01101

Extendible Hashing...

➤The bucket overflows, and the **local depth of bucket** < **Global depth** ($2 < 3$), directories are not doubled but, only the bucket is split and elements are rehashed. Finally, the output of hashing the given list of numbers is obtained.



Key	Binary Representation
16	10000
4	00100
6	00110
22	10110
24	11000
10	01010
31	11111
7	00111
9	01001
20	10100
26	01101

Pattern Matching

Pattern Matching:

- The Pattern Matching algorithms are also referred as String Searching Algorithms
- These algorithms are useful in the case of searching a pattern within the text or searching a substring within a String.

Text : A A B A A C A A D A A B A A B A

Pattern : A A B A

A	A	B	A							A	A	B	A		
A	A	B	A	A	C	A	A	D	A	A	B	A	A	B	A
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
												A	A	B	A

Pattern Found at 0, 9 and 12

Pattern Matching Algorithms:

1. Brute force algorithm
2. Boyer –Moore algorithm
3. Knuth-Morris-Pratt algorithm

Brute Force Algorithm

Brute force algorithm:

- The Brute Force algorithm compares the pattern to the text, one character at a time, until unmatching characters are found.
- If unmatched characters are found move the pattern down the text by one character.
- The algorithm can be designed to stop on either the first occurrence of the pattern, or upon reaching the end of the text.

Algorithm BruteForceMatch(T,P):

Input: Strings T with n characters and P with m characters

Output: String index of the first substring of T matching P, or an indication that P is not a substring of T

for i:=0 to n-m do //for each candidate index in T do //

{ j:=0

while (j<m and T[i+j]=P[j]) do j:=j+1

if j=m then return i

}

return “ there is no substring of T matching P.”

Brute Force Algorithm...

Example:

Sequence

C	T	C	A	C	T	G	C	C	T	G	C	C	T	A	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Pattern

C	T	G	C	C	T	A	G
---	---	---	---	---	---	---	---

|| || x

Sequence

C	T	C	A	C	T	G	C	C	T	G	C	C	T	A	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Pattern

C	T	G	C	C	T	A	G
---	---	---	---	---	---	---	---

x

Sequence

C	T	C	A	C	T	G	C	C	T	G	C	C	T	A	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Pattern

C	T	G	C	C	T	A	G
---	---	---	---	---	---	---	---

|| x

Sequence

C	T	C	A	C	T	G	C	C	T	G	C	C	T	A	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Pattern

C	T	G	C	C	T	A	G
---	---	---	---	---	---	---	---

x

Sequence

C	T	C	A	C	T	G	C	C	T	G	C	C	T	A	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Pattern

C	T	G	C	C	T	A	G
---	---	---	---	---	---	---	---

|| || || || || || x

Sequence

C	T	C	A	C	T	G	C	C	T	G	C	C	T	A	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Pattern

C	T	G	C	C	T	A	G
---	---	---	---	---	---	---	---

x

Sequence

C	T	C	A	C	T	G	C	C	T	G	C	C	T	A	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Pattern

C	T	G	C	C	T	A	G
---	---	---	---	---	---	---	---

|| || || || || || || ||

KMP Algorithm

- KMP Algorithm is one of the most popular patterns matching algorithms. KMP stands for **Knuth Morris Pratt**.
- KMP algorithm was invented by **Donald Knuth** and **Vaughan Pratt** together and independently by **James H Morris** in the year 1970. In the year 1977, all the three jointly published KMP Algorithm.
- KMP algorithm is used to find a "**Pattern**" in a "**Text**". This algorithm compares character by character from left to right. But whenever a mismatch occurs, it uses a pre-processed table called "**Prefix Table**" to skip characters comparison while matching.
- Prefix table is also known as **LPS Table**. Here LPS stands for "**Longest proper Prefix which is also Suffix**".

Steps for Creating LPS Table (Prefix Table)

Step 1 : Define a one dimensional array with the size equal to the length of the Pattern.
(LPS[size])

Step 2 : Define variables **i & j**. Set $i = 0$, $j = 1$ and $LPS[0] = 0$.

Step 3 : Compare the characters at **Pattern[i]** and **Pattern[j]**.

Step 4 : If both are matched then set $LPS[j] = i+1$ and increment both i & j values by one.
Goto to Step 3.

Step 5 : If both are not matched then check the value of variable ' i '. If it is '0' then set $LPS[j] = 0$ and increment ' j ' value by one, if it is not '0' then set $i = LPS[i-1]$. Goto Step 3.

Step 6: Repeat above steps until all the values of LPS[] are filled.

KMP Algorithm...

Example for creating KMP Algorithm's LPS Table (Prefix Table)

Consider the following Pattern

Pattern :

0	1	2	3	4	5	6
A	B	C	D	A	B	D

Let us define LPS[] table with size 7 which is equal to length of the Pattern

LPS

0	1	2	3	4	5	6

Step 1 - Define variables i & j. Set $i = 0$, $j = 1$ and $LPS[0] = 0$.

LPS

0	1	2	3	4	5	6
0						

$i = 0$ and $j = 1$

Step 2 - Compare Pattern[i] with Pattern[j] ==> A with B.

Since both are not matching and also " $i = 0$ ", we need to set $LPS[j] = 0$ and increment 'j' value by one.

LPS

0	1	2	3	4	5	6
0	0					

$i = 0$ and $j = 2$

KMP Algorithm...

Pattern :

0	1	2	3	4	5	6
A	B	C	D	A	B	D

Step 3 - Compare Pattern[i] with Pattern[j] ==> A with C.

Since both are not matching and also "i = 0", we need to set LPS[j] = 0 and increment 'j' value by one.

0 1 2 3 4 5 6
LPS

0	0	0				
---	---	---	--	--	--	--

i = 0 and j = 3

Step 4 - Compare Pattern[i] with Pattern[j] ==> A with D.

Since both are not matching and also "i = 0", we need to set LPS[j] = 0 and increment 'j' value by one.

0 1 2 3 4 5 6
LPS

0	0	0	0			
---	---	---	---	--	--	--

i = 0 and j = 4

Step 5 - Compare Pattern[i] with Pattern[j] ==> A with A.

Since both are matching set LPS[j] = i+1 and increment both i & j value by one.

0 1 2 3 4 5 6
LPS

0	0	0	0	1		
---	---	---	---	---	--	--

i = 1 and j = 5

KMP Algorithm...

Pattern :

0	1	2	3	4	5	6
A	B	C	D	A	B	D

Step 6 - Compare Pattern[i] with Pattern[j] ==> B with B.
Since both are matching set $LPS[j] = i+1$ and increment both i & j value by one.

LPS

0	1	2	3	4	5	6
0	0	0	0	1	2	

i = 2 and j = 6

Step 7 - Compare Pattern[i] with Pattern[j] ==> C with D.
Since both are not matching and $i \neq 0$, we need to set $i = LPS[i-1]$
==> $i = LPS[2-1] = LPS[1] = 0$.

LPS

0	1	2	3	4	5	6
0	0	0	0	1	2	

i = 0 and j = 6

Step 8 - Compare Pattern[i] with Pattern[j] ==> A with D.
Since both are not matching and also " $i = 0$ ", we need to set $LPS[j] = 0$ and increment 'j' value by one.

LPS

0	1	2	3	4	5	6
0	0	0	0	1	2	0

Here LPS[] is filled with all values so we stop the process. The final LPS[] table is as follows...

LPS

0	1	2	3	4	5	6
0	0	0	0	1	2	0

KMP Algorithm...

How to use LPS Table:

- We use the LPS table to decide how many characters are to be skipped for comparison when a mismatch has occurred.
- When a mismatch occurs, check the LPS value of the previous character of the mismatched character in the pattern. If it is '0' then start comparing the first character of the pattern with the next character to the mismatched character in the text.
- If it is not '0' then start comparing the character which is at an index value equal to the LPS value of the previous character to the mismatched character in pattern with the mismatched character in the Text.
- **Example :** Apply the KMP for the following

Text : ABC ABCDAB ABCDABCDABDE
Pattern : ABCDABD

LPS[] table for the above pattern is as follows...

	0	1	2	3	4	5	6
LPS	0	0	0	0	1	2	0

KMP Algorithm...

Text : ABC ABCDAB ABCDABCDABDE

Pattern : ABCDABD

LPS[] table for the above pattern is as follows...

	0	1	2	3	4	5	6
LPS	0	0	0	0	1	2	0

Step 1 - Start comparing first character of Pattern with first character of Text from left to right

Text	A	B	C		A	B	C	D	A	B		A	B	C	D	A	B	C	D	A	B	D	E
	0	1	2	3	4	5	6																
Pattern	A	B	C	D	A	B	D																

Here mismatch occurred at Pattern[3], so we need to consider LPS[2] value. Since LPS[2] value is '0' we must compare first character in Pattern with next character in Text.

KMP Algorithm...

LPS

0	1	2	3	4	5	6
0	0	0	0	1	2	0

Step 2 - Start comparing first character of Pattern with next character of Text.

Text

A	B	C		A	B	C	D	A	B		A	B	C	D	A	B	C	D	A	B	D	E
---	---	---	--	---	---	---	---	---	---	--	---	---	---	---	---	---	---	---	---	---	---	---

Pattern

0	1	2	3	4	5	6
A	B	C	D	A	B	D

Here mismatch occurred at Pattern[6], so we need to consider LPS[5] value. Since LPS[5] value is '2' we compare Pattern[2] character with mismatched character in Text.

Step 3 - Since LPS value is '2' no need to compare Pattern[0] & Pattern[1] values

Text

A	B	C		A	B	C	D	A	B		A	B	C	D	A	B	C	D	A	B	D	E
---	---	---	--	---	---	---	---	---	---	--	---	---	---	---	---	---	---	---	---	---	---	---

Pattern

0	1	2	3	4	5	6
A	B	C	D	A	B	D

Here mismatch occurred at Pattern[2], so we need to consider LPS[1] value. Since LPS[1] value is '0' we must compare first character in Pattern with next character in Text.

KMP Algorithm...

LPS 0 1 2 3 4 5 6

0	0	0	0	1	2	0
---	---	---	---	---	---	---

Step 4 - Compare Pattern[0] with next character in Text.

Text A B C A B C D A B A B C D A B C D A B D E

Pattern 0 1 2 3 4 5 6
 A B C D A B D

Here mismatch occurred at Pattern[6], so we need to consider LPS[5] value. Since LPS[5] value is '2' we compare Pattern[2] character with mismatched character in Text.

Step 5 - Compare Pattern[2] with mismatched character in Text.

Text

A	B	C		A	B	C	D	A	B		A	B	C	D	A	B	C	D	A	B	D	E
---	---	---	--	---	---	---	---	---	---	--	---	---	---	---	---	---	---	---	---	---	---	---

Pattern

															0	1	2	3	4	5	6
															A	B	C	D	A	B	D

Here all the characters of Pattern matched with a substring in Text which is starting from index value 15. So we conclude that given Pattern found at index 15 in Text.

Boyer Moore Algorithm

- It was developed by Robert S. Boyern and J Strother Moore in 1977.
- The Boyer-Moore algorithm is consider the most efficient string-matching algorithm.
- It works the fastest when the Text is moderately sized and the pattern is relatively long.
- Boyer Moore is a combination of following two approaches:
 - 1) Bad Character Heuristic
 - 2) Good Suffix Heuristic
- Both of the above heuristics can also be used independently to search a pattern in a text.
- It processes the pattern and creates different arrays for both heuristics. At every step, it slides the pattern by the max of the slides suggested by the two heuristics. So it uses best of the two heuristics at every step.
- Boyer Moore algorithm starts matching from the last character of the pattern.

Bad Character Heuristic:

- The character of the text which doesn't match with the current character of the pattern is called the **Bad Character**.
- Upon mismatch, we shift the pattern until –
 - 1) The mismatch becomes a match.
 - 2) Pattern P move past the mismatched character.

Boyer Moore Algorithm

Case 1 : Mismatch become match We will lookup the position of last occurrence of mismatching character in pattern and if mismatching character exist in pattern then we'll shift the pattern such that it get aligned to the mismatching character in text T.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
G	C	A	A	T	G	C	C	T	A	T	G	T	G	A	C	C
T	A	T	G	T	G											



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
G	C	A	A	T	G	C	C	T	A	T	G	T	G	A	C	C
		T	A	T	G	T	G									

Case 2: Pattern move past the mismatch character We'll lookup the position of last occurrence of mismatching character in pattern and if character does not exist we will shift pattern past the mismatching character.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
G	C	A	A	T	G	C	C	T	A	T	G	T	G	A	C	C



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
G	C	A	A	T	G	C	C	T	A	T	G	T	G	A	C	C
								T	A	T	G	T	G			

DATA STRUCTURES UNIT-1 QUESTION BANK		
	Short Questions	
S. No	Question	Blooms Taxonomy Level
1	Define the term algorithm and state the criteria the algorithm should satisfy?	Remember
2	What are characteristics of an algorithm?	Understand
3	What is a pseudocode?	Understand
4	Define asymptotic notations: big 'Oh', omega and theta?	Remember
5	Describe best case, average case and worst case efficiency of an algorithm?	Remember
6	How do you measure the algorithm time complexity?	Understand
7	Describe the role of space complexity and time complexity in measuring the performance of a program?	Understand
8	Define the term Data abstraction?	Remember
9	Define data structure?	Remember
10	List linear and nonlinear data structures?	Remember
11	Define Linked List?	Remember
12	State the different types of linked lists?	Remember
13	List the basic operations carried out in a linked list?	Remember
14	List the applications of the linked list	Remember
Long Questions		
1	Explain Performance Analysis in Detail.	Understand
2	Explain time and space complexities in detail	Understand
3	Explain the different operations on singly linked list	Remember
4	Explain circular linked list operations	Remember
5	Explain doubly linked list operations	Remember

6	Differentiate between Linear and non-linear data structures.	Analyse
7	Explain the following operations in a doubly linked list.	Remember
	(i) Insert an element	
	(ii) Delete an element	
	(iii) Reverse the list	
8	Explain how to create circular linked list and insert nodes at end	Apply

DATA STRUCTURES UNIT-2 QUESTION BANK

	Short Questions	
S. No	Question	Blooms Taxonomy Level
1	Define Stack?	Remember
2	List the applications of stack?	Remember
3	Define Queue?	Remember
4	List the applications of queue?	Remember
5	Differentiate Stack and Queue?	Understand
6	List out the basic operations that can be performed on a stack and queue?	Remember
7	List the different types of queues?	Remember
8	Define Circular Queue?	Remember
9	List the operations that can be performed on Circular Queue?	Remember
10	Define Circular Queue full condition?	Remember
11	Define DEQUEUE?	Remember
12	List the operations that can be performed on DEQUEUE?	Remember
13	State the different ways of representing expressions?	Remember
14	Convert the infix expression $(a+b)-(c*d)$ into post fix form?	Apply
15	List how Stacks and Queues are represented in data structure ?	Understand
	Long Questions	
1	Write an algorithm for basic operations on Stack	Remember
2	Explain the procedure to evaluate postfix expression	Remember
3	Evaluate the following postfix expression: $6\ 2\ 3\ +\ -\ 3\ 8\ 2\ /\ +\ * \ 2\ \ 3\ +$	Apply
4	Explain the procedure to convert infix expression into postfix expression	Remember
5	Convert the following expression $A + (B * C) - ((D * E + F) / G)$ into post fix	Apply
6	Explain the operations simple Queue	Remember

7	Write an algorithm for basic operations on circular queue	Remember
8	Write a program to implement the circular queue	Remember
9	Write a program to implement the queue using linked list	Remember
10	Write a program to implement the stack using linked list	Remember

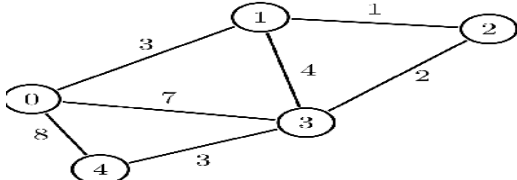
DATA STRUCTURES UNIT-3 QUESTION BANK

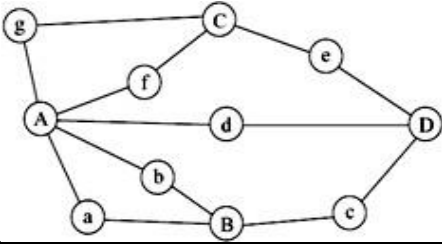
	Short Questions	
S. No	Question	Blooms Taxonomy Level
1	Define Tree?	Remember
2	Define the terms degree, siblings, depth/height, level?	Remember
3	Define path in a tree.	Remember
4	Define Binary Tree?	Remember
5	Define full binary tree?	Remember
6	Define complete binary tree?	Remember
7	State the properties of a Binary Tree?	Remember
8	Discuss how to represent Binary Tree?	Remember
9	List the different tree traversals?	Remember
10	Discuss threaded binary tree?	Remember
11	Define heap?	Remember
12	Differentiate Max-heap and Min-heap?	Understand
13	List the properties of Red black tree	Remember
14	Define the balanced factor in AVL Tree.	Remember
15	Differentiate B-tree and B+ Tree	Understand
	Long Questions	
1	Write inorder, preorder, post order traversal of the following tree <pre> graph TD 2((2)) --> 7((7)) 2 --> 5((5)) 7 --> 2_2((2)) 7 --> 6((6)) 5 --> 9((9)) 6 --> 5_5((5)) 6 --> 11((11)) 9 --> 4((4)) </pre>	Apply
2	What is a threaded binary tree ? Explain different types of threaded binary trees.	Understand
3	Construct AVL tree for following elements.63,9,19,27,18,108,99,81,44,26,91	Apply

4	Construct BST for G,H,A,B,D,J,T,R,S,E,K,L,M,V,C	Apply
5	Construct B-Tree of order 5 for the following elements: 13,14,7,1,8,5,11,17,23,6,33,12,20,26,4,16,18,19	Apply
6	Explain the Insertion and deletion cases in Red Black tree.	Understand
7	Insert the following elements in Red Black tree:25,14,45,51,55,40,32,60,57	Apply
8	Differentiate different types of Search trees .	Understand
9	Explain the Insertion and deletion operations in B+ Tree with suitable example.	Understand
10	write a program to implement tree traversals.	Apply

D

DATA STRUCTURES UNIT-4 QUESTION BANK

	Short Questions	
S. No	Question	Blooms Taxonomy Level
1	Define graph?	Remember
2	Discuss representation of graph with examples?	Understand
3	List the different graph traversals?	Remember
4	Differentiate BFS and DFS?	Understand
5	What is a spanning tree?	Remember
6	Define Minimum Spanning Tree.	Remember
7	What are the disadvantages of linear search?	Remember
8	List the algorithms used for finding minimum spanning trees.	Remember
9	What is adjacency matrix?	Remember
10	Write the algorithm for bubble sort.	Understand
11	Define Directed graph.	Remember
12	Write the algorithm for Insertion sort.	Understand
	Long Questions	
1	Write a program to implement the Quick Sort.	Understand
2	Write a program to implement the Quick Sort.	Understand
3	Construct the minimum spanning tree using Kruskals algorithm for the following graph 	Apply

4	Illustrate DFS and BFS traversals of following graph 	Apply
5	Apply the Quick sort on the following elements 54,26,20,17,44,31,77,55,93	Apply
6	Illustrate Merge Sort with suitable example.	Understand
7	Apply the Selection sort on the following elements 54,26,20,17,44,31,77,55,93	Apply
8	Apply the Heap sort on the following elements 54,26,20,17,44,31,77,55,93,8,34,22,76,100.4.33	Apply
9	Write a program to implement the Binary search	Understand
10	Write a program to implement the Linear search	Understand

DATA STRUCTURES UNIT-5 QUESTION BANK		
	Short Questions	
S. No	Question	Blooms Taxonomy Level
1	Define Hashing?	Remember
2	Explain Hash Function?	Remember
3	List different types of popular hash functions?	Remember
4	Define Collision?	Remember
5	State different types of collision resolving techniques?	Remember
6	Define Separate Chaining?	Remember
7	Define Open Addressing?	Remember
8	Define Linear probing?	Remember
9	Define Quadratic Probing?	Remember
10	Define Double Hashing?	Remember
11	Define rehashing?	Remember
12	What is the advantage of rehashing.	Remember
13	What is a pattern matching.	Remember

	Long Questions	
1	Use quadratic probing to fill the Hash table of size 11. Data elements are 23,0,52,61,78,33,100,8,90,10,14	Apply
2	Apply KMP algorithm on pattern “abaa” and text “abbbaababaab”	Apply
3	Analyze input (371, 323, 173, 199, 344, 679, 989) and hash function $h(x)=x \bmod 10$, Show the result Separate Chaining, linear probing	Apply
4	Explain Boyers Moore pattern matching algorithm with suitable example.	Understand
5	Analyze input (371, 323, 173, 199, 344, 679, 989) and hash function $h(x)=x \bmod 10$, Show the result using quadratic probing.	Apply
6	Explain Collision Resolutin techniques.	Understand

7	Apply quadratic hashing to fill the hash table of size 11 elements 20,5,10,22,33,40,50,30,51,31	Apply
8	Apply Brute force algorithm on pattern “abacab” and text “abacaabaccabacabaabb”	Apply
9	Show the each step of hash table entries for the given data set using Double hashing probing 12,45,67,88,27,78,20,62,36,55 (size=10), $h_1(x)=x \bmod 10$ and $h_2(x)=x \bmod 9$	Apply
10	Explain Extendible hashing with suitable example.	Understand